

УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ
ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ, ВСЕ

Дата генерации: 15.09.2026, 13:39:31
Файлов исходного кода: 111
Суммарный объём кода: 1.96 MB
Глав/билетов в пособии: 36

ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ

1. 1. Дерево отрезков с операциями на отрезках [id: ...]
2. 2. Двумерная разреженная матрица (Sparse Table)
3. 3. Декартово дерево (Treap) [id: treap]
4. 4. Splay-дерево [id: splay-tree]
5. 5. Двумерная матрица префиксных сумм [id: pref]
6. 6. Графы. Представление графов, DFS, BFS [id: ...]
7. 7. Графы. Планарные. Покраска [id: graph-plana]
8. 7. Графы. Планарные. Покраска [id: planarity-e]
9. 8. Графы. Компоненты связности [id: graph-comp]
10. 9. Графы. Топологическая сортировка [id: top-s]
11. 10. Графы. Компоненты сильной связности (SCC)
12. 11. Графы. Мосты [id: graph-bridges]
13. 11. Графы. Мосты [id: bridges-code]
14. 12. Графы. Точки сочленения [id: graph-articul]
15. 13. Графы. Эйлеров цикл [id: graph-euler]
16. 13. Графы. Эйлеров цикл [id: euler-path-vs-cyc]
17. Билет 14. Дейкстра [id: dijkstra]
18. Билет 15. Форд-Беллман [id: bellman-ford]
19. Билет 16. Флойд-Уоршелл [id: floyd]
20. 17. Графы. Ускорения поиска [id: pathfinding-a]
21. 17. Графы. Поиск кратчайшего пути. Ускорения [id: ...]
22. 18. Графы. Остовное дерево. Краскал [id: mst-k]
23. 18–20. Графы. Остовное дерево: Краскал, Прима, Б...
24. 19. Графы. Остовное дерево. Прима [id: mst-pri]
25. 20. Графы. Остовное дерево. Борувка [id: mst-b]

- 18. src/data/content_temp.ts
- 19. src/data/content.ts
- 20. src/data/debugRunner.ts
- 21. src/data/glossary.ts
- 22. src/data/graphContent.ts
- 23. src/data/quizzes.ts
- 24. src/data/vizStepBus.ts
- 25. src/data/vizSync.ts

Билеты (учебный контент)

- 26. src/data/tickets/ticket1_segtree.ts
- 27. src/data/tickets/ticket14_20.ts
- 28. src/data/tickets/ticket18_20_mst.ts
- 29. src/data/tickets/ticket21_24.ts
- 30. src/data/tickets/ticket6_13.ts

Интерактивные компоненты и симуляторы

- 31. src/components/ArticulationPointsViz.tsx
- 32. src/components/BellmanFordViz.tsx
- 33. src/components/BfsViz.tsx
- 34. src/components/ChapterImage.tsx
- 35. src/components/ChapterNav.tsx
- 36. src/components/ChapterQuiz.tsx
- 37. src/components/ChapterView.tsx
- 38. src/components/ComplexityViz.tsx
- 39. src/components/DfsBridgesSimulator.tsx
- 40. src/components/DijkstraViz.tsx
- 41. src/components/DPVisualizer.tsx
- 42. src/components/EulerSimulator.tsx
- 43. src/components/FloydViz.tsx
- 44. src/components/GraphComponentsViz.tsx
- 45. src/components/GraphTraversalViz.tsx
- 46. src/components/HeapViz.tsx
- 47. src/components/hints/demoKit.tsx
- 48. src/components/hints/demosExtra.tsx
- 49. src/components/hints/demosGraph.tsx
- 50. src/components/hints/demosStruct.tsx
- 51. src/components/hints/MiniDemo.tsx

86. src/layout/header.html

Утилиты

87. src/utils/cn.ts

88. src/utils/exportHtml.ts

89. src/utils/ticketLinks.ts

Скрипты сборки и экспорта

90. scripts/audit-coverage.mjs

91. scripts/audit-terms.mjs

92. scripts/export/build-guide-pdf.mjs

93. scripts/export/build-pdf.mjs

94. scripts/export/capture-viz-shots.mjs

95. scripts/export/collect-code.mjs

96. scripts/export/common.mjs

97. scripts/export/html-to-blocks.mjs

98. scripts/export/render-pages.mjs

99. scripts/verify/browser-checks.tsx

100. scripts/verify/check-links.mjs

101. scripts/verify/check-reference-code.mjs

102. scripts/verify/dom-env.mjs

103. scripts/verify/live-matrix.test.mjs

104. scripts/verify/probe-compiler.tsx

105. scripts/verify/README.md

106. scripts/verify/run-browser-checks.mjs

107. scripts/verify/run-probe.mjs

CI / автоматизация

108. .github/workflows/deploy-pages.yml

109. .github/workflows/rebuild-pdf.yml

110. .github/workflows/verify.yml

Прочее

111. .ai_skills/universal_guide_authoring/SKILL.md

```
contents: read
pages: write
id-token: write
```

```
concurrency:
  group: pages
  cancel-in-progress: true
```

```
jobs:
  build:
    name: Build static site
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v6

      - name: Setup Node.js
        uses: actions/setup-node@v7
        with:
          node-version: 24
          cache: npm

      - name: Install dependencies
        run: npm ci --no-audit --no-fund

      - name: Build for the repository subpath
        env:
          VITE_BASE: /algv0/
        run: npm run build

      - name: Make GitHub Pages fallback
        run: cp dist/index.html dist/404.html

      - name: Upload Pages artifact
        uses: actions/upload-pages-artifact@v5
        with:
          path: ./dist
```

```
+ group: pages
+ cancel-in-progress: true
+
+ jobs:
+   build:
+     name: Build static site
+     runs-on: ubuntu-latest
+     steps:
+       - name: Checkout
+         uses: actions/checkout@v6
+
+       - name: Setup Node.js
+         uses: actions/setup-node@v7
+         with:
+           node-version: 24
+           cache: npm
+
+       - name: Install dependencies
+         run: npm ci --no-audit --no-fund
+
+       - name: Build for the repository subpath
+         env:
+           VITE_BASE: /algv0/
+         run: npm run build
+
+       - name: Make GitHub Pages fallback
+         run: cp dist/index.html dist/404.html
+
+       - name: Upload Pages artifact
+         uses: actions/upload-pages-artifact@v5
+         with:
+           path: ./dist
+
+   deploy:
+     name: Publish to GitHub Pages
+     needs: build
+     runs-on: ubuntu-latest
+     environment:
```

```
      required: false
      default: "150"

concurrency:
  group: rebuild-pdf-`${{ github.ref }}
  cancel-in-progress: true

permissions:
  contents: write

jobs:
  rebuild-pdf:
    name: код → txt → png → pdf
    runs-on: ubuntu-latest
    # не реагируем на собственный коммит бота
    if: "!contains(github.event.head_commit.message, '[bot]')"

    steps:
      - name: Checkout
        uses: actions/checkout@v6
        with:
          fetch-depth: 0
          persist-credentials: true

      - name: Setup Node.js
        uses: actions/setup-node@v7
        with:
          node-version: "24"
          cache: npm

      - name: Install ImageMagick + DejaVu fonts
        run: |
          sudo apt-get update -qq
          sudo apt-get install -y --no-install-recommends imagemagick
          # На Ubuntu политика ImageMagick иногда запрещает конвертировать текст в картинку
          # разрешаем то, что нужно пайплайну (text -> png)
          for policy in /etc/ImageMagick-6/policy.xml /usr/share/ImageMagick-6/policy.xml; do
            if [ -f "$policy" ]; then
              sudo sed -i 's/<policy domain="resource" name="text" value="no"/<policy domain="resource" name="text" value="yes"/g' $policy
            fi
          done
```

```
      name: full-code-export
      path: |
        public/export/full_code_all_pages.pdf
        public/export/full_code_all_pages.txt
      if-no-files-found: error
      retention-days: 30

- name: Upload artifacts (PNG-страницы)
  uses: actions/upload-artifact@v7
  with:
    name: full-code-pages-png
    path: tmp/export-pages/*.png
    if-no-files-found: error
    retention-days: 7

- name: Commit rebuilt export back to the branch
  run: |
    git config user.name "github-actions[bot]"
    git config user.email "41898282+github-action@users.noreply.github.com"
    git add -- public/export/full_code_all_pages.*
    if git diff --cached --quiet; then
      echo "Экспорт не изменился — коммит не нужен"
      exit 0
    fi
    git commit -m "chore(export): пересборка full-code-export"
    git push origin "HEAD:${GITHUB_REF_NAME}"
  ....

### Patch относительно `main`

```diff
diff --git a/.github/workflows/rebuild-pdf.yml b/.github/workflows/rebuild-pdf.yml
index bdc3c33..ff49c0c 100644
--- a/.github/workflows/rebuild-pdf.yml
+++ b/.github/workflows/rebuild-pdf.yml
@@ -42,15 +42,15 @@ jobs:

 steps:
```

Универсальное пособие • полный исходный код

-----  
### Полное содержимое после изменений

```
````html
<!doctype html>
<html lang="ru" class="dark bg-slate-950 text-slate-100" >
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, height=device-height" />
    <link rel="icon" type="image/svg+xml" href="%BASE_URL%/public/favicon.svg" />
    <title> Дискретка для тупых – Интерактивный гид по дискреткам
  </head>
  <body class="bg-slate-950 text-slate-100 min-h-screen" >
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
````
```

### Patch относительно `main`

```
````diff
diff --git a/index.html b/index.html
index cdea4c7..a44c135 100644
--- a/index.html
+++ b/index.html
@@ -3,6 +3,7 @@
   <head>
     <meta charset="UTF-8" />
     <meta name="viewport" content="width=device-width, height=device-height" />
+    <link rel="icon" type="image/svg+xml" href="%BASE_URL%/public/favicon.svg" />
     <title> Дискретка для тупых – Интерактивный гид по дискреткам
   </head>
   <body class="bg-slate-950 text-slate-100 min-h-screen" >
````
```

## `public/favicon.svg`

### Полное содержимое после изменений



```

import { SimulatorHub } from "../components/SimulatorHub
import { PythonCompiler } from "../components/PythonComp

export default function App() {
 const [activeTab, setActiveTab] = useState<"guide" |
 const [activeChapterId, setActiveChapterId] = useStat
 const [modalVizId, setModalVizId] = useState<string |

 const index = Math.max(
 0,
 chapters.findIndex((c) => c.id === activeChapterId)
);
 const activeChapter = chapters[index] ?? chapters[0];
 const prev = index > 0 ? chapters[index - 1] : undefin
 const next = index < chapters.length - 1 ? chapters[in

 const goTo = useCallback((id: string) => {
 setActiveChapterId(id);
 window.scrollTo({ top: 0, behavior: "smooth" });
 }, []);

 // Стрелки ← → листают темы (как на обучающих сайтах)
 useEffect(() => {
 const onKey = (e: KeyboardEvent) => {
 const target = e.target as HTMLElement | null;
 if (target && /^(INPUT|TEXTAREA|SELECT)$/.test(ta
 if (e.metaKey || e.ctrlKey || e.altKey) return;
 if (e.key === "ArrowRight" && next) goTo(next.id)
 if (e.key === "ArrowLeft" && prev) goTo(prev.id);
 };
 window.addEventListener("keydown", onKey);
 return () => window.removeEventListener("keydown",
 }, [goTo, next, prev]);

 const progress = useMemo(() => ((index + 1) / chapter

 return (
 <div className="min-h-screen bg-slate-950 text-slate

```

```

 <ChapterView
 key={activeChapter.id}
 chapter={activeChapter}
 prev={prev}
 next={next}
 index={index}
 total={chapters.length}
 onGo={goTo}
 onOpenSimulator={setModalVizId}
 />
 </main>
</div>
</>
) : activeTab === "simulator" ? (
 <main className="px-4 sm:px-6 lg:px-10 py-6 lg:py-8" >
 <SimulatorHub
 onOpen={setModalVizId}
 onGoChapter={({id}) => {
 setActiveTab("guide");
 goTo(id);
 }}
 />
 </main>
) : (
 <PythonCompiler />
)
}

<SimulatorModal vizId={modalVizId} onClose={() => {
 setModalVizId(null);
}} />

<footer className="p-8 text-center text-slate-600" >
 © 2026 Universal Educational Guide. Интерактивное
</footer>
</div>
);
}
},

```

```
`src/components/Navbar.tsx`
```

```
Полное содержимое после изменений
```

```
```tsx
```

```
import React, { useState } from 'react';
import { BookOpen, Cpu, Sparkles, FileDown, FileText, C
import { chapters } from '../data/content';
import { downloadBookHtml } from '../utils/exportHtml';
```

```
interface NavbarProps {
  activeTab: 'guide' | 'simulator' | 'compiler';
  setActiveTab: (tab: 'guide' | 'simulator' | 'compiler
}
```

```
export const Navbar: React.FC<NavbarProps> = ({ activeT
  const [busy, setBusy] = useState(false);
```

```
  const saveBook = async () => {
    setBusy(true);
    try {
      await downloadBookHtml(chapters);
    } finally {
      setBusy(false);
    }
  };
};
```

```
  return (
    <header className="bg-slate-900 border-b border-sla
      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg
        ap-3 lg:gap-0">
          <div className="flex items-center gap-3 w-full
            <div className="bg-gradient-to-tr from-indigo
              k-0">
                <Sparkles className="w-6 h-6" />
              </div>
            <div className="min-w-0 flex-1">
```

```

        className={`flex-1 lg:flex-none flex items-
uration-200 whitespace-nowrap ${
            activeTab === 'compiler'
                ? 'bg-emerald-600 text-white shadow-lg
                : 'text-slate-400 hover:text-white'
        }}
    >
    <Terminal className="w-4 h-4" /> Python
</button>
<a
    id="download-pdf-btn"
    href={` ${import.meta.env.BASE_URL}export/full
download="full_code_all_pages.pdf"
    target="_blank"
    rel="noreferrer"
    className="flex items-center justify-center
over:bg-emerald-600/20 border border-emerald
    title="Скачать полный PDF сборник всех стра
    >
    <FileDown className="w-4 h-4" /> Скачать PD
</a>
<a
    id="download-txt-btn"
    href={` ${import.meta.env.BASE_URL}export/full
download="full_code_all_pages.txt"
    target="_blank"
    rel="noreferrer"
    className="flex items-center justify-center
: bg-sky-600/20 border border-sky-500/30 tra
    title="Скачать полный TXT файл со всем кодом
    >
    <FileText className="w-4 h-4" /> TXT
</a>
<button
    id="download-html-btn"
    onClick={saveBook}
    disabled={busy}
    className="flex items-center justify-center

```

```

+         <button
+             id="tab-compiler-btn"
+             onClick={() => setActiveTab('compiler')}
+             className={`flex-1 lg:flex-none flex items
duration-200 whitespace-nowrap ${
+                 activeTab === 'compiler'
+                     ? 'bg-emerald-600 text-white shadow-lg
+                     : 'text-slate-400 hover:text-white'
+             }`}
+         >
+         <Terminal className="w-4 h-4" /> Python
+     </button>
+     <a
+         id="download-pdf-btn"
+         href="/export/full_code_all_pages.pdf"
+         href={`${import.meta.env.BASE_URL}export/full
+         download="full_code_all_pages.pdf"
+         target="_blank"
+         rel="noreferrer"
@@ -73,7 +84,7 @@ export const Navbar: React.FC<NavbarP
    </a>
    <a
        id="download-txt-btn"
        href="/export/full_code_all_pages.txt"
        href={`${import.meta.env.BASE_URL}export/full
        download="full_code_all_pages.txt"
        target="_blank"
        rel="noreferrer"
    ....

```

`src/components/PythonCompiler.tsx`

Полное содержимое после изменений

```

````tsx
import { useCallback, useState } from "react";
import { CheckCircle2, ExternalLink, Info, Loader2, Play

```

```

export function PythonCompiler() {
 const [code, setCode] = useState(STARTER_CODE);
 const [stdin, setStdin] = useState("");
 const [output, setOutput] = useState("Нажмите «Запустить»");
 const [running, setRunning] = useState(false);
 const [runtimeReady, setRuntimeReady] = useState(false);
 const [runtimeMessage, setRuntimeMessage] = useState("");

 const runCode = useCallback(async () => {
 if (running) return;

 setRunning(true);
 setOutput("");
 setRuntimeMessage(runtimeReady ? "Выполняю код..." : "Готово: программа ничего не выполнила");

 const stdout: string[] = [];
 const stderr: string[] = [];
 const inputLines = stdin.split(/\r?\n/);

 try {
 const runtime = await getPythonRuntime();
 setRuntimeReady(true);
 setRuntimeMessage("Python готов — код выполняется");

 runtime.setStdout({ batched: (message) => stdout.push(message) });
 runtime.setStderr({ batched: (message) => stderr.push(message) });
 runtime.setStdin({ stdin: () => (inputLines.length > 0 ? inputLines.shift() : null) });
 await runtime.runPythonAsync(code);

 const text = [...stdout, ...stderr].join("");
 setOutput(text || "Готово: программа ничего не выполнила");
 } catch (error) {
 const captured = [...stdout, ...stderr].join("");
 setOutput(`${captured}${captured && !captured.endsWith("\n") ? "\n" : ""}`);
 setRuntimeMessage("Не удалось выполнить код — подробности в консоли");
 } finally {
 setRunning(false);
 }
 }
}
```

```

 <span className="text-[11px] text-slate
</div>
<div className="flex items-center gap-2">
 <button
 type="button"
 onClick={reset}
 className="inline-flex items-center gap-2 text-slate-800 transition-colors"
 >
 <RotateCcw className="w-3.5 h-3.5" />
 </button>
 <button
 type="button"
 onClick={() => void runCode()}
 disabled={running}
 className="inline-flex items-center gap-2 text-white text-xs font-bold transition-colors"
 >
 {running ? <Loader2 className="w-3.5 h-3.5" /> : "Запускаю..."}
 {running ? "Запускаю..." : "Запустить"}
 </button>
</div>
</div>

<textarea
 value={code}
 onChange={(event) => setCode(event.target.value)}
 onKeyDown={(event) => {
 if ((event.ctrlKey || event.metaKey) && event.defaultPrevented) {
 void runCode();
 }
 }}
 spellCheck={false}
 aria-label="Код Python"
 className="block w-full min-h-[360px] resize-none focus:ring-2 focus:ring-inset focus:ring-slate-500"
 placeholder="Напишите Python-код..."

```

```

 <div className="flex gap-2 rounded-xl border border-gray-200 p-4" >
 <Info className="w-4 h-4 shrink-0 text-indigo-600" >
 Ctrl или Cmd + Enter запускает код. Введите код в поле ввода.

 </div>
 <div className="flex gap-2 rounded-xl border border-gray-200 p-4" >
 <Info className="w-4 h-4 shrink-0 text-amber-600" >
 Первый запуск скачивает примерно несколько килобайт кода.

 </div>
 <div className="flex gap-2 rounded-xl border border-gray-200 p-4" >
 <Info className="w-4 h-4 shrink-0 text-emerald-600" >
 Код выполняется в браузере и не отправляется на сервер.

 </div>
 </div>
</div>
</main>
);
}
....
```

### Patch относительно `main`

```
````diff
diff --git a/src/components/PythonCompiler.tsx b/src/components/PythonCompiler.tsx
new file mode 100644
index 00000000..1113e0f
--- /dev/null
+++ b/src/components/PythonCompiler.tsx
@@ -0,0 +1,205 @@
+import { useCallback, useState } from "react";
+import { CheckCircle2, ExternalLink, Info, Loader2, Plus } from "lucide-react";
+
+const PYODIDE_VERSION = "0.27.7";
+const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/bin/pyodide.js`;
+const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/index.html`;
+
+const STARTER_CODE = `# Первый запуск загрузит Python и установит библиотеки
+name = "алгоритмы"
+
+def main():
+    print("Привет, мир!")
+    return "Ваш код выполнен успешно!"
+
+main()
+
+`;
```



```

+
+ const runCode = useCallback(async () => {
+   if (running) return;
+
+   setRunning(true);
+   setOutput("");
+   setRuntimeMessage(runtimeReady ? "Выполняю код..." :
+
+   const stdout: string[] = [];
+   const stderr: string[] = [];
+   const inputLines = stdin.split(/\r?\n/);
+
+   try {
+     const runtime = await getPythonRuntime();
+     setRuntimeReady(true);
+     setRuntimeMessage("Python готов – код выполняется");
+
+     runtime.setStdout({ batched: (message) => stdout
+     runtime.setStderr({ batched: (message) => stderr
+     runtime.setStdin({ stdin: () => (inputLines.length
+     await runtime.runPythonAsync(code);
+
+     const text = [...stdout, ...stderr].join("");
+     setOutput(text || "Готово: программа ничего не выв
+   } catch (error) {
+     const captured = [...stdout, ...stderr].join("")
+     setOutput(`${captured}${captured && !captured.en
+     setRuntimeMessage("Не удалось выполнить код – пр
+   } finally {
+     setRunning(false);
+   }
+ }, [code, stdin, running, runtimeReady]);
+
+ const reset = () => {
+   setCode(STARTER_CODE);
+   setStdin("");
+   setOutput("Нажмите «Запустить», чтобы увидеть резу.
+ };

```

```

late-800 transition-colors"
+
+       >
+       <RotateCcw className="w-3.5 h-3.5" />
+       </button>
+       <button
+       type="button"
+       onClick={() => void runCode()}
+       disabled={running}
+       className="inline-flex items-center
-60 text-white text-xs font-bold transition-colors"
+       >
+       {running ? <Loader2 className="w-3.5
+       {running ? "Запускаю..." : "Запустить"}
+       </button>
+       </div>
+     </div>
+
+     <textarea
+     value={code}
+     onChange={(event) => setCode(event.target
+     onKeyDown={(event) => {
+       if ((event.ctrlKey || event.metaKey) &
+       event.preventDefault();
+       void runCode();
+     }
+     }}
+     spellCheck={false}
+     aria-label="Код Python"
+     className="block w-full min-h-[360px] re
e-none focus:ring-2 focus:ring-inset focus:ring-emera
+     placeholder="Напишите Python-код..."
+     />
+
+     <div className="border-t border-slate-800
+     <label className="block px-4 pt-3 text-[
+       Ввод для input() • по одной строке на
+     </label>
+     <textarea

```

```

+         </div>
+         <div className="flex gap-2 rounded-xl border
+             <Info className="w-4 h-4 shrink-0 text-eme
+             <span>Код выполняется в браузере и не отпра
+         </div>
+     </div>
+ </div>
+ </main>
+ );
+}
+...

```

`vite.config.ts`

Полное содержимое после изменений

```

````typescript
import path from "path";
import { fileURLToPath } from "url";
import tailwindcss from "@tailwindcss/vite";
import react from "@vitejs/plugin-react";
import { defineConfig } from "vite";
import { viteSingleFile } from "vite-plugin-singlefile"

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

// https://vite.dev/config/
export default defineConfig({
 // На GitHub Pages приложение живёт в /algv0/, локальн
 // VITE_BASE можно переопределить, если репозиторий б
 base: process.env.VITE_BASE || "/",
 plugins: [react(), tailwindcss(), viteSingleFile()],
 resolve: {
 alias: {
 "@": path.resolve(__dirname, "src"),
 },
 },
},

```

Универсальное пособие • полный исходный код

-----  
КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 12 | РАЗМЕР: 7  
-----

# AI Agent Custom Instructions

You are the author of a Universal Educational Guide (Универсальное

**\*\*CRITICAL DIRECTIVE\*\***:

Before you start creating new chapters, editing content or using the `view_file` tool to read the skill instructions located at `.ai_skills/universal_guide_authoring/SKILL.md``

This file contains the mandatory rich HTML templates, styles and components required to maintain the high quality of this universal guide.

Do NOT generate plain markdown chapters or invent new HTML elements.

-----  
ФАЙЛ 3/111: GEMINI.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 16 | РАЗМЕР: 2  
-----

# Gemini App Canvas Instructions

Ты — AI-ассистент, работающий в Gemini App Canvas (или Gemini App Canvas). При запросе пользователя создать или сгенерировать новую страницу **\*\*строго\*\*** придерживаться готовых шаблонов разметки, стилей и компонентов.

## Правила для малых страниц (Билетов / Глав)

- \*\*Единый стиль:\*\*** Всегда используй Tailwind CSS и HTML-шаблоны.
- \*\*Структура JSON/TS:\*\*** Каждая новая малая страница должна быть описана в формате `{ "type": "page", "content": ... }` для вставки в ``src/data/content.json``.
- \*\*Строгая структура контента:\*\***
  - Заголовок с цветным бейджем (например, ``bg-indigo-100``).
  - Определение/правило в центре (``border-blue-500`` или ``border-purple-500``).
  - Обязательная сетка аналогий (2 колонки: неформальный язык и формальный язык).

---

ФАЙЛ 6/111: package.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 54 | РАЗМЕР: 2

---

```
{
 "name": "algv0-exam-guide",
 "private": true,
 "version": "0.0.0",
 "type": "module",
 "scripts": {
 "dev": "vite",
 "build": "vite build",
 "preview": "vite preview",
 "audit:content": "node scripts/audit-coverage.mjs",
 "audit:terms": "node scripts/audit-terms.mjs",
 "export:txt": "node scripts/export/collect-code.mjs",
 "export:png": "node scripts/export/render-pages.mjs",
 "export:pdf": "node scripts/export/build-pdf.mjs",
 "export": "npm run export:txt && npm run export:png",
 "export:guide": "node scripts/export/build-guide-pdf.mjs",
 "export:shots": "node scripts/export/capture-viz-shots.mjs",
 "verify:unit": "node scripts/verify/live-matrix.test.mjs",
 "verify:code": "node scripts/verify/check-reference.mjs",
 "verify:links": "node scripts/verify/check-links.mjs",
 "verify:dom": "node scripts/verify/run-browser-check.mjs",
 "verify:probe": "node scripts/verify/run-probe.mjs",
 "verify": "npm run verify:unit && npm run verify:links",
 },
 "dependencies": {
 "clsx": "2.1.1",
 "lucide-react": "^1.21.0",
 "motion": "^12.40.0",
 "pdfkit": "^0.20.1",
 "react": "19.2.6",
 "react-dom": "19.2.6",
 }
}
```

Универсальное пособие • полный исходный код

```

npm run dev # предпросмотр на http://0.0.0.0:5
npm run build # сборка в dist/ (single-file)
````
```

Экспорт кода: код → txt → png → pdf

Пайплайн собирает ****весь**** исходный код репозитория в о

| Команда | Шаг | Результат |
|----------------------|-----------|---|
| `npm run export:txt` | код → txt | `public/export/full_code_all_pages.txt` |
| `npm run export:png` | txt → png | `tmp/export-pages/` |
| `npm run export:pdf` | png → pdf | `public/export/full_code_all_pages.pdf` |
| `npm run export` | всё сразу | txt + png + pdf |

Скрипты лежат в `scripts/export/`. Требуется ImageMagick

```
```bash  
sudo apt-get install -y imagemagick fonts-dejavu-core
````
```

Настройки через переменные окружения:

- * `EXPORT\_DENSITY=200` – DPI растеризации (по умолчанию 300)
- * `EXPORT\_GRAYSCALE=1` – оттенки серого вместо 1-битного

Автоматизация (CI)

`github/workflows/rebuild-pdf.yml` запускается ****на каж**
пересобирает TXT → PNG → PDF, кладёт PNG-страницы и док
и коммитит обновлённые `public/export/full\_code\_all\_pages`
(с меткой `[skip ci]`, чтобы не заиклиться).

Приложение отдаёт эти файлы по ссылкам `/export/full\_code`

Аудит контента

```
```bash
```

## ## Заголовки

- \* Заголовки секций: Используют стандартный тег `<h2>` и
- \* Подзаголовки: Используют тег `<h3>`.

## ## Текст и параграфы

- \* Обычный текст содержится в тегах `<p>`.
- \* Блоки "Шиза" (Аналогии) и другая текстовая инфографика параграфы `<p>`.

## ## Математические формулы

В приложении используется MathJax.

- \* В самом исходном коде страницы (и внутри тегов) формулы.
- \* Таким образом, при извлечении текстового содержимого математически остаются в валидном Markdown-формате латеха (и

## ## Картинки и визуализации (Схемы)

Вместо обычных `<img>` в приложении используются векторы

- \* Они помещены внутрь оберток с классом `<div class="img">`
- \* Все отрисовки находятся внутри векторного тега `<svg>`
- \* Подпись к картинке, если она есть, идет сразу после `<img>`

## ## Списки

- \* Стандартные списки оформлены через `<ul class="list-n">`

## ## Валидный экспорт в Markdown (встроенный)

Экспорт перехватывает основные структурные компоненты HTML

1. Заголовки `<h2>` и `<h3>` заменяются на `##` и `###`
2. Элементы `.analogy-badge` получают выделение маркера
3. Теги `<svg>` заменяются строкой-заглушкой `[ Иллюстрация ]`.
4. Элементы списка `<li>` получают маркер `-`.
5. Результат извлекается в виде текста и чистится от лишнего

Универсальное пособие • полный исходный код

```

* `stack-dfs` → `StackViz.tsx` — **Стек и DFS**
* `queue-bfs` → `QueueViz.tsx` + `BfsViz.tsx` — **Очередь и BFS**
* `heap-beam-search` → `HeapViz.tsx` — **Куча и лучевой поиск**
* `segment-trees` → `SegmentTreeVisualizer.tsx` — **Деревья отрезков**
* `dynamic-programming` → `DPVisualizer.tsx` — **Динамическое программирование**

```

## Уровень 3. Есть 2D, но НЕТ бытовой аналогии («сухари»)

Глава	id	Комментарий
15. Планарность: K5, K3,3 и формула Эйлера	`planarity-euler`	т «на пальцах».
16. Флойд (Флойд–Уоршелл)	`floyd`	Есть матричный алгоритм
17. Алгоритм Джонсона	`johnson-algo`	Перевзвешивание
Обзор: Кратчайшие пути и Графы	`intro`	Вводная страница
Эйлер: Путь ≠ Цикл	`euler-path-vs-cycle`	Ключевая концепция

## Уровень 4. Есть аналогия, но НЕТ 2D-визуализации («сложно»)

> Замечание: «5. Двумерная матрица префиксных сумм» пока не реализована

Глава	id	Чего не хватает
3. Декартово дерево (Treap)	`treap`	Нет ни визуализации, ни кода
8. Графы. Компоненты связности	`graph-components`	Нет кода
24. Классы сложности, сведение задач	`complexity-classes`	Нет кода

## Сводная таблица по всем главам

#	Глава	Аналогий	2D-виз.	SVG	Картинка
---	---	---	---	---	---



ФАЙЛ 9/111: tsconfig.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 32 | РАЗМЕР: 6

---

```
{
 "compilerOptions": {
 "target": "ES2020",
 "useDefineForClassFields": true,
 "lib": ["ES2020", "DOM", "DOM.Iterable"],
 "module": "ESNext",
 "skipLibCheck": true,
 "types": ["node"],

 /* Bundler mode */
 "moduleResolution": "bundler",
 "allowImportingTsExtensions": true,
 "resolveJsonModule": true,
 "isolatedModules": true,
 "noEmit": true,
 "jsx": "react-jsx",

 /* Path mapping */
 "baseUrl": ".",
 "paths": {
 "@/*": ["src/*"]
 },

 /* Linting */
 "strict": true,
 "noUnusedLocals": true,
 "noUnusedParameters": true,
 "noFallthroughCasesInSwitch": true
 },
 "include": ["src", "vite.config.ts"]
}
```

```
 },
 });
}
```

---

## РАЗДЕЛ: ЯДРО ПРИЛОЖЕНИЯ

---

ФАЙЛ 11/111: src/App.tsx

КАТЕГОРИЯ: Ядро приложения | СТРОК: 218 | РАЗМЕР: 11.3 КБ

```
import { useCallback, useEffect, useMemo, useRef, useState } from "react";
import { PanelLeftClose, PanelLeftOpen } from "lucide-react";
import { chapters } from "../data/content";
import { Navbar } from "../components/Navbar";
import { Sidebar } from "../components/Sidebar";
import { MobileToc } from "../components/MobileToc";
import { ChapterView } from "../components/ChapterView";
import { ChapterNav } from "../components/ChapterNav";
import { SimulatorModal } from "../components/hints/SimulatorModal";
import { LiteGuide } from "../components/LiteGuide";
import { PythonCompiler } from "../components/PythonCompiler";
```

```
/**
```

```
 * «Лёгкий» режим для печати и внешних парсеров: `?lite`
```

```
 * или `?lite=1&topic=mst` (одна тема). Без навигации, без поиска.
```

```
 */
```

```
function readLiteMode(): { lite: boolean; topic?: string } {
 if (typeof window === "undefined") return { lite: false };
 const q = new URLSearchParams(window.location.search);
 const lite = q.has("lite") || q.get("view") === "lite";
 return { lite, topic: q.get("topic") ?? undefined };
}
```

```
export default function App() {
 const liteMode = useMemo(readLiteMode, []);
```

```

 setSidebarMode((m) => {
 const next = m === "wide" ? "compact" : m === "co
 window.localStorage.setItem("sidebarMode", next);
 return next;
 });
 }, []);
const sidebarCollapsed = sidebarMode === "hidden";

// Десктопный отступ контента = ширине панели компиля
const contentRef = useRef<HTMLDivElement>(null);
useEffect(() => {
 const apply = () => {
 const el = contentRef.current;
 if (!el) return;
 el.style.paddingRight = compilerOpen && window.inn
 };
 apply();
 window.addEventListener("resize", apply);
 return () => window.removeEventListener("resize", a
}, [compilerOpen, compilerWidth]);

const index = Math.max(
 0,
 chapters.findIndex((c) => c.id === activeChapterId)
);
const activeChapter = chapters[index] ?? chapters[0];
const prev = index > 0 ? chapters[index - 1] : undefi
const next = index < chapters.length - 1 ? chapters[in

const goTo = useCallback((id: string) => {
 setActiveChapterId(id);
 window.scrollTo({ top: 0, behavior: "smooth" });
}, []);

// Открыть компилятор и подогнать визуализацию вверх:
const openCompilerWithViz = useCallback(() => {
 setCompilerOpen(true);
 requestAnimationFrame(() => {

```

```

<div className="flex flex-col lg:flex-row max-w-
 {/* Сайдбар только на десктопе — на мобильном
 <div
 id="sidebar-root"
 className={`hidden lg:block shrink-0 border
 -all duration-200 print:hidden ${sidebarCol
 " : "lg:w-56"}`}
 >
 {!sidebarCollapsed && (
 <Sidebar selectedChapterId={activeChapterId}
)}
 </div>
 {/* Вкладка-переключатель содержания у левого
 <button
 type="button"
 onClick={toggleSidebar}
 className="hidden lg:flex items-center gap-1
 border-slate-700 bg-slate-900/90 px-1 py-3
 aria-label={sidebarMode === "wide" ? "Сжать
 ание"}
 title={sidebarMode === "wide" ? "Сжать содер
 }
 >
 {sidebarCollapsed ? <PanelLeftOpen className="
 <span className="text-[10px] font-bold [wri
 </button>

 <main id="main-content" className="flex-1 min
 {/* Шапка главы с быстрыми переходами */}
 <div className="flex items-center justify-b
 <div className="min-w-0">
 <div className="text-[10px] uppercase t
 {activeChapter.category || "Универсальн
 </div>
 <h2 className="text-base sm:text-xl font
 {activeChapter.title}
 </h2>
 </div>
 </div>

```

```
 onHeightChange={setCompilerHeight}
 onClose={() => setCompilerOpen(false)}
 />
))
 </div>
);
}
```

---

ФАЙЛ 12/111: src/hooks/useTermHints.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 129 | РАЗМЕР: 5.1 K

---

```
import { useEffect } from "react";
import { GLOSSARY_LABELS } from "../data/glossary";

/**
 * Ограничители, чтобы текст не рябил, но и чтобы подсказки
 *
 * Считаем не «сколько раз встретился термин», а «сколько
 * написание». Иначе в главе про splay первые же «Splay
 * лимит, и слова «Zig-Zag», «вращений», «AVL-дереве» о
 */
const MAX_PER_SURFACE = 2;
const MAX_PER_TERM = 8;

const SKIP_SELECTOR = "code, pre, script, style, a, text";

/** Числовые константы TreeWalker: глобального `NodeFile`
const SHOW_TEXT = 4;
const FILTER_ACCEPT = 1;
const FILTER_REJECT = 2;

const escapeRe = (s: string) => s.replace(/[\.*\+\?\^\$\{\}\(\)]/g, "\\$1");

/** Одна большая регулярка из всех меток: длинные раньше
function buildRegex(): RegExp {
```

```

 re.lastIndex = 0;
 if (!re.test(text)) continue;
 re.lastIndex = 0;

 const frag = document.createDocumentFragment();
 let last = 0;
 let m: RegExpExecArray | null;

 while ((m = re.exec(text)) !== null) {
 const surface = m[1].toLowerCase();
 const id = labelToId.get(surface);
 if (!id) continue;

 const usedTerm = perTerm.get(id) ?? 0;
 const usedSurface = perSurface.get(surface) ?? 0;
 if (usedTerm >= MAX_PER_TERM || usedSurface >= MAX_PER_SURFACE) continue;
 perTerm.set(id, usedTerm + 1);
 perSurface.set(surface, usedSurface + 1);

 if (m.index > last) frag.appendChild(document.createElement("span"));
 const span = document.createElement("span");
 span.className = "term-hint";
 span.dataset.termId = id;
 span.tabIndex = 0;
 span.setAttribute("role", "button");
 span.setAttribute("aria-label", `Подсказка: ${m[1]}`);
 span.textContent = m[1];
 frag.appendChild(span);
 last = m.index + m[1].length;
 }

 if (last === 0) continue;
 if (last < text.length) frag.appendChild(document.createElement("span"));
 textNode.parentNode?.replaceChild(frag, textNode);
}

/**

```

```
.neon-glow-blue {
 box-shadow: 0 0 15px rgba(59, 130, 246, 0.5);
}
.neon-glow-emerald {
 box-shadow: 0 0 15px rgba(16, 185, 129, 0.5);
}
.neon-glow-rose {
 box-shadow: 0 0 15px rgba(244, 63, 94, 0.5);
}
.neon-glow-yellow {
 box-shadow: 0 0 20px rgba(234, 179, 8, 0.8);
}
.no-scrollbar::-webkit-scrollbar {
 display: none;
}
.no-scrollbar {
 scrollbar-width: none;
}
}

@keyframes pulse-gold {
 0%,
 100% {
 stroke: #eab308;
 stroke-width: 5;
 filter: drop-shadow(0 0 2px rgba(234, 179, 8, 0.5))
 }
 50% {
 stroke: #fef08a;
 stroke-width: 8;
 filter: drop-shadow(0 0 12px rgba(234, 179, 8, 0.9))
 }
}

.animate-pulse-gold {
 animation: pulse-gold 1.5s infinite;
}
```

```
 color: #fff;
 outline: none;
}
@media (hover: none) {
 .term-hint {
 border-bottom-style: solid;
 background: rgba(99, 102, 241, 0.12);
 }
}

/* Ссылка на другой билет («...берите Форда–Беллмана (билет
.ticket-link {
 color: #a5b4fc;
 border-bottom: 1px dotted rgba(165, 180, 252, 0.7);
 text-decoration: none;
 border-radius: 3px;
 padding: 0 1px;
 cursor: pointer;
 transition:
 background-color 0.15s ease,
 color 0.15s ease;
}
.ticket-link:hover,
.ticket-link:focus-visible {
 background: rgba(99, 102, 241, 0.25);
 color: #fff;
 outline: none;
}

/* Контент главы приходит готовым HTML – чиним его под
.chapter-body {
 max-width: 100%;
 overflow-wrap: anywhere;
}

.chapter-body :where(section, div, p, li) {
 min-width: 0;
}
```



```
 line-height: 1.3 !important;
 }
 .chapter-body :where(h3) {
 font-size: 1.05rem !important;
 line-height: 1.35 !important;
 }
 .chapter-body :where(.p-6, .p-8) {
 padding: 0.9rem !important;
 }
 .chapter-body :where(.text-3xl) {
 font-size: 1.4rem !important;
 }
 .chapter-body :where(.text-2xl) {
 font-size: 1.2rem !important;
 }
 .chapter-body :where(.text-xl) {
 font-size: 1.05rem !important;
 }
}
```

```
/* Custom scrollbar styling for dark theme */
::-webkit-scrollbar {
 width: 8px;
 height: 8px;
}
::-webkit-scrollbar-track {
 background: #0f172a;
}
::-webkit-scrollbar-thumb {
 background: #334155;
 border-radius: 4px;
}
::-webkit-scrollbar-thumb:hover {
 background: #475569;
}
```

```
/* Индикатор длительности кадра в пошаговых визуализаци
@keyframes demoProgress {
```

```
main {
 padding: 0 !important;
 max-width: none !important;
}

/* текст темы – чёрным по белому, без теней и плашек
article.chapter-body .prose,
article.chapter-body .prose * {
 color: #111827 !important;
 background-color: transparent !important;
 background-image: none !important;
 box-shadow: none !important;
 text-shadow: none !important;
 border-color: #d1d5db !important;
}

article.chapter-body .prose pre,
article.chapter-body .prose code {
 background-color: #f3f4f6 !important;
 color: #111827 !important;
 font-size: 9pt;
}

article.chapter-body .prose table {
 border-collapse: collapse;
}

article.chapter-body .prose th,
article.chapter-body .prose td {
 border: 1px solid #9ca3af !important;
}

article.chapter-body .prose h2,
article.chapter-body .prose h3,
article.chapter-body .prose h4 {
 page-break-after: avoid;
}
```

Универсальное пособие • полный исходный код

---

ФАЙЛ 14/111: src/main.tsx

КАТЕГОРИЯ: Ядро приложения | СТРОК: 11 | РАЗМЕР: 230 В

---

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import "./index.css";
import App from "./App";

createRoot(document.getElementById("root")!).render(
 <StrictMode>
 <App />
 </StrictMode>
);
```

---

ФАЙЛ 15/111: src/types.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 9 | РАЗМЕР: 153 В

---

```
export interface Chapter {
 id: string;
 title: string;
 type: "html" | "markdown";
 content: string;
 description?: string;
 category?: string;
}
```

---

ФАЙЛ 16/111: src/vite-env.d.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 19 | РАЗМЕР: 324 В

---

```
/// <reference types="vite/client" />
```

```
<div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
```

```
<div class="space-y-8">
```

```
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
```

```
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 <p class="text-sm font-mono text-slate-300">
] [k] </p>
 </div>
```

```
<p class="text-slate-300 text-sm mb-4">Ячей
лины
">i. Любой отрезок накрывается дв
перация идемпотентна: min,
, побитовые И и ИЛИ.</p>
```

```
<div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-indigo-300 italic">
 <p class="text-xl font-mono text-white">
 <p class="text-sm font-mono text-slate-300">
вет = op из четырёх блоков kx × ky, накрыва
</div>
```

```
<p class="text-slate-300 text-sm mb-4">Двум
по столбцам. Отсюда и цена: уровней станов
s="font-mono text-emerald-300">log N
```

```
<div class="grid grid-cols-1 sm:grid-cols-3">
 <div class="bg-slate-900 border border-slate-700">
 <p class="text-[10px] uppercase tra
```

```

 <div class="absolute -top-3 left-4 border-rose-500 shadow-md">
 Аналогия 2: Торт-пирамидка
 </div>
 <p class="text-slate-300 text-sm mb-4">
 ия, зато красивый, как пирамидка: каждый из
 сех блоков $2^{k_x} \times 2^{k_y}$), и ответ на запрос
 ой операции нет: крем обратно в торт не запы
 $n(x, x) = x$), и цена $O(1)$ – сто коржей вмес
 <div class="rounded-lg bg-slate-950">
 <div>
 $n \ O(1)$ </div>
 <div>
 = 100 копий матрицы</div>
 </div>
 <p class="text-[12px] text-slate-400">
 шает). Менять элементы тоже нельзя. Сумму ч
 </div>
 </div>
 <p class="text-slate-400 text-sm mt-4">Поря
 ерению) → 2D: построение (уровни <span
 ос (прямоугольник раскладывается на чет
 </div>

<div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-rose-400">
 <p class="text-slate-300 text-sm mb-4">Эти
 за
 они решают разные задачи.</p>

<div class="bg-slate-800/70 p-5 rounded-lg">
 <p class="text-sm font-bold text-white">
 <p class="text-slate-300 text-sm mb-3">
 цы, у разреженной таблицы – размер, умножен
 <table class="w-full text-xs text-left">
 <thead>
 <tr class="text-slate-500 border">
 <th class="py-1.5 pr-2 font
```

Почему префиксная тонкая

</div>

<p class="text-slate-300 text-sm mb-4" style="border: 1px solid #000; padding: 10px; border-radius: 10px; background-color: #f0f0f0; width: fit-content; margin: 10px auto; font-family: monospace;">P[i][j]</p>  
ругой прямоугольник выражается через четыре

</div>

<div class="bg-slate-900 p-6 rounded-lg">

<div class="absolute -top-3 left-4 border-rose-500 shadow-md">

Почему разреженная жирная

</div>

<p class="text-slate-300 text-sm mb-4" style="border: 1px solid #000; padding: 10px; border-radius: 10px; background-color: #f0f0f0; width: fit-content; margin: 10px auto; font-family: monospace;">ум на его части не вычислишь. Приходится хранить  $\log N \times \log M$  на ка

</div>

</div>

<p class="text-slate-300 text-sm mt-4">Вывод: **лицу** – перекрывающиеся блоки посчитают **лицу** – у минимума нет обратной операции. Стр

</div>

<div class="bg-slate-700/50 p-6 rounded-xl border-rose-500/50">

<h3 class="text-xl font-bold text-amber-400">

<div class="bg-rose-950/30 p-5 rounded-lg border-rose-500/50">

<ul class="list-disc pl-5 space-y-2 text-slate-300">

<li><b>Сумма через разреженную таблицу префиксные (билет 5).</b></li>

<li><b>Память.</b>  $N = M = 1000$  даём  $10^6$  ячеек, считайте ячейки, потом пишите код.</li>

<li><b>Один log вместо двух.</b> В `std::vector` `st`  
<span class="font-mono text-emerald-300">st[0] = 0; st[1] = 1; st[2] = 1; st[3] = 2; st[4] = 3; st[5] = 4; st[6] = 5; st[7] = 6; st[8] = 7; st[9] = 8; st[10] = 9; st[11] = 10; st[12] = 11; st[13] = 12; st[14] = 13; st[15] = 14; st[16] = 15; st[17] = 16; st[18] = 17; st[19] = 18; st[20] = 19; st[21] = 20; st[22] = 21; st[23] = 22; st[24] = 23; st[25] = 24; st[26] = 25; st[27] = 26; st[28] = 27; st[29] = 28; st[30] = 29; st[31] = 30; st[32] = 31; st[33] = 32; st[34] = 33; st[35] = 34; st[36] = 35; st[37] = 36; st[38] = 37; st[39] = 38; st[40] = 39; st[41] = 40; st[42] = 41; st[43] = 42; st[44] = 43; st[45] = 44; st[46] = 45; st[47] = 46; st[48] = 47; st[49] = 48; st[50] = 49; st[51] = 50; st[52] = 51; st[53] = 52; st[54] = 53; st[55] = 54; st[56] = 55; st[57] = 56; st[58] = 57; st[59] = 58; st[60] = 59; st[61] = 60; st[62] = 61; st[63] = 62; st[64] = 63; st[65] = 64; st[66] = 65; st[67] = 66; st[68] = 67; st[69] = 68; st[70] = 69; st[71] = 70; st[72] = 71; st[73] = 72; st[74] = 73; st[75] = 74; st[76] = 75; st[77] = 76; st[78] = 77; st[79] = 78; st[80] = 79; st[81] = 80; st[82] = 81; st[83] = 82; st[84] = 83; st[85] = 84; st[86] = 85; st[87] = 86; st[88] = 87; st[89] = 88; st[90] = 89; st[91] = 90; st[92] = 91; st[93] = 92; st[94] = 93; st[95] = 94; st[96] = 95; st[97] = 96; st[98] = 97; st[99] = 98; st[100] = 99; st[101] = 100; st[102] = 101; st[103] = 102; st[104] = 103; st[105] = 104; st[106] = 105; st[107] = 106; st[108] = 107; st[109] = 108; st[110] = 109; st[111] = 110; st[112] = 111; st[113] = 112; st[114] = 113; st[115] = 114; st[116] = 115; st[117] = 116; st[118] = 117; st[119] = 118; st[120] = 119; st[121] = 120; st[122] = 121; st[123] = 122; st[124] = 123; st[125] = 124; st[126] = 125; st[127] = 126; st[128] = 127; st[129] = 128; st[130] = 129; st[131] = 130; st[132] = 131; st[133] = 132; st[134] = 133; st[135] = 134; st[136] = 135; st[137] = 136; st[138] = 137; st[139] = 138; st[140] = 139; st[141] = 140; st[142] = 141; st[143] = 142; st[144] = 143; st[145] = 144; st[146] = 145; st[147] = 146; st[148] = 147; st[149] = 148; st[150] = 149; st[151] = 150; st[152] = 151; st[153] = 152; st[154] = 153; st[155] = 154; st[156] = 155; st[157] = 156; st[158] = 157; st[159] = 158; st[160] = 159; st[161] = 160; st[162] = 161; st[163] = 162; st[164] = 163; st[165] = 164; st[166] = 165; st[167] = 166; st[168] = 167; st[169] = 168; st[170] = 169; st[171] = 170; st[172] = 171; st[173] = 172; st[174] = 173; st[175] = 174; st[176] = 175; st[177] = 176; st[178] = 177; st[179] = 178; st[180] = 179; st[181] = 180; st[182] = 181; st[183] = 182; st[184] = 183; st[185] = 184; st[186] = 185; st[187] = 186; st[188] = 187; st[189] = 188; st[190] = 189; st[191] = 190; st[192] = 191; st[193] = 192; st[194] = 193; st[195] = 194; st[196] = 195; st[197] = 196; st[198] = 197; st[199] = 198; st[200] = 199; st[201] = 200; st[202] = 201; st[203] = 202; st[204] = 203; st[205] = 204; st[206] = 205; st[207] = 206; st[208] = 207; st[209] = 208; st[210] = 209; st[211] = 210; st[212] = 211; st[213] = 212; st[214] = 213; st[215] = 214; st[216] = 215; st[217] = 216; st[218] = 217; st[219] = 218; st[220] = 219; st[221] = 220; st[222] = 221; st[223] = 222; st[224] = 223; st[225] = 224; st[226] = 225; st[227] = 226; st[228] = 227; st[229] = 228; st[230] = 229; st[231] = 230; st[232] = 231; st[233] = 232; st[234] = 233; st[235] = 234; st[236] = 235; st[237] = 236; st[238] = 237; st[239] = 238; st[240] = 239; st[241] = 240; st[242] = 241; st[243] = 242; st[244] = 243; st[245] = 244; st[246] = 245; st[247] = 246; st[248] = 247; st[249] = 248; st[250] = 249; st[251] = 250; st[252] = 251; st[253] = 252; st[254] = 253; st[255] = 254; st[256] = 255; st[257] = 256; st[258] = 257; st[259] = 258; st[260] = 259; st[261] = 260; st[262] = 261; st[263] = 262; st[264] = 263; st[265] = 264; st[266] = 265; st[267] = 266; st[268] = 267; st[269] = 268; st[270] = 269; st[271] = 270; st[272] = 271; st[273] = 272; st[274] = 273; st[275] = 274; st[276] = 275; st[277] = 276; st[278] = 277; st[279] = 278; st[280] = 279; st[281] = 280; st[282] = 281; st[283] = 282; st[284] = 283; st[285] = 284; st[286] = 285; st[287] = 286; st[288] = 287; st[289] = 288; st[290] = 289; st[291] = 290; st[292] = 291; st[293] = 292; st[294] = 293; st[295] = 294; st[296] = 295; st[297] = 296; st[298] = 297; st[299] = 298; st[300] = 299; st[301] = 300; st[302] = 301; st[303] = 302; st[304] = 303; st[305] = 304; st[306] = 305; st[307] = 306; st[308] = 307; st[309] = 308; st[310] = 309; st[311] = 310; st[312] = 311; st[313] = 312; st[314] = 313; st[315] = 314; st[316] = 315; st[317] = 316; st[318] = 317; st[319] = 318; st[320] = 319; st[321] = 320; st[322] = 321; st[323] = 322; st[324] = 323; st[325] = 324; st[326] = 325; st[327] = 326; st[328] = 327; st[329] = 328; st[330] = 329; st[331] = 330; st[332] = 331; st[333] = 332; st[334] = 333; st[335] = 334; st[336] = 335; st[337] = 336; st[338] = 337; st[339] = 338; st[340] = 339; st[341] = 340; st[342] = 341; st[343] = 342; st[344] = 343; st[345] = 344; st[346] = 345; st[347] = 346; st[348] = 347; st[349] = 348; st[350] = 349; st[351] = 350; st[352] = 351; st[353] = 352; st[354] = 353; st[355] = 354; st[356] = 355; st[357] = 356; st[358] = 357; st[359] = 358; st[360] = 359; st[361] = 360; st[362] = 361; st[363] = 362; st[364] = 363; st[365] = 364; st[366] = 365; st[367] = 366; st[368] = 367; st[369] = 368; st[370] = 369; st[371] = 370; st[372] = 371; st[373] = 372; st[374] = 373; st[375] = 374; st[376] = 375; st[377] = 376; st[378] = 377; st[379] = 378; st[380] = 379; st[381] = 380; st[382] = 381; st[383] = 382; st[384] = 383; st[385] = 384; st[386] = 385; st[387] = 386; st[388] = 387; st[389] = 388; st[390] = 389; st[391] = 390; st[392] = 391; st[393] = 392; st[394] = 393; st[395] = 394; st[396] = 395; st[397] = 396; st[398] = 397; st[399] = 398; st[400] = 399; st[401] = 400; st[402] = 401; st[403] = 402; st[404] = 403; st[405] = 404; st[406] = 405; st[407] = 406; st[408] = 407; st[409] = 408; st[410] = 409; st[411] = 410; st[412] = 411; st[413] = 412; st[414] = 413; st[415] = 414; st[416] = 415; st[417] = 416; st[418] = 417; st[419] = 418; st[420] = 419; st[421] = 420; st[422] = 421; st[423] = 422; st[424] = 423; st[425] = 424; st[426] = 425; st[427] = 426; st[428] = 427; st[429] = 428; st[430] = 429; st[431] = 430; st[432] = 431; st[433] = 432; st[434] = 433; st[435] = 434; st[436] = 435; st[437] = 436; st[438] = 437; st[439] = 438; st[440] = 439; st[441] = 440; st[442] = 441; st[443] = 442; st[444] = 443; st[445] = 444; st[446] = 445; st[447] = 446; st[448] = 447; st[449] = 448; st[450] = 449; st[451] = 450; st[452] = 451; st[453] = 452; st[454] = 453; st[455] = 454; st[456] = 455; st[457] = 456; st[458] = 457; st[459] = 458; st[460] = 459; st[461] = 460; st[462] = 461; st[463] = 462; st[464] = 463; st[465] = 464; st[466] = 465; st[467] = 466; st[468] = 467; st[469] = 468; st[470] = 469; st[471] = 470; st[472] = 471; st[473] = 472; st[474] = 473; st[475] = 474; st[476] = 475; st[477] = 476; st[478] = 477; st[479] = 478; st[480] = 479; st[481] = 480; st[482] = 481; st[483] = 482; st[484] = 483; st[485] = 484; st[486] = 485; st[487] = 486; st[488] = 487; st[489] = 488; st[490] = 489; st[491] = 490; st[492] = 491; st[493] = 492; st[494] = 493; st[495] = 494; st[496] = 495; st[497] = 496; st[498] = 497; st[499] = 498; st[500] = 499; st[501] = 500; st[502] = 501; st[503] = 502; st[504] = 503; st[505] = 504; st[506] = 505; st[507] = 506; st[508] = 507; st[509] = 508; st[510] = 509; st[511] = 510; st[512] = 511; st[513] = 512; st[514] = 513; st[515] = 514; st[516] = 515; st[517] = 516; st[518] = 517; st[519] = 518; st[520] = 519; st[521] = 520; st[522] = 521; st[523] = 522; st[524] = 523; st[525] = 524; st[526] = 525; st[527] = 526; st[528] = 527; st[529] = 528; st[530] = 529; st[531] = 530; st[532] = 531; st[533] = 532; st[534] = 533; st[535] = 534; st[536] = 535; st[537] = 536; st[538] = 537; st[539] = 538; st[540] = 539; st[541] = 540; st[542] = 541; st[543] = 542; st[544] = 543; st[545] = 544; st[546] = 545; st[547] = 546; st[548] = 547; st[549] = 548; st[550] = 549; st[551] = 550; st[552] = 551; st[553] = 552; st[554] = 553; st[555] = 554; st[556] = 555; st[557] = 556; st[558] = 557; st[559] = 558; st[560] = 559; st[561] = 560; st[562] = 561; st[563] = 562; st[564] = 563; st[565] = 564; st[566] = 565; st[567] = 566; st[568] = 567; st[569] = 568; st[570] = 569; st[571] = 570; st[572] = 571; st[573] = 572; st[574] = 573; st[575] = 574; st[576] = 575; st[577] = 576; st[578] = 577; st[579] = 578; st[580] = 579; st[581] = 580; st[582] = 581; st[583] = 582; st[584] = 583; st[585] = 584; st[586] = 585; st[587] = 586; st[588] = 587; st[589] = 588; st[590] = 589; st[591] = 590; st[592] = 591; st[593] = 592; st[594] = 593; st[595] = 594; st[596] = 595; st[597] = 596; st[598] = 597; st[599] = 598; st[600] = 599; st[601] = 600; st[602] = 601; st[603] = 602; st[604] = 603; st[605] = 604; st[606] = 605; st[607] = 606; st[608] = 607; st[609] = 608; st[610] = 609; st[611] = 610; st[612] = 611; st[613] = 612; st[614] = 613; st[615] = 614; st[616] = 615; st[617] = 616; st[618] = 617; st[619] = 618; st[620] = 619; st[621] = 620; st[622] = 621; st[623] = 622; st[624] = 623; st[625] = 624; st[626] = 625; st[627] = 626; st[628] = 627; st[629] = 628; st[630] = 629; st[631] = 630; st[632] = 631; st[633] = 632; st[634] = 633; st[635] = 634; st[636] = 635; st[637] = 636; st[638] = 637; st[639] = 638; st[640] = 639; st[641] = 640; st[642] = 641; st[643] = 642; st[644] = 643; st[645] = 644; st[646] = 645; st[647] = 646; st[648] = 647; st[649] = 648; st[650] = 649; st[651] = 650; st[652] = 651; st[653] = 652; st[654] = 653; st[655] = 654; st[656] = 655; st[657] = 656; st[658] = 657; st[659] = 658; st[660] = 659; st[661] = 660; st[662] = 661; st[663] = 662; st[664] = 663; st[665] = 664; st[666] = 665; st[667] = 666; st[668] = 667; st[669] = 668; st[670] = 669; st[671] = 670; st[672] = 671; st[673] = 672; st[674] = 673; st[675] = 674; st[676] = 675; st[677] = 676; st[678] = 677; st[679] = 678; st[680] = 679; st[681] = 680; st[682] = 681; st[683] = 682; st[684] = 683; st[685] = 684; st[686] = 685; st[687] = 686; st[688] = 687; st[689] = 688; st[690] = 689; st[691] = 690; st[692] = 691; st[693] = 692; st[694] = 693; st[695] = 694; st[696] = 695; st[697] = 696; st[698] = 697; st[699] = 698; st[700] = 699; st[701] = 700; st[702] = 701; st[703] = 702; st[704] = 703; st[705] = 704; st[706] = 705; st[707] = 706; st[708] = 707; st[709] = 708; st[710] = 709; st[711] = 710; st[712] = 711; st[713] = 712; st[714] = 713; st[715] = 714; st[716] = 715; st[717] = 716; st[718] = 717; st[719] = 718; st[720] = 719; st[721] = 720; st[722] = 721; st[723] = 722; st[724] = 723; st[725] = 724; st[726] = 725; st[727] = 726; st[728] = 727; st[729] = 728; st[730] = 729; st[731] = 730; st[732] = 731; st[733] = 732; st[734] = 733; st[735] = 734; st[736] = 735; st[737] = 736; st[738] = 737; st[739] = 738; st[740] = 739; st[741] = 740; st[742] = 741; st[743] = 742; st[744] = 743; st[745] = 744; st[746] = 745; st[747] = 746; st[748] = 747; st[749] = 748; st[750] = 749; st[751] = 750; st[752] = 751; st[753] = 752; st[754] = 753; st[755] = 754; st[756] = 755; st[757] = 756; st[758] = 757; st[759] = 758; st[760] = 759; st[761] = 760; st[762] = 761; st[763] = 762; st[764] = 763; st[765] = 764; st[766] = 765; st[767] = 766; st[768] = 767; st[769] = 768; st[770] = 769; st[771] = 770; st[772] = 771; st[773] = 772; st[774] = 773; st[775] = 774; st[776] = 775; st[777] = 776; st[778] = 777; st[779] = 778; st[780] = 779; st[781] = 780; st[782] = 781; st[783] = 782; st[784] = 783; st[785] = 784; st[786] = 785; st[787] = 786; st[788] = 787; st[789] = 788; st[790] = 789; st[791] = 790; st[792] = 791; st[793] = 792; st[794] = 793; st[795] = 794; st[796] = 795; st[797] = 796; st[798] = 797; st[799] = 798; st[800] = 799; st[801] = 800; st[802] = 801; st[803] = 802; st[804] = 803; st[805] = 804; st[806] = 805; st[807] = 806; st[808] = 807; st[809] = 808; st[810] = 809; st[811] = 810; st[812] = 811; st[813] = 812; st[814] = 813; st[815] = 814; st[816] = 815; st[817] = 816; st[818] = 817; st[819] = 818; st[820] = 819; st[821] = 820; st[822] = 821; st[823] = 822; st[824] = 823; st[825] = 824; st[826] = 825; st[827] = 826; st[828] = 827; st[829] = 828; st[830] = 829; st[831] = 830; st[832] = 831; st[833] = 832; st[834] = 833; st[835] = 834; st[836] = 835; st[837] = 836; st[838] = 837; st[839] = 838; st[840] = 839; st[841] = 840; st[842] = 841; st[843] = 842; st[844] = 843; st[845] = 844; st[846] = 845; st[847] = 846; st[848] = 847; st[849] = 848; st[850] = 849; st[851] = 850; st[852] = 851; st[853] = 852; st[854] = 853; st[855] = 854; st[856] = 855; st[857] = 856; st[858] = 857; st[859] = 858; st[860] = 859; st[861] = 860; st[862] = 861; st[863] = 862; st[864] = 863; st[865] = 864; st[866] = 865; st[867] = 866; st[868] = 867; st[869] = 868; st[870] = 869; st[871] = 870; st[872] = 871; st[873] = 872; st[874] = 873; st[875] = 874; st[876] = 875; st[877] = 876; st[878] = 877; st[879] = 878; st[880] = 879; st[881] = 880; st[882] = 881; st[883] = 882; st[884] = 883; st[885] = 884; st[886] = 885; st[887] = 886; st[888] = 887; st[889] = 888; st[890] = 889; st[891] = 890; st[892] = 891; st[893] = 892; st[894] = 893; st[895] = 894; st[896] = 895; st[897] = 896; st[898] = 897; st[899] = 898; st[900] = 899; st[901] = 900; st[902] = 901; st[903] = 902; st[904] = 903; st[905] = 904; st[906] = 905; st[907] = 906; st[908] = 907; st[909] = 908; st[910] = 909; st[911] = 910; st[912] = 911; st[913] = 912; st[914] = 913; st[915] = 914; st[916] = 915; st[917] = 916; st[918] = 917; st[919] = 918; st[920] = 919; st[921] = 920; st[922] = 921; st[923] = 922; st[924] = 923; st[925] = 924; st[926] = 925; st[927] = 926; st[928] = 927; st[929] = 928; st[930] = 929; st[931] = 930; st[932] = 931; st[933] = 932; st[934] = 933; st[935] = 934; st[936] = 935; st[937] = 936; st[938] = 937; st[939] = 938; st[940] = 939; st

```

 else
 st[r][c][kx][ky] = min(

```

```

// запрос минимума на прямоугольнике (R1,C1)..(R2,C2)
int kx = log2(R2 - R1 + 1);
int ky = log2(C2 - C1 + 1);
int ans1 = min(st[R1][C1][kx][ky], st[R1][C2 - (1 << ky) + 1][C1][kx][ky]);
int ans2 = min(st[R2 - (1 << kx) + 1][C1][kx][ky], st[R2][C1][kx][ky]);
int minimum = min(ans1, ans2);

```

Четыре запроса к четырем углам запроса и вместе с ним образуют минимальный квадрат. Но.

Сколько это памяти в числах

- 100 × 100: уровни 7 × 7 ≈ 49
- 1000 × 1000: уровни 10 × 10 = 100
- Префиксная сумма для той же матрицы

Почему? Потому что для каждого из нескольких сотен на сторону), а запросов очень много.

Сколько это памяти в числах

- 100 × 100: уровни 7 × 7 ≈ 49
- 1000 × 1000: уровни 10 × 10 = 100
- Префиксная сумма для той же матрицы

Почему? Потому что для каждого из нескольких сотен на сторону), а запросов очень много.

```
<div class="text-center my-6">
 <p class="text-slate-400 text-xs uppercase">
 <p class="my-1 font-bold leading-none">
 И
 </p>
 <p class="text-slate-300 font-mono text">
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Аналогия 1: Split – таможня
 </div>
 <p class="text-slate-300 text-sm">S
х». Спуск от корня: узел, который вместе с
скрытия – досматривается только одна ве
 </div>
```

```
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
 Аналогия 2: Merge – стыковка
 </div>
 <p class="text-slate-300 text-sm">M
аг – одно сравнение: чей у у корня больше,
сохраняет порядок x). Итого $O(h)$ сравнений
 </div>
 </div>
```

```
<div class="bg-slate-900/40 rounded-lg bord
 <p class="text-slate-300 text-sm mb-2">
 <p class="text-slate-400 text-sm mb-1">
быстрее $n \cdot \log n$.</p>
 <p class="text-slate-400 text-sm mb-1">
 <p class="text-slate-400 text-sm">• «У6
```



```
<summary class="p-4 font-bold text-slate-700 border-slate-700/50">
```

Код: вставка, merge, erase (Скрыто)

```
</summary>
```

```
<div class="p-5 text-sm text-slate-300">
```

```
<pre class="bg-slate-950 p-4 rounded">
```

```
def insert(root, key, pri):
```

```
 if root is None:
```

```
 return Node(key, pri)
```

```
 if key <= root.key: # равные x — левый
```

```
 root.left = insert(root.left, key, pri)
```

```
 else:
```

```
 root.right = insert(root.right, key, pri)
```

```
 return root
```

```
for key, pri in pairs_sorted_by_pri_desc: # порядок э
```

```
 root = insert(root, key, pri)</pre>
```

```
<pre class="bg-slate-950 p-4 rounded">
```

```
def merge(a, b): # все x в a < все x в b
```

```
 if not a or not b: return a or b
```

```
 if a.pri > b.pri:
```

```
 a.right = merge(a.right, b); return a
```

```
 b.left = merge(a, b.left); return b
```

```
def erase(root, key): # удалить произвольный элемент
```

```
 if key < root.key: root.left = erase(root.left, key)
```

```
 elif key > root.key: root.right = erase(root.right, key)
```

```
 else: root = merge(root.left, root.right)
```

```
 return root</pre>
```

```
</div>
```

```
</details>
```

```
</div>
```

```
</div>
```

```
</section>`,
```

```
},
```

```
{
```

```
 id: "splay-tree",
```

```
 title: "4. Splay-дерево",
```

по факту: ничего не хранит, дереву разр

```
<div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
```

### Аналогия 1: Стопка бумага

тировку не ведёшь, но через неделю самое ну  
ерх». </p>

```

 </div>
 <div class="bg-slate-900 p-6 rounded-lg" style="border: 1px solid #000000; border-radius: 10px; width: 100%; height: 100%; position: relative; margin-top: 10px;">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border-rose-500 shadow-md">

```

### Аналогия 2: Тропинка через га

`</div>`  
`<p class="text-slate-300 text-sm">`И,  
рно вдвое. Первый проход дорогой, зато опла  
н.`</p>`

</div>  
</div>

Среднее поддерево  $\beta$  (между  $x$  и  $p$ ) – единственное дерево, которое имеет  $x$  и под  $p$ .

```
<pre class="bg-slate-950 p-4 rounded-lg overflow-relaxed">
```

$$\begin{array}{ccc} \begin{array}{cc} / & \backslash \\ x & y \\ / & \backslash \\ \alpha & \beta \end{array} & \begin{array}{c} x \text{ выше} \\ \text{-----} \rightarrow \\ p \text{ выше} \\ \text{<-----} \end{array} & \begin{array}{cc} / & \backslash \\ \alpha & p \\ / & \backslash \\ \beta & y \end{array} \end{array}$$

Обход дерева поиска поворотами невозможен в момент «отстёжки», а врезка «Механика одного

#### class="text-lg font-bold text-white mt-4">

```
<pre class="bg-slate-950 p-4 rounded-lg">
 амбук из 8 узлов 8→7→6→5→4→3→2→1, ищем 1 (ч
наивно (x, потом x): 3 × «два зига» + Zig → высота 8
Zig-Zig (p, потом x): 3 × Zig-Zig + Zig → высота 8
бамбук из 16 узлов: наивно 16 → 16, Zig-Zig 16 → 10<
```

```
 <p class="text-slate-300 text-sm mt-3">
де: <code class="bg-slate-950 px-2 py-0.5 r
s="bg-slate-950 px-2 py-0.5 rounded font-mo
изации не будет.</p>
</div>
```

```
<p class="text-slate-300 text-sm mt-6">Полн
м 1. Спуск 8 узлов, затем три Zig-Zig по
8 до 6 – линия перестала быть линией. Наивно
о перевёрнутой. Важно не врать себе: на кор
ван не на одном запросе, а в сумме. Следующ
по пути. Отсюда амортизация: дорогой за
дом потенциала: потенциал Φ = сумма по узла
ь его; отсюда же теорема о статической опти
ь запросов).</p>
```

```
<div class="bg-slate-900/40 rounded-lg bord
 <p class="text-slate-300 text-sm mb-2">
 <p class="text-slate-400 text-sm mb-1">
га» – просто переворот.</p>
 <p class="text-slate-400 text-sm mb-1">
иза к верху; в симуляторе оно подсвечиваетс
 <p class="text-slate-400 text-sm">• «Sp
о амортизированная.</p>
</div>
```

```
<p class="text-slate-300 text-sm mt-4"><b c
ое <code class="bg-slate-950 px-2 py-0.5 ro
одиночная операция – до <code class="bg-sla
>O(n)</code>; n вставок подряд – амортизиро
-300 border border-slate-800">O(n log n)</c
```

```
<p class="text-slate-300 text-sm mt-4">Все
```

```

 rotate_up(x)
 else: # ZIG-ZAG: НИЖНЯЯ
 rotate_up(x)
 rotate_up(x)
 return x

def find(root, key): # поиск: спуск + s
 cur, last = root, None
 while cur:
 last = cur
 if key == cur.key: break
 cur = cur.left if key < cur.key else cur.right
 return splay(last) if last else None</pre>
 <p class="text-xs text-slate-400">В
 Одна строка – и вся амортизация.</p>
 </div>
</details>
</div>
</div>
</section>`,
 },
 {
 id: "prefix-sums-2d",
 title: "5. Двумерная матрица префиксных сумм",
 type: "html",
 description: "Сумма подматрицы за O(1): 1D и 2D, дв
 category: "Алгоритмы матрицы",
 content: `
<section id="prefix-sums-2d" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-v
 </div>

 <div class="space-y-8">

 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-emerald-4

```

```

 <td class="py-2 pr-3 font-mono">
 </td>
 </tr>
 </tbody>
</table>
<p class="text-slate-400 text-xs mt-2">
 ышли за границу» – поэтому в 2D почти всегда
</div>

<div class="grid grid-cols-1 sm:grid-cols-3">
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
 <p class="text-[11px] text-slate-500
 </div>
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
 <p class="text-[11px] text-slate-500
 </div>
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-rose-300 t
 <p class="text-[11px] text-slate-500
 </div>
</div>
</div>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-blue-400 m
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800
 <div class="absolute -top-3 left-4 b
 rder border-emerald-500 shadow-md">
 Аналогия 1: Сетка из вафельных
 </div>
 <p class="text-slate-300 text-sm mb
 о накопилось от угла (0,0) до меня». Хочешь
 у и верни дважды отрезанный угол. Вычитание

```

```

<div class="bg-slate-800 p-6 rounded-lg" style="border: 1px solid #000; border-radius: 10px; padding: 10px;">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
 Тонкая: одна матрица
 </div>
 <p class="text-slate-300 text-sm mb-2">
 одной матрицы (плюс рамка из нулей). Никаких вычитаний, только сложение. Прямоугольника от угла. Всё остальное получаем из
 </div>
 <div class="bg-slate-900 p-6 rounded-lg" style="border: 1px solid #000; border-radius: 10px; padding: 10px;">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-rose-500 shadow-md">
 Не умеет: всё, что нельзя вычислить с помощью
 </div>
 <p class="text-slate-300 text-sm mb-2">
 максимума или gcd вычитания не существует:
 е запросы – это
 платит она за это памятью
 </div>
 </div>

<div class="bg-slate-800/70 p-5 rounded-lg" style="border: 1px solid #000; border-radius: 10px; padding: 10px;">
 <p class="text-sm font-bold text-white">
 <table class="w-full text-xs text-left" style="width: 100%; border-collapse: collapse;">
 <thead>
 <tr class="text-slate-500 border-b border-slate-700">
 <th class="py-1.5 pr-2 font-normal">
 <th class="py-1.5 pr-2 font-normal">
 <th class="py-1.5 pr-2 font-normal">
 </tr>
 </thead>
 <tbody>
 <tr class="border-b border-slate-700">
 <td class="py-2 pr-3 font-normal">
 <td class="py-2 pr-3 font-normal">
 <td class="py-2 pr-3 font-normal">
 </tr>
 <tr class="border-b border-slate-700">

```

```

<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
 Код: 1D (тот же, что выполняет пане
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounde
 = [3, 1, 4, 1, 5, 9, 2, 6] # исходный ма
n = len(a) # префиксов будет n + 1
P = [0] # P[0] = 0 – пустой преф
for i in range(1, n + 1):
 P.append(P[i - 1] + a[i - 1])
print("n =", n, "| P =", P)</code></pre>
 <p class="text-slate-300 text-sm">П
- часть демонстрации: лента подсвечивает, из
 </div>
</details>

<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
 Код: 2D – построение и запрос прямо
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounde
 = [[1, 2, 3, 4], [5, 6, 7, 8], [9, 1, 2, 3
n, m = 4, 4

S = [] # рамка из нуле
for rr in range(n + 1):
 S.append([0] + [0] * m)
for i in range(1, n + 1):
 for j in range(1, m + 1):
 S[i][j] = A[i-1][j-1] + S[i-1][j] + S[i][j-1] -

r1, c1, r2, c2 = 1, 1, 2, 2 # прямоугольник
ans = S[r2+1][c2+1] - S[r1][c2+1] - S[r2+1][c1] + S[r1]
print("итого:", ans)</code></pre>

```

```

 },
 {
 id: "graph-planar-colors",
 title: "7. Графы. Планарные. Покраска",
 type: "html",
 description: "Формула Эйлера и теорема о 4 красках.",
 category: "Графы. Теория",
 content: `
<section id="graph-planar-colors" class="mb-12 scroll-m-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Графы
 <h2 class="text-2xl sm:text-3xl font-bold text-blue-400">Покраска
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-blue-400">Теорема о 4 красках
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">Любой планарный граф можно раскрасить 4 цветами.
 <p class="text-xl font-mono text-white">
 1. Планарный граф — граф, который можно нарисовать
 так, чтобы никакие две его ребра не пересекались.
 2. Планарный граф можно раскрасить 4 цветами.
 </p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Карта мира
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Планарный граф — граф, который можно нарисовать
 так, чтобы никакие две его ребра не пересекались.
 Границы не пересекаются в одной точке по-
 тому, что в противном случае они сливались
 цветами. Всегда можно всего 4 цвета.
 </p>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "top-sort",

```



```

 </div>
</section>`,
 },
 {
 id: "scc-kosaraju",
 title: "10. Графы. Компоненты сильной связности (SCC)",
 type: "html",
 description: "Разбиение орграфа на макро-вершины. Алгоритм Косарaju",
 category: "Графы. Продвинутое",
 content: `
<section id="scc-kosaraju" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Графы
 <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">10. Графы. Компоненты сильной связности (SCC)
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
 <h3 class="text-xl font-bold text-emerald-400">10.1. Компоненты сильной связности (SCC)
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">Сильная связность графа — это свойство, при котором для любых двух вершин u и v графа существует путь из u в v и обратно.
 <p class="text-xl font-mono text-white">Граф, состоящий из нескольких сильно связных компонент, можно преобразовать в ориентированный ациклический граф (DAG), инвертировав порядок топологической сортировки каждой компоненты.
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
 Аналогия 1: Улицы с односторонним движением
 </div>
 <p class="text-slate-300 text-sm mb-4">Представьте себе город, где все улицы имеют одностороннее движение. Если вы можете добраться из любой точки города в любую другую, то город является сильно связным.
 </div>
 <div class="bg-slate-800 p-6 rounded-lg">
 <p class="text-slate-300 text-sm mb-4">В графе с сильной связностью для любых двух вершин u и v существует путь из u в v и обратно. Это означает, что граф можно разбить на несколько компонент сильной связности (SCC), каждая из которых является сильно связным графом.
 </div>
 </div>
 </div>
 </div>
</section>`,
 },

```

```

 description: "Вершины, удаление которых убивает свя
 category: "Графы. Продвинутые",
 content: `
<section id="graph-articulation" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-rose-400">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-rose-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-rose-300 italic">
 <p class="text-xl font-mono text-white">
 теи > 1).</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
 order border-emerald-500 shadow-md">
 Аналогия 1: Центральный роутер
 </div>
 <p class="text-slate-300 text-sm mb-4">
 правильный роутер - и два сегмента офиса б
 </div>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "graph-euler",
 title: "13. Графы. Эйлеров цикл",
 type: "html",
 description: "Пройти по всем ребрам ровно 1 раз.",
 category: "Графы",
 content: `
<section id="graph-euler" class="mb-12 scroll-mt-10">

```

```

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border"
 <h3 class="text-xl font-bold text-blue-400"
 <div class="bg-slate-900 p-4 rounded-lg mb-1"
 <p class="text-lg text-blue-300 italic"
 <p class="text-xl font-mono text-white"
d-длина.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Аналогия: Копаем туннель под .
 </div>
 <p class="text-slate-300 text-sm mb
ней земли. Но если начать копать одновременно
в 2 РАЗА меньше работы (геометрически: площ
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "mst-kruskal",
 title: "18. Графы. Остовное дерево. Краскал",
 type: "html",
 description: "",
 category: "Графы. Остовные",
 content: `
<section id="mst-kruskal" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-2xl sm:text-3xl font-bold text-
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-emerald-4

```

```

 которое торчит из посещенных в непосещенных
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 ✦ Аналогия: Заражение вирусом
 </div>
 <p class="text-slate-300 text-sm mb-2">
 (или вирус). Мы стартуем из одного города, и
 Сеть растет только из одного связного куска
 </p>
 </div>
 </div>
</div>
</section>`,
 },
 {
 id: "mst-boruvka",
 title: "20. Графы. Остовное дерево. Борувка",
 type: "html",
 description: "",
 category: "Графы. Остовные",
 content: `
<section id="mst-boruvka" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 ается с соседом.</p>
 </div>
 </div>
 <div class="grid grid-cols-1 gap-6">

```

```

 rder border-emerald-500 shadow-md">
 Аналогия 1: Бумеранг (Префикс
 </div>
 <p class="text-slate-300 text-sm mb
идишь "АБРА". Если ты ошибешься при поиске
концовка "АБРА" совпадает с началом "АБРА"
меня перепроверок!</p>
 </div>
</div>
<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounde
vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
 int j = pi[i-1];
 while (j > 0 && s[i] != s[j]) j = pi[j-1];
 if (s[i] == s[j]) j++;
 pi[i] = j;
}</pre>
 </div>
 </details>
</div>
</div>
</section>`,
 },
 {
 id: "string-z-func",
 title: "22. Z-функция",
 type: "html",
 description: "",
 category: "Строки",
 content: `
<section id="string-z-func" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

```

```

 </div>
</section>`,
 },
 {
 id: "complexity-classes",
 title: "24. Классы сложности, сведение задач",
 type: "html",
 description: "",
 category: "Теория сложности",
 content: `
<section id="complexity-classes" class="mb-12 scroll-mt-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-purple-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-purple-300 italic">
 <p class="text-base sm:text-lg font-mono">
 P: класс decision-задач, решаемый за полиномиальное время.
 NP: класс decision-задач с сертификатом.
 </p>
 </div>
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Судoku (P vs NP)
 </div>
 <p class="text-slate-300 text-sm mb-4">
 пример сортировка чисел). Класс NP – класс задач, для которых существует полиномиальную сетку (ответ), он БЫСТРО (за класс P)
 </div>
 </div>
 </div>
 </h3>
 </div>
 </div>
 </div>

```

```

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border"
 <h3 class="text-xl font-bold text-blue-400"

 <div class="bg-slate-900 p-4 rounded-lg mb-4"
 <p class="text-lg text-blue-300 italic"
 <div class="text-left font-mono text-sm"
 <p><strong class="text-white">Эйлер
ин раз. Вершины повторять можно.</p>
 <p><strong class="text-white">Эйлер
 <div class="p-3 bg-slate-950 rounded"
 <p class="text-sm text-blue-300"
 <p><strong class="text-white">П
 <p><strong class="text-white">Ц
 </div>
 </div>
 </div>
</div>

<p class="text-slate-300 text-sm">
 Билет 13 (Различие пути и цикла

• <span class="text-green-400 font-
шел-вошёл-вышел).

• <span class="text-yellow-400 font
финиш).
</p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap"
 <div class="bg-slate-800 p-6 rounded-lg bor"
 <div class="absolute -top-3 left-4 bg-s
order-green-500 shadow-md">
 Аналогия 1: Путь – из дома в бар,
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Ты вышел из дома (нечётная)
 талый пришёл в бар (вторая нечётная
 Домой вернуться не смог, потому что
 </p>

```

```

 </div>
</section>`,
 },
 {
 id: "bridges-code",
 title: "11. Графы. Мосты",
 type: "html",
 description: "Мосты через DFS Тарьяна: tin/low, усл
 category: "Графы. Продвинутые",
 content: `<section id="bridges-code" class="mb-20 s
 <div class="flex items-center mb-6">
 <span class="bg-emerald-600 text-white px-4 py-
 <h2 class="text-3xl font-bold text-white">Мосты
 </div>

 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-emerald-400">Мосты

 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <div class="text-left font-mono text-sm">
 <p><strong class="text-white">tin[v
 <p><strong class="text-white">low[v
 <div class="p-3 bg-slate-950 rounded">
 <span class="text-rose-400 font
 <p class="text-xl font-bold tex
 <p class="text-xs text-slate-400
 </div>
 </div>
 </div>
 </div>
 </div>
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-slate-300 text-sm">
 Если из сына u и всех его потомков
 Единственная ниточка. Порвётся – граф р
 </p>
 </div>
 </div>
 }
}

```



```

<pre class="text-xs font-mono text-
#include <vector>;
#include <algorithm>;

using namespace std;

int n; // число вершин
vector<vector<int>>> adj; // список смежност
vector<bool> visited;
vector<int> tin, low;
int timer;

void dfs(int v, int p = -1) {
 visited[v] = true;
 tin[v] = low[v] = timer++; // устанавливаем

 for (int to : adj[v]) {
 if (to == p) continue; // не идём назад

 if (visited[to]) {
 // обратное ребро: обновляем low
 low[v] = min(low[v], tin[to]);
 } else {
 // ребро дерева DFS
 dfs(to, v); // рекурсивный вызов
 low[v] = min(low[v], low[to]);

 if (low[to] > tin[v]) {
 // ЭТО МОСТ!
 // bridge_list.push_back({v, to});
 }
 }
 }
}

void find_bridges() {
 timer = 0;
 visited.assign(n, false);
}
```

```

 <p><span class="text-emerald-400 font-
 <p><span class="text-rose-400 font-l
 </div>
</div>

<p class="text-slate-300 text-sm">
 Следствие: Для планарн
ode>.
 Если ребер больше – граф гарантированно
</p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap
 <div class="bg-slate-800 p-6 rounded-lg bord
 <div class="absolute -top-3 left-4 bg-s
order-green-500 shadow-md">
 K5: Полный граф на 5 вершинах
 </div>
 <p class="text-slate-300 text-sm mb-4">
 У K5: <code class="text-white">V=5,
-white">10 > 9</code> – БАХ, непланарен!
 Все 5 вершин соединены со всеми. На
 Теорема Куратовского: K5 и K3,3 – д
 Если внутри графа спрятано подра
 – плоской раскладки не будет.
 </p>
 </div>

<div class="bg-slate-900 p-6 rounded-lg bord
 <div class="absolute -top-3 left-4 bg-r
der-rose-500 shadow-md">
 K3,3: 3 дома и 3 колодца
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Три дома хотят соединиться с тремя
 Нарисовать без пересечений невозможно
 У него <code class="text-white">V=6
xt-white">9 ≤ 12</code> – проходит, но он в
```

```
<div class="grid grid-cols-1 xl:grid-cols-2"
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
```

Аналогия 1: Расписание экзамен

```
</div>
```

```
<p class="text-slate-300 text-sm">В
слот времени. Раскрасить граф = составить р
«берём предметы в каком-то порядке и ставим
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
border border-fuchsia-500 shadow-md">
```

Аналогия 2: Частоты вышек

```
</div>
```

```
<p class="text-slate-300 text-sm">С
цвет, интерференция = ребро. Диапазон частот
регистров в компиляторе: «живые» одновременно
</div>
```

```
</div>
```

```
<div class="overflow-x-auto my-6">
```

```
<table class="w-full text-xs text-left"
 <thead>
```

```
<tr class="text-slate-500 border-b"
 <th class="py-2 pr-3 font-b"
 <th class="py-2 pr-3 font-b"
 <th class="py-2 font-bold">
```

```
</tr>
```

```
</thead>
```

```
<tbody class="text-slate-300">
```

```
<tr class="border-b border-slate-800"
><td class="py-2 pr-3 font-mono"> $\chi \leq \Delta + 1$ <
а свободен</td></tr>
```

```
<tr class="border-b border-slate-800"
d><td class="py-2 pr-3 font-mono"> $\chi \leq \Delta$ </td>
/td></tr>
```

```

 веркой цветов соседей; проверка двудольности
border border-slate-800">0(V + E)</code>; т
-300 border border-slate-800">NP-трудно</co

<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
 Код: жадная раскраска и проверка
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounded
я раскраска: не больше $\Delta + 1$ цвета
color = [-1] * n # -1 = ещё не покрашен
for v in order: # порядок влияет на чи
 used = set()
 for to in graph[v]:
 if color[to] != -1:
 used.add(color[to]) # цвета уже покрашены
 c = 0
 while c in used: # минимальный свободны
 c += 1
 color[v] = c
chi = max(color) + 1 # χ , который выдал жад

Проверка двудольности = раскраска в 2 цвета (BFS)
from collections import deque
col = [-1] * n
bipartite = True
for start in range(n):
 if col[start] != -1:
 continue
 col[start] = 0
 dq = deque([start])
 while dq:
 v = dq.popleft()
 for to in graph[v]:
 if col[to] == -1:
 col[to] = col[v] ^ 1 # второй цвет

```

```
<p class="text-slate-300 text-sm">
```

```
 Важно: Это работает то-
 еплятся к точкам сочленения. То есть,
 ег - это тупик со степенью 1).
```

```
</p>
```

```
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2 gap-
```

```
 <div class="bg-slate-800 p-6 rounded-lg border-
```

```
 <div class="absolute -top-3 left-4 bg-slate-800
 rder-rose-500 shadow-md">
```

```
 Аналогия: Главный перекресток
```

```
 </div>
```

```
 <p class="text-slate-300 text-sm mb-4">
```

```
 Представь город, где два района соединены
 её перекроют (удалят вершину) — районы будут
 g>хрупкость системы.
```

```
 </p>
```

```
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg border-
```

```
 <div class="absolute -top-3 left-4 bg-emerald-500
 er border-emerald-500 shadow-md">
```

```
 Телеком: Центральный свитч
```

```
 </div>
```

```
 <p class="text-slate-300 text-sm mb-4">
```

```
 У тебя несколько компьютеров в одной комнате
 абелем (мостом). Сами свитчи — это точки сочле-
```

```
 </p>
```

```
</div>
```

```
</div>
```

```
<details class="bg-slate-900/40 rounded-lg border-
```

```
 <summary class="p-4 font-bold text-slate-400 border-
 order-slate-700/50">
```

```
 Душный мод: Код и корень DFS (Скрыто)
```

```
</summary>
```

```

 <ul class="list-disc list-inside text-sm text-rose-400">
 <strong class="text-rose-400">Точки
ong> и показывают физическую уязвимость свя
 <strong class="text-emerald-400">SCC
ависимые "конгломераты".
 Как точка сочленения рушит
Косарайю, свернув каждую SCC в одну супер-в
(сделаем неориентированным), то любая <strong
о и есть критическая SCC! Если удалить эту
. Одно слабое звено изолирует целые касты S

 </div>
</div>
</section>`,
 },
];

```

ФАЙЛ 19/111: src/data/content.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 145 | РАЗМ

```

import { Chapter } from "../types";
import { graphChapters } from "../graphContent";
import { additionalTickets } from "../additionalTickets";
import { tickets14to20 } from "../tickets/ticket14_20";
import { tickets21to24 } from "../tickets/ticket21_24";
import { tickets6to13 } from "../tickets/ticket6_13";
import { ticket1Segtree } from "../tickets/ticket1_segtr
import { tickets18to20Mst } from "../tickets/ticket18_20
import { chapters as newChapters } from "../content_temp

/**
 * Содержание пособия = билеты 1–24 из списка тем.
 *
 * Порядок массивов важен: при одинаковом id побеждает
 * версия главы. `newChapters` (content temp) перетирае

```

```
merged.forEach((c) => {
 if (!toRemove.includes(c.id)) {
 dedup.set(c.id, c);
 }
});

// Самые свежие версии страниц перетирают старые.
if (newChapters) {
 newChapters.forEach((c) => {
 if (!toRemove.includes(c.id)) dedup.set(c.id, c);
 });
}

const finalChapters = Array.from(dedup.values());

finalChapters.sort((a, b) => {
 // Главы без номера билета (введение) стоят ДО билетов
 const numOf = (t: string) => {
 const m = t.match(/(\d+)/);
 return m ? parseInt(m[0], 10) : -1;
 };
 return numOf(a.title) - numOf(b.title);
});

export const chapters: Chapter[] = finalChapters;

/**
 * Билеты, закрытые каждой страницей (для шапки, экспор
 * Страница `mst`, например, закрывает сразу 18, 19 и 20
 */
export const chapterTopics: Record<string, number[]> = {
 "segment-trees": [1],
 "sparse-table": [2],
 treap: [3],
 "splay-tree": [4],
 "prefix-sums-2d": [5],
 "graph-dfs-bfs": [6],
 "planarity-euler-formula": [7],

```

```

const tickets = chapterTopics[chapterId] ?? [];
// Глав без билетов (введение) относим к первому разд
if (!tickets.length) return SECTIONS[0];
const first = Math.min(...tickets);
return SECTIONS.find((sec) => first >= sec.from && fi
}

export const activeVizIds = [
 "euler-path-vs-cycle", "bridges-code", "planarity-euler
 "graph-dfs-bfs", "stack-dfs", "queue-bfs", "heap-beam-se
 "dijkstra", "bellman-ford", "floyd", "aho-corasick",
 "mst", "string-kmp", "string-z-func", "segment-trees", "s
 "dynamic-programming", "johnson-algo", "graph-articulat
 "complexity-classes", "treap", "splay-tree", "prefix-sum
];

```

-----  
ФАЙЛ 20/111: src/data/debugRunner.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 207 | РАЗМ

```

/**
 * Пошаговый дебаггер поверх Pyodide: CPython выполняет
 * под sys.settrace и на каждой строке записывает фрейм
 * как панель Variables в обычном отладчике.
 */

```

```

export type DebugValue = null | boolean | number | string

```

```

export interface DebugStep {
 /** Номер строки в коде пользователя (1-based). */
 line: number;
 /** Имя функции-фрейма ("<module>" – верхний уровень)
 func: string;
 /** Локальные переменные: имя → герг (усечённый), для
 locals: Record<string, string>;
 /**

```



```

__steps = []
__final = {}
__final_values = {}
__CAP = ${DEBUG_STEP_CAP}
__was_truncated = False

class __TraceLimit(Exception):
 pass

def __safe(v):
 try:
 r = repr(v)
 except Exception:
 r = "<...>"
 return r if len(r) <= 80 else r[:77] + "..."

def __snapshot(v, depth=0):
 """Небольшой JSON-снимок: UI использует его без парсера"""
 if v is None or isinstance(v, (bool, int, str)):
 return v
 if isinstance(v, float):
 return v if v == v and v not in (float("inf"), float("-inf")) else "inf/-inf"
 if depth >= 4:
 return __safe(v)
 limit = 80 if depth < 2 else 32
 if isinstance(v, (list, tuple)):
 return [__snapshot(x, depth + 1) for x in list(v)]
 if isinstance(v, (set, frozenset)):
 return [__snapshot(x, depth + 1) for x in list(v)]
 if isinstance(v, dict):
 # 8x8 Sparse Table содержит 64+49+25+1 = 139 элементов
 # предел 80 незаметно обрезал уровни 4x4 и 8x8
 dict_limit = 192 if depth == 0 else limit
 return {str(k): __snapshot(x, depth + 1) for k, x in list(v.items())}
 # deque, heap-подобные и другие итерируемые учебные
 if type(v).__name__ == "deque":
 return [__snapshot(x, depth + 1) for x in list(v)]
 return __safe(v)

```

```

 # PEP 709: с 3.12 компрехенشنы встроены во фреймворк
 # на своей строке каждую итерацию. Склеиваем по
 if __steps and __steps[-1]["line"] == frame.f_lineno:
 __steps[-1]["locals"] = loc
 __steps[-1]["values"] = values
 else:
 __steps.append({"line": frame.f_lineno, "func": frame.f_code.co_name})
 if len(__steps) >= __CAP:
 # Нельзя просто отключить trace: бесконечный цикл
 # тогда продолжит работать вечно. Мягко прерываем
 __was_truncated = True
 raise __TraceLimit()
 # Финальные локалы программы (после последней строки)
 if event == "return" and frame.f_code.co_filename == __FILE__:
 loc, values = __capture(frame)
 __final.update(loc)
 __final_values.update(values)
 return __tracer

__err = ""
try:
 __sys.settrace(__tracer)
 exec(compile(__src, "<user-code>", "exec"), {"__name__": "__main__"})
except __TraceLimit:
 pass
except Exception:
 import traceback as __tb
 __err = __tb.format_exc(limit=3)
finally:
 __sys.settrace(None)

__OUT = __json.dumps({
 "steps": __steps,
 "finalLocals": __final,
 "finalValues": __final_values,
 "error": __err,
 "truncated": __was_truncated,
})

```

---

ФАЙЛ 21/111: src/data/glossary.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 639 | РАЗМ

---

```
import type { DemoKind } from "../components/hints/Mini"
```

```
/**
```

```
 * Глоссарий «википедийных» подсказок.
```

```
 *
```

```
 * Любое слово из `labels`, встреченное в тексте главы,
```

```
 * в подчёркнутый термин: при наведении (или тапе на мо
```

```
 * с объяснением на пальцах и мини-визуализацией.
```

```
 */
```

```
export interface GlossaryEntry {
```

```
 id: string;
```

```
 /** Варианты написания в тексте (регистр не важен). */
```

```
 labels: string[];
```

```
 title: string;
```

```
 /** Объяснение «на пальцах» – 1–2 фразы. */
```

```
 short: string;
```

```
 /** Строгая формулировка / что важно не забыть. */
```

```
 formal?: string;
```

```
 complexity?: string;
```

```
 /** Какая мини-анимация показывается в карточке. */
```

```
 demo?: DemoKind;
```

```
 /** id полного симулятора (см. vizRegistry) – кнопка
```

```
 simulator?: string;
```

```
}
```

```
export const GLOSSARY: GlossaryEntry[] = [
```

```
 {
```

```
 id: "bfs",
```

```
 labels: ["BFS", "обход в ширину", "поиск в ширину",
```

```
 title: "BFS – обход в ширину",
```

```
 short:
```

"Не отсортированный список, а «мешок, из которого  
formal:

"Полное бинарное дерево в массиве: дети узла  $i$  ле

complexity: "вставка/извлечение  $O(\log n)$ , «посмотре

demo: "heap",

simulator: "heap-beam-search",

},

{

id: "stack",

labels: ["стек", "стека", "стеке", "стеком", "LIFO"]

title: "Стек (LIFO)",

short: "Стопка тарелок: кладёшь сверху и берёшь свер

formal: "push / pop / top за  $O(1)$ . На стеке живёт р

demo: "stack",

simulator: "stack-dfs",

},

{

id: "queue",

labels: ["очередь", "очереди", "очередью", "FIFO"]

title: "Очередь (FIFO)",

short: "Очередь в столовой: кто первым встал, тот п

formal: "push\_back / pop\_front за  $O(1)$ . Основа BFS

demo: "queue",

simulator: "queue-bfs",

},

{

id: "dsu",

labels: ["DSU", "СНМ", "система непересекающихся мн

title: "DSU — система непересекающихся множеств",

short:

"«Кто твой староста?» Каждый элемент знает своего

formal:

"find со сжатием путей + union по рангу/размеру. I

complexity: "почти  $O(1)$  — обратная функция Аккерман

demo: "dsu",

simulator: "mst",

},

{

```

 short:
 "«Куда откатиться, если следующая буква не подошла",
 строки.",
 formal: "link(v) = go(link(parent(v)), ch). Считает",
 demo: "suffix-link",
 simulator: "aho-corasick",
},
{
 id: "toposort",
 labels: ["топологическая сортировка", "топсорт", "топ",
 title: "Топологическая сортировка",
 short:
 "Порядок «сначала штаны, потом ботинки»: каждая с",
 циклов.",
 formal: "DFS + вершины в обратном порядке выхода, л",
 complexity: "O(V + E)",
 demo: "toposort",
 simulator: "top-sort",
},
{
 id: "relaxation",
 labels: ["релаксация", "релаксацию", "релаксации",
 title: "Релаксация ребра",
 short:
 "«А через соседа не быстрее?» Если известный путь",
 ,
 formal: "if d[v] > d[u] + w(u,v) → d[v] = d[u] + w(u,v)",
 ан и Флойд.",
 demo: "relax",
 simulator: "dijkstra",
},
{
 id: "prefix-sum",
 labels: ["префиксные суммы", "префиксная сумма", "пре",
 title: "Префиксные суммы",
 short:
 "Заранее посчитанные «сколько всего набежало от н",
 formal: "P[0]=0, P[i]=P[i-1]+a[i-1]. Сумма a[l..r] = P[r] - P[l-1]"

```

```

 complexity: "O(V + E) одним DFS",
 demo: "bridge",
 simulator: "bridges-code",
},
{
 id: "articulation",
 labels: ["точка сочленения", "точки сочленения", "то"],
 title: "Точка сочленения",
 short:
 "Вершина-незаменимый-сотрудник: уволь её — и коллапс",
 formal: "Для не-корня: v — точка сочленения Э ребро",
 demo: "articulation",
 simulator: "graph-articulation",
},
{
 id: "mst",
 labels: [
 "остовное дерево", "остовного дерева", "остовное",
 "минимальное остовное дерево", "MST",
],
 title: "Минимальное остовное дерево (MST)",
 short:
 "Связать все точки проводами так, чтобы суммарная",
 formal: "V-1 ребро, никаких циклов. Краскал (жадно и",
 "но).",
 demo: "mst",
 simulator: "mst",
},
{
 id: "amortized",
 labels: [
 "амортизированная", "амортизированный", "амортизир",
 "амортизированное", "амортизированного", "амортизир",
],
 title: "Амортизированная сложность",
 short:
 "Средняя цена операции «по абонементу»: одна опер",
 formal: "Метод потенциалов: дорогая операция оплачи
```

```

{
 id: "prefix-function",
 labels: [
 "префикс-функция", "префикс-функции", "префикс-фу",
 "префикс", "префикса", "суффиксом", "суффикс", "с",
 "подстроки", "подстроку", "подстрока",
],
 title: "Префикс-функция π (КМП)",
 short:
 "Для каждой позиции запоминаем: какой кусок начался",
 "от кусок.",
 formal:
 " $\pi[i]$ — длина наибольшего собственного префикса s ",
 "возвращая указатель по тексту.",
 complexity: " $O(n + m)$ ",
 demo: "prefix-function",
 simulator: "string-kmp",
},
{
 id: "z-function",
 labels: ["Z-функция", "Z-функции", "Z-функцию", "z-"],
 title: "Z-функция",
 short:
 "Для каждой позиции: сколько символов подряд, начи",
 "аждой букве.",
 formal:
 " $z[i]$ = длина наибольшего общего префикса s и $s[i$ ",
 complexity: " $O(n)$ ",
 demo: "prefix-function",
 simulator: "string-z-func",
},
{
 id: "dp",
 labels: [
 "динамическое программирование", "динамики", "дин",
 "мемоизация", "мемоизацию", "подзадач", "подзадач",
],
 title: "Динамическое программирование",

```

```

 short:
 "На каждом шаге хватаем то, что лучше прямо сейчас"
 formal:
 "Корректность обычно доказывается «леммой о разрезе»"
 demo: "mst",
 simulator: "mst",
},
{
 id: "euler",
 labels: ["эйлеров цикл", "эйлеров путь", "эйлерова теорема"],
 title: "Эйлеров путь и цикл",
 short:
 "Пройти по каждому ребру ровно один раз, не отрываясь"
 formal:
 "Связный граф: эйлеров цикл все степени чётные; эйлеров путь все степени нечётные"
 simulator: "euler-path-vs-cycle",
},
{
 id: "planarity",
 labels: ["планарный", "планарность", "планарного", "планаризация"],
 title: "Планарность",
 short:
 "Граф планарен, если его можно нарисовать на листе бумаги"
 formal:
 "V - E + F = 2 для связного планарного графа. Курсивом выделены вершины, рёбра и области"
 simulator: "planarity-euler-formula",
},
{
 id: "splay",
 labels: [
 "splay", "splay-дерево", "AVL", "AVL-дерево", "всесторонний поиск"
],
 title: "Splay и повороты",
 short:
 "Каждый раз, когда к узлу обратились, его «вытягивают» к корню"
 formal:

```



```

 "После k-й итерации гарантированно верны все кратчайшие пути, которые не содержат отрицательный цикл.",
 complexity: "O(V · E)",
 demo: "relax",
 simulator: "bellman-ford",
 },
 {
 id: "floyd",
 labels: [
 "Флойд", "Флойда", "Флойд-Уоршелл", "Флойда-Уоршелла",
 "промежуточная вершина", "промежуточную вершину",
],
 title: "Флойд-Уоршелл: разрешаем вершину k",
 short:
 "Внешний цикл — это не «шаг», а «какие пересадки сделать, чтобы пересадку в k?",
 formal:
 "D[i][j] = min(D[i][j], D[i][k] + D[k][j]). Порядок пересадки выбирается среди всех вершин.",
 complexity: "O(V³) времени, O(V²) памяти",
 demo: "floyd",
 simulator: "floyd",
 },
 {
 id: "prim",
 labels: ["Прима", "Прим", "Prim"],
 title: "Алгоритм Прима",
 short:
 "Остов растёт как плесень из одной точки: на каждом шаге добавляется ребро, не создающее циклов.",
 formal:
 "Множество посещённых + куча рёбер на границе. Добавляем ребро с минимальным весом, не создающее циклов.",
 complexity: "O(E log V)",
 demo: "mst",
 simulator: "mst",
 },
 {
 id: "boruvka",

```

```
{
 id: "graph",
 labels: [
 "граф", "графа", "графе", "графы", "графов", "граф",
 "вершины", "вершина", "вершин", "ребра с весом",
],
 title: "Граф",
 short:
 "Точки (вершины) и связи между ними (рёбра). Города",
 formal:
 " $G = (V, E)$. Ребро бывает ориентированным (улица с односторонним движением)",
 demo: "graph",
 simulator: "graph-dfs-bfs",
},
{
 id: "coord-compress",
 labels: ["сжатие координат", "сжатия координат", "сжатие координат"],
 title: "Сжатие координат",
 short:
 "Координаты бывают гигантские и дробные, а нам нужно хранить их в виде целочисленного порядкового номера.",
 formal:
 "sort + unique + binary search: значение → индекс",
 complexity: " $O(n \log n)$ ",
 demo: "coord-compress",
},
{
 id: "treap",
 labels: ["Treap", "декартово дерево", "декартова д-во"],
 title: "Treap = Tree + Heap",
 short:
 "Одно дерево живёт по двум правилам сразу: по ключу и по приоритету. Случайность и держит его сбалансированным.",
 formal:
 "Базис — split(t, x) (разрезать по ключу) и merge(t1, t2) (соединить два дерева через них. Ожидаемая высота $O(\log n)$).",
 complexity: "ожидаемо $O(\log n)$ ",
}
```

```

 demo: "np",
 },
 {
 id: "range",
 labels: ["отрезок", "отрезка", "отрезке", "отрезки"],
 title: "Запрос на отрезке",
 short:
 "Спрашиваем не про один элемент, а про кусок массива

 предподсчёты.",
 formal:
 "Суммы — префиксные суммы $O(1)$. Минимум без изменений.",
 demo: "prefix-sum",
 simulator: "prefix-sums-2d",
 },
 {
 id: "string",
 labels: ["строки", "строке", "строку", "строка", "строкам"],
 title: "Строка как массив символов",
 short:
 "Строковые алгоритмы держатся на трёх вопросах: где

 (Z-функция) и где встречаются слова словаря (бордер-лист).",
 formal:
 "Наивный поиск подстроки — $O(N \cdot M)$. Префикс-функция — $O(N)$.",
 demo: "prefix-function",
 simulator: "string-kmp",
 },
 {
 id: "rotation",
 labels: ["поворот", "повороты", "поворота", "поворотом"],
 title: "Поворот (rotation) в дереве",
 short:
 "Локальная перевеска двух узлов: ребёнок встанет на

 дившемся месту. Порядок ключей при этом не ломается.",
 formal:
 "Правый поворот вокруг p: $x = p.left; p.left = x$.

 g и Zig-Zag.",
 complexity: "O(1)",
 demo: "zig",
 },

```

Универсальное пособие • полный исходный код

```

/** Быстрый доступ по id. */
export const GLOSSARY_BY_ID = new Map(GLOSSARY.map((e) => {

/** Все метки, отсортированные от длинных к коротким (ч
export const GLOSSARY_LABELS: { label: string; id: string }[] =
 e.labels.map((label) => ({ label, id: e.id })))
).sort((a, b) => b.label.length - a.label.length);

```

ФАЙЛ 22/111: src/data/graphContent.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 411 | РАЗМЕР: 1.5 КБ

```

import { Chapter } from "../types";

export const graphChapters: Chapter[] = [
 {
 id: "intro",
 title: "Введение. Базовая база: куча, очередь, бинарное
 type: "html",
 category: "Введение",
 content: `
 <section id="intro-content" class="mb-12">
 <div class="flex items-center mb-6 flex-wrap gap-2">
 Базовая база
 <h2 class="text-3xl font-bold text-white">Базовая база</h2>
 </div>

 <div class="bg-slate-900 border border-indigo-500/40 p-6">
 <p class="text-slate-200 text-sm leading-relaxed">
 <b class="text-indigo-300">Базовая база — это структура данных, которая
 работает как массив, куча, очередь и стек, бинарное дерево.
 Кодим по минимуму: там, где за нас уже написали.
 </p>
 </div>

 <div class="space-y-8">
```

```
h = []
heapq.heappush(h, 5); heapq.heappush(h, 1); heapq.heappop(h)
h[0] # 1 – минимум виден без всякого обхода
heapq.heappop(h) # 1, куча починилась за $O(\log n)$
```

<p class="text-slate-300 text-sm">Куча – движок, который поддерживает кучу, раз, и список с <span class="font-mono text-sm">heapq</span>.</p>

<div class="bg-slate-700/50 p-6 rounded-xl border-l-4 border-r-4 border-b-4 border-t-4">  
<h3 class="text-xl font-bold text-sky-400 mb-4">Очереди и стеки</h3>  
<p class="text-slate-300 text-sm mb-4"><b>Очереди</b> (BFS) отличаются от стеков (DFS) тем, что первый вышел) отличаются одной операцией снятия элемента</span>: оба конца работают за  $O(1)$ . Обычный список сдвигает весь хвост и стоит  $O(n)$ .</p>

```
<pre class="bg-slate-950 border border-slate-700">from collections import deque
q = deque([1, 2])
q.append(3); q.popleft() # 1 – очередь (BFS)
s = [1, 2]; s.append(3); s.pop() # 3 – стек (DFS)</pre>
```

<p class="text-slate-300 text-sm">Вся разница между BFS (волнами), стек – DFS (вглубь).</p>

<div class="bg-slate-700/50 p-6 rounded-xl border-l-4 border-r-4 border-b-4 border-t-4">  
<h3 class="text-xl font-bold text-amber-400 mb-4">Бинарный поиск</h3>  
<p class="text-slate-300 text-sm mb-4">Если массив отсортирован, то можно найти элемент за  $O(\log n)$  с помощью бинарного поиска. Если массив не отсортирован, то нужно сначала отсортировать, что займёт  $O(n \log n)$ . Если массив отсортирован, то можно найти элемент за  $O(\log n)$  с помощью бинарного поиска. Если массив не отсортирован, то нужно сначала отсортировать, что займёт  $O(n \log n)$ .</p>

```
<pre class="bg-slate-950 border border-slate-700">from bisect import bisect_left
a = [10, 20, 30, 40, 50]
bisect_left(a, 30) # 2 – индекс первого элемента, не меньшего 30
```

<p class="text-slate-300 text-sm">Без сортировки бинарный поиск не работает.</p>

<div class="bg-slate-700/50 p-6 rounded-xl border-l-4 border-r-4 border-b-4 border-t-4">  
<h3 class="text-xl font-bold text-rose-400 mb-4">Сортировка</h3>  
<p class="text-slate-300 text-sm mb-4">Писать сортировку можно по-разному. В Python есть встроенная функция sort, которая сортирует список in-place. Если нужно получить отсортированную копию, то можно использовать sorted().</p>

```

left(x) = z # z становится левым р
высоты пересчитываются только у z и x — ост
</div>
</details>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl border-l
 <h3 class="text-xl font-bold text-teal-400 mb-4
 <p class="text-slate-300 text-sm mb-4">Оба обхо
 тару». Вся разница в таре: стек уходит вглуби
 ext-emerald-300">0(V + E): каждая вершн
 <pre class="bg-slate-950 border border-slate-700
def go(graph, start, tank): # tank: список-стек
 seen, tank = {start}, tank([start])
 while tank:
 v = tank.pop() if isinstance(tank, list) else t
 for to in graph[v]:
 if to not in seen:
 seen.add(to); tank.append(to)
 return seen</pre>
 <p class="text-slate-300 text-sm">Дальше эта па
 надстройки над этими восемью строками.</p>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl border-l
 <h3 class="text-xl font-bold text-slate-300 mb-
 <div class="grid grid-cols-1 xl:grid-cols-2 gap
 <div class="bg-slate-800 p-6 rounded-lg bord
 <div class="absolute -top-3 left-4 bg-s
 border-indigo-500 shadow-md">
 Аналогия 1: Схема кинозала
 </div>
 <p class="text-slate-300 text-sm">Дерев
 азателей-«usher'ов» не нужно. Место n, дети
 рху вниз и слева направо.</p>
 </div>
 <div class="bg-slate-800 p-6 rounded-lg bord
 <div class="absolute -top-3 left-4 bg-s

```

```

title: "Билет 14. Дейкстра",
type: "html",
category: "Графы",
content: `<section id="dijkstra-content" class="mb-
<div class="flex items-center mb-6">
 <span class="bg-blue-600 text-white px-4 py-1 rounded
 <h2 class="text-3xl font-bold text-white">Дейкст
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border
 <h3 class="text-xl font-bold text-emerald-400">Аналогия 1:
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">Аналогия 1:
 <p class="text-xl font-mono text-white">Аналогия 1:
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Позитивное планирование
 </div>
 <p class="text-slate-300 text-sm mb-4">Аналогия 1: Позитивное планирование
 (нельзя поехать и заработать). Он всегда выбирает кратчайший путь.
 <div id="slot-chapter-image-dijkstra" class="h-400px">
 </div>

 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Ограничения
 </div>
 <p class="text-slate-300 text-sm mb-4">Ограничения
 (неожиданное ограничение: нельзя ехать по одной и той же улице дважды),
 (нельзя поехать и заработать). Он всегда выбирает кратчайший путь.
 Дейкстра сломается, так как не может учесть ограничение на количество

```

```

<div class="grid grid-cols-1 xl:grid-cols-2"
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 ♂ Аналогия 1: Параноик
 </div>
 <p class="text-slate-300 text-sm mb
еряет ВСЕ возможные дороги V-1 раз подряд.
 <div id="slot-chapter-image-bellman
 </div>

 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
 Отрицательный цикл
 </div>
 <p class="text-slate-300 text-sm mb
г станет ЕЩЕ короче – значит мы попали во в
 </div>
 </div>

 <div id="slot-bellman-viz" class="mt-8"></div>
</div>
</section>`
},
{
 id: "floyd",
 title: "Билет 16. Флойд-Уоршелл",
 type: "html",
 category: "Графы",
 content: `<section id="floyd-content" class="mb-12"
<div class="flex items-center mb-6">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-3xl font-bold text-white">Флойд

```



```

 },
 {
 id: "aho-corasick",
 title: "Билет 23. Алгоритм Ахо-Корасик",
 type: "html",
 category: "Строки",
 content: `<section id="aho-corasick" class="mb-12 s
<div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-indigo-600 text-white px-4 py-1 round
 <h2 class="text-2xl sm:text-3xl font-bold text-white
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border-l
 <h3 class="text-xl font-bold text-blue-400 mb-4">

 <div class="bg-slate-900 p-4 rounded-lg mb-6 text
 <p class="text-sm text-blue-300 italic mb-2">0с
 <p class="text-lg font-mono text-white">Бор (Tr
 </div>
 <p class="text-slate-300 text-sm">Позволяет найти
 -mono text-amber-300 bg-black/30 px-1 rounded">
 y.</p>
 </div>

<!-- Аналогии -->
<div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg border bo
 <div class="absolute -top-3 left-4 bg-slate-700
 emerald-500 shadow-md">
 Аналогия 1: Паутина (Бор)
 </div>
 <p class="text-slate-300 text-sm mb-4">Представ
 мы двигаемся по веткам. Если нужного продолж
 ("нити страховки") в самый длинный из извест
 </div>

 <div class="bg-slate-900 p-6 rounded-lg border-2

```

```

while (!q.empty()) {
 int v = q.front(); q.pop();
 for (auto const& [ch, child] : nodes[v].trie) {
 // Суффиксная ссылка ребёнка — это переход
 nodes[child].link = nodes[nodes[v].link].go

 // Если суффиксная ссылка сама является сло
 // Иначе наследуем terminal_link
 if (nodes[nodes[child].link].is_terminal) {
 nodes[child].term_link = nodes[child].l
 } else {
 nodes[child].term_link = nodes[nodes[ch
 }

 q.push(child);
 }
}
}
</div>
</div>
</details>
</div>
</section>`
},
{
 id: "alg-map",
 title: "Алгоритмы на карте",
 type: "html",
 category: "Графы",
 content: `<section id="alg-map-content" class="mb-1
<div class="flex items-center mb-6">
 <span class="bg-purple-600 text-white px-4 py-1
 <h2 class="text-3xl font-bold text-white">Алгори
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord

```

```

 options: [
 "Да: минимум и сумма считаются одним приёмом",
 "Нет: блоки перекрываются, а вычесть перекрытие",
 "Да, если заранее хранить суммы в нулевом уровне",
],
 correctIndex: 1,
 explanation: "Два (в 2D – четыре) блока накрывают
 обратная операция, но применить её к перекрытию
 ммы, билет 5."
},
{
 question: "Какие операции подходят разреженной та
 options: [
 "Любые ассоциативные",
 "Идемпотентные: min, max, gcd, побитовые И и ИЛИ",
 "Только сумма и произведение"
],
 correctIndex: 1,
 explanation: "Нужно $op(x, x) = x$: тогда дважды на
 приём и уходит в префиксные суммы."
},
{
 question: "Между запросами элементы матрицы меняю
 options: [
 "Разреженную таблицу: она перестраивается за $O(n^2)$ ",
 "Префиксную матрицу: достаточно поправить одну
 "Дерево отрезков: и таблица, и префиксы статичны",
],
 correctIndex: 2,
 explanation: "Обе структуры статичны: изменение э
 ксной. С обновлениями работает дерево отрезков,
 }
},
"prefix-sums-2d": [
 {
 question: "Сколько памяти занимает двумерная преф
 options: [
 "N × M – как исходная матрица, плюс рамка из ну

```

```

 ной операции нет. Для min, max и gcd берут разро
 },
 {
 question: "Изменили один элемент матрицы. Что ста
 options: [
 "Поправится одна клетка за $O(1)$ ",
 "Нужен пересчёт $O(N \cdot M)$: от этой клетки зависит
 "Ничего не станет: таблица остаётся верной"
],
 correctIndex: 1,
 explanation: "Каждая клетка $S[i][j]$ включает в се
 я частые, это задача дерева отрезков (билет 1).
 }
],
"euler-path-vs-cycle": [
 {
 question: "В связном графе ровно 2 вершины с нечёт
 options: [
 "Есть эйлеров цикл (вернись домой!)",
 "Есть эйлеров путь (из одной нечётной в другую)
 "Ни пути, ни цикла нет — это полный провал"
],
 correctIndex: 1,
 explanation: "Если нечётных вершин ровно 2 — это
 ётной, заканчиваем в другой."
 },
 {
 question: "Почему гамильтонов цикл — это NP-полна
 options: [
 "Потому что Гамильтон — болезнь, и лечить её т
 "Потому что для эйлерова цикла есть простой кри
 "Потому что Эйлер был умнее Гамильтона"
],
 correctIndex: 1,
 explanation: "Для эйлерова цикла работает критери
 ого критерия. Это NP-полная задача, которую не
 },
 {

```

```

 "0(2^V) – полный перебор"
],
 correctIndex: 1,
 explanation: "DFS обходит каждую вершину и каждое
}
],
"planarity-euler-formula": [
{
 question: "Планарный граф имеет V=6, E=12. Возможн
 options: [
 "Да, если это двудольный граф",
 "Нет, нарушается неравенство $E \leq 3V - 6$ ",
 "Да, если нет треугольных граней"
],
 correctIndex: 1,
 explanation: " $E \leq 3V - 6 \Rightarrow 12 \leq 12$ – на грани пр
 Так что такой граф непланарен."
 },
 {
 question: "Сколько граней у планарного графа с V=
 options: [
 "3 грани",
 "5 граней",
 "10 граней"
],
 correctIndex: 1,
 explanation: " $F = E - V + 2 = 10 - 7 + 2 = 5$. Не
 },
 {
 question: "Почему K5 непланарен?",
 options: [
 "Потому что $10 > 3*5 - 6 = 9$ – нарушение",
 "Потому что Эйлер был против",
 "Потому что 5 вершин – несчастливое число"
],
 correctIndex: 0,
 explanation: "У K5: V=5, E=10, $3V - 6 = 9$. $10 > 9$
 }
}

```

```

/** Транслировать реальные переменные визуализациям тек
export function emitVizState(
 chapterId: string,
 snapshot: Omit<VizStateEventDetail, "chapterId">
) {
 if (typeof window === "undefined") return;
 const detail: VizStateEventDetail = { chapterId, ...s
 latestByChapter.set(chapterId, detail);
 window.dispatchEvent(new window.CustomEvent<VizStateE
}

/** Вернуть демонстрацию в ручной режим при закрытии/ре
export function clearVizState(chapterId: string) {
 if (typeof window === "undefined") return;
 latestByChapter.delete(chapterId);
 window.dispatchEvent(new window.CustomEvent<VizStateE
}

/** Текущий снимок компилятора для прямой отрисовки мас
export function useVizRuntime(): VizStateEventDetail | n
 const chapterId = useContext(VizChapterContext);
 const [runtime, setRuntime] = useState<VizStateEventD
 chapterId ? latestByChapter.get(chapterId) ?? null
);

 useEffect(() => {
 setRuntime(chapterId ? latestByChapter.get(chapterId
 if (!chapterId) return;
 const handler = (e: Event) => {
 const detail = (e as CustomEvent<VizStateEventDet
 if (detail?.chapterId === chapterId) {
 setRuntime(detail.variables || detail.line !==
 }
 };
 window.addEventListener(CHANNEL, handler);
 return () => window.removeEventListener(CHANNEL, ha
 }, [chapterId]);

```

```

 }, [chapterId, currentStep, maxStep, goToStep, selectStep]
 }
}

```

```

/** Утилиты без небезопасных cast-ов для визуализаторов
export const vizNumber = (value: DebugValue | undefined) => {
 typeof value === "number" && Number.isFinite(value) ? value : null;
}

```

```

export const vizString = (value: DebugValue | undefined) => {
 typeof value === "string" ? value : null;
}

```

```

export const vizArray = (value: DebugValue | undefined) => {
 Array.isArray(value) ? value : null;
}

```

```

export const vizRecord = (value: DebugValue | undefined) => {
 value !== null && typeof value === "object" && !Array.isArray(value)
 ? (value as Record<string, DebugValue>)
 : null;
}

```

```

/* — Матрицы из компилятора: СВОЯ размерность, а не дефолтная

```

```

/**
 * Нормализованная матрица, пришедшая из выполненного Рун-тайма
 *
 * Размерность берётся из самих данных: если пользователь ввёл
 * 5 вершин, визуализация рисует 5x5, а не подставляет его же
 * 7x7 (раньше ячейки «не той» размерности превращались в 0)
 *
 * Поддерживаются все учебные способы хранения:
 * list[list] d = [[0, 4], [999, 0]]
 * list[dict] d = [{0: 0, 1: 4}, {1: 0}]
 * dict[dict] d = {0: {0: 0, 1: 4}}
 * dict[(i, j)] d = {(0, 1): 4} — ключ после запятой
 */

```

```

export interface LiveMatrix {
 rows: number;
 cols: number;
 /** Значение ячейки; null — таких данных в коде пользователя нет
 at: (i: number, j: number) => number | null;
}

```

```

 if (n === null) return;
 cells.set(`${i}:${j}`, n);
 rows = Math.max(rows, i + 1);
 cols = Math.max(cols, j + 1);
 };

const asArray = vizArray(value);
if (asArray) {
 asArray.slice(0, MAX_MATRIX_SIDE).forEach((row, i) => {
 const inner = vizArray(row);
 if (inner) {
 inner.slice(0, MAX_MATRIX_SIDE).forEach((cell, j) => {
 return;
 });
 }
 const record = vizRecord(row);
 if (record) {
 for (const [key, cell] of Object.entries(record)) {
 const j = Number(key);
 if (Number.isInteger(j)) put(i, j, cell);
 }
 }
 });
} else {
 const record = vizRecord(value);
 if (!record) return null;
 for (const [key, cell] of Object.entries(record)) {
 const pair = parsePairKey(key);
 if (pair) {
 put(pair[0], pair[1], cell);
 continue;
 }
 const i = Number(key);
 if (!Number.isInteger(i)) continue;
 const inner = vizArray(cell);
 if (inner) {
 inner.slice(0, MAX_MATRIX_SIDE).forEach((v, j) => {
 continue;
 });
 }
 }
}

```



-----  
/\* — Выбор демонстрации внутри страницы (вкладки 1D / 2D)

```
const DEMO_CHANNEL = "algo:viz-demo";
const latestDemoByChapter = new Map<string, string>();

export interface VizDemoEventDetail {
 chapterId: string;
 /** base-запись страницы – demoId пустой; подписанные */
 demoId: string;
}

/** Демонстрация сообщает, какая её вкладка сейчас открыта */
export function emitVizDemo(chapterId: string, demoId: string) {
 if (typeof window === "undefined") return;
 latestDemoByChapter.set(chapterId, demoId);
 window.dispatchEvent(new window.CustomEvent<VizDemoEventDetail>("viz-demo", {
 detail: { chapterId, demoId }
 }));
}

/**
 * Подписка: панель компилятора следует за активной вкладкой
 * Сразу отдаём последнее значение: компилятор часто отстает
 * как пользователь переключился с 1D на 2D.
 */
export function onVizDemo(chapterId: string, cb: (demoId: string) => void) {
 if (typeof window === "undefined") return () => {};
 const latest = latestDemoByChapter.get(chapterId);
 if (latest !== undefined) cb(latest);
 const handler = (e: Event) => {
 const detail = (e as CustomEvent<VizDemoEventDetail>).detail;
 if (detail && detail.chapterId === chapterId) cb(detail.demoId);
 };
 window.addEventListener(DEMO_CHANNEL, handler);
 return () => window.removeEventListener(DEMO_CHANNEL, handler);
}
```

## Универсальное пособие • полный исходный код

```

tree = [0] * (2 * n)
i = 0 # лист: tree[n + i]

v, l, r = 1, 0, n - 1 # вершина и её отрезок
m = (l + r) // 2 # граница детей 2v, 2v+1
print(f"v={v} [{l}..{r}] mid={m}")`,
 variables: [
 { name: "n", role: "длина исходного массива; листы"},
 { name: "i", role: "индекс элемента массива; его"},
 { name: "v", role: "текущая вершина дерева отрезка"},
 { name: "l, r", role: "границы отрезка, за который"},
 { name: "m", role: "середина отрезка $m = (l + r) // 2$ "},
],
},
treap: {
 vizTitle: "Декартово дерево: build → split → merge",
 stepNote: "Шаг = один спуск в split/merge/erase (переходимся целиком.",
 code: `points = [[45, 8], [3, 6], [6, 5], [2, 4], [1, 3]]
n = len(points) # сколько пар (x; y) – столько
mid = 7 # разрез для split: $x \leq mid < y$
key = 9 # какую точку удаляем в erase
step = 0 # номер шага – листает кадры
height = 0 # высота построенного дерева
root = None

--- тело алгоритма ---
class Node:
 def __init__(s, x, y):
 s.x = x; s.y = y; s.l = None; s.r = None

def build(pts):
 """Декартово дерево из отсортированных по x точек:
 pts = sorted(pts, key=lambda p: p[0])
 stack = []
 for x, y in pts:
 cur = Node(x, y)
 last = None
```



```

while x.p:
 p = x.p; g = p.p
 step += 1
 if g is None:
 case = "zig" # родители
 rot(x)
 elif (g.l is p) == (p.l is x):
 case = "zig-zig" # x и p -
 rot(p); rot(x)
 else:
 case = "zig-zag" # змейка
 rot(x); rot(x)
 xk, pk, gk = x.k, p.k, (g.k if g else 0)
 print("шаг", step, "->", case, "| x =", xk, "p = ", pk)
 root = x
 print(" корень теперь", x.k, "| обход:", inorder(root))
 return x

```

# 1) Zig: родитель x – уже корень (дерево 40 -> 20)  
 root = N(40); root.l = N(20); root.l.p = root  
 splay(root.l)

# 2) Zig-Zig: бамбук 60 -> 40 -> 20, оба сына левые  
 root = N(60); root.l = N(40); root.l.p = root; root.l.l = N(20)  
 splay(root.l.l)

# 3) Zig-Zag: змейка 60 -> 20 -> 40 (40 – правый сын 20)  
 root = N(60); root.l = N(20); root.l.p = root; root.l.r = N(40)  
 splay(root.l.r),

```

variables: [
 { name: "case", role: "какой случай сработал: \"zig-zig\", \"zig-zag\" или \"zig\" },
 { name: "step", role: "номер применения случая – от 1 до 3"},
 { name: "xk / x", role: "ключ узла, который поднимали"},
 { name: "pk / p", role: "ключ родителя x перед поворотом"},
 { name: "gk / g", role: "ключ деда – нужен в Zig-Zig"},
 { name: "root", role: "корень дерева после расшейки"},
]

```

```

 stepNote: "Шаг = заполнение очередного $P[i] = P[i-1]$
code: `a = [3, 1, 4, 1, 5, 9, 2, 6]` # исходный массив
n = len(a) # длина: префиксов будет n
i = 0 # текущая граница префикса

P = [0] # $P[0] = 0$ – пустой префикс
for i in range(1, n + 1):
 P.append(P[i - 1] + a[i - 1])
print("n =", n, "| P =", P)`
variables: [
 { name: "a", role: "исходный массив 1D-демонстрация" },
 { name: "n", role: "длина массива a; префиксов будет n" },
 { name: "P", role: "префиксные суммы: $P[i] = a[0] + \dots + a[i-1]$ " },
 { name: "i", role: "граница префикса: на шаге считаем до i" }
],
},
"dynamic-programming": {
 vizTitle: "Динамическое программирование (мемоизация)",
 stepNote: "Шаг = событие внутри fibМемо(k): вызов,
code: `n = 5`
memo = {}

def fib(k):
 if k in memo:
 return memo[k]
 if k <= 2:
 return 1
 print(f"считаем fib({k-1}) + fib({k-2})")
 r1 = fib(k - 1)
 r2 = fib(k - 2)
 memo[k] = r1 + r2
 return memo[k]

print("итог:", fib(n))`
variables: [
 { name: "n", role: "целевой номер числа Фибоначчи" },
 { name: "k", role: "аргумент текущего рекурсивного вызова" },
 { name: "memo", role: "кэш уже посчитанных значений" }
]
```

```

code: `from collections import deque

q = deque(["A"]) # очередь BFS
u = "B" # сосед, найденный из текущ
q.append(u) # enqueue соседа – в конец
front = q[0] # голова очереди: её обрабо
v = q.popleft() # шаг: достали голову и обр
print("front =", front, "| v =", v, "| очередь:", list(
 variables: [
 { name: "front", role: "голова очереди – вершина,
 { name: "v", role: "вершина, извлечённая из очере
 { name: "u", role: "сосед вершины v, которого кла
],
},
"everyday-basics": {
 vizTitle: "Бытовой тренажёр: стек, очередь, куча",
 stepNote: "Шаг = одна бытовая операция: тарелка на
 code: `from collections import deque

st, q = [], deque() # тарелки (стек) и очередь
st.append(" "); top = st.pop()
q.append(" "); q.append(" "); first = q.popleft()

i = 0 # счётчик выполненных опера
for op in ("push", "pop", "enqueue", "dequeue"):
 i += 1
print("тарелка:", top, "| первый из очереди:", first, "
 variables: [
 { name: "top / front", role: "верх стопки (стек)
 { name: "i", role: "индекс в массиве кучи; родите
],
},
"graph-dfs-bfs": {
 vizTitle: "DFS и BFS на графе",
 stepNote: "Шаг = переход из текущей вершины v к неп
 code: `from collections import deque

adj = {"A": ["B", "C"], "B": ["D"], "C": [], "D": []}
```

## Универсальное пособие • полный исходный код

```

v = -1 # вершина, в которой обход начался
stack = [] # фронт обхода (стек для DFS)
i = 0 # номер шага — двигает по какому-то ребру

--- тело алгоритма ---
for start in range(n):
 if comp[start] != -1:
 continue
 comps += 1 # непокрашенная вершина => новая компонента
 c = comps
 stack = [start]
 comp[start] = c
 used[start] = True
 v = start
 while stack:
 v = stack.pop() # для BFS: v = stack.pop(0)
 i += 1
 for to in adj[v]:
 if comp[to] == -1:
 comp[to] = c
 used[to] = True
 stack.append(to)
print("компонент:", comps)
print("разметка comp:", comp)`
variables: [
 { name: "n", role: "число вершин; в демонстрации — 10" },
 { name: "adj", role: "списки смежности — тот же граф, что в примере" },
 { name: "comp", role: "разметка: номер компоненты", range: "-1 ... comps" },
 { name: "used", role: "булев вариант разметки — для BFS" },
 { name: "comps", role: "счётчик найденных компонент" },
 { name: "c", role: "номер компоненты, которую красим" },
 { name: "v", role: "вершина, где обход находится" },
 { name: "stack", role: "фронт обхода: стек (DFS) или очередь (BFS)" },
 { name: "i / step", role: "номер шага — перематываем кадр" },
],
},
"top-sort": {

```

```
def dfs1(v):
 used[v] = True
 for to in adj[v]:
 if not used[to]:
 dfs1(to)
 order.append(v) # выходим из вершины — она

for v in adj:
 if not used[v]:
 dfs1(v)
 print("DFS из", v, "закончен | order:", order)

транспонирование: каждое ребро разворачиваем
adjT = {}
for v in adj:
 adjT[v] = []
for v in adj:
 for to in adj[v]:
 adjT[to].append(v)
print("обратный граф:", adjT)

фаза 2: идём по order С КОНЦА, DFS по adjT красит одн
comp = {}
c = 0
for i in range(len(order) - 1, -1, -1):
 v = order[i]
 if v in comp:
 continue
 stack = [v]
 comp[v] = c
 while stack:
 u = stack.pop()
 for to in adjT[u]:
 if to not in comp:
 comp[to] = c
 stack.append(to)
 print("SCC", c, "собрана из", v, ":", comp)
```



```

 },
 "bridges-code": {
 vizTitle: "Мосты: DFS + tin/low",
 stepNote: "Шаг = строка псевдокода слева; ребро (v, to) проверяется, является ли мостом. Если да, то его можно удалить, не нарушая связности графа.",
 code: `tin, low = {"A": 1, "B": 2, "G": 3}, {"A": 1, "B": 2, "G": 3}
timer = 3
v, to = "B", "G"
is_bridge = low[to] > tin[v]
print(f"ребро {v}-{to}: мост? {is_bridge}")`,
 variables: [
 { name: "v", role: "текущая вершина DFS", range: "A, B, G"},
 { name: "to", role: "сосед; ребро (v, to) проверяется", range: "A, B, G"},
 { name: "tin[v]", role: "время входа в вершину", range: "1, 2, 3"},
 { name: "low[v]", role: "минимум tin, куда можно вернуться", range: "1, 2, 3"},
 { name: "timer", role: "глобальный счётчик времени", range: "1, 2, 3"}
],
 },
 "euler-path-vs-cycle": {
 vizTitle: "Эйлеров маршрут: алгоритм Иера",
 stepNote: "Шаг = переход по неиспользованному ребру, если он существует. Если нет, то идти нельзя.",
 code: `n = 5
edges = [(0, 1), (0, 2), (1, 3), (2, 4), (1, 2), (3, 4)]
adj = {v: [] for v in range(n)}
for i, (u, v) in enumerate(edges):
 adj[u].append((v, i))
 adj[v].append((u, i))
deg = {v: len(adj[v]) for v in range(n)}
odd = [v for v in range(n) if deg[v] % 2 == 1]
start = odd[0] if odd else 0
used = [False] * len(edges)
path = [start]
last = start
stack = [start]
v = start
step = 0`
 },
}

```

```

V = n # вершин для формулы
E = sum(len(a) for a in adj) // 2 # рёбер: в списке смежн
F = 2 - V + E # граней: из $V - E + F = 2$

--- тело алгоритма ---
print("V =", V, "| E =", E, "| F =", F, "| $V - E + F = 2$ ")
for v in range(n):
 used = set()
 for u in adj[v]:
 if color[u] != 0:
 used.add(color[u])
 c = 1
 while c in used: # наименьший свободный цвет
 c += 1
 color[v] = c
 chi = max(chi, c)
 i += 1
 print("v =", v, "занято:", sorted(used), "-> цвет", color[v])
print("chi =", chi, "| delta =", delta, "| chi <= delta")
variables: [
 { name: "n", role: "число вершин графа на демонстрации" },
 { name: "adj", role: "списки смежности — тот же граф" },
 { name: "color", role: "массив цветов вершин (0 = не занят)" },
 { name: "v", role: "вершина, которую красим сейчас" },
 { name: "u", role: "сосед вершины v, чей цвет уже занят" },
 { name: "delta", role: "максимальная степень $\Delta = \max \deg v$ " },
 { name: "chi", role: "сколько цветов реально понадобится" },
 { name: "i / step", role: "номер шага — перематываем" },
 { name: "V, E, F", role: "для вкладки «Планарность»" },
]
},
dijkstra: {
 vizTitle: "Дейкстра",
 stepNote: "Шаг = старт, извлечение ближайшей вершины",
 code: `import heapq

```

## Универсальное пособие • полный исходный код

```

nodes = ["S", "A", "B", "C", "D", "E", "F"]
n = len(nodes) # 7 вершин => n - 1 = 6 работ
m = len(E) # 9 рёбер
```

```
dist = {v: INF for v in nodes}
dist["S"] = 0 # старт – База
```

```
--- тело алгоритма ---
```

```
last_changed = ""
for i in range(1, n): # итерация i = 1 ... n-1
 last_changed = ""
 for u, v, w in E: # шаг: релаксация ребра (u, v)
 if dist[u] != INF and dist[u] + w < dist[v]:
 dist[v] = dist[u] + w
 last_changed = v
 if last_changed == "":
 print(f"итерация {i}: изменений нет – сошлось для всех вершин")
 break
 print(f"итерация {i}: " + ", ".join(f"{k}={dist[k]}" for k in nodes))
```

```
контрольный (n-й) проход: улучшение здесь означает отрицательный цикл
cycle_edge = None
for u, v, w in E:
 if dist[u] != INF and dist[u] + w < dist[v]:
 cycle_edge = (u, v, w)
 break
```

```
if cycle_edge:
 print("отрицательный цикл: улучшается ребро", cycle_edge)
else:
```

```
 print("отрицательных циклов нет:", {k: dist[k] for k in nodes})
 variables = [
 { name: "n", role: "число вершин графа – ровно n-1"},
 { name: "m", role: "число рёбер, по которым проходят"},
 { name: "i", role: "номер итерации (после i итераций)"},
 { name: "u, v, w", role: "текущее ребро: откуда, куда, вес"},
 { name: "dist[v]", role: "расстояние; если обновилось, то новое"},
],
```

```

 на-Форда" },
 { name: "u, v", role: "ребро (u, v) перевзвешивает",
 { name: "d[v]", role: "расстояние Дейкстры в новом графе"
 },
],
},
mst: {
 vizTitle: "Остовное дерево: Краскал и DSU",
 stepNote: "Шаг = следующее ребро из отсортированных",
 code: `n = 9 # вершин: A..I — те, что в графе
edges = [(2, "A", "B"), (3, "D", "E"), (4, "B", "C"), (5, "A", "E"),
 (7, "E", "F"), (8, "D", "G"), (9, "C", "F"), (10, "B", "G"),
 (12, "H", "I"), (13, "F", "I")]
edges.sort() # Краскал: рёбра по возрастанию веса
parent = {v: v for v in "ABCDEFGH I"} # DSU: представителю
rank = {v: 0 for v in "ABCDEFGH I"} # высота дерева
mst = [] # рёбра, взятые в остов
total = 0 # вес остова (на этом графе)
i = 0 # номер текущего ребра
u, v, w = None, None, None # концы и вес ребра, которое

--- тело алгоритма ---
def find(x):
 while parent[x] != x:
 parent[x] = parent[parent[x]] # сжатие путей
 x = parent[x]
 return x

for i, (w, a, b) in enumerate(edges):
 u, v = a, b
 ra, rb = find(a), find(b)
 if ra != rb: # разные компоненты
 parent[ra] = rb
 if rank[ra] == rank[rb]:
 rank[rb] += 1
 mst.append((w, a, b))
 total += w
print("вес остова:", total, "| рёбер:", len(mst))

```

```

vizTitle: "Z-функция",
stepNote: "Шаг = вычисление z[i]: внутри Z-блока [l, r]",
code: `s = "abacaba"
n = len(s)
i, l, r = 1, 0, 0 # правый Z-блок [l, r]
z = [None] * n # пустые клетки: значения по умолчанию
z[0] = 0

for i in range(1, n): # шаг: вычисляем z[i]
 if i <= r:
 z[i] = min(r - i + 1, z[i - l]) # из блока
 else:
 z[i] = 0 # свежая клетка: с нуля
 while i + z[i] < n and s[z[i]] == s[i + z[i]]:
 z[i] += 1 # досчёт в лоб
 if i + z[i] - 1 > r:
 l, r = i, i + z[i] - 1
print(z)`
variables: [
 { name: "n", role: "длина строки s", range: "n = len(s)" },
 { name: "i", role: "позиция, для которой считаем z[i]" },
 { name: "l, r", role: "правый крайний Z-блок: отрезок [l, r]" },
 { name: "z[i]", role: "длина наибольшего общего префикса s[i:] и s[0:z[i]]" }
],
},
"aho-corasick": {
 vizTitle: "Ахо-Корасик: бор, fail-ссылки и автомат",
 stepNote: "Шаг = один символ текста: state = go(state, c). Если не найдено, то переходят в корень. Если совпадают с картинкой «водопада».",
 code: `words = ["HERS", "HIS", "SHE", "HE"] # образцы
text = "USHERS" # текст, в котором ищем
n = len(text) # длина текста
state = 0 # текущее состояние автомата
i = 0 # индекс текущего символа
c = "" # сам символ
nodes = 1 # сколько вершин в автомате
queue = [] # очередь BFS, к которой добавляем новые вершины
fail = {} # fail[v] = вершина, в которую нужно перейти, если не найдено

```

```

 queue.append(u)
print("вершин в боре:", nodes)
print("fail-ссылки:", fail)

def go(v, ch):
 """Переход автомата: сначала по бору, иначе спускаем
 while v != 0 and (v, ch) not in nxt:
 v = fail[v]
 return nxt.get((v, ch), 0)

for i, c in enumerate(text):
 state = go(state, c)
 v = state
 while v != 0: # собираем все образцы
 if v in term:
 matches.append((i, term[v]))
 v = fail[v]
 print(f"i={i} c='{c}' state={state} найдено={matches}")

print("всего вхождений:", len(matches), matches),
variables: [
 { name: "words", role: "набор образцов – по нему"},
 { name: "text", role: "сканируемый текст – подста"},
 { name: "state", role: "текущее состояние автомата"},
 { name: "i / c", role: "индекс и сам символ текста"},
 { name: "nxt", role: "переходы бора: nxt[(v, ch)]"},
 { name: "fail", role: "fail[v] – вершина наибольшей"},
 { name: "term", role: "в каких вершинах заканчивае"},
 { name: "queue", role: "очередь BFS, которой натя"},
 { name: "matches", role: "найденные вхождения: (ин"},
 { name: "nodes", role: "сколько вершин в боре (ко"},
],
},
"complexity-classes": {
 vizTitle: "Классы сложности: рост по n и ops",
 stepNote: "Шаг = одна точка (n, ops): визуализация",
 code: `n = 1 # размер входа

```

```

 },
 intro: {
 // мнемодкартки без шагов алгоритма
 variables: [],
 },

 /* — Дополнительные демонстрации внутри страниц (вкл.)

 "planarity-euler-formula#greedy": {
 vizTitle: "Жадная покраска вершин ($\chi \leq \Delta + 1$)",
 stepNote: "Шаг = вершина v получает наименьший цвет",
 code: `n = 8 # вершин в графе
adj = [[1, 2], [0, 2, 3], [0, 1, 4], [1, 4, 5], [2, 3, 4]]
delta = max(len(a) for a in adj) # максимальная степень
color = [0] * n # 0 = ещё не покрашена
chi = 0 # сколько цветов понадобилось
v = -1 # вершина, которую красим сейчас
i = 0 # номер шага

--- тело алгоритма ---
for v in range(n):
 used = set()
 for u in adj[v]:
 if color[u] != 0:
 used.add(color[u])
 c = 1
 while c in used: # наименьший свободный цвет
 c += 1
 color[v] = c
 chi = max(chi, c)
 i += 1
 print("v =", v, "занято:", sorted(used), "-> цвет", color[v])
print("chi =", chi, "| delta =", delta, "| chi <= delta")
variables: [
 { name: "color", role: "цвета вершин (0 = не покрашено)", range: "0 ... n" },
 { name: "v", role: "текущая вершина", range: "0 ... n-1" },
 { name: "delta / chi", role: "максимальная степень / количество цветов", range: "1 ... n" },
 { name: "i / step", role: "номер кадра", range: "0 ... n-1" },
]
 `
 },

```

```

],
},
"planarity-euler-formula#edges": {
 vizTitle: "Покраска рёбер (Визинг: $\Delta \leq \chi' \leq \Delta + 1$)",
 stepNote: "Шаг = ребро e получает наименьший цвет, <= \Delta",
 code: `n = 8
edges = [(0, 1), (0, 2), (1, 2), (1, 3), (2, 4), (3, 4)]
adj = [[1, 2], [0, 2, 3], [0, 1, 4], [1, 4, 5], [2, 3, 4]]
delta = max(len(a) for a in adj)
ecolor = [0] * len(edges) # цвет каждого ребра
inc = [dict() for _ in range(n)] # inc[v][цвет] = ребро
e = -1 # номер текущего ребра
v = u = -1 # его концы
chi_e = 0

--- тело алгоритма ---
for e, (a, b) in enumerate(edges):
 v, u = a, b
 used = set(inc[a].values()) | set(inc[b].values())
 c = 1
 while c in used:
 c += 1
 ecolor[e] = c
 inc[a][e] = c
 inc[b][e] = c
 chi_e = max(chi_e, c)
 print("ребро", (a, b), "занято:", sorted(used), "-> ", c)
print("chi' =", chi_e, "| delta =", delta, "| delta <= \Delta")
variables: [
 { name: "ecolor", role: "цвета рёбер – рёбра на графе"},
 { name: "e", role: "номер текущего ребра", range: "0..n-1"},
 { name: "v, u", role: "концы текущего ребра – подмножества V"},
 { name: "inc", role: "для каждой вершины: какие цвета рёбер"},
 { name: "chi_e / delta", role: "\chi' – сколько цветов"},
]
},
"graph-components#dfs": {
 vizTitle: "Компоненты связности: DFS-покраска",

```



```

 },
 "graph-components#dsu": {
 vizTitle: "Компоненты связности: DSU по рёбрам",
 stepNote: "Шаг = одно ребро (u, v): union склеивает",
 code: `n = 10 # вершин в графе
edges = [(0, 1), (1, 2), (2, 3), (0, 3), (1, 3), (4, 5)]
parent = list(range(n)) # DSU: parent[v] = представи
rank = [0] * n # высота дерева – по ней реш
comps = n # сначала каждая вершина – о
e = -1 # номер обрабатываемого ребра
u, v = -1, -1 # концы текущего ребра
step = 0 # номер шага – двигает по ка

--- тело алгоритма ---
def find(x):
 while parent[x] != x:
 parent[x] = parent[parent[x]] # сжатие путей
 x = parent[x]
 return x

def union(a, b):
 global comps
 ra, rb = find(a), find(b)
 if ra == rb:
 return False # ребро внутри
 if rank[ra] < rank[rb]:
 ra, rb = rb, ra
 parent[rb] = ra
 if rank[ra] == rank[rb]:
 rank[ra] += 1
 comps -= 1
 return True

for e, (a, b) in enumerate(edges):
 u, v = a, b
 step = e + 1
 union(a, b)
print("компонент:", comps)

```

```

k = 3 # размер клики, которую ищем

--- тело алгоритма ---
Сведение 3-SAT <=p КЛИКА: вершина = {номер дизъюнкта,
ребро = два литерала из РАЗНЫХ дизъюнктов, не противоречивые
clauses = [(1, 2, -3), (-1, 3, 4), (2, -3, -4)]
V = [(i, l) for i, cl in enumerate(clauses) for l in cl]
E = [(a, b) for a in V for b in V if a[0] < b[0] and a[1] != b[1]]
for step in range(7):
 print("кадр", step, "| вершин:", len(V), "| рёбер:", len(E))
print("формула выполнима <=> в графе есть клика размера k")
variables: [
 { name: "step", role: "номер кадра сведения (0..6)" },
 { name: "cls", role: "класс задачи A, которую сводят к КЛИКА" },
 { name: "n", role: "размер входа x; после сведения к КЛИКА" },
 { name: "k", role: "размер клики, которую ищет рекурсия" },
],
},
"complexity-classes#verify": {
 vizTitle: "Верификатор 3-SAT: проверка сертификата",
 stepNote: "Шаг = один дизъюнкт: верификатор подставляет значения в формулу",
 code: `clauses = [(1, 2, -3), (-1, 3, 4), (2, -3, -4)]
cert = [1, 0, 1, 0] # сертификат: x1=1, x2=0, x3=1
k = 0 # номер проверяемого дизъюнкта
ok = True # вердикт верификатора

--- тело алгоритма ---
for k, cl in enumerate(clauses):
 hit = False
 for l in cl:
 val = cert[l - 1] if l > 0 else 1 - cert[-l - 1]
 if val == 1:
 hit = True
 ok = ok and hit
 print("дизъюнкт", k + 1, cl, "выполнен" if hit else "не выполнен")
print("сертификат", "принят" if ok else "отвергнут")`,
 variables: [
 { name: "cert", role: "сертификат – назначение переменных" },
],
},

```

```

edges.sort() # Краскал: рёбра по возрастанию веса
parent = {v: v for v in "ABCDEFGHI"} # DSU: представителем компонента
rank = {v: 0 for v in "ABCDEFGHI"} # высота дерева
mst = [] # рёбра, взятые в остов
total = 0 # вес остова (на этом графе 51)
i = 0 # номер текущего ребра
u, v, w = None, None, None # концы и вес ребра, которое рассматриваем

--- тело алгоритма ---
def find(x):
 while parent[x] != x:
 parent[x] = parent[parent[x]] # сжатие путей
 x = parent[x]
 return x

for i, (w, a, b) in enumerate(edges):
 u, v = a, b
 ra, rb = find(a), find(b)
 if ra != rb: # разные компоненты
 parent[ra] = rb
 if rank[ra] == rank[rb]:
 rank[rb] += 1
 mst.append((w, a, b))
 total += w
print("вес остова:", total, "| рёбер:", len(mst))
print(mst)`
 variables: [
 { name: "i", role: "номер текущего ребра – перебираем"},
 { name: "u, v, w", role: "концы и вес рассматриваемого ребра"},
 { name: "parent", role: "DSU-массив: цвета компонентов"},
 { name: "mst", role: "рёбра, уже взятые в остов", "51"},
 { name: "total", role: "вес остова (51 на этом графе)"},
],
},
"mst#prim": {
 vizTitle: "Остовное дерево: Прим (рост из вершины E)",
 stepNote: "Шаг = из кучи достаётся самое лёгкое ребро",
 code: `import heapq

```

```

vizTitle: "Остовное дерево: Борувка (фазы-«пятiletки»)"
stepNote: "Шаг = одна фаза: каждая компонента ОДНОВЕРШИНОВА"
code: `n = 9 # вершин: A...I
edges = [(2, "A", "B"), (3, "D", "E"), (4, "B", "C"), (5, "C", "D"),
 (7, "E", "F"), (8, "D", "G"), (9, "C", "F"), (10, "F", "G"),
 (12, "H", "I"), (13, "F", "I")]
comp = {v: v for v in "ABCDEFGH I"} # DSU: корень компоненты
cheapest = {} # корень компоненты -> самое дешёвое ребро
mst = [] # рёбра остова
total = 0 # вес остова (= 51)
phase = 0 # номер фазы («пятiletки») — от 0 до 4
u, v, w = None, None, None # ребро-кандидат

--- тело алгоритма ---
def find(x):
 while comp[x] != x:
 comp[x] = comp[comp[x]]
 x = comp[x]
 return x

while len(mst) < n - 1:
 phase += 1
 cheapest = {}
 for (w, a, b) in edges: # КАЖДАЯ компонента ОДНОВЕРШИНОВА
 ra, rb = find(a), find(b)
 if ra == rb:
 continue
 u, v = a, b
 if ra not in cheapest or w < cheapest[ra][0]:
 cheapest[ra] = (w, a, b)
 if rb not in cheapest or w < cheapest[rb][0]:
 cheapest[rb] = (w, a, b)
 if not cheapest:
 break # граф несвязен
 for (w, a, b) in cheapest.values():
 ra, rb = find(a), find(b)
 if ra == rb:
 continue # две компоненты

```

```

 { name: "k", role: "уровень: блоки $2^k \times 2^k$ (шаг
 { name: "st2[{r,c,k,k}]", role: "минимум квадратн
 },
],
},
"sparse-table#2d-query": {
 vizTitle: "Разреженная таблица 2D: запрос",
 stepNote: "Демонстрация кликабельна вручную: выбери
 code: `# st2[{r, c, kx, ky}] из вкладки «построение
r1, c1, r2, c2 = 1, 1, 6, 6

h, w = r2 - r1 + 1, c2 - c1 + 1
kx, ky = h.bit_length() - 1, w.bit_length() - 1
print(f"прямоугольник {h}x{w} кроется блоками уровня ({
 variables: [
 { name: "r1, c1, r2, c2", role: "углы запрашиваем
 { name: "kx, ky", role: "уровень самого крупного
],
},
"prefix-sums-2d#2d": {
 vizTitle: "Префиксные суммы 2D (прямоугольники)",
 stepNote: "Демонстрация кликабельна вручную: наведи
 code: `A = [[1, 2, 3, 4], [5, 6, 7, 8], [9, 1, 2, 3
n, m = 4, 4
i, j = 0, 0
S = [] # строки таблицы: рамка и
for rr in range(n + 1):
 S.append([0] + [None] * m)
S[0] = [0] * (m + 1)
for i in range(1, n + 1):
 for j in range(1, m + 1):
 S[i][j] = A[i - 1][j - 1] + S[i - 1][j] + S[i][j - 1]

r1, c1, r2, c2 = 1, 1, 2, 2
ans = S[r2 + 1][c2 + 1] - S[r1][c2 + 1] - S[r2 + 1][c1]
print("итого:", ans)`
 variables: [
 { name: "i, j", role: "текущая ячейка таблицы S (

```

```

const keep: string[] = [];
for (const line of code.split("\n")) {
 const t = line.trim();
 if (t === "# --- тело алгоритма ---" || /^(for|while)/.test(t)) {
 keep.push(line);
 }
}
// убрать пустые строки с конца
while (keep.length > 0 && keep[keep.length - 1].trim() === "") {
 keep.pop();
}

/**
 * Дефолтное содержимое редактора: шапка + ТОЛЬКО инициализация
 * переменные демо (без всего кода алгоритма).
 */
export function buildInitTemplate(chapterId: string, chapterTitle: string, sync: Sync) {
 const sync = getPageSync(chapterId, demoId);

 const head = sync.vizTitle
 ? `# «${chapterTitle}»\n# демо: ${sync.vizTitle}\n#`
 : `# «${chapterTitle}»\n# демо на странице нет – св

 if (!sync.code) return head;
 const init = extractInit(sync.code);
 return `${head}\n${init}\n`;
}

```

## РАЗДЕЛ: БИЛЕТЫ (УЧЕБНЫЙ КОНТЕНТ)

=====

-----  
ФАЙЛ 26/111: src/data/tickets/ticket1\_segtree.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 217 | РАЗМЕР: 1.5 КБ

-----  
import { Chapter } from '../../types';

```

span class="font-mono text-emerald-300">[0, 1
тям <span class="font-mono text-emerald-300
– это одиночные элементы массива. В узле л
чество единиц, И/ИЛИ – любая операция, кото
>).</p>

```

```

<p class="text-slate-300 text-sm mb-4">Отсю
а каждом уровне встречает один из трёх случ
не пересекается – возвращаем нейтральный э
склеиваем. Частичных пересечений на уровне
одного элемента – спуск к листу и пересчёт

```

```

<div class="grid grid-cols-1 sm:grid-cols-3
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
 </div>
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
 </div>
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
 </div>
</div>

```

```

<div class="grid grid-cols-1 xl:grid-cols-2
 <div class="bg-slate-800 p-6 rounded-lg
 <div class="absolute -top-3 left-4 l
 <div class="border border-emerald-500 shadow-md">

```

Аналогия 1: Корпоративная пре

```

 </div>
 <p class="text-slate-300 text-sm">Б
икам департаментов: «внутри всем +5 %». Нач
nt-mono text-emerald-300">lazy[v])</div> и
свой баланс. Это и есть отложенное обнов
</div>

```

```
<p class="text-slate-500 text-xs mt-3">
кивают в строгом порядке: сначала присваива
</div>
```

```
<div class="bg-slate-900/40 rounded-lg border
 <p class="text-slate-300 text-sm mb-2">
 <p class="text-slate-400 text-sm mb-1">
за $O(1)$, но строятся один раз и умирают при
ми.</p>
```

```
 <p class="text-slate-400 text-sm mb-1">
идемпотентные запросы (min, gcd, И) по ста
 <p class="text-slate-400 text-sm mb-1">
n ячеек (не $2n!$) – иначе на $n = 6$ поймать п
n ... $2n-1$] и n – степень двойки (или использ
 <p class="text-slate-400 text-sm">•
ть лень во всё дерево – это $O(n)$ и весь смы
</div>
```

```
<p class="text-slate-300 text-sm mt-4"><b c
-2 py-0.5 rounded text-xs font-mono text-em
ся один раз), запрос на отрезке <code class=
-slate-800"> $O(\log n)$ </code>, одиночное обно
-300 border border-slate-800"> $O(\log n)$ </code>
s font-mono text-emerald-300 border border-
xt-xs font-mono text-emerald-300 border bord
ounded text-xs font-mono text-emerald-300 b
```

```
<p class="text-slate-500 text-xs mt-3 leading
в редакторах и базах, сжатие координат + «
оить», дерево отрезков по дереву (HLD), пер
</p>
```

```
<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
```

Код: построение и запрос суммы (С

```
</summary>
```

```
<div class="p-5 text-sm text-slate-300
```



```

 y[4 * MAXN]; // «обещание»: прибавит

void push(int v, int l, int r) {
 if (lazy[v] == 0) return;
 int m = (l + r) / 2;
 tree[2 * v] += lazy[v] * (m - l + 1); // п
 lazy[2 * v] += lazy[v];
 tree[2 * v + 1] += lazy[v] * (r - m); // и
 lazy[2 * v + 1] += lazy[v];
 lazy[v] = 0; // о
}

void add(int v, int l, int r, int L, int R, int x) { /
 if (r < L || R < l) return;
 if (L <= l && r <= R) {
 tree[v] += x * (r - l + 1);
 lazy[v] += x; // д
 return;
 }
 push(v, l, r); // о
 int m = (l + r) / 2;
 add(2 * v, l, m, L, R, x);
 add(2 * v + 1, m + 1, r, L, R, x);
 tree[v] = tree[2 * v] + tree[2 * v + 1];
}

```

<p class="text-slate-400 text-xs">Пр  
 г накопиться. Забыл push – получил правильн  
 </div>  
 </details>

<details class="bg-slate-900/40 rounded-lg l  
 <summary class="p-4 font-bold text-slate  
 -b border-slate-700/50">  
 Код: итеративное дерево (2n, снизу)  
 </summary>  
 <div class="p-5 text-sm text-slate-300  
 <pre class="bg-slate-950 p-4 rounded  
 # для удобств

```
</section>
```

```
`,
 },
];
```

---

ФАЙЛ 27/111: src/data/tickets/ticket14\_20.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 1218 | РАЗМЕР: 10000 Б

---

```
import { Chapter } from '../types';
```

```
export const tickets14to20: Chapter[] = [
 {
```

```
 id: "dijkstra",
 title: "14. Графы. Поиск кратчайшего пути. Дейкстра",
 type: "html",
 category: "Графы. Пути",
 content: `
```

```
<section id="dijkstra-content" class="mb-12 scroll-mt-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 14. Графы. Поиск кратчайшего пути. Дейкстра
 <h2 class="text-2xl sm:text-3xl font-bold text-slate-900">14. Графы. Поиск кратчайшего пути. Дейкстра</h2>
 </div>
```

```
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-700">
 <h3 class="text-xl font-bold text-emerald-400">14.1. Поиск кратчайшего пути. Дейкстра</h3>

 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">Дейкстра — алгоритм поиска кратчайшего пути в графе с неотрицательными весами.</p>
 <p class="text-xl font-mono text-white">Дейкстра</p>
 </div>
```

```
 <p class="text-slate-300 text-sm mb-4">Дейкстра — алгоритм поиска кратчайшего пути в графе с неотрицательными весами.</p>
 <p>Вершины V и ребра E графа G = (V, E) удовлетворяют требованию к графу — неотрицательные веса.</p>
```

```
ext-emerald-300">w ≥ 0}, зафиксирова
ию расстояния!</p>
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg
 <div class="absolute -top-3 left-4
border-rose-500 shadow-md">
```

Ограничения: минус ломает жадно

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4"
class="font-mono text-emerald-300">s→a = 2</
ono text-emerald-300">b→a = -2. Верн
начала вытащит <span class="font-mono text-
o text-emerald-300">3 - 2 = 1 придёт
Джонсона (билет 17).</p>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
<div class="bg-slate-700/50 p-6 rounded-xl bord
<h3 class="text-xl font-bold text-blue-400 m
```

```
<p class="text-slate-300 text-sm mb-4">Демон
D, Дом 2 E, ТЦ F, Метро G) и 11 одностор
ршины <span class="font-mono text-emerald-3
(u → v): переменные <span class="font
>,
аут картинку напрямую.</p>
```

```
<ol class="list-decimal pl-5 space-y-2 text
 Инициализация. ∞; в кучу кладём prev пуст.
```

```
 Извлечь минимум. Достаём пару du. Если done – пара устарела
```

```
 Зафиксировать. du
меняется.
```

```
</div>
```

```
<div class="bg-slate-800/70 p-5 rounded-lg">
 <p class="text-sm font-bold text-white">
 <table class="w-full text-xs text-left">
 <thead>
 <tr class="text-slate-500 border">
 <th class="py-1.5 pr-2 font">
 <th class="py-1.5 pr-2 font">
 <th class="py-1.5 font-bold">
 </tr>
 </thead>
 <tbody class="text-slate-300">
 <tr class="border-b border-slate-
y-2 pr-2">лучший известный путь $s \rightarrow v$ </td><td>
 <tr class="border-b border-slate-
y-2 pr-2">предок v на текущем лучшем пути</td><td>
 <tr class="border-b border-slate-
lass="py-2 pr-2">вершины с окончательным $dist[v]$ </td></tr>
 <tr class="align-top"><td class="text-slate-300">
ие, вершина)</td><td class="py-2">извлечения
 </tbody>
 </table>
 </div>
```

```
</div>
```

```
<div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-indigo-400">Извлечение из кучи

 <p class="text-slate-300 text-sm mb-4">Инициализация
 s извлекается из кучи, $s \rightarrow u$. Отсюда
 $dist[v] = \delta(s, v)$ для всех достижимых v .
 $dist[v] = \infty$ для недостижимых v .
```

```
<details class="bg-slate-900/40 rounded-lg">
 <summary class="p-4 font-bold text-slate-300">Извлечение из кучи
```

```

 </div>
 </details>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl border-2 border-slate-800">
 <h3 class="text-xl font-bold text-emerald-400">Дейкстра на графе
 <div class="overflow-x-auto my-4">
 <table class="w-full text-xs text-left">
 <thead>
 <tr class="text-slate-500 border-b">
 <th class="py-2 pr-3 font-b">
 <th class="py-2 pr-3 font-b">
 <th class="py-2 font-bold">
 </tr>
 </thead>
 <tbody class="text-slate-300">
 <tr class="border-b border-slate-800">
 <td class="text-slate-300">priority_queue (C++)</td><td class="py-2 pr-3 font-mono">
 й выбор</td></tr>
 <tr class="border-b border-slate-800">
 <td class="text-slate-300">ease-key</td><td class="py-2 pr-3 font-mono">
 очереди</td></tr>
 <tr class="border-b border-slate-800">
 <td class="text-slate-300">поиск минимума</td><td class="py-2 pr-3 font-mono">
 </tr>
 <tr class="border-b border-slate-800">
 <td class="text-slate-300">><td class="py-2 pr-3 font-mono">0(V + E)</td></tr>
 <tr class="align-top"><td class="text-slate-300 font-mono">0(E + V · C)</td><td class="py-2">ц
 </tr>
 </tbody>
 </table>
 </div>

<details class="bg-slate-900/40 rounded-lg border-2 border-slate-800">
 <summary class="p-4 font-bold text-slate-300 border-b border-slate-700/50">» Код: Дейкстра на
 <div class="p-5 text-sm text-slate-300"

```

```

 "E": [{"F", 1}],
 "F": [],
 "G": [{"D", 1}, {"E", 2}],
}
dist, prev = dijkstra(g, "A")
print(dist) # A 0, B 3, C 2, D 6, E 7,
print(restore(prev, "A", "F")) # ['A', 'C', 'G', 'E',
 <p class="text-slate-400 text-xs">П
 же вершины, потому что читает ваши <span c
nt-mono">done.</p>
 </div>
</details>

<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">> Код: C++ (priority
 <div class="p-5 text-sm text-slate-300
<pre class="bg-slate-950 p-4 rounded overflow-x-auto te
 t;
#include <queue>;
using namespace std;
const long long INF = 4e18; // не INT_MAX:

int n;
vector<vector<pair<int,int>>>> g;
vector<long long> d;
vector<int> p;

void dijkstra(int s) {
 d.assign(n, INF); p.assign(n, -1); d[s] = 0;
 priority_queue<pair<long long,int>;,
 vector<pair<long long,int>>&
 greater<pair<long long,int>>
 pq.push({0, s});
 while (!pq.empty()) {
 auto [du, u] = pq.top(); pq.pop();
 if (du > d[u]) continue; // устарев
 for (auto [v, w] : g[u]) {

```

```
used[u] = True # фиксируем d[u]
for v in range(n): # релаксируем v
```

```
 if d[u] + w[u][v] < d[v]:
 d[v] = d[u] + w[u][v]
```

Т. Память – матрица

```
</div>
</details>
</div>
```

```
<div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-amber-400">

```

```
<div class="bg-rose-950/30 p-5 rounded-lg border">
 <p class="text-sm font-bold text-rose-300">

```

```
<ul class="list-disc pl-5 space-y-2 text-rose-300">
 Отрицательные веса. Алгоритм релаксации вершины необратима. Нужен Форд–Беллмановский цикл.
 Отрицательные циклы. При наличии отрицательного цикла не существует минимального числа. Сначала проверьте граф Фордом–Беллманом.
 Ленивое удаление. Без предварительной обработки вершина обрабатывается несколько раз:
 Переполнение INF. u уходит в отрицательное число и «улучшается».

```

```
Ориентированность. Демонстрация алгоритма на графе: каждое ребро добавляется в оба списка.
 Дейкстра ≠ BFS. BFS – частный случай Дейкстры. С разными весами FIFO ломается, нужна куча.
 «Одна пара». Алгоритм сходит к одному выходу при извлечении цели (билет 17), но не к началу.

```

```
</div>
```

```
<p class="text-slate-300 text-sm mt-4 mb-4">
 о скоростью O(V^3); б
 после перевзвешивания потенциалов). Представим граф с
```

```

<div class="grid grid-cols-1 sm:grid-cols-3">
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
 </div>
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
 </div>
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
 </div>
</div>

```

```

<div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Патруль Бэтмена на м
 </div>
 <div class="my-3 rounded-lg overflow">
 все ули
овно 1 выходной в году, когда он отде
-ю ночь Бэтмен делает контрольный проезд: е
о-то района снова уменьшилось – значит, Джо
 </div>

```

```

 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Отрицательный цикл: временная п
 </div>

```



```

 </table>
 <p class="text-slate-400 text-xs mt-3">
бном» порядке: S→A,
порядок рёбер местами – и понадобится до ше
</div>

<div class="bg-slate-800/70 p-5 rounded-lg"
 <p class="text-sm font-bold text-white"
 <p class="text-slate-300 text-sm mb-3">
тава A имеет суммарный вес <span cla
веет» на 2.</p>
 <table class="w-full text-xs text-left"
 <thead>
 <tr class="text-slate-500 borde
 <th class="py-1.5 pr-2 font
 <th class="py-1.5 pr-2 font
 <th class="py-1.5 pr-2 font
 <th class="py-1.5 pr-2 font
 <th class="py-1.5 font-bold
 </tr>
 </thead>
 <tbody class="text-slate-300">
 <tr class="border-b border-slate
-2 font-mono">0</td><td class="py-2 pr-2 fo
td></tr>
 <tr class="border-b border-slate
-2 font-mono">-2</td><td class="py-2 pr-2 f
/td></tr>
 <tr class="border-b border-slate
-2 font-mono">-6</td><td class="py-2 pr-2 f
</td></tr>
 <tr class="border-b border-slate
"py-2 pr-2 font-mono">-10</td><td class="py
nt-mono">-2</td></tr>
 <tr class="align-top"><td class
12</td><td class="py-2 pr-2 font-mono">-9</
ont-bold">всё ещё уменьшается → отрицательны
 </tbody>

```

```

<div class="overflow-x-auto">
 <table class="w-full text-xs text-l
 <thead>
 <tr class="text-slate-500 b
 <th class="py-1.5 pr-2
 <th class="py-1.5 pr-2
 <th class="py-1.5 pr-2
 <th class="py-1.5 font-l
 </tr>
 </thead>
 <tbody class="text-slate-300">
 <tr class="border-b border-
2">одна вершина → все</td><td class="py-2 p
 <tr class="border-b border-
font-mono">0((V+E) log V)</td><td class="p
 <tr class="border-b border-
ass="py-2 pr-2 text-rose-300">нет</td><td c
 </tr>
 <tr class="border-b border-
ass="py-2 pr-2 text-rose-300">молча ошибётся
emerald-300">обнаруживает (d[i][i] < 0)<
 <tr class="border-b border-
ass="py-2 pr-2">список смежности</td><td cl
 <tr class="align-top"><td c
φ, веса ≥ 0</td><td class="py-2 pr-2">минус
 </tbody>
 </table>
</div>
</div>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-emerald-4

 <div class="bg-slate-900/60 p-4 rounded-lg l
 <p class="text-xs font-bold text-amber-:
 :</p>
 <pre class="bg-slate-950 p-3 rounded text-xs font-mono

```

```

if changed_vertex == -1:
 return dist, prev, None # отрицательны

changed_vertex улучшен на n-й итерации:
поднимаемся по prev на n шагов – гарантированно з
y = changed_vertex
for _ in range(n):
 y = prev[y]

cycle = [y]
v = prev[y]
while v != y:
 cycle.append(v)
 v = prev[v]
cycle.append(y)
cycle.reverse()
return dist, prev, cycle

демо-граф страницы (9 рёбер, режим с отрицательным ци
nodes = ["S", "A", "B", "C", "D", "E", "F"]
idx = {name: i for i, name in enumerate(nodes)}
E = [{"S", "A", 4}, {"S", "B", 5}, {"B", "C", -2}, {"A", "C", 1},
 {"C", "D", 3}, {"C", "E", 4}, {"D", "F", 2}, {"E", "F", 1}]
E = [(idx[u], idx[v], w) for u, v, w in E]

dist, prev, cyc = bellman_ford(len(nodes), E + [(idx["D",
print([dist[i] for i in range(len(nodes))]) # [0, 4,
print(cyc) # None –

dist2, _, cyc2 = bellman_ford(len(nodes), E + [(idx["D",
print([nodes[i] for i in cyc2]) # ['D',
print([dist2[i] for i in range(len(nodes))]) # все
 <p class="text-slate-400 text-xs">П
 имая вершина с INF<
 ля float('inf')</sp
 </div>
</details>

```

```

 vector<int> cycle;
 for (int v = y;; v = p[v]) {
 cycle.push_back(v);
 if (v == y && cycle.size() > 1) break;
 }
 reverse(cycle.begin(), cycle.end());
 return cycle;
 }

```

```

int main() {
 int s = 0;
 int x = bellman_ford(s);
 if (x == -1) cout <<< "отрицательных циклов не

 else {
 cout <<< "цикл: ";
 for (int v : negative_cycle(x)) cout <<< v

 cout <<< "\n";
 }
}
</code></pre>

```

</div>

</details>

<details class="bg-slate-900/40 rounded-lg

<summary class="p-4 font-bold text-slate

-b border-slate-700/50">» Оптимизации: ранни

<div class="p-5 text-sm text-slate-300

```

<pre class="bg-slate-950 p-4 rounded overflow-x-auto te
 rt deque

```

```

def spfa(n, adj, s):
 """adj[u] = [(v, w), ...]. Релаксируем только «живые»
 INF = float("inf")
 dist = [INF] * n
 in_queue = [False] * n
 counter = [0] * n # сколько раз вершина
 dist[s] = 0
 q = deque([s])
 in_queue[s] = True

```

```

 отрицательным циклом
 Переполнение INF.
 INT_MAX превращается в отрицательное значение, что приводит к переполнению
 запасом и long long
 Неориентированный граф с минимальным циклом -3 – это цикл длины 3, который встречается в отрицательном цикле. Минусы в неориентированном графе.
 Цикл есть, но не от старта. -3 . Нужна проверка всего графа – добавляем ребро (1, 2) и запускаем от неё.
 «Ответ $-\infty$ ». Если отрицательный цикл, то dist[v] = -\infty: печатать ответ, если не отрицательный цикл, то ходом в обратном графе.
 SPFA как «быстрый Форд–Беллман». В условии V, E \le 10^5

 </div>

<p class="text-slate-300 text-sm mt-4">Сложные задачи: поиск кратчайшего пути. Потенциалы (15) решаются Дейкстрой. Флойд–Уоршелл (16) решает все задачи. Поиск кратчайшего пути в DAG – билет 9.</p>
</div>
</div>
</section>`
},
{
 id: "floyd",
 title: "16. Графы. Поиск кратчайшего пути. Флойд",
 type: "html",
 category: "Графы. Пути",
 content: `
<section id="floyd-content" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 16. Графы. Поиск кратчайшего пути. Флойд
 <h2 class="text-2xl sm:text-3xl font-bold text-slate-300">16. Графы. Поиск кратчайшего пути. Флойд</h2>
 </div>
`

```

```

<div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
</div>
<div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
</div>
<div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
</div>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2
 <div class="bg-slate-800 p-6 rounded-lg
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Аналогия 1: Таблица пересадок в
 </div>
 <p class="text-slate-300 text-sm mb
це только соседние станции. Затем говорите:
 через 1 там, где
плакат окончательный. Порядок «разрешений»
an>.</p>
 </div>

 <div class="bg-slate-900 p-6 rounded-lg
 <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
 Аналогия 2: Почему k обязан быть
 </div>
 <p class="text-slate-300 text-sm mb
s="font-mono">k один раз и бо
садку не соберётся. На демо-графе этой стра
ono text-emerald-300">d[2][1] = ∞, х
пользует «поздние» посредники). Проверка пер
 </div>

```

```

 </tbody>
 </table>
 <p class="text-slate-400 text-xs mt-3">
mono text-emerald-300">0 → {0, 4, 7, 2, 6, 1}
"font-mono">0 → 3 → 4 → 6 (2 + 4 + 1
всего в финальной матрице 11 таких пар. Диа
</div>

<div class="bg-slate-800/70 p-5 rounded-lg border">
 <p class="text-sm font-bold text-white">
 <table class="w-full text-xs text-left">
 <thead>
 <tr class="text-slate-500 border">
 <th class="py-1.5 pr-2 font">
 <th class="py-1.5 pr-2 font">
 <th class="py-1.5 font-bold">
 </tr>
 </thead>
 <tbody class="text-slate-300">
 <tr class="border-b border-slate-
y-2 pr-2 font-mono">конечное</td><td class="
 <tr class="border-b border-slate-
y-2 pr-2 font-mono">INF</td><td class="py-2
 <tr class="border-b border-slate-
y-2 pr-2 font-mono">0</td><td class="py-2">
 <tr class="align-top"><td class="
se-300">< 0</td><td class="py-2 text-rose-400">
делены</td></tr>
 </tbody>
 </table>
 </div>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-indigo-400">

 <details class="bg-slate-900/40 rounded-lg border">
 <summary class="p-4 font-bold text-slate-300">

```

```

d = [row[:] for row in M]
for a in range(n):
 for b in range(n):
 for c in range(n):
 k, i, j = {"kij": (a, b, c),
 "ijk": (c, a, b),
 "ikj": (c, a, b)}[order]
 if d[i][k] + d[k][j] < d[i][j]:
 d[i][j] = d[i][k] + d[k][j]
return d

good = floyd(M, "kij")
bad = floyd(M, "ijk")
print(good[2][1], bad[2][1]) # 4 и 999 – путь 2→4→6→1
print(sum(1 for i in range(7) for j in range(7) if good
4 клетки отличаются; при порядке (i,k,j) их 6</code><
 <p class="text-slate-400 text-xs">В
угих «почти работают» (большинство клеток с
 </div>
</details>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl border
 <h3 class="text-xl font-bold text-emerald-4

 <details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
 -b border-slate-700/50">> Код: Флойд–Уоршелл.
 <div class="p-5 text-sm text-slate-300
<pre class="bg-slate-950 p-4 rounded overflow-x-auto te

def floyd_warshall(n, edges, directed=True):
 d = [[INF] * n for _ in range(n)]
 nxt = [[-1] * n for _ in range(n)]
 for i in range(n):
 d[i][i] = 0 # диагональ об

 for u, v, w in edges:

```



```

print(neg) # [] — от

</div>
</details>

<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
 -b border-slate-700/50">> Код: C++ (матрица
 <div class="p-5 text-sm text-slate-300
<pre class="bg-slate-950 p-4 rounded overflow-x-auto te
 t;
#include <algorithm>;
using namespace std;
const long long INF = 4e18;

int n;
vector<vector<long long>>> d(n, vector<long long>(n, INF));

void floyd() {
 for (int i = 0; i < n; ++i) d[i][i] = 0;
 for (int k = 0; k < n; ++k) // k
 for (int i = 0; i < n; ++i) {
 if (d[i][k] == INF) continue; // защи
 for (int j = 0; j < n; ++j) {
 if (d[k][j] == INF) continue;
 d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
 }
 }
}

bool has_negative_cycle() {
 for (int i = 0; i < n; ++i) if (d[i][i] < 0)
 return true;
 return false;
}

long long diameter() { // сама
 long long res = 0;
 for (int i = 0; i < n; ++i)
 for (int j = 0; j < n; ++j)

```

```

 <tr class="border-b border-
ой сети</td><td class="py-2 pr-2 font-mono">
и отсутствии положительных циклов</td></tr>
 <tr class="align-top"><td c
o">if d[i][k]+d[k][j] == d[i][j]: cnt[i][j]
а переполнением</td></tr>
 </tbody>
</table>
 <p class="text-slate-400 text-xs mt
> при r[i][k]
V³).</p>
 </div>
</details>
</div>

```

```

<div class="bg-slate-700/50 p-6 rounded-xl bord
<h3 class="text-xl font-bold text-amber-400

```

```

<div class="bg-rose-950/30 p-5 rounded-lg b
 <p class="text-sm font-bold text-rose-3
 <ul class="list-disc pl-5 space-y-2 tex
 Порядок циклов. <span cl
ono">(i, j, k) на демо-графе 4 клетки
] = 999 вместо 4).
 Диагональ. <span class="
«починит» её только при наличии цикла, а п
 Переполнение INF. <span c
уходит в минус и «улучшает» всё подряд. Пр
no">long long.
 Параллельные рёбра. Во в
no">d[u][v] стартует с худшего веса.
 Отрицательные циклы. Форм
ых с циклом, ответ
t-mono">d[i][j] = -∞, если существуе
 d[i][k] < ∞</sp
 Недостижимость ≠ 0. В ор
таких пар из 49). Выводить 999 или 0 как «р
 Память. <span class="fon

```

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">Дейк
в лоб. На экзамене спрашивают, что сдела
строчки перевзвешивания» до промышленных у
```

```
<div class="grid grid-cols-1 xl:grid-cols-2
 <div class="bg-slate-800 p-6 rounded-lg
 <div class="absolute -top-3 left-4
 <div class="border border-emerald-500 shadow-md">
```

Аналогия 1: Такси с навигатором

```
</div>
```

```
<p class="text-slate-300 text-sm mb
ы одинаково. А* — тот же таксист, но с нави
/span> прибавляется оценка остатка пути по
т туда, где сумма меньше. Навигатор обязан
/span>): занижать можно, завышать нельзя —
0">h = 0 это снова Дейкстра.</p>
```

```
</div>
```

```
<div class="bg-slate-800 p-6 rounded-lg
 <div class="absolute -top-3 left-4
 <div class="border border-emerald-500 shadow-md">
```

Аналогия 2: Две бригады туннеля

```
</div>
```

```
<p class="text-slate-300 text-sm mb
пускают две: от А и от В. Каждая зона роста
я не в момент встречи, а когда сумма
— иначе можно пропустить более дешёвый обх
ирует.</p>
```

```
</div>
```

```
</div>
```

```
<div class="overflow-x-auto my-6">
```

```
<table class="w-full text-xs text-left
 <thead>
```

```
<tr class="text-slate-500 borde
```

```
<th class="py-2 pr-3 font-b
```

```
<th class="py-2 pr-3 font-b
```

```

 <p class="text-xl font-mono text-white">
</div>
<div class="grid grid-cols-1 gap-6 mb-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 b
er border-amber-500 shadow-md">
 ⚡ Суть перевзвешивания
 </div>
 <p class="text-slate-300 text-sm mb-3">
 Дейкстра (билет 14) быстрая, но
ленный. Алгоритм Джонсона делает финт: пере
ь теми же.
 </p>
 <ol class="text-sm text-slate-300 sp>
 Добавляем фиктивную вершн
 Запускаем от s Форда-Бел
ажный факт:</i> Джонсон НЕ «удаляет» отрица
кой и прекращает работу, так как при отрица
 Перевзвешиваем: <span class=
тво следует из того, что $h(v) \leq h(u) + w(u,$
 Магия: для любого пу
) – $h(\text{финиш})$, то есть все пути между фик
еняется.
 Запускаем быструю Дейкстр
lass="font-mono text-emerald-300"> $d(u, v) =$

 <p class="text-slate-400 text-xs mt-3">
 еженном графе ($E \approx V$) это $O(V^2 \log V)$ проти
ойд – для маленьких или очень плотных.</p>
 </div>
</div>

```

```

<h3 class="text-lg font-bold text-white mt-3">
<p class="text-slate-300 text-sm mb-3">Если
азад из t (по перевёрнутым рёбрам). Волны р
: в «широком» графе (дорожная сеть) это эконо
огда волны «встретились» в одной вершине; н
огда

```

```

 <p class="text-slate-400 text-sm mb-1">
именимой к отрицательным весам и даёт все п
 <p class="text-slate-400 text-sm mb-1">
in_f + min_b ≥ μ. Остановка по встрече даёт
 <p class="text-slate-400 text-sm mb-1">
ок сломается. Там нужен Dial.</p>
 <p class="text-slate-400 text-sm">•
кτικе меньше бесполезных релаксаций.</p>
</div>

<p class="text-slate-300 text-sm mt-4">с
py-0.5 rounded text-xs font-mono text-emera
class="bg-slate-950 px-2 py-0.5 rounded tex
м числом раскрытий, 0-1 BFS <code class="bg
te-800">0(V + E)</code>, Dial <code class="
late-800">0(E + V·C)</code>, Дейкстра с куч
merald-300 border border-slate-800">0(E + V
o text-rose-300 border border-slate-800">0('

<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
 Код: Джонсон целиком (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounded
 heapq
INF = float("inf")

def johnson(n, edges):
 # edges: (u, v, w)
 # 1) потенциалы: Форд–Беллман от фиктивной вершины
 h = [0] * (n + 1)
 for s in range(n):
 # из s все рёбра
 # V итераций: V-
 changed = False
 for u, v, w in edges:
 if h[u] + w < h[v]:
 h[v] = h[u] + w
 changed = True

```

```

def bidirectional(g, rg, s, t): # rg – граф с пер
 INF = float("inf")
 df = {s: 0}; db = {t: 0}
 qf = [(0, s)]; qb = [(0, t)]
 mu = INF
 while qf and qb:
 if qf[0][0] + qb[0][0] >= mu: # вот пра
 return mu
 dist, v = heapq.heappop(qf)
 if dist > df.get(v, INF): continue
 for to, w in g[v]:
 nd = dist + w
 if nd < df.get(to, INF):
 df[to] = nd; heapq.heappush(qf, (nd, to)
 if to in db:
 mu = min(mu, nd + db[to]) # встреча:
 dist, v = heapq.heappop(qb)
 if dist > db.get(v, INF): continue
 for to, w in rg[v]:
 nd = dist + w
 if nd < db.get(to, INF):
 db[to] = nd; heapq.heappush(qb, (nd, to)
 if to in df:
 mu = min(mu, nd + df[to])
 return mu

```

# --- A\*: приоритет f = g + h ---

```

def astar(g, s, t, h): # h(v) – допустим
 INF = float("inf")
 gdist = {s: 0}
 heap = [(h(s), 0, s)] # (f, g, v)
 closed = set()
 while heap:
 f, gv, v = heapq.heappop(heap)
 if v == t:
 return gv # ранний выход:
 if v in closed:
 continue

```

```
 <p>• Зачем нужен ранний выход в Дейкстры
 смысла нет.</p>
 <p>• Почему 0-1 BFS работает без кучи, если
 же отсортирована.</p>
 </div>
</details>
</div>
</div>
</section>
 },
];
```

---

ФАЙЛ 28/111: src/data/tickets/ticket18\_20\_mst.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 224 | РАЗМЕР: 1.1 КБ

---

```
import { Chapter } from '../../types';

/**
 * Билеты 18 · 19 · 20 – одна страница на три алгоритма
 * минимального остовного дерева (MST). Так же, как в симуляторе
 * KruskalSimulator, три алгоритма живут на одном графе
 * переключаются вкладками «Краскал / Прима / Борувка».
 *
 * Синхронизация с компилятором: PAGE_SYNC["mst"] + подстраницы
 * "mst#kruskal", "mst#prim", "mst#boruvka" (вкладка со
 * через emitVizDemo, код в панели следует за ней).
 */
export const tickets18to20Mst: Chapter[] = [
 {
 id: "mst",
 title: "18–20. Графы. Остовное дерево: Краскал, Прима, Борувка",
 type: "html",
 description:
 "Три способа построить минимальное остовное дерево",
 },
```

```

 <tr class="border-b border-slate-200">
 <td class="py-2 pr-3 font-bolder">Рёбра</td>
 <td class="py-2 pr-3">Рёбра</td>
 <td class="py-2 pr-3 font-mono">Рёбра</td>
 <td class="py-2 pr-3 font-mono">Рёбра</td>
 <td class="py-2">рёбра даны</td>
 </tr>
 <tr class="border-b border-slate-200">
 <td class="py-2 pr-3 font-bolder">Растёт</td>
 <td class="py-2 pr-3">Растёт</td>
 <td class="py-2 pr-3 font-mono">Растёт</td>
 <td class="py-2 pr-3 font-mono">Растёт</td>
 <td class="py-2">плотный гр</td>
 </tr>
 <tr class="align-top">
 <td class="py-2 pr-3 font-bolder">Растёт</td>
 <td class="py-2 pr-3">Все к</td>
 <td class="py-2 pr-3 font-mono">Растёт</td>
 <td class="py-2 pr-3 font-mono">Растёт</td>
 <td class="py-2">параллельны</td>
 </tr>
 </tbody>
 </table>
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-3">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-indigo-500 shadow-md">
 Аналогия к билету 18 • Краск
 </div>
 <p class="text-slate-300 text-sm">С
 >. По всей карте растут независимые кусочки
 дерево. Порядок рёбер важен, порядок верши
 </div>

 <div class="bg-slate-800 p-6 rounded-lg"

```





```

<div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounded
 = list(range(n))
def find(v):
 while parent[v] != v:
 parent[v] = parent[parent[v]]
 v = parent[v]
 return v

comps = n
total = 0
while comps > 1:
 best = {} # компонента
 for w, u, v in edges:
 cu, cv = find(u), find(v)
 if cu == cv:
 continue # уже внутри
 if cu not in best or w < best[cu][0]:
 best[cu] = (w, u, v)
 if cv not in best or w < best[cv][0]:
 best[cv] = (w, u, v)
 if not best:
 break # граф несвя
 for w, u, v in best.values():
 if find(u) == find(v):
 continue # дубль: два
 parent[find(u)] = find(v)
 comps -= 1
 total += w
print(total)</pre>

```

<p class="text-slate-400 text-xs">3  
ну), значит фаз не больше  $\log_2 V$ .</p>

</div>  
</details>

<details class="bg-slate-900/40 rounded-lg l  
<summary class="p-4 font-bold text-slate  
-b border-slate-700/50">

```
<div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
```

```
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 сом, оканчивающимся в позиции i.</p>
 </div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
```

♂ Аналогия 1: Поиск без отка

```
</div>
<p class="text-slate-300 text-sm mb-4">
 Мы идём по строке слева направо. И
 строки СЕЙЧАС закончился под моим пальцем?»
 Представь, ты ищешь в тексте слова
 x</code>. Тупой алгоритм начнет искать заголовок
 cab</code> суффикс <code>ab</code> – это ро
 буквы шаблона». Это поиск за $O(N)$ без возв
```

```
</p>
```

```
</div>
```

```
<!-- Аналогия 2 -->
<div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
```

Аналогия 2: LLM стриминг

```
</div>
<p class="text-slate-300 text-sm mb-4">
 Для LLM, которая стримит токены
```

```
for (int i = 1; i < s.length(); i++) {
 int j = pi[i-1];
 while (j > 0 && s[i] != s[j]) j = pi[j-1];
 if (s[i] == s[j]) j++;
 pi[i] = j;
}
```

</div>  
</details>

<h3 class="text-lg font-bold text-white mt-4">Поиск подстроки КМР (Скрыто)  
<div class="bg-slate-900 p-4 rounded-lg mb-4">  
 <p class="text-xl font-mono text-white">Пример кода  
</p>  
</div>

<p class="text-slate-300 text-sm mb-4">Префикс — это часть строки, которая встречается в строке один раз, затем идёт палиндром. Если префикс совпал с палиндромом — тогда  $j += 1$ . Если не совпало — откатываемся на  $j - 1$  и заново сравниваем его не нужно. Когда  $j$  достигнет 0, делаем  $j = \pi[j - 1]$ .</p>

<p class="text-slate-300 text-sm mb-4"><b>Палиндром — это строка, которая читается одинаково как слева направо, так и справа налево. Например, «abba» — это палиндром. Каждый откат  $j$  уменьшает. Значит суммарно откатов будет не больше, чем длина строки. Это классическая амортизационная оценка, её любят.</b></p>

<div class="bg-slate-900/40 rounded-lg border-1 border-slate-700">  
 <p class="text-slate-300 text-sm mb-2">Пример кода  
 <p class="text-slate-400 text-sm mb-1">Пример кода  
 <p class="text-slate-400 text-sm mb-1">Пример кода  
 <p class="text-slate-400 text-sm mb-1">Пример кода  
</div>

<p class="text-slate-400 text-sm mb-1">Пример кода  
 <p class="text-slate-400 text-sm">Пример кода  
 <p class="text-slate-400 text-sm">Пример кода  
 <p class="text-slate-400 text-sm">Пример кода  
</div>

<div class="bg-slate-900/40 rounded-lg border-1 border-slate-700">  
 <summary class="p-4 font-bold text-slate-300">Пример кода  
 </summary>  
</div>

Код: поиск подстроки КМР (Скрыто)

</summary>

```

 </details>
 </div>
 </div>
 </section>`
},
{
 id: "string-z-func",
 title: "22. Z-функция — Сдвиг и наложение",
 type: "html",
 category: "Строки",
 content: `
<section id="string-z-func" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 ca 0) и её суффиксом, начинающимся с i.</p>
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
 rder border-emerald-500 shadow-md">
 Аналогия 1: Линейка-шаблон
 </div>
 <p class="text-slate-300 text-sm mb">
 Тут мы берём всю строку и «прик
 я начну читать строку отсюда, насколько длин
 Это самый быстрый способ найти
 >, считаешь Z-функцию, и там, где <code>Z[i
 </p>

```

Код C++ (Скрыто)

</summary>

<div class="p-5 text-sm text-slate-300"

<pre class="bg-slate-950 p-4 rounded"

```
int l = 0, r = 0;
```

```
for (int i = 1; i < n; i++) {
```

```
 if (i <= r) z[i] = min(r - i + 1, z[i - 1]);
```

```
 while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
```

```
 if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
```

```
}</pre>
```

</div>

</details>

</div>

</div>

</section>

},

{

id: "aho-corasick",

title: "23. Алгоритм Ахо-Корасик",

type: "html",

category: "Строки",

content: `

<section id="aho-corasick" class="mb-12 scroll-mt-10">

<div class="flex items-center mb-6 flex-wrap gap-3">

<span class="bg-indigo-600 text-white px-4 py-1 rounded">

<h2 class="text-2xl sm:text-3xl font-bold text-white">

</div>

<div class="space-y-8">

<div class="bg-slate-700/50 p-6 rounded-xl border-l">

<h3 class="text-xl font-bold text-blue-400 mb-4">

<div class="bg-slate-900 p-4 rounded-lg mb-6 text">

<p class="text-sm text-blue-300 italic mb-2">0с

<p class="text-lg font-mono text-white">Бор (Tr

</div>

<p class="text-slate-300 text-sm">Позволяет найти

го один проход.</p>



```

 <tr class="text-slate-500 border-b" style="border-bottom: 1px solid black;">
 <th class="py-2 pr-3 font-b" style="text-align: left; padding: 2px 30px; font-weight: bold;">
 <th class="py-2 pr-3 font-b" style="text-align: left; padding: 2px 30px; font-weight: bold;">
 <th class="py-2 font-b" style="text-align: left; padding: 2px 30px; font-weight: bold;">
 </tr>
 </thead>
 <tbody class="text-slate-300">
 <tr class="border-b border-slate-500" style="border-bottom: 1px solid black; background-color: #f0f0f0;">
 <td class="py-2 pr-3" style="padding: 2px 30px;">детерминированно за $O(n^k)$ </td><td class="py-2 pr-3" style="padding: 2px 30px;">удольности, максимальный поток</td><td class="py-2 pr-3" style="padding: 2px 30px;"></td></tr>
 <tr class="border-b border-slate-500" style="border-bottom: 1px solid black; background-color: #f0f0f0;">
 <td class="py-2 pr-3" style="padding: 2px 30px;">сертификат проверяется за полиномиальную (не решающая версия), коммивояжёр «есть ли тур ≤ k»</td><td class="py-2 pr-3" style="padding: 2px 30px;"></td><td class="py-2 pr-3" style="padding: 2px 30px;"></td></tr>
 <tr class="border-b border-slate-500" style="border-bottom: 1px solid black; background-color: #f0f0f0;">
 <td class="py-2 pr-3" style="padding: 2px 30px;">сертификат есть у ответа «НЕТ» (не решающая версия)</td><td class="py-2 pr-3" style="padding: 2px 30px;"></td><td class="py-2 pr-3" style="padding: 2px 30px;"></td></tr>
 <tr class="border-b border-slate-500" style="border-bottom: 1px solid black; background-color: #f0f0f0;">
 <td class="py-2 pr-3" style="padding: 2px 30px;">не проще любой задачи из NP-полной (не решающая версия)</td><td class="py-2 pr-3" style="padding: 2px 30px;">фа, TSP-оптимизация (не решающая версия)</td><td class="py-2 pr-3" style="padding: 2px 30px;"></td></tr>
 <tr class="border-b border-slate-500" style="border-bottom: 1px solid black; background-color: #f0f0f0;">
 <td class="py-2 pr-3" style="padding: 2px 30px;">NP-трудные И из NP</td><td class="py-2 pr-3" style="padding: 2px 30px;"></td><td class="py-2 pr-3" style="padding: 2px 30px;">сумм</td></tr>
 <tr class="border-b border-slate-500" style="border-bottom: 1px solid black; background-color: #f0f0f0;">
 <td class="py-2 pr-3" style="padding: 2px 30px;">полиномиальная память, время не зависит от размера</td><td class="py-2 pr-3" style="padding: 2px 30px;">й на полиномиальном поле</td><td class="py-2 pr-3" style="padding: 2px 30px;"></td></tr>
 <tr class="border-b border-slate-500" style="border-bottom: 1px solid black; background-color: #f0f0f0;">
 <td class="py-2 pr-3" style="padding: 2px 30px;">за $2^{(n^k)}$ </td><td class="py-2 pr-3" style="padding: 2px 30px;">обобщённые</td><td class="py-2 pr-3" style="padding: 2px 30px;"></td></tr>
 </tbody>
</table>
</div>

```

```

<h3 class="text-lg font-bold text-white mt-3">
<p class="text-slate-300 text-sm mb-3">Стандартная задача — сертификат полиномиального размера и проверка сертификата — «все вершины по разу и каждая соседняя пара вершин образует треугольник».
 <p class="text-slate-300 text-sm mb-3">Теорема 1.1. Задача SAT NP-полна. Доказывается «в лоб»: вычисление

```



```

 ass="bg-slate-950 px-2 py-0.5 rounded text-slate-900">
 еребор сертификатов <code class="bg-slate-950 px-2 py-0.5 rounded text-slate-900">
 ^poly(n))</code>; сведение обязано работать
 0 border border-slate-800">0(poly(n))</code>

 <details class="bg-slate-900/40 rounded-lg border border-slate-800">
 <summary class="p-4 font-bold text-slate-900">
 -b border-slate-700/50">
 Код: верификатор сертификата и пр
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <pre class="bg-slate-950 p-4 rounded">
 ьтонов цикл NP-полон: сертификат = порядок
def verify_hamilton(n, edges, cert):
 if sorted(cert) != list(range(n)): # каждая
 return False
 allowed = {(u, v) for u, v in edges} | {(v, u) for
 for i in range(n):
 a, b = cert[i], cert[(i + 1) % n] # замык
 if (a, b) not in allowed:
 return False
 return True

3-SAT ≤p КЛИКА: формула с k дизъюнктов → граф из 3k верш
def reduce_3sat_to_clique(clauses):
 # вершина = (номер дизъюнкта, литерал); ребро, если
 verts = [(ci, lit) for ci, cl in enumerate(clauses)
 edges = []
 for i in range(len(verts)):
 for j in range(i + 1, len(verts)):
 ci, li = verts[i]; cj, lj = verts[j]
 if ci == cj: # из од
 continue
 if li.lstrip("-") == lj.lstrip("-") and (li
 continue # x и n
 edges.append((i, j))
 return verts, edges, len(clauses) # клика

```

```
 <p>• Чем NP-трудная задача отличается от задачи (проблема остановки)</p>
 </div>
 </details>
 </div>
</div>
</section>`
 }
};
```

---

ФАЙЛ 30/111: src/data/tickets/ticket6\_13.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 576 | РАЗМЕР: 1.5 КБ

---

```
import { Chapter } from "../../types";

export const tickets6to13: Chapter[] = [
 {
 id: "graph-dfs-bfs",
 title: "6. Графы. Представление графов, DFS, BFS",
 type: "html",
 description: "Представление графов и базовые обходы",
 category: "Графы. База",
 content: `
<section id="graph-dfs-bfs" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 6. Графы
 <h2 class="text-2xl sm:text-3xl font-bold text-slate-300">Представление графов, DFS, BFS
 </div>
 <div class="space-y-8">
 {/* База представлений */}
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-slate-300">Базовые обходы
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-slate-300 italic">Глубина (DFS) и ширина (BFS) обходы графов.
 <p class="text-xl font-mono text-white">Глубина (DFS) и ширина (BFS) обходы графов.
 </div>
 </div>
</section>`
 }
];
```

```

 Где используется:
 Топологическая сортировка (l
 Поиск мостов и точек сочленен
 Компоненты сильной связности
 Детектирование циклов (через

</div>

<!-- Аналогия BFS -->
<div class="bg-slate-900 p-6 rounded-lg" style="border: 1px solid #000; border-radius: 10px; margin: 10px 0;">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
 BFS (Поиск в ширину) = Волна
 </div>
 <p class="text-slate-300 text-sm mb-0">
 Что делает: Распространение по соседям. Работает через Очередь (FIFO). Гарантирует минимальное расстояние от источника.</p>
 <ul class="text-xs text-emerald-300">
 Где используется:
 Кратчайший путь в Невзвешенном графе
 Проверка двудольности графа
 Алгоритм Ахо-Корасик (построение автомата)
 Двунаправленный BFS для поиска пути (вектора).

 </div>
</div>

<details class="bg-slate-900/40 rounded-lg" style="border: 1px solid #000; border-radius: 10px; margin: 10px 0;">
 <summary class="p-4 font-bold text-slate-300" style="border: 1px solid #000; border-radius: 10px; margin: 0 0 10px 0;">
 Код обходов C++ (Основа)
 </summary>
 <div class="grid grid-cols-1 md:grid-cols-2" style="border: 1px solid #000; border-radius: 10px; margin: 0 0 10px 0;">
 <pre class="bg-slate-950 p-3 rounded" style="border: 1px solid #000; border-radius: 10px; margin: 0 0 10px 0;">
// DFS (Рекурсия с возвратом)
vector<bool> vis;

```

```


 <h2 class="text-2xl sm:text-3xl font-bold text-white">
</div>
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-2xl font-mono text-white">
 <p class="text-xs text-slate-400 mt-1">
 </div>

 <div class="space-y-4 text-slate-300 text-sm">
 <p>Главные неравенства на число рёбер</p>
 <ul class="list-disc list-inside space-y-2">
 Для простого планарного графа
 <i>Доказательство:</i> Каждая грань

 Для простого двудольного планарного графа

 Доказательство: В двудольном графе
 $2E \geq 4F \Rightarrow F \leq E/2$

 <div class="bg-slate-950 p-4 rounded-lg">
 <p class="text-amber-300 font-bold">Примеры</p>
 <p class="text-slate-300 text-xs leading-5">
 1. K5 (полный граф на 5 вершинах)
 $10 \leq 3(5) - 6 = 9$ – не выполняется
 2. K3,3 (полный двудольный 3×3)
 $9 \leq 12$ – формально выполняется
 $2V - 4$: $9 \leq 10$
 </p>
 </div>
 </div>
 </div>
 <p>Теорема Куратовского (1930) / Вагнера (1936)</p>

```

```

 type: "html",
 description:
 "Компоненты связности: обходами DFS/BFS за $O(V + E)$ ",
 category: "Графы. База",
 content: `
<section id="graph-components" class="mb-12 scroll-mt-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 рой вершин есть путь. Внешний цикл «для каж
 » даёт и разметку, и количество компонент.<
 </div>

 <p class="text-slate-300 text-sm mb-4">Два р
 ной запускаем DFS или BFS и присваиваем все
 mp[v] = c. Счётчик c увеличивается р
 Время $O(V + E)$, память $O(V)$. Чере
 каждое ребро делаем union(u, v), в конце чи
 ту и после каждого добавления спрашиваю

 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 b
 rder border-emerald-500 shadow-md">
 Аналогия 1: Острова
 </div>
 <p class="text-slate-300 text-sm">Т
 ались серые (неисследованные) зоны? Прыгаеш
 и компонент. Краска – это comp[v], прыжок ч
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">

```

```

 </div>
</section>`,
 },
 {
 id: "top-sort",
 title: "9. Графы. Топологическая сортировка",
 type: "html",
 description: "Топологическая сортировка ДАГ и детект",
 category: "Графы",
 content: `
<section id="top-sort" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 перед.</p>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
 rder border-emerald-500 shadow-md">
 Аналогия 1: Учеба в вузе
 </div>
 <p class="text-slate-300 text-sm mb-4">
 еская сортировка – это расписание, которое
 <p>
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
 border-rose-500 shadow-md">
 Ограничения: Циклы
 </div>

```

```

 // Переход в серую вершину = НАЙДЕН ЦИКЛ!
 has_cycle = true;
 return;
 }
}
color[v] = 2; // Вышли – вершина чёрная
ans.push_back(v); // Сохраняем в момент ВЫХОДА (pos
}

```

```

bool topological_sort(int n) {
 fill(color, color + n + 1, 0);
 has_cycle = false;
 ans.clear();
 for (int i = 1; i <= n; ++i) {
 if (color[i] == 0) dfs(i);
 }
 if (has_cycle) return false; // Граф содержит цикл!
 reverse(ans.begin(), ans.end()); // Разворачиваем р
 return true;
}

```

```

 </div>
 </details>
</div>
</div>
</section>`,
},
{
 id: "scc-kosaraju",
 title: "10. Графы. Компоненты сильной связности (SCC)",
 type: "html",
 description: "Разбиение орграфа на макро-вершины. Алгоритм Косарая",
 category: "Графы. Продвинутые",
 content: `
<section id="scc-kosaraju" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 10. Графы. Компоненты сильной связности (SCC)
 <h2 class="text-2xl sm:text-3xl font-bold text-white">Графы. Продвинутые</h2>
 </div>

```

```

</section>`,
 },
 {
 id: "graph-bridges",
 title: "11. Графы. Мосты",
 type: "html",
 description: "Рёбра, удаление которых увеличивает ч",
 category: "Графы. Продвинутые",
 content: `
<section id="graph-bridges" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-rose-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-rose-300 italic">
 <p class="text-xl font-mono text-white">
 <p class="text-xs text-slate-400 mt-1">
 </div>

 <div class="bg-slate-950 p-4 rounded-lg border">
 <p class="text-amber-300 font-bold text">
 <p class="text-slate-300 text-xs leading">
 Проверка <span class="font-mono text-
тиграфе между u и v может быть 2 параллельны
азуют цикл длиной 2.

 Решение: передавать в DFS не v
rald-300">p_edge_id (индекс ребра, по
 </p>
 </div>

 <details class="bg-slate-900/40 rounded-lg border">
 <summary class="p-4 font-bold text-slate-
-b border-slate-700/50">
 Код поиска мостов C++ (Скрыто)
 `
 }
}

```



```

<h2 class="text-2xl sm:text-3xl font-bold text-rose-400">
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border-1 border-slate-800">
 <h3 class="text-xl font-bold text-rose-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-rose-300 italic">
 <div class="text-left font-mono text-sm">
 <p><strong class="text-white">Точка
 рои увеличивает число компонент связности.</p>
 <div class="p-3 bg-slate-950 rounded">

 <p class="text-xl font-bold tex">
 <p class="text-xs text-slate-400">
 </p>
 </div>
 </div>
 </div>
 </div>
 </div>
 <div class="p-4">
 <p class="text-slate-300 text-sm">
 Особый случай: Корень дерева DFS
 моего ребёнка в дереве DFS.
 </p>
 <details class="bg-slate-900/40 rounded-lg">
 <summary class="p-4 font-bold text-slate-300">
 -b border-slate-700/50">
 Код поиска точек сочленения (Скрыт)
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <pre class="bg-slate-950 p-4 rounded">
void dfs_cutpoints(int v, int p = -1) {
 tin[v] = low[v] = ++timer;
 int children = 0;
 for (int to : adj[v]) {
 if (to == p) continue;
 if (tin[to]) {

```

```
ности!</p>
</div>
```

```
<div class="grid grid-cols-1 md:grid-cols-2
 <!-- Неориентированные графы -->
 <div class="bg-slate-800 p-5 rounded-lg
 <p class="text-indigo-300 font-bold
 <ul class="list-disc list-inside sp
 Эйлеров цикл: связан
-emerald-300">deg[v] % 2 == 0.
 Эйлеров путь: связан
o 2 – путь начинается в одной нечётной и за

 </div>
```

```
 <!-- Ориентированные графы -->
 <div class="bg-slate-800 p-5 rounded-lg
 <p class="text-indigo-300 font-bold
 <ul class="list-disc list-inside sp
 Эйлеров цикл: слабос
ald-300">in[v] == out[v].
 Эйлеров путь: слабос
s] = 1 (старт), у одной <span class=
an class="font-mono text-emerald-300">in[v]

 </div>
</div>
```

```
<h3 class="text-lg font-bold text-white mt-
<p class="text-slate-300 text-sm mb-3">Испо
 → рекурсивный переход. При возврате (на вы
ты: <span class="font-mono text-emerald-300
```

```
<details class="bg-slate-900/40 rounded-lg
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
 Код построения Эйлерова цикла C++
 </summary>
```

```

 x: number;
 y: number;
 label: string;
}

interface Edge {
 u: number;
 v: number;
}

const nodes: Node[] = [
 { id: 0, x: 250, y: 50, label: "0" },
 { id: 1, x: 100, y: 150, label: "1" },
 { id: 2, x: 250, y: 250, label: "2" },
 { id: 3, x: 400, y: 150, label: "3" },
 { id: 4, x: 550, y: 150, label: "4" },
 { id: 5, x: 700, y: 50, label: "5" },
 { id: 6, x: 700, y: 250, label: "6" },
];

const edges: Edge[] = [
 { u: 0, v: 1 },
 { u: 1, v: 2 },
 { u: 2, v: 0 },
 { u: 2, v: 3 },
 { u: 3, v: 4 }, // 3 and 4 are articulation points (3
 { u: 4, v: 5 },
 { u: 5, v: 6 },
 { u: 6, v: 4 },
];

// Adjacency list
const adj: number[][] = Array.from(
 { length: Object.keys(nodes).length },
 () => [],
);

edges.forEach((e) => {
 adj[e.u].push(e.v);
});

```

$$\begin{aligned} & \} \} ; \\ & \} ; \end{aligned}$$

```
const dfs = (v: number, p: number = -1) => {
 visited.add(v);
 currentTin[v] = currentLow[v] = timer++;
 let children = 0;
```

```
recordStep(`Заходим в вершину ${v}. tin[${v}]=${c
```

```
for (const to of adj[v]) {
 if (to === p) continue;
```

```
if (visited.has(to)) {
 currentLow[v] = Math.min(currentLow[v], currentLow[to]);
 recordStep(
 `Обратное ребро ${v}-${to}. Обновляем low[${v}]`
);
}
```

```

} else {
 children++;
 dfs(to, v);
 currentLow[v] = Math.min(currentLow[v], curre

```

```
recordStep(
 `Возврат в ${v} из ${to}. Обновляем low[${v}
 v,
):
```

```

if (currentLow[to] >= currentTin[v] && p !=
 ap.add(v);
 recordStep(
 ` Найдена точка сочленения: вершина ${v}
 v,
);
}

```

}

```

 if (isPlaying && currentStepIndex < steps.length -
 interval = setInterval(() => {
 setCurrentStepIndex((prev) => prev + 1);
 }, 1500);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearInterval(interval);
}, [isPlaying, currentStepIndex, steps.length]);

const togglePlay = () => setIsPlaying(!isPlaying);
const reset = () => {
 setIsPlaying(false);
 setCurrentStepIndex(0);
};

return (
 <div className="bg-slate-900 rounded-xl border border-
 <div className="bg-slate-800 p-4 border-b border-
 <h3 className="font-bold text-white flex items-
 <Info className="w-5 h-5 text-rose-400" />
 Моделирование поиска точек сочленения
 </h3>

 <div className="flex items-center gap-2 bg-slate-
 <button
 onClick={togglePlay}
 className="flex items-center gap-2 px-3 py-
 text-white"
 >
 {isPlaying ? (
 <Pause className="w-4 h-4 ml-1" />
) : (
 <Play className="w-4 h-4 ml-1" />
)}
 {isPlaying ? "Пауза " : "Старт "}
 </button>

```

```

 { /* Nodes */ }
 { nodes.map({ node } => {
 const isCurrent = currentStep.currentNode === node.id
 const isVisited = currentStep.visited.has(node.id)
 const isAp = currentStep.ap.has(node.id);

 const fill = isCurrent
 ? "#3b82f6"
 : isAp
 ? "#e11d48"
 : isVisited
 ? "#10b981"
 : "#1e293b";
 const stroke = isCurrent
 ? "#60a5fa"
 : isAp
 ? "#f43f5e"
 : isVisited
 ? "#34d399"
 : "#334155";
 const r = isAp ? 24 : 20;

 return (
 <g key={node.id}>
 <circle
 cx={node.x}
 cy={node.y}
 r={r}
 fill={fill}
 stroke={stroke}
 strokeWidth="3"
 className="transition-all duration-300"
 />
 <text
 x={node.x}
 y={node.y}
 dy=".3em"
 textAnchor="middle"

```

```

 }}}}
 </svg>
</div>

<div className="bg-slate-950 p-6 border-t lg:bo
 <h4 className="text-sm font-bold text-slate-400">
 Статус алгоритма
 </h4>

 <div className="bg-slate-900 border border-sl
 <p className="text-white font-medium min-h-
 {currentStep.desc}
 </p>
 </div>

 <div className="space-y-4 flex-1 overflow-y-a
 <div className="mb-4">
 <h5 className="text-xs text-slate-500 font
 <div className="flex items-center gap-2 t
 <div className="w-3 h-3 rounded-full bg
 Точка сочленения
 </div>
 <div className="flex items-center gap-2 t
 <div className="w-3 h-3 rounded-full bg
 Посещена
 </div>
 <div className="flex items-center gap-2 t
 <div className="w-3 h-3 rounded-full bg
 Текущая
 </div>
 <div className="flex items-center gap-2 t
 <div className="w-8 h-1 bg-rose-500 bor
 Мост
 </div>
 </div>

 <div>
 <h5 className="text-xs text-slate-500 font

```

```
import { useState, useEffect } from 'react';
import { useVizRuntime, vizNumber, vizRecord, vizString } from 'viz';
import { LiveDistTable } from './LiveDistTable';

interface Node { id: string; x: number; y: number; name: string; }
interface Edge { u: string; v: string; w: number; }
interface Step {
 iteration: number;
 checkingEdge: { u: string; v: string; w: number } | null;
 distances: { [key: string]: number };
 previous: { [key: string]: string | null };
 log: string;
 hasNegativeCycle?: boolean;
 activeCodeLine: number;
}

/** Подсказка для кнопок, когда шаги ведёт компилятор.
 * const HINT_TEXT = "Шаги сейчас ведёт панель Python: ← →
 * и листать вручную.";
 */

export default function BellmanFordViz() {
 const [hasCycle, setHasCycle] = useState(false);
 const [steps, setSteps] = useState<Step[]>([]);
 const [currentStepIndex, setCurrentStepIndex] = useState(0);
 const runtime = useVizRuntime();
 const [isPlaying, setIsPlaying] = useState(false);

 const nodes: Node[] = [
 { id: 'S', x: 60, y: 150, name: 'База (S)' },
 { id: 'A', x: 160, y: 60, name: 'Застава (A)' },
 { id: 'B', x: 160, y: 240, name: 'Топь 1 (B)' },
 { id: 'C', x: 260, y: 150, name: 'Мост (C)' },
 { id: 'D', x: 350, y: 60, name: 'Топь 2 (D)' },
 { id: 'E', x: 350, y: 240, name: 'Лес (E)' },
 { id: 'F', x: 450, y: 150, name: 'Склад (F)' },
];
};
```



```

 { line: 9, text: ' return "ОБНАРУЖЕН ОТПР'
 { line: 10, text: ' return dist' },
];

```

```

useEffect(() => {
 const simulationSteps: Step[] = [];
 let dist: { [key: string]: number } = { S: 0, A: 999 };
 let prev: { [key: string]: string | null } = { S: null };

 simulationSteps.push({
 iteration: 0, checkingEdge: null,
 distances: { ...dist }, previous: { ...prev },
 log: ' Фура заведена. Начинаем с Базы (S). Расст',
 activeCodeLine: 2,
 });

 const vCount = nodes.length;
 for (let iter = 1; iter < vCount; iter++) {
 let anyUpdate = false;
 for (const edge of edges) {
 const uDist = dist[edge.u];
 const vDist = dist[edge.v];

 let updated = false;
 if (uDist !== 999 && uDist + edge.w < vDist) {
 dist = { ...dist, [edge.v]: uDist + edge.w };
 prev = { ...prev, [edge.v]: edge.u };
 updated = true;
 anyUpdate = true;
 }

 simulationSteps.push({
 iteration: iter, checkingEdge: { u: edge.u, v: edge.v },
 distances: { ...dist }, previous: { ...prev },
 log: `Итерация ${iter}: Проверяем ${edge.u} → ${edge.v}
 ${updated ? 'Релаксация успешна! dist[${edge.v}] = ' + dist[edge.v] : ''}`,
 activeCodeLine: updated ? 6 : 5,
 });
 }
 }

```



```

<div className="w-full lg:w-2/3 bg-blue-950/20
 y-center min-h-[400px]">
 <div className="absolute top-3 left-3 bg-blue
 ld shadow-sm">
 Итерация {currentStep.iteration} (Шаг {currentStep.step})
 </div>

 {currentStep.hasNegativeCycle && (
 <div className="absolute top-3 right-3 bg-rose-950/20
 animate-pulse shadow-lg shadow-rose-600/50">
 ⚠ Отрицательный цикл!
 </div>
)}

<svg className="w-full h-auto max-h-[500px]" >
 <defs>
 <marker id="arrow-bf-normal" markerWidth="10" markerHeight="10"
 <path d="M0,0 L6,3 L0,6 z" fill="#64748b" />
 <marker id="arrow-bf-active" markerWidth="10" markerHeight="10"
 <path d="M0,0 L6,3 L0,6 z" fill="#f59e0b" />
 <marker id="arrow-bf-cycle" markerWidth="10" markerHeight="10"
 <path d="M0,0 L6,3 L0,6 z" fill="#e11d48" />
 </defs>

 {edges.map((edge, idx) => {
 const uNode = nodes.find((n) => n.id === edge.u);
 const vNode = nodes.find((n) => n.id === edge.v);
 const isChecking = currentStep.checkingEdges === idx;

 const isNeg = edge.w < 0;
 const isCyclePart = hasCycle && ((edge.u === 'D' || edge.v === 'D'));

 let strokeColor = '#475569';
 let marker = 'url(#arrow-bf-normal)';
 if (isChecking) { strokeColor = '#f59e0b'; marker = 'url(#arrow-bf-active)'; }
 else if (currentStep.hasNegativeCycle && isNeg) { strokeColor = '#e11d48'; marker = 'url(#arrow-bf-cycle)'; }

 return (
 <g key={idx}>
 <rect x={uNode.x} y={uNode.y} x2={vNode.x} y2={vNode.y} stroke={strokeColor} stroke-width={1} />
 <path d={`M${uNode.x},${uNode.y} L${vNode.x},${vNode.y}`} stroke={strokeColor} stroke-width={1} />
 <marker data-bbox={marker} />
 </g>
);
 })}

```

```

 { /* Список смежности */ }
 <div className="w-full lg:w-1/3 bg-emerald-950/
 <h4 className="text-lg font-bold text-emerald
 <div className="space-y-2 text-sm font-mono t
 {Object.keys(adjList).map(u => {
 <div key={u} className="flex flex-wrap g
 -colors">
 <span className="font-bold text-white
 {adjList[u].map((edge, idx) => {
 const isChk = currentStep.checkingE
 return (
 <span key={idx} className={`px-2
 edge.w < 0 ? 'bg-rose-900/80 text-rose-300
 {edge.v} ({edge.w})

)
)
 })}
 </div>
)
)
 })}
 </div>
)
 </div>
</div>

<div className="flex flex-col lg:flex-row gap-6">
 { /* Таблица расстояний */ }
 <div className="w-full lg:w-1/2 bg-rose-950/10
 <div>
 <h4 className="text-lg font-bold text-rose-
 <div className="grid grid-cols-4 gap-2 text
 {nodes.map(n => {
 const d = currentStep.distances[n.id];
 const isTgt = currentStep.checkingEdge
 return (
 <div key={n.id} className={`p-2 round
 : 'border-slate-700/60 bg-slate-800/40'}` }>
 <div className="text-xs text-slate
 <div className={`font-mono font-bo
 v>

```

```

 { id: 1, label: '1', x: 150, y: 150 },
 { id: 2, label: '2', x: 450, y: 150 },
 { id: 3, label: '3', x: 75, y: 250 },
 { id: 4, label: '4', x: 225, y: 250 },
 { id: 5, label: '5', x: 375, y: 250 },
 { id: 6, label: '6', x: 525, y: 250 },
];

const EDGES = [
 { from: 0, to: 1 }, { from: 0, to: 2 },
 { from: 1, to: 3 }, { from: 1, to: 4 },
 { from: 2, to: 5 }, { from: 2, to: 6 }
];

const ADJ: { [key: number]: number[] } = {
 0: [1, 2], 1: [3, 4], 2: [5, 6], 3: [], 4: [], 5: [],
};

interface Step {
 activeLine: number;
 queue: number[];
 visited: number[];
 currentNode: number | null;
 logs: string;
}

const generateBfsSteps = (): Step[] => {
 const steps: Step[] = [];
 const queue: number[] = [];
 const visited = new Set<number>();

 // Init
 steps.push({
 activeLine: 1,
 queue: [],
 visited: [],
 currentNode: null,
 logs: "Инициализация: начинаем с корня (0)."
 });
};
```

```

 currentNode: curr,
 logs: `Рассматриваем соседа: ${n}. Он еще не
 });

 visited.add(n);
 queue.push(n);

 steps.push({
 activeLine: 9,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,
 logs: `Помечаем ${n} как посещенную и добавляем
 });
 }
}
} else {
 steps.push({
 activeLine: 6,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,
 logs: `Соседей нет или все уже посещены.`
 });
}
}

steps.push({
 activeLine: 12,
 queue: [],
 visited: Array.from(visited),
 currentNode: null,
 logs: "Очередь пуста. Алгоритм завершен!"
});

return steps;
};

```

```

 onClick={() => { setIsPlaying(false); setSt
 className="p-2 text-slate-400 hover:text-wh
 title="Сброс"
 >
 <RotateCcw className="w-5 h-5" />
 </button>
 <button
 onClick={() => setIsPlaying(p => !p)}
 className="p-2 text-blue-400 hover:text-bl
 title={isPlaying ? 'Пауза' : 'Воспроизведен
 >
 {isPlaying ? <Pause className="w-5 h-5" />
 </button>
 <button
 onClick={handleNext}
 disabled={stepIdx >= STEPS.length - 1}
 className="p-2 text-emerald-400 hover:text-
 ed:hover:bg-transparent"
 title="Шаг вперед"
 >
 <StepForward className="w-5 h-5" />
 </button>
 </div>
 </div>

<div className="grid grid-cols-1 lg:grid-cols-2 g
 {/* Визуал графа */}
 <div className="bg-slate-800 border border-slate
 50px]">
 <svg className="w-full h-full min-h-[350px]" *
 {/* Рёбра */}
 {EDGES.map((e, idx) => {
 const n1 = NODES.find(n => n.id === e.from
 const n2 = NODES.find(n => n.id === e.to)
 const isVisited = step.visited.includes(n
 return (
 <line
 key={idx}

```

```

 r="20"
 fill={fill}
 stroke={stroke}
 strokeWidth="3"
 className="transition-colors duration-200"
 />
 <text
 x={n.x} y={n.y}
 textAnchor="middle"
 dy=".3em"
 className="fill-white font-bold text-sm"
 >
 {n.label}
 </text>
 </g>
);
}
</svg>
</div>

```

```

{/* Код и Очередь */}
<div className="flex flex-col gap-4">
 <div className="bg-slate-900 border border-slate-800 p-4">
 <div className="bg-slate-800 text-slate-400 p-2 font-mono font-size-sm">
 bfs(graph, start)
 var {step}
 </div>
 <div className="p-4">
 {[
 { line: 1, code: 'queue = Queue()' },
 { line: 2, code: 'queue.push(start); visit(start);' },
 { line: 3, code: '' },
 { line: 4, code: 'while not queue.isEmpty():' },
 { line: 5, code: ' node = queue.pop()' },
 { line: 6, code: ' for neighbor in graph[node]:' },
 { line: 7, code: ' if neighbor not in visited:' }
]}
 </div>
 </div>

```



```
 </div>
 </div>
);
}
```

---

ФАЙЛ 34/111: src/components/ChapterImage.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import dijkstraImg from '../assets/dijkstra-freakazoid.jpg';
import bellmanImg from '../assets/bellman-ford-batman.jpg';
import floydImg from '../assets/floyd-network.jpg';
```

```
const imageMap: Record<string, { src: string; alt: string; caption: string; borderColor: string; captionColor: string }> = {
 dijkstra: {
 src: dijkstraImg,
 alt: 'Декстер Дуглас и Фриказоид',
 caption: 'Декстер Дуглас и Фриказоид – волна вируса',
 borderColor: 'border-blue-500/30',
 captionColor: 'text-blue-400',
 },
 'bellman-ford': {
 src: bellmanImg,
 alt: 'Бэтмен на машине Генри Форда',
 caption: 'Бэтмен на машине Форда – n-1 ночей патрулирует',
 borderColor: 'border-rose-500/30',
 captionColor: 'text-rose-400',
 },
 floyd: {
 src: floydImg,
 alt: 'Все ко всем Флойд',
 caption: 'Все связаны со всеми!',
 borderColor: 'border-emerald-500/30',
 captionColor: 'text-emerald-400',
 },
};
```

```

export const ChapterNav: React.FC<Props> = ({ prev, next,
 if (compact) {
 return (
 <div className="flex items-center gap-2 text-xs">
 <button
 disabled={!prev}
 onClick={() => prev && onGo(prev.id)}
 title={prev ? prev.title : "Это первая тема"}
 className="flex items-center gap-1 px-2.5 py-1
 :bg-slate-700 enabled:hover:text-white disabled:opacity-50"
 >
 <ChevronLeft className="w-4 h-4" />
 Назад
 </button>

 {index + 1} / {total}

 <button
 disabled={!next}
 onClick={() => next && onGo(next.id)}
 title={next ? next.title : "Это последняя тема"}
 className="flex items-center gap-1 px-2.5 py-1
 :bg-slate-700 enabled:hover:text-white disabled:opacity-50"
 >
 Вперёд
 <ChevronRight className="w-4 h-4" />
 </button>
 </div>
);
 }

 return (
 <nav className="grid grid-cols-1 sm:grid-cols-2 gap-2" />
 {prev ? (
 <button
 onClick={() => onGo(prev.id)}
 className="group text-left p-4 rounded-xl bg-white
 transition-all"

```

```
interface Props {
 chapterId: string;
}

/**
 * Блиц по прочитанной теме: 3–5 коротких вопросов с не
 *
 * Вопросы живут в `src/data/quizzes.ts` (по id главы).
 * компонент-викторина существовали, но не были подклю
 * контент был, а добраться до него было нельзя. Теперь
 * демонстрации: прочитал → посмотрел, как работает → п
 *
 * В текстовой версии (`?lite=1`) и в PDF те же вопросы
 * с ответами и разбором – там интерактив не нужен.
 */
export const ChapterQuiz: React.FC<Props> = ({ chapterId
 const list = quizzes[chapterId];
 const [idx, setIdx] = useState(0);
 const [picked, setPicked] = useState<number | null>(n
 const [score, setScore] = useState(0);
 const [finished, setFinished] = useState(false);

 // другая тема – другой блиц с нуля
 useEffect(() => {
 setIdx(0);
 setPicked(null);
 setScore(0);
 setFinished(false);
 }, [chapterId]);

 if (!list || list.length === 0) return null;

 const q = list[idx];
 const answered = picked !== null;
 const correct = answered && picked === q.correctIndex

 const pick = (i: number) => {
 if (answered) return;
```

```

 : score * 2 >= list.length
 ? "Половина и больше верно – стоит пер
 : "Стоит вернуться к тексту страницы:
 </p>
 <p className="text-2xl font-bold text-white
 {score} / {list.length}
 </p>
 <button
 type="button"
 onClick={restart}
 className="inline-flex items-center gap-1
 -bold text-indigo-200 hover:bg-indigo-500/2
 >
 <RotateCcw className="w-3.5 h-3.5" /> Про
 </button>
 </div>
) : (
 <>
 <p className="text-sm font-semibold text-wh
 <div className="space-y-2">
 {q.options.map((option, i) => {
 const isRight = i === q.correctIndex;
 const chosen = picked === i;
 const tone = !answered
 ? "border-slate-600 bg-slate-800/70 t
 : isRight
 ? "border-emerald-500 bg-emerald-50
 : chosen
 ? "border-rose-500 bg-rose-500/15
 : "border-slate-700 bg-slate-800/
 return (
 <button
 key={i}
 type="button"
 onClick={() => pick(i)}
 disabled={answered}
 className={`w-full text-left flex i
 lons disabled:cursor-default ${tone}`}

```

```

export default ChapterQuiz;

```

ФАЙЛ 37/111: src/components/ChapterView.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import React, { useCallback, useEffect, useRef, useState } from "react";
import { Check, Download, FileText, Loader2, Maximize2, } from "lucide-react";
import type { Chapter } from "../types";
import { useTermHints } from "../hooks/useTermHints";
import { TermPopover, type HintAnchor } from "../hints/TermHint";
import { getViz } from "../vizRegistry";
import { VizChapterContext } from "../data/vizStepBus";
import { VizLiveStatus } from "../VizLiveStatus";
import { ChapterNav } from "../ChapterNav";
import { ChapterQuiz } from "../ChapterQuiz";
import { downloadChapterHtml } from "../utils/exportHtml";
import { useTicketLinks } from "../utils/ticketLinks";
```

```
interface Props {
 chapter: Chapter;
 prev?: Chapter;
 next?: Chapter;
 index: number;
 total: number;
 onGo: (id: string) => void;
 onOpenSimulator: (vizId: string) => void;
 /** Перейти во вкладку компилятора – там уже подставлены значения */
 onOpenCompiler?: () => void;
}
```

```
const isTouch = () => typeof window !== "undefined" && "ontouchstart" in window;
```

```
export const ChapterView: React.FC<Props> = ({ chapter,
 const contentRef = useRef<HTMLDivElement>(null);
 const [anchor, setAnchor] = useState<HintAnchor | null>(null);
```

```

 });
 };
 const restore = () => {
 each().forEach((d) => {
 if (d.dataset.wasOpen === "0") d.open = false;
 delete d.dataset.wasOpen;
 });
 };
 window.addEventListener("beforeprint", openAll);
 window.addEventListener("afterprint", restore);
 return () => {
 window.removeEventListener("beforeprint", openAll);
 window.removeEventListener("afterprint", restore);
 };
}, [chapter.id]);

const saveHtml = useCallback(async () => {
 setSaving("work");
 try {
 await downloadChapterHtml(chapter);
 setSaving("done");
 window.setTimeout(() => setSaving("idle"), 2500);
 } catch {
 setSaving("idle");
 }
}, [chapter]);

const open = useCallback(
 (el: HTMLElement) => {
 const termId = el.dataset.termId;
 if (!termId) return;
 cancelHide();
 setAnchor({ termId, rect: el.getBoundingClientRect() });
 },
 [cancelHide]
);

const handlePointerOver = (e: React.MouseEvent) => {

```

```

<button
 onClick={saveHtml}
 disabled={saving === "work"}
 title="Сохранить эту тему одним автономным файлом"
 className="flex items-center gap-1.5 text-x-300 hover:text-white hover:border-emerald-500"
>
 {saving === "work" ? (
 <Loader2 className="w-3.5 h-3.5 animate-spin" />
) : saving === "done" ? (
 <Check className="w-3.5 h-3.5 text-emerald-500" />
) : (
 <Download className="w-3.5 h-3.5" />
)}
 {saving === "done" ? "Файл сохранён" : "Скачать"}
</button>
<button
 onClick={() => window.print()}
 title="Распечатать или сохранить эту тему в PDF-формате, без шапки, содержания и панели комментариев"
 className="flex items-center gap-1.5 text-x-300 hover:text-white hover:border-indigo-500"
>
 <Printer className="w-3.5 h-3.5" /> PDF этой темы
</button>
<a
 href={`?lite=1&topic=${encodeURIComponent(currentTopic)}`}
 target="_blank"
 rel="noreferrer"
 title="Текстовая версия темы без навигации, комментариев и панели поиска"
 className="flex items-center gap-1.5 text-x-300 hover:text-white hover:border-sky-500"
>
 <FileText className="w-3.5 h-3.5" /> Текстовая версия

 Эта тема доступна в текстовом формате
</div>

```

```

 title="Открыть Python: код выполняе
рации"
 className="flex items-center gap-1.5
t-emerald-400 hover:text-white hover:border
 >
 <Terminal className="w-3.5 h-3.5" />
 </button>
 })
</div>
</div>
{viz.hint && <p className="text-xs text-sla
<VizChapterContext.Provider value={chapter.
 <VizLiveStatus demoTitle={viz.title} clas
 <viz.Component />
</VizChapterContext.Provider>
</section>
}}

{/* прочитал → посмотрел, как работает → провер
<ChapterQuiz chapterId={chapter.id} />

 <ChapterNav prev={prev} next={next} index={inde
</article>

<TermPopover
 anchor={anchor}
 onClose={() => setAnchor(null)}
 onOpenSimulator={(id) => {
 setAnchor(null);
 onOpenSimulator(id);
 }}
 onCardEnter={cancelHide}
 onCardLeave={scheduleHide}
/>
</>
);
};

```



```
const CLASSES = [
 { id: "EXP", label: "EXP", sub: "2^{n^k}", color: "#0000FF",
 {кванторные формулы}}],
 { id: "PSPACE", label: "PSPACE", sub: "полиномиальная
 с полной информацией"}],
 { id: "NP", label: "NP", sub: "сертификат проверяется за
 AT", "гамильтонов цикл", "раскраска в 3 цвета", "клика"},
 { id: "P", label: "P", sub: "решается за O(n^k)", color: "#00FF00",
 {ид", "MST", "2-SAT, двудольность"}],
];
```

```
const NPHARD_NOTE = "NP-трудные — всё, к чему сводится NP-трудная задача.
 гамильтонов цикл. Проблема остановки NP-трудна, но в NP."
```

```
/* ----- Сведение -----
```

```
const REDUCE_STEPS = [
 { title: "Дано: задача A, для которой быстрого алгоритма нет.
 — то есть B NP-трудная.", boxes: ["A"], active: 0 },
 { title: "Берём вход x задачи A", note: "x — конкретный экземпляр
 \"x\"], active: 1 },
 { title: "Строим f(x) = y за полиномиальное время", note: "f(x) —
 шинами из разных дизъюнктов, если литералы не противоречат друг
 ve: 2 },
 { title: "Решаем B на входе y", note: "Гипотетический алгоритм B
 — но допустим, что он есть.", boxes: ["A", "x", "f(x)", "y"], active: 3 },
 { title: "Переводим ответ обратно: A(x) = B(f(x))", note: "A(x) = B(f(x))
 ⇔ формула выполнима. Полиномиальное время сохранилось.", boxes: ["A", "x", "f(x)", "y", "B", "ans"], active: 5 },
 { title: "Вывод: B не проще A", note: "Раз A ≤p B и A NP-полна, то B
 -полна.", boxes: ["A", "x", "f(x)", "y", "B", "ans"], active: 6 },
];
```

```
const BOX_LABEL: Record<string, { t: string; c: string }> = {
 A: { t: "задача A\n(NP-полная)", c: "#f43f5e" },
 x: { t: "вход x\n|x| = n", c: "#64748b" },
 f: { t: "f: сведения\nO(poly(n))", c: "#f59e0b" },
 y: { t: "вход y\n|y| = poly(n)", c: "#64748b" },
 B: { t: "задача B\n(P-полная)", c: "#28a745" },
 ans: { t: "ответ", c: "#28a745" },
};
```

```

const chapterId = useContext(VizChapterContext);
const runtime = useVizRuntime();
const vars = runtime?.variables;
const [tab, setTab] = useState<Tab>("hierarchy");
const [step, setStep] = useState(0);
const [playing, setPlaying] = useState(false);
const [samples, setSamples] = useState<Sample[]>([]);

useEffect(() => {
 if (chapterId) emitVizDemo(chapterId, tab);
}, [chapterId, tab]);

const maxStep = tab === "reduce" ? REDUCE_STEPS.length : 0;
useEffect(() => {
 setStep(0);
 setPlaying(false);
}, [tab]);

useEffect(() => {
 if (!playing) return;
 const t = window.setInterval(() => {
 setStep((s) => {
 if (s >= maxStep) {
 setPlaying(false);
 return s;
 }
 return s + 1;
 });
 }, 1400);
 return () => window.clearInterval(t);
}, [playing, maxStep]);

/* живые переменные компилятора */
const liveStep = vars ? (vizNumber(vars.step) ?? vizNumber(0)) : null;
useEffect(() => {
 if (liveStep === null || !Number.isFinite(liveStep))
 setStep(Math.max(0, Math.min(maxStep, liveStep)));
}, [liveStep, maxStep]);

```

```

 const q = pts.slice(half);
 if (q.length < 2) return slope;
 const lx = q.map((p) => Math.log(p.n));
 const ly = q.map((p) => Math.log(p.ops));
 const ax = lx.reduce((a, b) => a + b, 0) / lx.length;
 const ay = ly.reduce((a, b) => a + b, 0) / ly.length;
 let nn = 0;
 let dd = 0;
 for (let i = 0; i < lx.length; i++) {
 nn += (lx[i] - ax) * (ly[i] - ay);
 dd += (lx[i] - ax) ** 2;
 }
 return dd === 0 ? slope : nn / dd;
 })();
 const cls =
 slopeTail > slope + 0.8 || slopeTail > 4.5
 ? { name: "экспонента / суперполином", color: "#f59e02", hint: "полиномиальный" }
 : slope <= 1.25
 ? { name: "≈ линейный, $O(n)$ ", color: "#10b981", hint: "линейный" }
 : slope <= 1.8
 ? { name: "≈ $O(n \log n)$ ", color: "#06b6d4", hint: "логарифмический" }
 : slope <= 2.4
 ? { name: "≈ квадратичный, $O(n^2)$ ", color: "#f59e02", hint: "квадратичный" }
 : slope <= 3.4
 ? { name: "≈ кубический, $O(n^3)$ ", color: "#fb9238", hint: "кубический" }
 : { name: "полином высокой степени", color: "#fb9238", hint: "полиномиальный" };
 return { slope, slopeTail, ...cls };
}, [samples]);

```

```

const reduceFrame = REDUCE_STEPS[Math.min(step, REDUCE_STEPS.length - 1)];
const verifyIdx = Math.min(step, VERIFY_STEPS.length);
const verifyDone = verifyIdx >= VERIFY_STEPS.length;
const allOk = VERIFY_STEPS.every((s) => s.ok);

```

```

return (
 <div className="bg-slate-800/90 p-5 sm:p-6 rounded-2xl" style={style} >
 <div className="flex flex-wrap items-center justify-center" >

```

```

 <span className="text-[11px] text-slate-500 m
 </div>
})

```

```

{/* ----- ИЕРАРХИЯ -----
{tab === "hierarchy" && (
 <div className="space-y-4">
 <div className="bg-slate-950 rounded-xl p-3 b
 <svg viewBox="0 0 500 380" className="w-ful
 {CLASSES.map((c) => {
 const on = liveCls ? liveCls.toUpperCas
 return (
 <g key={c.id}>
 <ellipse cx={c.x} cy={c.y} rx={c.rx
 {on ? 3 : 1.5} strokeDasharray={on ? undefi
 <text x={c.x} y={c.y - c.ry + 18} t
 <text x={c.x} y={c.y - c.ry + 32} t
 </g>
);
 });
 });
 /* примеры задач – внутри своих овалов */
 {CLASSES.map((c) =>
 c.tasks.map((t, i) => {
 <text key={t} x={c.x} y={c.y + (c.id :
 tAnchor="middle" fill="#e2e8f0" fontSize="8
 · {t}
 </text>
));
 });
 /* NP-полные = пересечение */
 <text x={250} y={352} textAnchor="middle"
 <text x={250} y={368} textAnchor="middle"
 рхии времени)</text>
 </svg>
 </div>
 <p className="text-xs text-slate-400 leading-
 <p className="text-[11px] text-slate-500">
 Подсказка: выполни в компиляторе <code clas

```

```

 })

 { /* ----- ВЕРИФИКАТОР -----
 {tab === "verify" && (
 <div className="space-y-3">
 <div className="bg-slate-950 rounded-xl p-4 b
 <p className="text-[10px] uppercase tracking
 <div className="flex flex-wrap gap-2 mb-3">
 {CERT.map((v, i) => {
 <span key={i} className={"font-mono tex
 ld-600/50 text-emerald-300" : "bg-slate-900
 x{i + 1} = {liveCert && typeof liveCe

)))
 {liveCert && <span className="font-mono t
 le-950/40">cert из кода}
 </div>
 <div className="space-y-2">
 {CLAUSES.map((cl, i) => {
 const checked = liveK !== null ? i < li
 const current = liveK !== null ? i ===
 const st = VERIFY_STEPS[i];
 return (
 <div
 key={i}
 className={"flex items-center gap-3
 amber-950/20" : checked ? {st.ok ? "border-
 slate-800 bg-slate-900/40"}}
 >
 <span className="text-[10px] font-b
 <span className="font-mono text-xs
 span>
 <span className="font-mono text-[10
 ")}
 <span className={"text-[11px] font-l
 0")}>
 {checked ? {st.ok ? "✓ истинно" :


```

```

<text x={16} y={110} fill="#64748b" fontS
{GROWTH_LINES.map((g) => {
 const pts: string[] = [];
 for (let i = 0; i <= 20; i++) {
 const n = 1 + i * 0.8;
 const v = g.fn(n);
 const x = 44 + (Math.log(n) / Math.lo
 const y = 200 - (Math.log(Math.max(1,
 pts.push(`${x.toFixed(1)} ${Math.max(
 }
 return <polyline key={g.id} points={pts
acity={0.55} />;
)))
 {samples.map((s) => {
 const x = 44 + (Math.log(Math.max(1, s.n
 const y = 200 - (Math.log(Math.max(1, s
 return <circle key={` ${s.n}`} cx={Math.n
strokeWidth={2} />;
)))
 {GROWTH_LINES.map((g, i) => (
 <g key={g.id + "l"}>
 <rect x={330 + (i % 3) * 58} y={16 + 1
 <text x={343 + (i % 3) * 58} y={24 + 1
 </g>
)))
</svg>
{samples.length === 0 && (
 <p className="text-xs text-slate-500 text
 Пусто. Запусти в компиляторе код с пере
t-mono text-purple-300">ops</code> – напри
 </p>
)}
</div>
<div className={"rounded-xl border p-3.5 " +
/60"}}>
 <p className="text-sm font-bold text-white
 {verdict ? `Похуже на ${verdict.name}` :
 </p>

```

ФАЙЛ 39/111: src/components/DfsBridgesSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import { useState, useEffect, useRef } from "react";
import { useVizStepSync, vizNumber, vizString } from ".";
import { Undo, Play, Pause, SkipForward, RefreshCw } from "react-icons/ri";
```

```
interface CodeLine {
 line: string;
 highlight: string;
}
```

```
const CODE_LINES: CodeLine[] = [
 { line: "void dfs(int v, int p = -1) {", highlight: "normal" },
 { line: " visited[v] = true;", highlight: "normal" },
 { line: " tin[v] = low[v] = timer++;", highlight: "normal" },
 { line: " for (int to : adj[v]) {", highlight: "normal" },
 { line: " if (to == p) continue;", highlight: "normal" },
 { line: " if (visited[to]) {", highlight: "normal" },
 { line: " low[v] = min(low[v], tin[to]);", highlight: "normal" },
 { line: " } else {", highlight: "normal" },
 { line: " dfs(to, v);", highlight: "normal" },
 { line: " low[v] = min(low[v], low[to]);", highlight: "normal" },
 { line: " }", highlight: "normal" },
 { line: " if (low[to] > tin[v]) {", highlight: "normal" },
 { line: " // ЭТО МОСТ!", highlight: "normal" },
 { line: " }", highlight: "normal" },
 { line: " }", highlight: "normal" },
 { line: "}", highlight: "normal" },
];
```

```
interface DfsStep {
 currentNode: number | null;
}
```

```

 activeCodeLine: [0,1,2]
},
{
 currentNode: 1, visitedIndices: [0,1],
 tin: {0:1,1:2}, low: {0:1,1:2}, stack: [0,1], bridge: null,
 activeEdge: [0,1], backEdge: null,
 log: "Идём (A,B). В впервые. tin[B]=2, low[B]=2.",
 activeCodeLine: [4,5,7,10,11]
},
{
 currentNode: 6, visitedIndices: [0,1,6],
 tin: {0:1,1:2,6:3}, low: {0:1,1:2,6:3}, stack: [0,1,6],
 activeEdge: [1,6], backEdge: null,
 log: "Из B идём в G. tin[G]=3, low[G]=3.",
 activeCodeLine: [4,5,7,10,11]
},
{
 currentNode: 1, visitedIndices: [0,1,6],
 tin: {0:1,1:2,6:3}, low: {0:1,1:2,6:3}, stack: [0,1,6],
 activeEdge: null, backEdge: null,
 log: "У G нет соседей (кроме B). Возврат. low[G]=3 : ",
 activeCodeLine: [13,14,15]
},
{
 currentNode: 2, visitedIndices: [0,1,6,2],
 tin: {0:1,1:2,6:3,2:4}, low: {0:1,1:2,6:3,2:4}, stack: [0,1,6,2],
 activeEdge: [1,2], backEdge: null,
 log: "Из B идём в C. tin[C]=4, low[C]=4.",
 activeCodeLine: [4,5,7,10,11]
},
{
 currentNode: 2, visitedIndices: [0,1,6,2],
 tin: {0:1,1:2,6:3,2:4}, low: {0:1,1:2,6:3,2:1}, stack: [0,1,6,2],
 activeEdge: null, backEdge: [2,0],
 log: "Видим соседа A (уже посещён)! Обратное ребро ",
 activeCodeLine: [7,8,9]
},
{

```



```

 log: "Возврат E→D. low[D]=min(6,5)=5. (D,E) – не мост",
 activeCodeLine: [11,13,16,17]
 },
 {
 currentNode: 2, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7},
 activeEdge: null, backEdge: null,
 log: "Возврат D→C. low[5] = 5 > tin[2] = 4! (C,D) – мост",
 activeCodeLine: [13,14,15,16,17]
 },
 {
 currentNode: 1, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:3,2:4,3:5,4:6,5:7},
 activeEdge: null, backEdge: null,
 log: "Возврат C→B. low[B]=min(low[B],low[C])=1. (B,C) – мост",
 activeCodeLine: [11,13,16,17]
 },
 {
 currentNode: 0, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:3,2:4,3:5,4:6,5:7},
 activeEdge: null, backEdge: null,
 log: "Возврат B→A. DFS завершён! Мосты: (B,G) и (C,D)",
 activeCodeLine: [11,13,16,17,18]
 },
 {
 currentNode: null, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:3,2:4,3:5,4:6,5:7},
 activeEdge: null, backEdge: null,
 log: "Готово! Найдены мосты: (B,G) и (C,D). Алгоритм DFS завершён",
 activeCodeLine: []
 },
];

export function DfsBridgesSimulator() {
 const [dfsIdx, setDfsIdx] = useState(0);
 useVizStepSync(dfsIdx, setDfsIdx, DFS_STEPS.length - 1);
 const rawV = vizString(vars.v) ?? (vizNumber(vars.v) ?? 0);
 const rawTo = vizString(vars.to) ?? (vizNumber(vars.to) ?? 0);

```

```

 className="p-1.5 hover:bg-slate-700 rounded
 <Undo className="h-3.5 w-3.5" />
 </button>
 <button onClick={() => setDfsAuto(!dfsAuto)}
 className={`px-2.5 py-1 rounded text-[10px]
 dfsAuto ? "bg-rose-600 text-white" : "bg-
 }`>
 {dfsAuto ? <><Pause className="h-3 w-3" />
 </button>
 <button onClick={() => { setDfsAuto(false); s
 disabled={dfsIdx === DFS_STEPS.length-1}
 aria-label="Шаг вперёд" title="Шаг вперёд"
 className="p-1.5 hover:bg-slate-700 rounded
 <SkipForward className="h-3.5 w-3.5" />
 </button>
 <button onClick={() => { setDfsAuto(false); s
 aria-label="Сбросить DFS" title="Сбросить D
 className="p-1.5 hover:bg-slate-700 rounded
 <RefreshCw className="h-3.5 w-3.5" />
 </button>
</div>
</div>

<div className="grid grid-cols-1 md:grid-cols-5 g
 {/* Graph visualization */}
 <div className="md:col-span-3 bg-slate-950 roun
 <div className="absolute inset-0 bg-[radial-g
 >
 <svg className="w-full h-[280px] z-10 relative
 {GRAPH_EDGES.map(e => {
 const u = GRAPH_NODES.find(n => n.id ===
 const v = GRAPH_NODES.find(n => n.id ===
 const bridge = step.bridges.includes(e.ke
 const activeTree = step.activeEdge && (
 (step.activeEdge[0]===e.u && step.active
 (step.activeEdge[0]===e.v && step.active
));
 const activeBack = step.backEdge && {

```

```

 </g>;
 }}}}
 </svg>
 <div className="flex items-center justify-between" >
 Шаг {dfsIdx+1}/{DFS_STEPS.length}
 <div className="flex-1 mx-2 bg-slate-950 h-10" >
 <div className="bg-indigo-600 h-full transition-all duration-200" >
 <div>
 </div>
 </div>
 </div>
 </div>

 {/* Code panel + stack */}
 <div className="md:col-span-2 space-y-3">
 {/* Code panel */}
 <div className="bg-slate-950 rounded-lg border border-slate-800" >
 <div className="bg-slate-900 px-3 py-1.5 text-slate-300" >
 <div className="p-2 overflow-x-auto" >
 <pre className="text-[10px] font-mono leading-relaxed" >
 {CODE_LINES.map(({cl, i}) => {
 const hl = step.activeCodeLine.includes(cl)
 return <div key={i}
 className={`px-1 transition-all duration-200
 ${hl ? "bg-indigo-600/20 text-indigo-300" : "text-slate-400"}
 >

 {cl.line || ' '}
 </div>;
 })}
 </pre>
 </div>
 </div>
 </div>

 {/* Stack */}
 <div className="bg-slate-900/50 rounded-xl border border-slate-800" >
 <div className="text-[10px] font-bold text-slate-400" >
 Стек DFS
 </div>
 </div>
 </div>
 </div>
</div>

```

```
import { useState, useEffect } from 'react';
import { useVizRuntime, vizArray, vizNumber, vizRecord,
import { LiveDistTable } from './LiveDistTable';
```

```
interface Node { id: string; x: number; y: number; name
interface Edge { u: string; v: string; w: number; }
interface Step {
 currentNode: string | null;
 checkingEdge: { u: string; v: string } | null;
 distances: { [key: string]: number };
 visited: { [key: string]: boolean };
 previous: { [key: string]: string | null };
 log: string;
 activeCodeLine: number;
}
```

```
/** Подсказка для кнопок, когда шаги ведёт компилятор.
const HINT_TEXT = "Шаги сейчас ведёт панель Python: ← →
 и листать вручную.";
```

```
export default function DijkstraViz() {
 const nodes: Node[] = [
 { id: 'A', x: 60, y: 150, name: 'Пиццерия (A)' },
 { id: 'B', x: 160, y: 60, name: 'Дом 1 (B)' },
 { id: 'C', x: 160, y: 240, name: 'Парк (C)' },
 { id: 'D', x: 340, y: 60, name: 'Офис (D)' },
 { id: 'E', x: 340, y: 240, name: 'Дом 2 (E)' },
 { id: 'F', x: 450, y: 150, name: 'ТЦ (F)' },
 { id: 'G', x: 250, y: 150, name: 'Метро (G)' },
];
```

```
 const edges: Edge[] = [
 { u: 'A', v: 'B', w: 4 },
 { u: 'A', v: 'C', w: 2 },
 { u: 'C', v: 'B', w: 1 },
 { u: 'C', v: 'G', w: 3 },
```

```

const prev: Record<string, string | null> = { A: null };

simulationSteps.push({
 currentNode: null, checkingEdge: null,
 distances: { ...dist }, visited: { ...visited },
 log: ` Дейкстра готов к доставке. Начинаем из ПИ,
 activeCodeLine: 2,
});

for (let iter = 0; iter < nodes.length; iter++) {
 let minD = 999;
 let u = null;
 for (const n of nodes) {
 if (!visited[n.id] && dist[n.id] < minD) {
 minD = dist[n.id];
 u = n.id;
 }
 }

 if (!u) break;

 visited[u] = true;
 simulationSteps.push({
 currentNode: u, checkingEdge: null,
 distances: { ...dist }, visited: { ...visited },
 log: ` Выбираем вершину ${u} с мин. непосещённой,
 activeCodeLine: 5,
 });

 const neighbors = edges.filter(e => e.u === u);
 for (const edge of neighbors) {
 const v = edge.v;
 if (visited[v]) continue;

 simulationSteps.push({
 currentNode: u, checkingEdge: { u, v },
 distances: { ...dist }, visited: { ...visited },
 log: ` Смотрим соседа ${v}. Текущее расстояние: ${dist[u]},

```

```

const baseStep = steps[currentStepIndex] || null;

if (!baseStep) return <div className="text-white">3arg
const vars = runtime?.variables;
const liveU = vizString(vars?.u);
const liveV = vizString(vars?.v);
const liveDist = vizRecord(vars?.dist);
const livePrev = vizRecord(vars?.prev);
const liveDone = vizArray(vars?.done) ?? vizArray(var
const compilerLinked = liveU !== null || liveV !== nu
const currentStep: Step = compilerLinked
 ? {
 ...baseStep,
 currentNode: liveU,
 checkingEdge: liveU && liveV ? { u: liveU, v: l
 distances: Object.fromEntries(nodes.map((node) =>
 previous: Object.fromEntries(nodes.map((node) =>
 visited: Object.fromEntries(nodes.map((node) =>
 log: `Компилятор: u=${liveU ?? "-"}, v=${liveV
 }
 : baseStep;

return (
 <div className="bg-slate-800/90 p-6 rounded-2xl bor
 { /* Шапка симулятора */ }
 <div className="flex flex-wrap items-center justi
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-indigo-600 text-whit

 </div>
 <div>
 <h3 className="text-2xl font-bold text-whit
 <p className="text-sm text-indigo-300">Полн
 </div>
 </div>
 </div>

 <div className="flex flex-wrap items-center jus

```

```

/>

```

```

<div className="flex flex-col lg:flex-row gap-6 m
 {/* Граф */}
 <div className="w-full lg:w-2/3 bg-indigo-950/20
 justify-center min-h-[400px]">
 <div className="absolute top-3 left-3 bg-indigo-950/20
 font-bold shadow-sm">
 Шаг {currentStepIndex + 1} из {steps.length}
 </div>

 <svg className="w-full h-auto max-h-[500px]" >
 <defs>
 <marker id="arrow-dh-normal" markerWidth=
 <path d="M0,0 L6,3 L0,6 z" fill="#47556b" />
 </marker>
 <marker id="arrow-dh-active" markerWidth=
 <path d="M0,0 L6,3 L0,6 z" fill="#f59e0b" />
 </marker>
 <marker id="arrow-dh-opt" markerWidth="6"
 <path d="M0,0 L6,3 L0,6 z" fill="#10b981" />
 </marker>
 </defs>
 {/* Рёбра */}
 {edges.map((edge, idx) => {
 const uNode = nodes.find((n) => n.id === edge.u)
 const vNode = nodes.find((n) => n.id === edge.v)
 const isChecking =
 currentStep.checkingEdge &&
 (currentStep.checkingEdge.u === edge.u &&
 currentStep.checkingEdge.v === edge.v)

 const isOptimal = currentStep.previous[edge.u] === edge.v

 let marker = 'url(#arrow-dh-normal)';
 if (isChecking) marker = 'url(#arrow-dh-active)';
 else if (isOptimal) marker = 'url(#arrow-dh-opt)';

 return (

```

```

 stroke={isCurrent ? '#a5b4fc' : isV
 strokeWidth={isCurrent ? 4 : 2}
 className="transition-all duration-
 />
 <text x={node.x} y={node.y + 5} fill=
 {node.id}
 </text>
 <text x={node.x} y={node.y - 28} fill=
 "middle" className="bg-indigo-950 px-1 py-0
 {dist === 999 ? '∞' : `d=${dist}`}
 </text>
 <text x={node.x} y={node.y + 35} fill=
 {node.name.split(' ')[0]}
 </text>
 </g>
);
 }}}
</svg>
<div className="w-full bg-slate-950/80 p-4 ro
 <p className="font-semibold text-amber-400
 <p className="min-h-[40px] flex items-cente
</div>
</div>

{/* Список смежности */}
<div className="w-full lg:w-1/3 bg-emerald-950/
 <h4 className="text-lg font-bold text-emerald
 <div className="space-y-2.5 flex-1 overflow-y
 {Object.keys(adjList).map((u) => {
 const isCurrentNode = currentStep.current
 return (
 <div key={u} className={`p-3 rounded-lg
 isCurrentNode ? 'bg-emerald-900/60
 t-slate-300'
 }`>
 <div className="flex items-center gap
 <span className={`px-2 py-0.5 round
 te-300'}`>

```



```

 </tr>
 </thead>
 <tbody className="text-sm">
 {nodes.map((n) => {
 const dist = currentStep.distances[n.id]
 const prev = currentStep.previous[n.id]
 const vis = currentStep.visited[n.id]
 const isCurr = currentStep.currentNode === n.id

 return (
 <tr key={n.id} className={`border-1
 isCurr ? 'bg-indigo-950/60 font-medium' : ''}>
 <td className="py-2.5 px-3 flex items-center">

 {n.name}
 </td>
 <td className="py-2.5 px-3 font-medium">
 {dist === 999 ? '∞' : dist}
 </td>
 <td className="py-2.5 px-3 text-sm">
 {prev || '-'}
 </td>
 </tr>
);
 })}
 </tbody>
 </table>
 </div>
</div>
</div>
</div>

{/* Код алгоритма */}
<div className="w-full lg:w-1/2 bg-slate-900/60 rounded-lg p-4">
 <div>
 <h4 className="text-lg font-bold text-slate-200">Алгоритм
 <div className="bg-slate-950/80 rounded-lg p-4">
 {codeLines.map((item) => {

```

```
const DP_CODE_LINES = [
 /* 1 */ "function fibMemo(n, memo = {}) {",
 /* 2 */ " if (n in memo) return memo[n];",
 /* 3 */ " if (n <= 2) return 1;",
 /* 4 */ " memo[n] = fibMemo(n - 1, memo) + fibMemo",
 /* 5 */ " return memo[n];",
 /* 6 */ "}"
];
```

```
export function DPVisualizer() {
 const [targetN, setTargetN] = useState<number>(5);
 const [steps, setSteps] = useState<DPStep[]>([]);
 const [currentStepIndex, setCurrentStepIndex] = useSt
 const runtime = useVizRuntime();
 const [isPlaying, setIsPlaying] = useState<boolean>(f
 const [speed] = useState<number>(800);
 const [activeTab, setActiveTab] = useState<'table' |
```

```
useEffect(() => {
 const generatedSteps: DPStep[] = [];
 let stepId = 0;
 const memoMap: { [key: number]: number } = {};
```

```
function simulateFib(n: number): number {
 generatedSteps.push({
 id: stepId++,
 n,
 action: 'call',
 desc: `Вызов fibMemo(${n}). Проверяем наличие в
 codeLine: 2,
 memoState: { ...memoMap }
 });
```

```
if (n in memoMap) {
 generatedSteps.push({
 id: stepId++,
 n,
```

```

 codeLine: 5,
 memoState: { ...memoMap }
 });

 return memoMap[n];
}

const finalRes = simulateFib(targetN);

generatedSteps.push({
 id: stepId++,
 n: targetN,
 action: 'done',
 desc: ` Расчет завершен! fibMemo(${targetN}) = $
 codeLine: 6,
 memoState: { ...memoMap }
});

setSteps(generatedSteps);
setCurrentStepIndex(0);
setIsPlaying(false);
}, [targetN]));

useEffect(() => {
 let timer: any = null;
 if (isPlaying && currentStepIndex < steps.length -
 timer = setTimeout(() => {
 setCurrentStepIndex(prev => prev + 1);
 }, speed);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps, speed]));

const baseStep = steps[currentStepIndex] || null;
const vars = runtime?.variables;
const liveK = vizNumber(vars?.k) ?? vizNumber(vars?.n

```

```

 onChange={(e) => setTargetN(Number(e.target.value))}
 disabled={isPlaying}
 className="bg-transparent text-white font-mono"
 >
 <option value={4}>N = 4</option>
 <option value={5}>N = 5</option>
 <option value={6}>N = 6</option>
 <option value={7}>N = 7</option>
 </select>
 </div>

 <div className="flex items-center bg-slate-950" >
 <button
 onClick={() => { setIsPlaying(false); setTargetN(0); }}
 className="p-2 text-slate-400 hover:text-white"
 title="Сбросить"
 >
 <RotateCcw className="w-4 h-4" />
 </button>
 <button
 onClick={() => { setIsPlaying(false); setTargetN(0); }}
 disabled={currentStepIndex === 0}
 className="p-2 text-slate-400 hover:text-white"
 title="Шаг назад"
 >
 <ChevronLeft className="w-5 h-5" />
 </button>
 <button
 onClick={() => setIsPlaying(!isPlaying)}
 className={`flex items-center gap-1.5 px-2 py-2
 ${isPlaying
 ? 'bg-amber-500 text-slate-950 hover:bg-amber-600'
 : 'bg-emerald-600 text-white hover:bg-emerald-700'}
 `}
 >
 {isPlaying ? <Pause className="w-4 h-4" /> : <Play className="w-4 h-4" />}
 {isPlaying ? 'Пауза' : 'Пуск'}
 </button>
 </div>
 </div>
</div>

```

```

 });
 }
 }
 </div>

 {currentStep && (
 <div className="mt-6 pt-4 border-t border-slate-400">
 <div className="text-sm font-medium text-slate-400">

 {currentStep.desc}
 </div>
 <div className="text-xs font-mono bg-slate-400 p-2">
 Текущий N: <strong className="text-emerald-400">{currentStep.n}
 </div>
 </div>
)}
</div>

{/* Tabs */}
<div className="flex items-center gap-2 border-b border-slate-400">
 <button
 onClick={() => setActiveTab('table')}
 className={`flex items-center gap-2 px-6 py-3 border rounded-md
 ${activeTab === 'table' ? 'border-emerald-500 text-emerald-400 bg-slate-400' : 'border-transparent text-slate-400 hover:text-slate-300'}
 `}
 >
 <Eye className="w-4 h-4" />
 Шаг-бай-шаг состояние
 </button>
 <button
 onClick={() => setActiveTab('code')}
 className={`flex items-center gap-2 px-6 py-3 border rounded-md
 ${activeTab === 'code' ? 'border-emerald-500 text-emerald-400 bg-slate-400' : 'border-transparent text-slate-400 hover:text-slate-300'}
 `}
 >
 </div>

```

```

 isActive
 ? 'bg-emerald-600/20 border-l-4
 : 'text-slate-300 hover:bg-slate-
 }` }
 >
 <span className="w-8 text-xs text-s
 {line}
 </div>
);
 }}}
</pre>
<div className="p-6 bg-slate-900/50 border-
 <span className="p-1.5 bg-emerald-500/20
 {currentStep?.desc}
</div>
</div>
 })
</div>

{/* Progress Bar */}
<div className="mt-8 pt-6 border-t border-slate-8
 <div className="flex items-center justify-betwe
 Прогресс выполнения
 № {currentStepIndex + 1} из {steps.len
 </div>
 <div className="h-2 bg-slate-950 rounded-full o
 <div
 className="h-full bg-gradient-to-r from-eme
 style={{ width: `${((currentStepIndex + 1)
 ></div>
 </div>
</div>
</div>
);
}
```

```

*/
export function EulerSimulator() {
 const chapterId = useContext(VizChapterContext);
 const runtime = useVizRuntime();
 const vars = runtime?.variables;

 const [c1Path, setC1Path] = useState<number[]>([]);
 const [c1VisitedEdges, setC1VisitedEdges] = useState<
 const [c1Msg, setC1Msg] = useState("Кликни на вершину
 const [c1Done, setC1Done] = useState(false);

 useEffect(() => {
 if (chapterId) emitVizDemo(chapterId, "walk");
 }, [chapterId]);

 const resetC1 = () => {
 setC1Path([]); setC1VisitedEdges([]); setC1Msg("Кли
 };

 const clickC1 = (vId: number) => {
 if (c1Done && c1Path.length > 0) return;
 if (c1Path.length === 0) {
 setC1Path([vId]);
 setC1Msg("Старт! Кликай соседние вершины, чтобы п
 return;
 }
 const last = c1Path[c1Path.length - 1];
 const edge = C1_EDGES.find(e => (e.u === last && e.v
 if (!edge) {
 setC1Msg("Нет ребра между этими вершинами! Выбери
 return;
 }
 const key = ekey(last, vId);
 if (c1VisitedEdges.includes(key)) {
 setC1Msg("Это ребро уже пройдено! Эйлер не прощае
 setC1Done(true); return;
 }
 const nextEdges = [...c1VisitedEdges, key];
 }
}

```

```

 if (path.length === 0 && usedFlags.length === 0 &&
 // рёбра маршрута: пары соседних вершин
 const keys: string[] = [];
 for (let i = 0; i + 1 < path.length; i++) keys.push
 usedFlags.forEach((f, i) => {
 if (f && C1_EDGES[i]) keys.push(ekey(C1_EDGES[i]).
 }));
 return {
 path,
 keys: Array.from(new Set(keys)),
 deg,
 odd,
 last: last !== null && last >= 0 && last < C1_NODES,
 step,
 complete: keys.length >= C1_EDGES.length,
 };
 }, [vars]));

const path = live && live.path.length ? live.path : c
const visitedKeys = live ? live.keys : c1VisitedEdges
const deg = live?.deg ?? DEG;
const odd = live?.odd ?? ODD;
const lastId = live ? live.last : path.length ? path[
const done = live ? live.complete : c1Done && visited
const msg = live
 ? live.complete
 ? `Компилятор прошёл все ${C1_EDGES.length} рёб
 шин ${odd.length} → это ${odd.length === 0 ? "ц
 : `Маршрут из кода: ${live.path.length ? live.pat
 ys.length}/${C1_EDGES.length}`
 : c1Msg;

const verdict =
 odd.length === 0
 ? { text: "0 нечётных → ЭЙЛЕРОВ ЦИКЛ есть (старт
 }
 : odd.length === 2
 ? { text: `2 нечётные (${odd.map((id) => C1_NODES

```



```

 </g>;
 }}}
 {C1_NODES.map(n => {
 const isLast = lastId === n.id;
 const visited = path.includes(n.id);
 const isOdd = odd.includes(n.id);
 return <g key={n.id} className="cursor-point
 {isLast && <circle cx={n.x} cy={n.y} r={11}
 <circle cx={n.x} cy={n.y} r={11}
 fill={isLast ? "#6366f1" : visited ? "#
 stroke={isLast ? "#fff" : visited ? "#6
 strokeWidth={2.5}
 className="transition-all duration-200
 <text x={n.x} y={n.y + 4} textAnchor="mid
 label}</text>
 <text x={n.x} y={n.y - 16} textAnchor="mi
 one">deg {deg[n.id]}{isOdd ? " • нечёт" : "
 </g>;
 }}}
 </svg>
 <div className="absolute top-2 left-2 bg-slate-
 Peбëp: {visitedKeys.length}/{C1_EDGES.length}
 </div>
</div>

<div className={`p-3 rounded-xl border text-sm ${
 done && visitedKeys.length === C1_EDGES.length
 ? "bg-emerald-950/40 border-emerald-500/30 te
 : c1Done && !live
 ? "bg-rose-950/40 border-rose-500/30 text-ros
 : "bg-slate-900/50 border-slate-700 text-slate
 `}>
 {msg}
</div>

```

/\* Критерий Эйлера \*/

```

<div className={`p-3 rounded-xl border text-xs fo
 Критерий Эйлера (для связного графа): {verdict.

```

```

import { useState, useEffect } from 'react';
import { liveMatrix, pickLiveVar, useVizRuntime, vizNum

interface Step {
 k: number; i: number; j: number;
 matrix: (number | null)[][]; updated: boolean;
 log: string; activeCodeLine: number;
}

/** Подсказка для кнопок, когда шаги ведёт компилятор.
const HINT_TEXT = "Шаги сейчас ведёт панель Python: ← →
 ва листать вручную.";

export default function FloydViz() {
 const [steps, setSteps] = useState<Step[]>([]);
 const [currentStepIndex, setCurrentStepIndex] = useSt
 const runtime = useVizRuntime();
 // k/i/j/d приходят напрямую; номер строки произвольн
 const [isPlaying, setIsPlaying] = useState(false);

 const nodeNames = ['А 0', 'Б 1', 'В 2', 'Г 3', 'Д 4',
 const nodesSvg = [
 { id: 0, name: 'Аэропорт 0', x: 100, y: 60 },
 { id: 1, name: 'Аэропорт 1', x: 250, y: 60 },
 { id: 2, name: 'Аэропорт 2', x: 400, y: 60 },
 { id: 3, name: 'Аэропорт 3', x: 100, y: 180 },
 { id: 4, name: 'Аэропорт 4', x: 400, y: 180 },
 { id: 5, name: 'Аэропорт 5', x: 175, y: 280 },
 { id: 6, name: 'Аэропорт 6', x: 325, y: 280 },
];

 const initialMatrix = [
 [0, 4, 999, 2, 999, 999, 999],
 [999, 0, 3, 999, 999, 3, 999],
 [999, 999, 0, 999, 2, 999, 999],
 [999, 999, 999, 0, 4, 2, 999],
 [999, 999, 999, 999, 0, 999, 1],
 [999, 999, 999, 999, 999, 0, 5],
```

```

 log: `+ Внешний цикл. Разрешаем пути через пром
 activeCodeLine: 3,
 });

 for (let i = 0; i < N; i++) {
 for (let j = 0; j < N; j++) {
 if (i === j || i === k || j === k) continue;

 const oldVal = dist[i][j];
 const viaVal = dist[i][k] + dist[k][j];
 if (dist[i][k] !== 999 && dist[k][j] !== 999 &
 dist[i][j] = viaVal;
 simulationSteps.push({
 k, i, j, matrix: dist.map(r => [...r]), u
 log: ` Улучшение пути [${i}]→[${j}] чере
 ∞'}.`,
 activeCodeLine: 7,
 });
 }
 }
 }

 simulationSteps.push({
 k: 7, i: -1, j: -1, matrix: dist.map(r => [...r])
 log: ` Все пары отработаны. Алгоритм Флойда завер
 activeCodeLine: 8,
 });

 setSteps(simulationSteps);
 setCurrentStepIndex(0);
 setIsPlaying(false);
}, []);

useEffect(() => {
 let timer: any = null;
 if (isPlaying && currentStepIndex < steps.length -
 timer = setTimeout(() => setCurrentStepIndex((p)

```

```

 k: liveK ?? -1,
 i: liveI ?? -1,
 j: liveJ ?? -1,
 matrix: shownMatrix,
 updated: !!runtime?.changed?.some(({name} => name === liveI || name === liveJ)) : "", Подсветка взята из реальных пер
 }
 : baseStep;

return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border" >
 <div className="flex flex-wrap items-center justify-center" >
 <div className="flex items-center gap-3" >
 <div className="p-2.5 bg-emerald-600 text-white" >

 </div>
 <div>
 <h3 className="text-2xl font-bold text-white" >
 <p className="text-sm text-emerald-300" >Полн
 </div>
 </div>

 <div className="flex items-center gap-2" >
 <button onClick={() => { setIsPlaying(false);
 TEXT : "Вернуться к первому шагу" } className="text-white
 d:cursor-not-allowed" >Сброс</button>
 <button onClick={() => setIsPlaying(!isPlaying)} >
 нную демонстрацию" } className={ `px-4 py-2 rounded
 wed ${isPlaying ? 'bg-amber-600' : 'bg-emerald-600'}
 <button onClick={() => { setIsPlaying(false);
 ; }} disabled={compilerLinked} title={compilerLinked}
 ld-600 text-white rounded-lg text-sm disabled
 </div>
 </div>

 <div className="flex flex-col lg:flex-row gap-6 mt-6" >
 { /* Граф */ }

```



```

 labelOf(idx)) {idx}</th>
)))
 </tr></thead>
 <tbody className="text-xs font-mono">
 {currentStep.matrix.map((row, i) => (
 <tr key={i} className="border-b border-
 <td className={`p-2 bg-rose-950/40
 tStep.k === i ? 'text-amber-400' : ''}`}>
 {labelOf(i)} {i}
 </td>
 {row.map((val, j) => {
 const isUpd = currentStep.i === i
 const isVia = currentStep.k !== -
 tep.j === j));
 let bg = 'bg-slate-900/40 text-slate-900'
 if (isUpd) bg = 'bg-emerald-900/80 text-emerald-900'
 else if (isVia) bg = 'bg-indigo-950/60 text-indigo-950'
 else if (currentStep.i === -1 && currentStep.k === -1)
 bg = 'bg-indigo-950/60 text-indigo-950'
 else if (i === j) bg = 'bg-slate-900/40 text-slate-900'
 const shown = val === null ? '.' : val
 return (<td key={j} className={`p-2 ${bg}`}>{shown})
)))
 </tr>
)))
 </tbody>
 </table>
 </div>
 </div>

```

```

 {/* Код алгоритма */}
 <div className="w-full lg:w-1/2 bg-slate-900/60 rounded-lg p-4">
 <div>
 <h4 className="text-lg font-bold text-slate-900">Алгоритм
 <div className="bg-slate-950/80 rounded-lg p-4">
 {codeLines.map(item => (
 <div key={item.line} className={`py-1.5 px-2
 e === item.line ? 'bg-blue-900/80 text-amber-400' : 'bg-slate-950/80 text-slate-900'}`}>

```

```

*/

type Mode = "dfs" | "dsu";

interface Node {
 id: number;
 label: string;
 x: number;
 y: number;
}

/** Граф из 10 вершин и трёх компонент — ровно как в те
const NODES: Node[] = [
 { id: 0, label: "0", x: 90, y: 80 },
 { id: 1, label: "1", x: 175, y: 45 },
 { id: 2, label: "2", x: 205, y: 140 },
 { id: 3, label: "3", x: 105, y: 175 },
 { id: 4, label: "4", x: 330, y: 70 },
 { id: 5, label: "5", x: 420, y: 45 },
 { id: 6, label: "6", x: 380, y: 145 },
 { id: 7, label: "7", x: 130, y: 290 },
 { id: 8, label: "8", x: 230, y: 320 },
 { id: 9, label: "9", x: 330, y: 275 },
];

const EDGES: [number, number][] = [
 [0, 1], [1, 2], [2, 3], [0, 3], [1, 3], // компонента
 [4, 5], [5, 6], [4, 6], // компонента
 [7, 8], [8, 9], // компонента
];

const ADJ: number[][] = NODES.map(() => []);
EDGES.forEach(([u, v]) => {
 ADJ[u].push(v);
 ADJ[v].push(u);
});
ADJ.forEach((a) => a.sort((x, y) => x - y));

```

```

 continue;
 }
 comp[next] = c;
 stack.push(next);
 push(next, stack, c, `${v} → ${next}: красим в цвет ${c}, кладём ${next} в стек.`, [v, next]);
}
push(null, [], c, `Компонента ${c} готова`, `Стек пуст. Ищем следующую серую вершину.`);
}
push(null, [], c, "Готово: компонент = " + c, `Внешние вершины. Вязности.`);
return frames;
}

```

```

interface DsuFrame {
 title: string;
 note: string;
 parent: number[];
 comps: number;
 edge: [number, number] | null;
 merged: boolean;
}

```

```

function buildDsuFrames(): DsuFrame[] {
 const frames: DsuFrame[] = [];
 const parent = NODES.map((n) => n.id);
 const find = (v: number): number => (parent[v] === v ? v : find(parent[v]));
 let comps = NODES.length;
 const push = (title: string, note: string, edge: [number, number] | null) => {
 frames.push({ title, note, parent: [...parent], comps, edge });
 push("Старт: каждая вершина сама по себе", `parent[v] = ${v}.`, null, false);
 EDGES.forEach(([u, v], i) => {
 const ru = find(u);
 const rv = find(v);
 if (ru === rv) {
 push(`Ребро ${i}: (${u}, ${v}) – корни совпали`,

```



```

/* — вкладка сообщает компилятору, какой код подстав.
useEffect(() => {
 if (chapterId) emitVizDemo(chapterId, mode);
}, [chapterId, mode]);

/* — автопроигрывание — */
useEffect(() => {
 if (!playing) return;
 timer.current = window.setInterval(() => {
 setStep((s) => {
 if (s >= maxStep) {
 setPlaying(false);
 return s;
 }
 return s + 1;
 });
 }, 900);
 return () => {
 if (timer.current) window.clearInterval(timer.cur
 };
}, [playing, maxStep]);

const goTo = useCallback((n: number) => setStep(Math.r

/* — переменные компилятора, которые рисуются НАПРЯМ.
const vars = runtime?.variables;
/** comp / color / used из кода пользователя → своя р
const liveComp = useMemo(() => {
 const arr = vars ? (vizArray(vars.comp) ?? vizArray
 if (!arr) return null;
 return arr.slice(0, NODES.length).map((v) => (typeo
}, [vars]);
/** parent / p из кода → DSU-множества по корням. */
const liveParent = useMemo(() => {
 const arr = vars ? (vizArray(vars.parent) ?? vizArr
 if (!arr || arr.length < NODES.length) return null;
 return arr.slice(0, NODES.length).map((v) => vizNum

```

```

const edge = frame.edge;
const title = runtime && vars ? `${frame.title} • Контент` : 'Контент';

const switchMode = (m: Mode) => {
 setMode(m);
 setStep(0);
 setPlaying(false);
};

return (
 <div className="bg-slate-800/90 p-5 sm:p-6 rounded-2xl">
 <div className="flex flex-wrap items-center justify-between">
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-emerald-600 text-white rounded-2xl">
 <Network className="w-5 h-5" />
 </div>
 <div>
 <h3 className="text-lg sm:text-xl font-bold">Дерево поиска
 <p className="text-xs text-emerald-300">10 шагов
 </div>
 </div>
 <div className="flex items-center bg-slate-950 p-2.5 rounded-2xl">
 <button
 onClick={() => switchMode("dfs")}
 className={"flex items-center gap-1.5 px-3 py-2 bg-slate-600 text-white shadow" : "text-slate-400 hover:bg-slate-700"}
 >
 <Layers className="w-3.5 h-3.5" /> 06ход (DFS)
 </button>
 <button
 onClick={() => switchMode("dsu")}
 className={"flex items-center gap-1.5 px-3 py-2 bg-slate-600 text-white shadow" : "text-slate-400 hover:bg-slate-700"}
 >
 <Network className="w-3.5 h-3.5" /> DSU (пёрышки)
 </button>
 </div>
 </div>
 </div>
);

```

```

return (
 <line
 key={i}
 x1={a.x} y1={a.y} x2={b.x} y2={b.y}
 stroke={active ? "#fbbf24" : sameComp ? "#fbbf24" : "#fbbf24"}
 strokeWidth={active ? 4 : sameComp ? 2 : 1}
 strokeDasharray={sameComp ? undefined : [5, 5]}
 className="transition-all duration-300"
 opacity={sameComp || active ? 1 : 0.55}
 />
);
}}}
{NODES.map((n) => {
 const c = compOf(n.id);
 const color = c > 0 ? COLORS[(c - 1) % COLORS.length] : "#fbbf24";
 const isCur = cur === n.id;
 const inFrontier = frontier.includes(n.id);
 return (
 <g key={n.id} onClick={() => goTo(step)} >
 {isCur && <circle cx={n.x} cy={n.y} r={13} fill={color} />}
 {inFrontier && !isCur && <circle cx={n.x} cy={n.y} r={13} fill="none" stroke="#fbbf24" />}
 </g>
 <circle cx={n.x} cy={n.y} r={13} fill={color} />
 <text x={n.x} y={n.y + 4} textAnchor="middle">
 {n.label}</text>
 {c > 0 && <text x={n.x} y={n.y - 19} textAnchor="middle">
 c{c}</text>}
 </g>
);
}})
{ /* стек/очередь и DSU-таблица справа */ }
<g>
 <text x={470} y={24} textAnchor="end" fill="black">
 {mode === "dfs" ? "стек DFS" : "parent[]"}
 </text>
 {mode === "dfs"
 ? frontier.slice(-8).map((v, i) => (

```

```

)
) : (

 открой панель Python – переменные <code cla
 erald-400">parent</code>, <code className="
 comps</code> покрасят этот граф сами

)}
</div>
</div>
);
};

```

```
export default GraphComponentsViz;
```

ФАЙЛ 45/111: src/components/GraphTraversalViz.tsx  
 КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import { useState, useEffect, useMemo } from 'react';
import { useVizStepSync, vizArray, vizString } from '...';
import { Play, Pause, SkipForward, Undo, RefreshCw } from '...';

```

```

type Node = { id: string; label: string; x: number; y: number };
type Edge = { u: string; v: string };

```

```

const nodes: Node[] = [
 { id: 'A', label: 'A', x: 200, y: 40 },
 { id: 'B', label: 'B', x: 100, y: 120 },
 { id: 'C', label: 'C', x: 300, y: 120 },
 { id: 'D', label: 'D', x: 50, y: 200 },
 { id: 'E', label: 'E', x: 150, y: 200 },
 { id: 'F', label: 'F', x: 250, y: 200 },
 { id: 'G', label: 'G', x: 350, y: 200 },
];

```

```

 .stack], visitedNodes: Array.from(vis) }));
 dfs(u);
 }
}

stack.pop();
steps.push({ type: 'backtrack', node: v, desc:
 ueOrStack: [...stack], visitedNodes: Array.fr
}

dfs(start);
return steps;
}

function generateBFSSteps(start: string) {
 const steps: Action[] = [];
 const vis = new Set<string>();
 const q: string[] = [];

 q.push(start);
 vis.add(start);
 steps.push({ type: 'visit', node: start, desc: `Начн
 ray.from(vis)] }));

 while (q.length > 0) {
 const v = q[0];
 steps.push({ type: 'process', node: v, desc: `Д
 q], visitedNodes: [...Array.from(vis)] }));
 q.shift();

 for (const u of adj[v]) {
 if (!vis.has(u)) {
 vis.add(u);
 q.push(u);
 steps.push({ type: 'visit', node: u, de
 tack: [...q], visitedNodes: [...Array.from(
 }
 }
 }
 }
}

```

```

 setStepIdx(p => {
 if (p >= steps.length - 1) {
 setAutoPlay(false);
 return p;
 }
 return p + 1;
 });
 }, 1200);
 return () => clearInterval(t);
}
}, [autoPlay, steps.length]));

const step = steps[stepIdx] || steps[0];

const isVisited = (nodeId: string) => step.visitedNodes.includes(nodeId);
const isCurrent = (nodeId: string) => step.node === nodeId;

return (
 <div className="space-y-4">
 <div className="flex bg-slate-900 border border-slate-800 p-4">
 <button onClick={() => setMode("dfs")}
 className={`px-4 py-2 text-xs font-medium rounded-md
 text-slate-400 hover:text-slate-300`} `>
 DFS (В глубину)
 </button>
 <button onClick={() => setMode("bfs")}
 className={`px-4 py-2 text-xs font-medium rounded-md
 text-slate-400 hover:text-slate-300`} `>
 BFS (В ширину / Волна)
 </button>
 </div>

 <div className="flex flex-col md:flex-row gap-4">
 {/* SVG Graph View */}
 <div className="bg-slate-900 border border-slate-800 p-4
 flex flex-1 min-h-[300px]">
 <svg className="w-full h-full max-w-[1000px] max-h-[500px]">
 {/* Edges */}

```

```

 className={`stroke-rose-500`}
 <text x={n.x} y={n.y}
 className={`text-rose-500 font-mono font-weight-normal`}
 {n.label}
 </text>

 {/* Ripple for BFS */}
 {current && mode === 'bfs' ? (
 <circle cx={n.x} cy={n.y}
 className="stroke-rose-500 stroke-width-2"
 >
)}
 </g>
)
}}
</svg>
</div>

{/* Structure View (Stack / Queue) */}
<div className="bg-slate-900 border border-rose-500 p-10"
 r">
 <div className={`absolute -top-3 left-3 right-3 bottom-3
 ${mode === 'dfs' ? 'bg-slate-800' : 'bg-slate-900'}
 border border-emerald-500`} `>
 {mode === 'dfs' ? 'Стек вызовов' : 'Очередь'}
 </div>

 <div className="flex flex-col gap-2 mt-5"
 {step.queueOrStack && step.queueOrStack.length > 0 ? (
 {step.queueOrStack.map((item, idx) => (
 <div key={idx} className={`w-full flex justify-between items-center
 ${mode === 'dfs' ? 'bg-slate-800' : 'bg-slate-900'}
 text-emerald-200`}
 >
 {item}
 <div>
 ${idx === (mode === 'dfs' ? step.queueOrStack.length - 1 : 0)
 ? `Вызван` : ''}
 </div>
 </div>
)}
)}
) : (
 <div>
 Узел: {item}
 {idx === (mode === 'dfs' ? step.queueOrStack.length - 1 : 0)
 ? `Вызван` : ''}
 </div>
)}
 </div>
</div>

```

```
 <div className="text-sm text-slate-500">
 {step.desc}
 </div>
 </div>
 </div>
 </div>
);
}
```

---

ФАЙЛ 46/111: src/components/HeapViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useState, useCallback } from 'react';
import { Heart } from 'lucide-react';
import { useVizRuntime, vizArray, vizNumber } from '../viz';
```

```
interface Patient {
 id: string;
 name: string;
 emoji: string;
 symptom: string;
 priority: number; // 1 to 99 (Severity)
 animState: 'in' | 'idle' | 'winner' | 'out';
}
```

```
// Max-heap helpers for Patients
```

```
function siftUp(arr: Patient[]): Patient[] {
 const a = arr.map(x => ({ ...x }));
 let idx = a.length - 1;
 while (idx > 0) {
 const parent = Math.floor((idx - 1) / 2);
 if (a[parent].priority < a[idx].priority) {
 [a[parent], a[idx]] = [a[idx], a[parent]];
 idx = parent;
 } else break;
 }
}
```





```

 };

 const nextHeap = heapInsert(heap, newPatient).map(x => {
 // Mark specifically this new patient as 'in' in the heap
 const target = nextHeap.find(x => x.priority === pr.priority);
 const marked = nextHeap.map(x => x.id === (target?.id));

 setHeap(marked);
 addLog(` Поступил пациент: ${finalName} с тяжестью ${pr.priority}`);

 setTimeout(() => {
 setHeap(nextHeap);
 setBusy(false);
 }, 500);
 }, [busy, heap]);

 const handleRandomInsert = () => {
 const preset = PATIENT_PRESETS[Math.floor(Math.random() * PATIENT_PRESETS.length)];
 const randomShift = Math.floor(Math.random() * 10);
 const prio = Math.max(10, Math.min(99, preset.priority + randomShift));
 insertPatient(preset.name, prio);
 };

 const handleCustomInsert = (e: React.FormEvent) => {
 e.preventDefault();
 if (!customName.trim()) return;
 insertPatient(customName, customPriority);
 setCustomName('');
 };

 // --- EXTRACT MAX: Направить самого тяжелого в реанимацию
 const extractMax = useCallback(() => {
 if (busy || heap.length === 0) {
 setShake(true);
 addLog('⚠ Нет пациентов для экстренной госпитализации');
 setTimeout(() => setShake(false), 400);
 return;
 }
 }, [busy, heap]);

```

```

 setHeap(INITIAL_PATIENTS.map(x => ({ ...x, animStat: 0 })));
 setLog([' База данных скорой помощи перезагружена.']);
 setHealing(null);
 setTimeout(() => setHeap(INITIAL_PATIENTS.map(x => ({ ...x, animStat: 0 }))));
 });

// --- Render binary tree lines ---
const edges = visualHeap.flatMap((_, idx) => {
 const res: React.ReactElement[] = [];
 const p = nodePos(idx);
 const l = 2 * idx + 1;
 const r = 2 * idx + 2;
 if (l < visualHeap.length) {
 const lp = nodePos(l);
 res.push(
 <line key={`el${idx}`}
 x1={` ${p.x}%`} y1={` ${p.y + 4}%`}
 x2={` ${lp.x}%`} y2={` ${lp.y - 4}%`}
 stroke="#475569" strokeWidth="2"
 />
);
 }
 if (r < visualHeap.length) {
 const rp = nodePos(r);
 res.push(
 <line key={`er${idx}`}
 x1={` ${p.x}%`} y1={` ${p.y + 4}%`}
 x2={` ${rp.x}%`} y2={` ${rp.y - 4}%`}
 stroke="#475569" strokeWidth="2"
 />
);
 }
 return res;
});

return (
 <div className="flex flex-col gap-6">
 {/* МНЕМОНИКА / ПРАВИЛО */}
 </div>
);

```

```

 />
 <div className="flex flex-col items-center min-w-[300px]">

 <input
 type="range"
 min="10"
 max="99"
 value={customPriority}
 onChange={e => setCustomPriority(Number(e.target.value))}
 className="w-20 accent-emerald-500 cursor-pointer"
 />
 </div>
 <button
 type="submit"
 disabled={busy || !customName.trim() || heap.length === 0}
 className="bg-teal-600 hover:bg-teal-500 disabled:bg-slate-700 text-white px-4 rounded-xl shadow-lg active:scale-95 transition-all cursor-pointer"
 >
 Везти
 </button>
 </form>

 {/* Забрать самого тяжелого */}
 <button
 onClick={extractMax}
 disabled={busy || heap.length === 0}
 className="flex-1 min-w-[170px] bg-gradient-to-r from-teal-600 to-teal-500 disabled:cursor-not-allowed text-white font-mono transition-all cursor-pointer flex items-center justify-center"
 >
 В реанимацию (EXTRACT MAX)
 </button>

 <button
 onClick={reset}
 className="px-5 py-3.5 rounded-2xl bg-slate-800 text-white cursor-pointer border border-slate-700"
 >

```

```

 width: isRoot ? 60 : 52,
 height: isRoot ? 60 : 52,
 }}
 >

 {patient.priority}%

 {/* Tooltip */}
 <div className="absolute bottom-full m-0 shadow-xl">
 <div className="bg-slate-950 border
 <div className="font-bold text-white">
 <div className="text-emerald-400">
 </div>
 </div>
 </div>
 </div>
);
 }}}
</div>

<div className="w-full h-3 bg-gradient-to-r from-blue-500 to-purple-500">
</div>

{/* ПРАВАЯ КОЛОНКА: ОПЕРАЦИОННЫЙ СТОЛ */}
<div className="lg:col-span-4 bg-slate-900 rounded-2xl">
 <div className="absolute top-4 left-4 text-[10px] font-mono">
 Операционная койка
 </div>

 <div className="flex-1 flex flex-col justify-between">
 {healingPatient ? (
 <div className="flex flex-col items-center">
 <div className="relative">


```

```
 </div>
)})
 </div>
 </div>
);
};
```

---

ФАЙЛ 47/111: src/components/hints/demoKit.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { createContext, useContext, useEffect, use
```

  

```
/** Общие примитивы для мини-визуализаций подсказок. */
export const W = 260;
export const H = 130;

/**
 * Общий множитель скорости. Кадры были слишком короткими
 * заметить, что именно переехало, и анимация читалась
 */
export const SPEED = 2.1;

/** Длительность переезда узлов. Должна быть заметно меньше
export const MOVE_MS = 800;
export const EASE = "cubic-bezier(.34,.01,.2,1)";
export const MOVE = `all ${MOVE_MS}ms ${EASE}`;

/** Наведение на карточку ставит анимацию на паузу – мож
export const DemoPauseContext = createContext(false);

export function useLoop(steps: number, ms = 900) {
 const [step, setStep] = useState(0);
 const paused = useContext(DemoPauseContext);
 const period = Math.round(ms * SPEED);
```

```
};
```

```
export const Node: React.FC<{
 x: number;
 y: number;
 r?: number;
 fill: string;
 label?: string | number;
 stroke?: string;
}> = ({ x, y, r = 13, fill, label, stroke }) => (
 <g style={{ transition: MOVE }}>
 <circle cx={x} cy={y} r={r} fill={fill} stroke={stroke}
 {label !== undefined && (
 <text x={x} y={y} textAnchor="middle" dominantBaseline="middle">
 {label}
 </text>
)}
 </g>
);
```

```
export const Edge: React.FC<{
 a: [number, number];
 b: [number, number];
 color?: string;
 width?: number;
 dashed?: boolean;
}> = ({ a, b, color = C.edge, width = 2, dashed }) => (
 <line
 x1={a[0]}
 y1={a[1]}
 x2={b[0]}
 y2={b[1]}
 stroke={color}
 strokeWidth={width}
 strokeDasharray={dashed ? "4 3" : undefined}
 style={{ transition: MOVE }}
 />
);
```

```

 4
 </text>

 <Node x={pos.i[0]} y={pos.i[1]} label="i" fill={C}
 <Node x={pos.k[0]} y={pos.k[1]} label="k" fill={v}
 <Node x={pos.j[0]} y={pos.j[1]} label="j" fill={r}
 </Svg>
);
};

/** Компоненты связности: несколько «островов», которые
export const ComponentsDemo: React.FC = () => {
 const comps: [string, string][][] = [
 ["A", "B"], ["B", "C"],
 ["D", "E"],
 ["F", "G"], ["G", "H"], ["F", "H"],
];
 const pos: Record<string, [number, number]> = {
 A: [26, 40], B: [62, 92], C: [26, 92],
 D: [120, 40], E: [120, 92],
 F: [190, 34], G: [232, 78], H: [178, 96],
 };
 const colors = [C.active, C.warn, C.done];
 const step = useLoop(comps.length + 1, 900);
 const compOf = (v: string) => comps.findIndex((e) =>

 return (
 <Svg caption={step === 0 ? "Граф распался на острова"
 {comps.flatMap((edges, ci) =>
 edges.map(([u, v], i) => (
 <Edge key={` ${ci} - ${i}`} a={pos[u]} b={pos[v]}
))
)}
 >
 {Object.entries(pos).map(([k, [x, y]]) => {
 const ci = compOf(k);
 return <Node key={k} x={x} y={y} r={12} label={
 }}}
 </Svg>
)

```



```

)))
 {Object.entries(pos).map(([k, [x, y]]) => {
 <Node key={k} x={x} y={y} label={k} fill={C.idle} />
 })}
 </Svg>
);
};

```

/\*\* Сжатие координат: разрежённые числа → плотные индексы

```

export const CoordCompressDemo: React.FC = () => {
 const raw = [1000, 5, 74000, 5, 300];
 const sorted = [5, 300, 1000, 74000];
 const step = useLoop(3, 1200);
 const boxW = 46;
 const startX = (260 - raw.length * (boxW + 4)) / 2;

 return (
 <Svg
 caption={
 step === 0
 ? "Исходные координаты – огромные и с дырами"
 : step === 1
 ? "Сортируем уникальные значения"
 : "Каждое число заменяем его номером → плотные индексы"
 }
 >
 {raw.map((v, i) => {
 <g key={i}>
 <rect x={startX + i * (boxW + 4)} y={20} width={boxW} height={20} />
 <text x={startX + i * (boxW + 4) + boxW / 2} y={30}>
 {v}
 </text>
 </g>
 })}

 {step >= 1 &&
 sorted.map((v, i) => {
 <g key={v}>

```

```

 key={i}
 x1={f.pos[a][0]}
 y1={f.pos[a][1]}
 x2={f.pos[b][0]}
 y2={f.pos[b][1]}
 stroke={C.edge}
 strokeWidth={1.8}
 style={{ transition: MOVE }}
 />
)))
 {Object.entries(f.pos).map(([id, [x, y]]) => (
 <g key={id} style={{ transform: `translate(${x}, ${y})` }>
 <circle r={13} fill={color(id)} stroke={C.idle} />
 <text textAnchor="middle" dominantBaseline="bottom">
 {id}
 </text>
 </g>
))}
 </Svg>
);
};

```

```

/** Zig – один поворот, когда родитель уже корень. */
export const ZigDemo: React.FC = () => (
 <SplayFrames
 captions={["p – корень, x под ним", "Один поворот: "]}
 frames={[
 { pos: { p: [130, 32], x: [78, 92] }, edges: [{"p", "x"}]},
 { pos: { x: [130, 32], p: [182, 92] }, edges: [{"x", "p"}]}
]}
 />
);

```

```

/** Zig-Zig – x и p с одной стороны, крутим сначала вершину p. */
export const ZigZigDemo: React.FC = () => (
 <SplayFrames
 captions={["Линия g → p → x («бамбук»)", "Поворот В"]}
 frames={[

```

```

const visited = new Set(order.slice(0, step).flat());
const wave = step > 0 && step <= order.length ? order
return (
 <Svg caption={`Уровень ${Math.max(0, Math.min(step,
 {GRAPH_EDGES.map(([u, v], i) => {
 <Edge key={i} a={GRAPH_POS[u]} b={GRAPH_POS[v]}
 }}}
 {Object.entries(GRAPH_POS).map(([k, [x, y]]) => {
 <Node key={k} x={x} y={y} label={k}
 fill={wave.includes(k) ? C.active : visited.h
 stroke={wave.includes(k) ? "#a5b4fc" : undefi
 }}}
 </Svg>
);
};

export const DfsDemo: React.FC = () => {
 const path = ["A", "B", "D", "E", "F", "C"];
 const step = useLoop(path.length + 1, 750);
 const visited = path.slice(0, step);
 const head = visited[visited.length - 1];
 return (
 <Svg caption={head ? `Идём вглубь: ${visited.join("
 {GRAPH_EDGES.map(([u, v], i) => {
 const iu = visited.indexOf(u);
 const iv = visited.indexOf(v);
 const onPath = iu >= 0 && iv >= 0 && Math.abs(i
 return <Edge key={i} a={GRAPH_POS[u]} b={GRAPH_I
 }}}
 {Object.entries(GRAPH_POS).map(([k, [x, y]]) => {
 <Node key={k} x={x} y={y} label={k} fill={k ===
 }}}
 </Svg>
);
};

export const ToposortDemo: React.FC = () => {
 const nodes = [

```

```

return (
 <Svg caption={improved ? "12 > 3 + 4 → пишем 7. Путь"
 <Edge a={[60, 70]} b={[200, 70]} color={improved ? "red" : "black"} />
 <text x={130} y={54} textAnchor="middle" fontSize={16} />
 <Node x={60} y={70} r={20} fill={C.active} label="3" />
 <Node x={200} y={70} r={20} fill={improved ? C.done : C.active} label="4" />
 <text x={60} y={106} textAnchor="middle" fontSize={16} />
 <text x={200} y={106} textAnchor="middle" fontSize={16} />
 <text x={130} y={22} textAnchor="middle" fontSize={16} />
 </Svg>
);
};

export const BridgeDemo: React.FC = () => {
 const step = useLoop(2, 1300);
 const pos: Record<string, [number, number]> = {
 A: [40, 40], B: [40, 95], C: [95, 68], D: [170, 68]
 };
 const edges: [string, string][] = [
 ["A", "B"], ["A", "C"], ["B", "C"], ["C", "D"], ["D", "A"]
];
 const cut = step === 1;
 return (
 <Svg caption={cut ? "Убрали C-D → граф развалился на 2 компонента"
 {edges.map(([u, v], i) => {
 const isBridge = u === "C" && v === "D";
 if (isBridge && cut) return null;
 return <Edge key={i} a={pos[u]} b={pos[v]} color={isBridge ? "red" : "black"} />
 })}
 {Object.entries(pos).map(([k, [x, y]]) => {
 <Node key={k} x={x} y={y} label={k} fill={cut ? "red" : "black"} />
 })}
 </Svg>
);
};

export const ArticulationDemo: React.FC = () => {
 const step = useLoop(2, 1300);

```

```

 <text x={{pos[e.u][0] + pos[e.v][0]} / 2} y=
 textAnchor="middle" fontSize={9} fontWeigh
 </g>
);
 }
 }
 {Object.entries(pos).map([k, [x, y]] => (<Node
</Svg>
);
};

export const SccDemo: React.FC = () => {
 const pos: Record<string, [number, number]> = {
 A: [45, 35], B: [110, 35], C: [77, 95], D: [185, 45]
 };
 const edges: [string, string][] = [["A", "B"], ["B",
 const step = useLoop(2, 1400);
 const scc = ["A", "B", "C"];
 const inScc = (u: string, v: string) => step === 1 &&
 return (
 <Svg caption={step === 1 ? "{A,B,C} – цикл: из кажд
 {edges.map([u, v], i) => {
 <Edge key={i} a={pos[u]} b={pos[v]} color={inSc
 }}}
 {Object.entries(pos).map([k, [x, y]] => {
 <Node key={k} x={x} y={y} label={k} fill={step
 }}}
 </Svg>
);
};

```

ФАЙЛ 50/111: src/components/hints/demosStruct.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИ

```

import React from "react";
import { C, Edge, Node, Svg, useLoop } from "../demoKit"

```

```

export const QueueDemo: React.FC = () => {
 const states = [["A", "B"], ["A", "B", "C"], ["B", "C"], ["C"]];
 const step = useLoop(states.length, 800);
 const items = states[step];
 return (
 <Svg caption="pop_front слева, push_back справа – к"
 <defs>
 <marker id="mdArrow" markerWidth="6" markerHeight="6"
 <path d="M0,0 L6,3 L0,6 Z" fill={C.done} />
 </marker>
 </defs>
 <text x={12} y={44} fontSize={9} fill={C.dim}>ВЫХОД</text>
 <text x={206} y={44} fontSize={9} fill={C.dim}>ВХОД</text>
 {items.map((v, i) => (
 <g key={v} style={{ transition: "all .3s" }}>
 <rect x={30 + i * 54} y={56} width={46} height={16} fill={C.dim} />
 <text x={53 + i * 54} y={71} textAnchor="middle">{v}</text>
 </g>
))}
 <line x1={24} y1={71} x2={10} y2={71} stroke={C.dim} />
 </Svg>
);
};

```

```

export const DsuDemo: React.FC = () => {
 const parents = [[0, 1, 2, 3], [0, 0, 2, 3], [0, 0, 2, 3], [0, 0, 2, 3]];
 const step = useLoop(parents.length, 950);
 const p = parents[step];
 const pos: [number, number][] = [[50, 95], [105, 95], [160, 95], [215, 95]];
 const rootColor = ["#6366f1", "#10b981", "#f59e0b", "#f44336"];
 const root = (i: number): number => (p[i] === i ? i : root(p[i]));
 return (
 <Svg caption="find = «кто твой староста», union = объединение"
 {p.map((par, i) => (
 par !== i ? (
 <path key={i} d={`M ${pos[i][0]} ${pos[i][1]} L ${pos[root(i)][0]} ${pos[root(i)][1]}`
 fill="none" stroke={rootColor[root(i)]} strokeDash={root(i) === 0 ? [4, 4]} : [] />
)
))}
 </Svg>
);
};

```

```

 return <Edge key={n.id} a={[p.x, p.y]} b={[n.x,
]}]
]}]
 {TRIE_NODES.map((n) => {
 <Node key={n.id} x={n.x} y={n.y} label={n.ch}
 fill={wave.includes(n.id) ? C.active : doneId
]}]
]}]
 <text x={6} y={12} fontSize={9} fill={C.dim}>очер
</Svg>
]];
};

```

```

export const SuffixLinkDemo: React.FC = () => {
 const links: [number, number][] = [[1, 0], [2, 0], [3
 const step = useLoop(links.length + 1, 900);
 return (
 <Svg caption="Суффиксная ссылка: «куда откатиться,
 {TRIE_NODES.filter((n) => n.parent !== null).map(
 const p = TRIE_NODES[n.parent as number];
 return <Edge key={n.id} a={[p.x, p.y]} b={[n.x,
]}]
]}]
 {links.slice(0, step).map(([from, to], i) => {
 const a = TRIE_NODES[from];
 const b = TRIE_NODES[to];
 return (
 <path key={i} d={`M ${a.x} ${a.y} Q ${a.x + 1
 fill="none" stroke={C.warn} strokeWidth={1.
]];
]}]
 {TRIE_NODES.map((n) => {
 <Node key={n.id} x={n.x} y={n.y} label={n.ch} f
]}]
]}]
 </Svg>
]];
};

```

```

export const SegmentTreeDemo: React.FC = () => {
 const step = useLoop(3, 1000);
 const nodes = [

```

```

)))
 <text x={4} y={72} fontSize={8} fill={C.dim}>P</text>
 {pref.map((v, i) => (
 <g key={i}>
 <rect x={20 + i * 33} y={78} width={29} height={10}
 fill={show && i === l ? C.bad : show && i === l ? C.dim : C.idle}
 stroke={C.idleStroke} style={{ transition: 'fill 0.2s' }}
 <text x={34 + i * 33} y={90} textAnchor="middle">{v}</text>
 </g>
 </g>
))}
 </Svg>
);
};

```

```

export const SparseTableDemo: React.FC = () => {
 const step = useLoop(3, 1000);
 const len = [1, 2, 4][step];
 const count = 8 - len + 1;
 return (
 <Svg caption={`Уровень j=${step}: готовые ответы для`}>
 {Array.from({ length: 8 }).map((_, i) => (
 <g key={i}>
 <rect x={12 + i * 30} y={18} width={26} height={10}
 fill={C.idle} stroke={C.idleStroke} style={{ transition: 'fill 0.2s' }}
 <text x={25 + i * 30} y={29} textAnchor="middle">{i}</text>
 </g>
 </g>
))}
 {Array.from({ length: count }).map((_, i) => (
 <rect key={i} x={12 + i * 30} y={52 + (i % 4) * 10}
 fill={C.active} opacity={i === 0 ? 1 : 0.4} style={{ transition: 'fill 0.2s' }}
 </rect>
))}
 </Svg>
);
};
};

```

```

export const AmortizedDemo: React.FC = () => {
 const costs = [1, 1, 1, 8, 1, 1, 1, 1];
 const step = useLoop(costs.length + 1, 550);

```



```

 {s.split("").map((ch, idx) => (
 <g key={idx}>
 <rect x={18 + idx * 33} y={26} width={28} height={24}
 fill={step > 0 && (idx < len || (idx > i - 1)) ? C.warn : C.idle}
 style={{ transition: "fill .25s" }} />
 <text x={32 + idx * 33} y={40} textAnchor="middle" font-size={10} fill="black">
 {ch}
 </text>
 <text x={32 + idx * 33} y={70} textAnchor="middle" font-size={10} fill="black">
 {ch}
 </text>
 </g>
))}
 <text x={4} y={40} font-size={8} fill={C.dim}>s</text>
 <text x={4} y={70} font-size={8} fill={C.dim}>π</text>
 <text x={130} y={106} textAnchor="middle" font-size={10} fill="black">
 π[i] – длина самого длинного «префикс = суффикс»
 </text>
 </Svg>
);
};

```

```

export const DpDemo: React.FC = () => {
 const rows = 3, cols = 5;
 const step = useLoop(rows * cols + 1, 320);
 return (
 <Svg caption={`Заполняем таблицу подзадач: готово $
 {Array.from({ length: rows }).map((_, r) =>
 Array.from({ length: cols }).map((_, c) => {
 const idx = r * cols + c;
 const filled = idx < step;
 return (
 <g key={idx}>
 <rect x={40 + c * 38} y={22 + r * 32} width={38} height={32}
 fill={idx === step - 1 ? C.active : filled ? C.warn : C.idle}
 style={{ transition: "fill .25s" }} />
 {filled && (
 <text x={57 + c * 38} y={36 + r * 32} text-align="center" font-size={10} font-weight="700" fill="black">
 {ch}
 </text>
)}
 </g>
)
)}
)}
 </Svg>
);
};

```

```

 heap: HeapDemo,
 stack: StackDemo,
 queue: QueueDemo,
 dsu: DsuDemo,
 trie: TrieDemo,
 "trie-bfs": TrieBfsDemo,
 "suffix-link": SuffixLinkDemo,
 toposort: ToposortDemo,
 relax: RelaxDemo,
 "prefix-sum": PrefixSumDemo,
 "sparse-table": SparseTableDemo,
 "segment-tree": SegmentTreeDemo,
 bridge: BridgeDemo,
 articulation: ArticulationDemo,
 mst: MstDemo,
 amortized: AmortizedDemo,
 np: NpDemo,
 scc: SccDemo,
 "prefix-function": PrefixFunctionDemo,
 dp: DpDemo,
 floyd: FloydDemo,
 components: ComponentsDemo,
 graph: GraphBasicsDemo,
 "coord-compress": CoordCompressDemo,
 zig: ZigDemo,
 "zig-zig": ZigZigDemo,
 "zig-zag": ZigZagDemo,
 };

```

```

/** Список работающих мини-демо — его проверяет `npm run`
export const MINI_DEMO_KINDS = Object.keys(REGISTRY);

```

```

export const MiniDemo: React.FC<{ kind: DemoKind }> = ({
 const [paused, setPaused] = useState(false);
 const Cmp = REGISTRY[kind];
 if (!Cmp) return null;
 return (
 <DemoPauseContext.Provider value={paused}>

```

```

 return () => {
 window.removeEventListener("keydown", onKey);
 document.body.style.overflow = "";
 };
 }, [vizId, onClose]);

const viz = getViz(vizId ?? undefined);
if (!vizId || !viz) return null;
const Component = viz.Component;

return createPortal(
 <div className="fixed inset-0 z-[110] flex items-end">
 <div className="absolute inset-0 bg-slate-950/80">
 <div className="relative w-full sm:max-w-5xl max-h-
 ded-2xl shadow-2xl flex flex-col">
 <div className="flex items-center gap-3 px-4 py-4">
 <div className="min-w-0 flex-1">
 <h3 className="font-bold text-white text-sm">
 {viz.hint && <p className="text-[11px] text-white">
 <VizLiveStatus demoTitle={viz.title} className="text-white">
 </div>
 <button
 onClick={onClose}
 className="p-2 rounded-lg bg-slate-800 hover:bg-slate-700
 aria-label="Заккрыть симулятор"
 >
 <X className="w-5 h-5" />
 </button>
 </div>
 <div className="overflow-y-auto overscroll-contain">
 {/* Тот же id главы: развёрнутая демонстрация */}
 <VizChapterContext.Provider value={vizId}>
 <Component />
 </VizChapterContext.Provider>
 </div>
 </div>
 </div>,
 document.body
)

```

```

 setPos(null);
 return;
 }
 const vw = window.innerWidth;
 const vh = window.innerHeight;
 const width = Math.min(CARD_W, vw - 16);
 const height = cardRef.current?.offsetHeight ?? 300

 let left = anchor.rect.left + anchor.rect.width / 2
 left = Math.max(8, Math.min(left, vw - width - 8));

 const below = anchor.rect.top < height + GAP + 8;
 const top = below ? anchor.rect.bottom + GAP : anchor

 setPos({ top: Math.max(8, Math.min(top, vh - height
 }, [anchor]));

 useEffect(() => {
 if (!anchor) return;
 const onKey = (e: KeyboardEvent) => {
 if (e.key === "Escape") onClose();
 };
 window.addEventListener("keydown", onKey);
 window.addEventListener("scroll", onClose, true);
 return () => {
 window.removeEventListener("keydown", onKey);
 window.removeEventListener("scroll", onClose, true);
 };
 }, [anchor, onClose]);

 if (!anchor) return null;
 const entry = GLOSSARY_BY_ID.get(anchor.termId);
 if (!entry) return null;
 const viz = getViz(entry.simulator);
 const simulatorId = entry.simulator;

 return createPortal(
 <div

```

```

 </div>
)}

 {viz && simulatorId && (
 <button
 onClick={() => onOpenSimulator(simulatorId)}
 className="w-full flex items-center justify-center
 rounded-lg py-2 transition-colors"
 >
 <Play className="w-3.5 h-3.5" /> Показать
 </button>
)}
 </div>
 </div>
</div>,
document.body
);
};

```

ФАЙЛ 54/111: src/components/JohnsonViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import { useState } from 'react';
import { RotateCcw, SkipForward, Undo } from 'lucide-react';
import { useVizStepSync, vizString } from '../data/vizSteps';

export function JohnsonViz() {
 const [step, setStep] = useState(0);
 const MAX_STEP = 7;
 useVizStepSync(step, setStep, MAX_STEP, (vars) => {
 const edge = `${vizString(vars.u) ?? ""}${vizString(vars.v) ?? ""}`;
 return edge === "AB" ? 4 : edge === "BC" ? 5 : 0;
 });

 const nodes = [

```

```

 if (e.id === 'AB') {
 if (step === 4) return "stroke-amber-400 stroke-width-2px";
 if (step > 4) return "stroke-emerald-500";
 }
 if (e.id === 'BC') {
 if (step === 5) return "stroke-amber-400 stroke-width-2px";
 if (step > 5) return "stroke-emerald-500";
 }
 if (e.id === 'CA') {
 if (step === 6) return "stroke-amber-400 stroke-width-2px";
 if (step > 6) return "stroke-emerald-500";
 }

 if (step >= 4) return "stroke-slate-600"; // otherwise
 return "stroke-slate-500";
 };

const getMarkerUrl = (e: any) => {
 if (e.u === 'S') {
 if (step === 0 || step === 7) return "";
 if (step === 1) return "url(#arrow-amber)";
 if (step === 2) return "url(#arrow-pink)";
 return "url(#arrow-slate-dim)";
 }
 if (e.id === 'AB' && step === 4) return "url(#arrow-amber)";
 if (e.id === 'AB' && step > 4) return "url(#arrow-emerald)";
 if (e.id === 'BC' && step === 5) return "url(#arrow-amber)";
 if (e.id === 'BC' && step > 5) return "url(#arrow-emerald)";
 if (e.id === 'CA' && step === 6) return "url(#arrow-amber)";
 if (e.id === 'CA' && step > 6) return "url(#arrow-emerald)";

 if (step >= 4) return "url(#arrow-slate-dim)";
 return "url(#arrow-slate)";
};

const getEdgeContent = (e: any) => {
 if (e.u === 'S') return e.weight;

```

```

 Если мы запустим быстрый алгоритм Д
 </div>
);
case 1: return (
 <div>
 Шаг 1: Точка отсчета.

 Добавляем фиктивную вершину S. Соединяем ее с остальными вершинами.
 Это наша база для расчета потенциалов.
 </div>
);
case 2: return (
 <div>
 Шаг 2: Ищем потенциалы $h(v)$.

 Запускаем медленный, но мощный алгоритм для всех вершин:

 </div>
);
case 3: return (
 <div>
 Шаг 3: Магическая формула.

 Всем ребрам графа назначаем новый вес.
 <div className="text-center font-monospace">
 Это повышает или понижает вес ребра.
 </div>
 </div>
);
case 4: return (
 <div>
 Шаг 3.1: Пересчет ребра A → B.

 Старый вес:
 Вычисляем: $-2 + 0 - (-2) =$
 </div>
);
case 5: return (
 <div>
 Шаг 3.2: Пересчет ребра B → C.

 Старый вес: 1.


```

```

 <marker id="arrow-slate-dim
uto-start-reverse">
 <path d="M 0 0 L 10 5 L
 </marker>
 <marker id="arrow-amber" vie
start-reverse">
 <path d="M 0 0 L 10 5 L
 </marker>
 <marker id="arrow-pink" vie
tart-reverse">
 <path d="M 0 0 L 10 5 L
 </marker>
 <marker id="arrow-emerald"
o-start-reverse">
 <path d="M 0 0 L 10 5 L
 </marker>
 </defs>

```

```

 { /* Edges */ }
 { edges.map((e, i) => {
 const u = nodes.find(n => n
 const v = nodes.find(n => n

 const isS = e.u === 'S';
 if (isS && (step === 0 || s

 const midX = (u.x + v.x) / 2;
 const midY = (u.y + v.y) / 2;

 const edgeColorClass = getE
 const textHigh = isEdgeText
 const textEmer = isEdgeText

 return {
 <g key={i}>
 <line
 x1={u.x} y1={u.y}
 className={`str

```



```

 <g transform={`
fade-in zoom-in duration-500">
 <rect x="-1
k-500 stroke-1" />
 <text x="0"
font-bold fill-pink-400">
 {pot}
 </text>
 </g>
)}
 </g>
)
 }}}
 </svg>
 </div>

```

```

<div className="w-full md:w-[320px] flex
 {/* Log Box */}
 <div className="bg-[#0f172a] p-5 ro
l justify-center leading-relaxed text-sm te
 {getDesc()}
 </div>

```

```

 {/* Controls */}
 <div className="flex bg-slate-900 b
 <button onClick={() => setStep(1
 className="p-3 bg-slate-800
-slate-400 transition-colors" title="Сброси
 <RotateCcw strokeWidth={3}
 </button>
 <button onClick={() => setStep(1
 className="p-3 bg-slate-800
-slate-400 transition-colors" title="Вар на
 <Undo strokeWidth={3} size=
 </button>
 <button onClick={() => setStep(
 className={`flex-1 font-bol
 ${step === MAX STEP

```

```

 y: number;
 label: string;
}

interface GEdge {
 u: number;
 v: number;
}

const initialNodes: GNode[] = [
 { id: 0, x: 150, y: 100, label: "0" },
 { id: 1, x: 300, y: 100, label: "1" },
 { id: 2, x: 225, y: 220, label: "2" }, // 0->1->2->0
 { id: 3, x: 450, y: 160, label: "3" },
 { id: 4, x: 600, y: 160, label: "4" }, // 3 <-> 4 (Se
];

const initialEdges: GEdge[] = [
 { u: 0, v: 1 },
 { u: 1, v: 2 },
 { u: 2, v: 0 },
 { u: 2, v: 3 }, // Connection edge
 { u: 3, v: 4 },
 { u: 4, v: 3 },
];

interface AlgoStep {
 phase: "init" | "dfs1" | "transpose" | "dfs2" | "fini
 desc: string;
 visited: Set<number>;
 currentNode: number | null;
 order: number[];
 transposed: boolean;
 sccMap: Record<number, number>; // node -> sccId
 currentSccId: number;
 activeEdges: GEdge[];
}

```

```

 ? initialEdges.map((e) => ({ u: e.v, v: e.u }
 : [...initialEdges],
 }));
};

recordStep(
 "init",
 "Инициализация алгоритма Косарайю. Начинаем первый
 null,
 false,
 -1,
);

// Adj list
const adj: number[][] = Array.from({ length: 5 }, () => []);
initialEdges.forEach((e) => adj[e.u].push(e.v));

// DFS 1: post-order list
const dfs1 = (u: number) => {
 visited.add(u);
 recordStep(
 "dfs1",
 `DFS 1: зашли в вершину ${u}, помечаем посещение
 u,
 false,
 -1,
);

 for (const to of adj[u]) {
 if (!visited.has(to)) {
 dfs1(to);
 }
 }

 order.push(u);
 recordStep(
 "dfs1",
 `DFS 1: вершина ${u} полностью обработана. Запи

```

```

 true,
 currentScc,
);

 for (const to of adjT[u]) {
 if (!visited.has(to)) {
 dfs2(to, currentScc);
 }
 }
};

// Run DFS2 using the order (reversed from back to front)
for (let i = order.length - 1; i >= 0; i--) {
 const u = order[i];
 if (!visited.has(u)) {
 sccId++;
 recordStep(
 "dfs2",
 `DFS 2: берем следующую непосещенную из стека`,
 u,
 true,
 sccId,
);
 dfs2(u, sccId);
 } else {
 recordStep(
 "dfs2",
 `DFS 2: вершина ${u} из стека уже покрашена в`,
 u,
 true,
 sccId,
);
 }
}

recordStep(
 "finished",
 `Алгоритм успешно завершен! Найдено ${sccId} компонент`,
);

```

```

const getSccColor = (id: number) => {
 if (id === 1) return "fill-emerald-600 stroke-emera
 if (id === 2) return "fill-indigo-600 stroke-indigo
 return "fill-slate-800 stroke-slate-500";
};

return (
 <div className="bg-slate-900 rounded-xl border bord
 <div className="bg-slate-800 p-4 border-b border-
 <h3 className="font-bold text-white flex items-
 <Layers className="w-5 h-5 text-emerald-400"
 Визуализатор Алгоритма Косарайю (Поиск SCC)
 </h3>

 <div className="flex items-center gap-2 bg-slate
 <button
 onClick={togglePlay}
 className="flex items-center gap-2 px-3 py-
 d-500 text-white"
 >
 {isPlaying ? {
 <Pause className="w-4 h-4 ml-1" />
 } : {
 <Play className="w-4 h-4 ml-1" />
 }}
 {isPlaying ? "Пауза " : "Старт "}
 </button>

 <button
 onClick={reset}
 aria-label="Сбросить обход"
 title="Сбросить обход"
 className="flex items-center justify-center
 ors"
 >
 <RotateCcw className="w-4 h-4" />
 </button>

```

```

 orient="auto-start-reverse"
 >
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#6
 </marker>
</defs>

{/* Edges */}
{initialEdges.map((edge, idx) => {
 const u = initialNodes.find((n) => n.id ==
 const v = initialNodes.find((n) => n.id ==

 // If transposed, draw reversed coordinates
 const x1 = step.transposed ? v.x : u.x;
 const y1 = step.transposed ? v.y : u.y;
 const x2 = step.transposed ? u.x : v.x;
 const y2 = step.transposed ? u.y : v.y;

 const isSccEdge =
 step.sccMap[edge.u] &&
 step.sccMap[edge.v] &&
 step.sccMap[edge.u] === step.sccMap[edge.v];
 let stroke = "#475569";
 let markerId = "arrow";

 if (step.transposed) {
 stroke = isSccEdge ? "#818cf8" : "#4f46e5";
 markerId = "arrow-transposed";
 } else {
 if (
 step.currentNode === edge.u ||
 step.currentNode === edge.v
) {
 stroke = "#10b981";
 markerId = "arrow-active";
 }
 }
})

// Adjust slightly for bidirectional link

```

```

 <g key={node.id}>
 <circle
 cx={node.x}
 cy={node.y}
 r="20"
 className={`transition-all duration
 strokeWidth="3"
 />
 <text
 x={node.x}
 y={node.y}
 dy=".3em"
 textAnchor="middle"
 fill="white"
 fontSize="13px"
 fontWeight="bold"
 className="select-none"
 >
 {node.label}
 </text>

 {sccId && (
 <text
 x={node.x}
 y={node.y - 28}
 textAnchor="middle"
 fill="#10b981"
 fontSize="10px"
 fontWeight="bold"
 >
 SCC #{sccId}
 </text>
)}
 </g>
);
 }
 }
</svg>

```

```

 </div>
))
 })
 </div>
</div>

{/* List of actions log */}
<div>

 ЛОГ ОПЕРАЦИЙ:

 <div className="space-y-1.5 max-h-[150px]">
 {steps
 .slice(0, currentStepIdx + 1)
 .reverse()
 .slice(0, 5)
 .map(({s, idx}) => {
 <div
 key={idx}
 className={`text-[11px] p-1.5 rounded`
 >
 {s.desc}
 </div>
)}}
 </div>
 </div>
</div>

<div className="mt-4 pt-3 border-t border-slate-200">

 Шаг {currentStepIdx + 1} из {steps.length}

 <div className="flex gap-1.5">
 <button
 disabled={currentStepIdx === 0}
 onClick={() => setCurrentStepIdx((prev) => prev - 1)}
 className="bg-slate-900 border border-slate-800 text-white px-2 py-1 rounded"
 >
 Назад
 </button>
 <button
 disabled={currentStepIdx === steps.length - 1}
 onClick={() => setCurrentStepIdx((prev) => prev + 1)}
 className="bg-slate-900 border border-slate-800 text-white px-2 py-1 rounded"
 >
 Далее
 </button>
 </div>
</div>

```



```

 id: string;
 u: number;
 v: number;
 weight: number;
 status: 'pending' | 'inspecting' | 'included' | 'reje
 color: string;
}

```

```

interface StepLog {
 title: string;
 description: string;
 type: 'info' | 'success' | 'warning' | 'error';
}

```

// 9 Vertices layout

```

const initialVertices: Vertex[] = [
 { id: 0, label: 'A', x: 20, y: 20, color: 'none' },
 { id: 1, label: 'B', x: 50, y: 13, color: 'none' },
 { id: 2, label: 'C', x: 80, y: 20, color: 'none' },
 { id: 3, label: 'D', x: 18, y: 50, color: 'none' },
 { id: 4, label: 'E', x: 50, y: 50, color: 'none' },
 { id: 5, label: 'F', x: 82, y: 50, color: 'none' },
 { id: 6, label: 'G', x: 20, y: 80, color: 'none' },
 { id: 7, label: 'H', x: 50, y: 87, color: 'none' },
 { id: 8, label: 'I', x: 80, y: 80, color: 'none' },
];

```

// 12 Edges connecting the 9 vertices

```

const initialEdges: Edge[] = [
 { id: '0-1', u: 0, v: 1, weight: 2, status: 'pending' },
 { id: '3-4', u: 3, v: 4, weight: 3, status: 'pending' },
 { id: '1-2', u: 1, v: 2, weight: 4, status: 'pending' },
 { id: '0-3', u: 0, v: 3, weight: 5, status: 'pending' },
 { id: '1-4', u: 1, v: 4, weight: 6, status: 'pending' },
 { id: '4-5', u: 4, v: 5, weight: 7, status: 'pending' },
 { id: '3-6', u: 3, v: 6, weight: 8, status: 'pending' },
 { id: '2-5', u: 2, v: 5, weight: 9, status: 'pending' },
 { id: '6-7', u: 6, v: 7, weight: 10, status: 'pending' },
];

```

```

 title: 'Успех: Ребро A-B в осто́ве',
 description: 'Вершины A and B окрашены в красный цвет',
 type: 'success'
 }, ...prev));
},
// Step 3: Inspect D-E (3)
() => {
 setEdges(prev => prev.map(e => e.id === '3-4' ? {
 setLogs(prev => [{
 title: 'Шаг 2: Берем второе ребро D-E (вес 3)',
 description: 'Оно соединяет две другие бесцветные вершины',
 type: 'info'
 }, ...prev]));
},
// Step 4: Include D-E
() => {
 setVertices(prev => prev.map(v => v.id === 3 || v.id === 4 ? {
 setEdges(prev => prev.map(e => e.id === '3-4' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро D-E в осто́ве',
 description: 'Вершины D и E окрашены в синий цвет',
 type: 'success'
 }, ...prev]));
},
// Step 5: Inspect B-C (4)
() => {
 setEdges(prev => prev.map(e => e.id === '1-2' ? {
 setLogs(prev => [{
 title: 'Шаг 3: Берем ребро B-C (вес 4)',
 description: 'Оно соединяет красную вершину B и синюю вершину C',
 type: 'info'
 }, ...prev]));
},
// Step 6: Include B-C
() => {
 setVertices(prev => prev.map(v => v.id === 2 ? {
 setEdges(prev => prev.map(e => e.id === '1-2' ? {
 setLogs(prev => [{

```

```

 title: 'Отказ: Цикл обнаружен!',
 description: 'Мы выбрасываем ребро B-E. Иначе по-прежнему не получится',
 type: 'error'
 }, ...prev]));
},
// Step 11: Inspect E-F (7)
() => {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {
 setLogs(prev => [{
 title: 'Шаг 6: Берем ребро E-F (вес 7)',
 description: 'Оно соединяет фиолетовую вершину E с фиолетовой вершиной F',
 type: 'info'
 }, ...prev]));
},
// Step 12: Include E-F
() => {
 setVertices(prev => prev.map(v => v.id === 5 ? {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро E-F в осто́ве',
 description: 'Вершина F окрашена в фиолетовый цвет',
 type: 'success'
 }, ...prev]));
},
// Step 13: Inspect D-G (8)
() => {
 setEdges(prev => prev.map(e => e.id === '3-6' ? {
 setLogs(prev => [{
 title: 'Шаг 7: Берем ребро D-G (вес 8)',
 description: 'Оно соединяет фиолетовую вершину D с фиолетовой вершиной G',
 type: 'info'
 }, ...prev]));
},
// Step 14: Include D-G
() => {
 setVertices(prev => prev.map(v => v.id === 6 ? {
 setEdges(prev => prev.map(e => e.id === '3-6' ? {
 setLogs(prev => [{

```

```

 description: 'Вершина H перекрашена в фиолетовый',
 type: 'success'
 }, ...prev]);
},
// Step 19: Inspect E-H (11) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '4-7' ? {
 setLogs(prev => [{
 title: 'Шаг 10: Берем ребро E-H (вес 11)',
 description: 'Оба конца E и H уже фиолетовые. Соединяет их!',
 type: 'warning'
 }, ...prev]));
},
// Step 20: Reject E-H
() => {
 setEdges(prev => prev.map(e => e.id === '4-7' ? {
 setLogs(prev => [{
 title: 'Отказ: Цикл обнаружен!',
 description: 'Выбрасываем ребро E-H.',
 type: 'error'
 }, ...prev]));
},
// Step 21: Inspect H-I (12)
() => {
 setEdges(prev => prev.map(e => e.id === '7-8' ? {
 setLogs(prev => [{
 title: 'Шаг 11: Берем ребро H-I (вес 12)',
 description: 'Соединяет фиолетовую H и последнюю I',
 type: 'info'
 }, ...prev]));
},
// Step 22: Include H-I
() => {
 setVertices(prev => prev.map(v => v.id === 8 ? {
 setEdges(prev => prev.map(e => e.id === '7-8' ? {
 setLogs(prev => [{
 title: 'Успех: MST построено Краскалом!',
 description: 'Все 9 вершин окрашены в единый фиолетовый цвет'
 }

```

```

 type: 'success'
 }, ...prev]);
},
// Step 4: Pop A-D (5)
() => {
 setEdges(prev => prev.map(e => e.id === '0-3' ? {
 setLogs(prev => [{
 title: 'Шаг 3: Куча выдает ребро A-D (вес 5)',
 description: 'Плесень расширяется в сторону верш
 type: 'info'
 }, ...prev]);
},
// Step 5: Include A-D, add A's edges to heap
() => {
 setVertices(prev => prev.map(v => v.id === 0 ? {
 setEdges(prev => prev.map(e => e.id === '0-3' ? {
 eap' } : e));
 setHeapQueue(['A-B (2)', 'B-E (6)', 'E-F (7)', 'D
 setLogs(prev => [{
 title: 'Успех: Плесень поглотила вершину A',
 description: 'Новое ребро A-B(2) добавлено в куч
 type: 'success'
 }, ...prev]);
},
// Step 6: Pop A-B (2)
() => {
 setEdges(prev => prev.map(e => e.id === '0-1' ? {
 setLogs(prev => [{
 title: 'Шаг 4: Куча выдает ребро A-B (вес 2)',
 description: 'Плесень стремительно бежит к верш
 type: 'info'
 }, ...prev]);
},
// Step 7: Include A-B, add B's edges
() => {
 setVertices(prev => prev.map(v => v.id === 1 ? {
 setEdges(prev => prev.map(e => e.id === '0-1' ? {
 eap' } : e));

```

```

() => {
 setEdges(prev => prev.map(e => e.id === '1-4' ? {
 setHeapQueue(['E-F (7)', 'D-G (8)', 'C-F (9)', 'E
 setLogs(prev => [{
 title: 'Отказ: Игнорируем внутреннее ребро',
 description: 'Ребро B-E отброшено. Плесень не р
 type: 'error'
 }, ...prev]));
},
// Step 12: Pop E-F (7)
() => {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {
 setLogs(prev => [{
 title: 'Шаг 7: Куча выдает ребро E-F (вес 7)',
 description: 'Плесень из центра Токио (E) тянет
 type: 'info'
 }, ...prev]));
},
// Step 13: Include E-F, add F's edges
() => {
 setVertices(prev => prev.map(v => v.id === 5 ? {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {
 eap' } : e));
 setHeapQueue(['D-G (8)', 'C-F (9 - цикл)', 'E-H (
 setLogs(prev => [{
 title: 'Успех: Вершина F захвачена',
 description: 'Ребро F-I(13) добавлено в кучу. Р
 type: 'success'
 }, ...prev]));
},
// Step 14: Pop D-G (8)
() => {
 setEdges(prev => prev.map(e => e.id === '3-6' ? {
 setLogs(prev => [{
 title: 'Шаг 8: Куча выдает ребро D-G (вес 8)',
 description: 'Плесень расширяется вниз к G.',
 type: 'info'
 }, ...prev]));
}

```

```

 type: 'info'
 }, ...prev]);
},
// Step 19: Include G-H, add H's edges
() => {
 setVertices(prev => prev.map(v => v.id === 7 ? {
 setEdges(prev => prev.map(e => e.id === '6-7' ? {
 eap' } : e));
 setHeapQueue(['E-H (11 - цикл)', 'H-I (12)', 'F-I
 setLogs(prev => [{
 title: 'Успех: Вершина H захвачена',
 description: 'В кучу добавлено H-I(12).',
 type: 'success'
 }, ...prev]);
},
// Step 20: Pop E-H (11) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '4-7' ? {
 setLogs(prev => [{
 title: 'Шаг 11: Куча выдает E-H (вес 11)',
 description: 'Обе вершины E и H уже в плесени.'
 type: 'warning'
 }, ...prev]);
},
// Step 21: Reject E-H
() => {
 setEdges(prev => prev.map(e => e.id === '4-7' ? {
 setHeapQueue(['H-I (12)', 'F-I (13)']));
 setLogs(prev => [{
 title: 'Отказ: Игнорируем E-H',
 description: 'Выбрасываем из кучи.',
 type: 'error'
 }, ...prev]);
},
// Step 22: Pop H-I (12)
() => {
 setEdges(prev => prev.map(e => e.id === '7-8' ? {
 setLogs(prev => [{

```

```

 description: 'Каждая из 9 деревень одновременно
 А выбирает A-B(2), D выбирает D-E(3), а G вы
 !',
 type: 'info'
 }, ...prev]);
},
// Step 2: Concurrently merge into Kolkhozes (Component)
() => {
 // Concurrently build chosen roads. Merge into:
 // Component 1: {A, B, C} (colored red)
 // Component 2: {D, E, F, G, H, I} (colored gold/
 setVertices(prev => prev.map(v =>
 [0, 1, 2].includes(v.id) ? { ...v, color: 'red'
 }));
 setEdges(prev => prev.map(e => {
 if (['0-1', '1-2'].includes(e.id)) {
 return { ...e, status: 'included', color: 'red'
 }
 if (['3-4', '4-5', '3-6', '6-7', '7-8'].includes(e.id)) {
 return { ...e, status: 'included', color: 'blue'
 }
 return e;
 }));
 setLogs(prev => [{
 title: 'Слияние: Деревни объединились в колхозы
 description: 'Все выбранные дороги построены ра
 оз {D, E, F, G, H, I}.',
 type: 'success'
 }, ...prev]);
},
// Step 3: Five-Year Plan 2 (Inspecting bridge between
() => {
 // Now both kolkhozes look at all outgoing roads
 // Outgoing roads between {A, B, C} and {D, E, F,
 // A-D (5), B-E (6), C-F (9).
 // Cheapest is A-D (5).
 setEdges(prev => prev.map(e => e.id === '0-3' ? {
 setLogs(prev => [{

```



```

 setStepIndex(stepIndex + 1);
 }
};

const introLog = {algo: 'kruskal' | 'prim' | 'boruvka'
 if (algo === 'kruskal') {
 return [{
 title: 'Введение в раскраску (Краскал)',
 description: 'Представь, что все 9 вершин графа
 type: 'info',
 }]];
 }
 if (algo === 'prim') {
 return [{
 title: 'Введение в Плесень (Прима)',
 description: 'Логика «Плесени»: выбираем старто
 ь (Heap) и захватываем соседей!',
 type: 'info',
 }]];
 }
 return [{
 title: 'Введение в Коллективизацию (Борувка)',
 description: 'Логика «Борувки»: Каждая из 9 дерев
 ы объединиться в колхоз.',
 type: 'info',
 }]];
};

const handleReset = {algo: 'kruskal' | 'prim' | 'boru
 setActiveAlgo(algo);
 setVertices(initialVertices);
 setEdges(initialEdges);
 setStepIndex(0);
 setHeapQueue([]);
 setLogs(introLog(algo));
};

```

/\* ————— связь с Python-компилятором —————

```

const c = name.trim().toUpperCase().replace(/[^A-I]/g, '');
return c ? c.charCodeAt(0) - 65 : null;
};

/** Разбор одного ребра из кода: (2, 'A', 'B') / [2, 'A', 'B']
const parseEdge = (item: unknown): { u: number; v: number; w: string | null } | null {
 if (item === null || item === undefined) return null;
 if (typeof item === 'string') {
 const nums = item.match(/-?\d+(?:\.\d+)?/g);
 const letters = item.match(/[A-Ia-i]/g);
 let u: number | null = null;
 let v: number | null = null;
 if (letters && letters.length >= 2) { u = idOf(letters[0]); v = idOf(letters[1]); }
 else if (nums && nums.length >= 2) { u = idOf(nums[0]); v = idOf(nums[1]); }
 if (u === null || v === null) return null;
 return { u, v, w: nums && nums.length ? Number(nums[0]) : null };
 }
 if (Array.isArray(item)) {
 const letters = item.filter((x) => typeof x === 'string');
 const nums = item.filter((x) => typeof x === 'number');
 if (letters.length >= 2) {
 const u = idOf(letters[0]); const v = idOf(letters[1]);
 if (u === null || v === null) return null;
 return { u, v, w: nums.length ? nums[0] : null };
 }
 if (nums.length >= 2) {
 const u = idOf(nums[nums.length - 2]); const v = idOf(nums[nums.length - 1]);
 if (u === null || v === null) return null;
 return { u, v, w: nums.length > 2 ? nums[0] : null };
 }
 }
 return null;
};

```

```

const PALETTE = ['red', 'blue', 'purple', 'mold'];
/** Цвет по корне компоненты: одинаковый корень → одинаковый цвет
const colorOfComp = (root: string | undefined): string {
 if (!root) return 'none';

```

```

const compOfVertex = new Map<number, string>();
rawComp.forEach((label, id) => {
 if (parentIdx.size === 0) {
 compOfVertex.set(id, label);
 return;
 }
 let cur = id;
 const seen = new Set<number>([id]);
 for (let hop = 0; hop < 16; hop++) {
 const nxt = parentIdx.get(cur);
 if (nxt === undefined || nxt === cur || seen.has(nxt)) {
 seen.add(nxt);
 cur = nxt;
 }
 compOfVertex.set(id, String.fromCharCode(65 + cur));
 });
const takenRaw = vizArray(vars.mst) ?? vizArray(vars.mst);
const takenEdges = (takenRaw ?? []).map(parseEdge).filter(e => e !== null);
const takenVertices = (vizArray(vars.taken) ?? vizArray(vars.taken)).map((x) => idOf(typeof x === 'string' ? x : vizNumber(x))).filter((x): x is number => x !== null);
const curU = idOf(vizString(vars.u) ?? vizNumber(vars.u));
const curV = idOf(vizString(vars.v) ?? vizNumber(vars.v));
const curW = vizNumber(vars.w);
const heapRaw = vizArray(vars.heap) ?? vizArray(vars.heap);
const heapEdges = (heapRaw ?? []).map(parseEdge).filter(e => e !== null);
const cheapestRaw = vizRecord(vars.cheapest) ?? vizRecord(vars.cheapest);
const cheapestEdges = cheapestRaw ? Object.values(cheapestRaw).filter(e => e !== null) : [];
const total = vizNumber(vars.total) ?? vizNumber(vars.total);
const any = compOfVertex.size > 0 || takenEdges.length > 0 || (curU !== null && curV !== null) || heapEdges.length > 0;
return any ? { compOfVertex, takenEdges, takenVertices, total, curU, curV, curW, heapEdges, cheapestEdges } : { compOfVertex, takenEdges, takenVertices, total, curU, curV, curW, heapEdges, cheapestEdges }, [vars]);

```

```

/** Рёбра, которые рисуем: код пользователя важнее ка
const effEdges: Edge[] = useMemo(() => {

```

```

 return 'bg-slate-700 border-emerald-500 text-slate-50';
 }
 return 'bg-slate-700 border-slate-500 text-slate-50';
}
};

```

```

const getEdgeStroke = (status: string, color: string) => {
 if (status === 'inspecting') return '#f59e0b';
 if (status === 'rejected') return '#ef4444';
 if (status === 'in_heap') return '#0284c7';
 if (status === 'included') {
 if (color === 'red') return '#ef4444';
 if (color === 'blue') return '#3b82f6';
 if (color === 'purple') return '#a855f7';
 if (color === 'mold') return '#10b981';
 }
 return '#475569';
};

```

```

return (
 <div className="space-y-12">

 {/* 3-in-1 MST Simulator Box */}
 <div className="bg-slate-900/90 border border-slate-800" >
 <div className="flex flex-col lg:flex-row justify-between" >
 <div>
 <div className="flex items-center gap-2 mb-2" >

 Супер-Интерактив 3 в 1

 <h3 className="text-2xl font-bold text-white" >
 MST Simulator
 </h3>
 </div>
 <p className="text-slate-400 text-sm" >
 Наблюдай за работой Краскала (Раскраска),
 </p>
 </div>
 </div>
 </div>
 </div>
)

```

```

 >
 <Users className="w-3.5 h-3.5" />
 Борувка (Билет 20)
 </button>
 </div>

 <button
 onClick={() => handleReset(activeAlgo)}
 className="flex items-center justify-cent
-200 font-semibold text-xs rounded-xl borde
 >
 <RotateCcw className="w-3.5 h-3.5" />
 Сбросить
 </button>

 <button
 onClick={handleNextStep}
 disabled={stepIndex >= currentSteps.length}
 className={
 "flex items-center justify-center gap-2
nitial cursor-pointer " +
 {stepIndex >= currentSteps.length
 ? "bg-slate-800 text-slate-500 border
 : activeAlgo === 'kruskal'
 ? "bg-indigo-600 hover:bg-indigo-500
 : activeAlgo === 'prim'
 ? "bg-emerald-600 hover:bg-emerald-500
 : "bg-rose-600 hover:bg-rose-500 text
 }
 >
 <SkipForward className="w-4 h-4" />
 {stepIndex === 0 ? 'Начать' : stepI
 </button>
 </div>
</div>

{/* Что компилятор сообщил графу прямо сейчас */
<div className="mb-6 flex flex-wrap items-cente

```

```

 вес остова = {live.total}

 })
</>
) : (
 <span className="text-[11px] text-slate-500"
 открой панель Python и запусти код вкладки
 ка' }>: переменные
 {' '}<code className="font-mono text-purple-300"
 e className="font-mono text-purple-300">u</code>
 mono text-purple-300">w</code> перекрасят э

 })
</div>

<div className="grid grid-cols-1 lg:grid-cols-3"
 { /* Graph Canvas */ }
 <div className="lg:col-span-2 bg-slate-950 rounded-3xl
 active min-h-[480px] overflow-hidden">

 { /* Legend */ }
 <div className="absolute top-4 left-4 bg-slate-900
 -center gap-3 text-xs">
 <div className="flex items-center gap-1.5">
 <div className="w-3 h-3 rounded-full bg-white" />
 </div>
 {activeAlgo === 'kruskal' ? (
 <>
 <div className="flex items-center gap-1.5">
 <div className="w-3 h-3 rounded-full bg-white" />
 </div>
 <div className="flex items-center gap-1.5">
 <div className="w-3 h-3 rounded-full bg-white" />
 </div>
 <div className="flex items-center gap-1.5">
 <div className="w-3 h-3 rounded-full bg-white" />
 </div>
 </>
) : (
 <div className="flex items-center gap-1.5">
 <div className="w-3 h-3 rounded-full bg-white" />
 </div>
)
 }
 </div>
 </div>
</>

```

```

const isInHeap = edge.status === 'in_heap'

return (
 <g key={edge.id}>
 <line
 x1={u.x + "%"}
 y1={u.y + "%"}
 x2={v.x + "%"}
 y2={v.y + "%"}
 stroke={strokeColor}
 strokeWidth={isInspecting ? 4.5 : 1}
 strokeDasharray={isRejected ? '4,4' : ''}
 className={"transition-all duration-300"}
 />
 <text
 x={({u.x + v.x} / 2 + "%")}
 y={({u.y + v.y} / 2 + "%")}
 fill={isInspecting ? '#f59e0b' : '#94a3b8'}
 fontSize="4.5"
 fontFamily="monospace"
 fontWeight="bold"
 textAnchor="middle"
 dominantBaseline="middle"
 className="select-none filter drop-shadow"
 dy="-1.5"
 >
 {edge.weight}
 </text>
 </g>
);
)}
</svg>

```

```

{/* HTML Overlay for Vertices */}
<div className="absolute inset-0 w-full h-full" >
 {effVertices.map(vertex => (
 <div

```

```

 <p className="text-[11px] font-bold text-slate-800">
 ✂ Приоритетная очередь (Min-Heap):
 </p>
 <div className="flex flex-wrap gap-1.5">
 {heapQueue.length === 0 ? (

 Очередь пуста

) : (
 heapQueue.map((item, idx) => (

 {item}

))
)}
 </div>
 </div>
)}

<div className="flex-1 p-4 overflow-y-auto">
 {logs.map((log, idx) => (
 <div
 key={idx}
 className={
 "p-4 rounded-xl border transition-all" +
 (log.type === 'success'
 ? 'bg-emerald-950/40 border-emerald-500'
 : log.type === 'warning'
 ? 'bg-amber-950/40 border-amber-500'
 : log.type === 'error'
 ? 'bg-rose-950/40 border-rose-500'
 : 'bg-slate-900/60 border-slate-700')
 }
 >
 <div className="flex items-center gap-2">
 {log.type === 'success' && <CheckCircle className="text-emerald-500"/>
 {log.type === 'warning' && <HelpCircle className="text-amber-500"/>
 {log.type === 'error' && <XCircle className="text-rose-500"/>
 {log.type === 'info' && <Play className="text-slate-500"/>
 </div>
 {log.message}
 >
)}
</div>
)

```



Борувка (Билет 20)

</button>

</div>

</div>

```
{activeCheatTab === 'kruskal' && (
```

```
<div className="grid grid-cols-1 lg:grid-cols-
```

```
<div className="lg:col-span-1 space-y-4">
```

```
<div className="bg-slate-950 p-4 rounded-
```

```
<p className="text-slate-400 text-xs for
```

```
<Clock className="w-3.5 h-3.5 text-in
```

```
</p>
```

```
<p className="text-2xl font-black text-
```

```
<p className="text-slate-400 text-xs mt
```

```
</div>
```

```
<div className="bg-slate-950 p-4 rounded-
```

```
<p className="text-slate-400 text-xs for
```

```
<Award className="w-3.5 h-3.5 text-in
```

```
</p>
```

```
<p className="text-2xl font-black text-
```

```
<p className="text-slate-400 text-xs mt
```

```
</div>
```

```
<div className="bg-slate-950 p-4 rounded-
```

```
<p className="text-slate-400 text-xs for
```

```
ожении:</p>
```

```
<p className="text-xs text-slate-300 le
```

```
ди и красим вершины в один цвет, бракуя цик
```

```
</div>
```

```
</div>
```

```
<div className="lg:col-span-2">
```

```
<div className="bg-slate-950 p-4 rounded-
```

```
<p className="text-slate-400 text-xs for
```

```
<Code className="w-3.5 h-3.5 text-ind
```

```
</p>
```

```
<pre className="text-xs text-emerald-40
```

```
80 flex-1">
```

```
{`# 1. Инициализируем DSU
```

```
def find(i):
```

```

 <p className="text-slate-400 text-xs mt-4">
 </div>
 <div className="bg-slate-950 p-4 rounded-1">
 <p className="text-slate-400 text-xs font-medium">
 ложении:</p>
 <p className="text-xs text-slate-300 leading-5">
 ие чистые вершины через самое дешевое доступное ребро.</p>
 </div>
 </div>
 <div className="lg:col-span-2">
 <div className="bg-slate-950 p-4 rounded-1">
 <p className="text-slate-400 text-xs font-medium">
 <Code className="w-3.5 h-3.5 text-emerald-400">
 </p>
 <pre className="text-xs text-emerald-400 font-mono">
80 flex-1">
{`import heapq

def prim(graph, start):
 visited = [False] * len(graph)
 heap = [(0, start, -1)] # (weight, node, parent)
 mst = []

 while heap:
 w, u, prev = heapq.heappop(heap)
 if visited[u]: continue
 visited[u] = True
 if prev != -1: mst.append((prev, u, w))

 for next_w, v in graph[u]:
 if not visited[v]:
 heapq.heappush(heap, (next_w, v, u))
 return mst`}</pre>
 </div>
 </div>
 </div>
})

```

```
 cheapest = [-1] * n
 for u, v, w in edges:
 ru, rv = find(u), find(v)
 if ru != rv:
 if cheapest[ru] == -1 or cheapest[ru][2] > w:
 cheapest[ru] = [u, v, w]
 if cheapest[rv] == -1 or cheapest[rv][2] > w:
 cheapest[rv] = [u, v, w]
 # Объединяем все колхозы по выбранным мостам
 for bridge in cheapest:
 if bridge != -1 and union(bridge[0], bridge[1]):
 mst.append(bridge)
 </pre>
</div>
</div>
 })
</div>
 })
};
};
```

---

ФАЙЛ 57/111: src/components/LiteGuide.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useMemo } from "react";
import { chapters, chapterTopics } from "../data/content";
import { getViz } from "../vizRegistry";
import { PAGE_SYNC } from "../data/vizSync";
import { linkifyTicketsHtml } from "../utils/ticketLink";
import { quizzes } from "../data/quizzes";
```

```
/**
```

```
 * «Лёгкая» текстовая версия пособия.
```

```
 *
```

```
 * Открывается по адресу `?lite=1` (весь список тем) или
```

```

 .lite-toc ol { margin: .4rem 0 0 1.2rem; }
 @media print {
 .lite-wrap { max-width: none; padding: 0; }
 h2 { page-break-before: always; }
 h2:first-of-type { page-break-before: avoid; }
 pre, table, details { page-break-inside: avoid; }
 }
 `;

/** Раскрываем все спойлеры и вычищаем служебную разметку
function toPlainTextHtml(html: string): string {
 return html
 .replace(/<details(?:[>]*\bopen\b)/g, "<details open")
 // кнопки/иконки внутри текста не нужны в текстовой версии
 .replace(/<button[\s\S]*?<\/button>/gi, "")
 .replace(/<svg[\s\S]*?<\/svg>/gi, "")
 // интерактивные подсказки терминов оставляем как есть
 .replace(/\\sdata-term-id="[^"]*" /g, "")
 .replace(/class="term-hint"[^"]*" /g, "")
 // ссылки на другие билеты должны вести в текстовую версию
 .replace(/href="\\?topic=/g, 'href="?lite=1&topic=')
 .replace(/class="ticket-link"/g, "");
}

interface Props {
 /** id одной темы; undefined – все темы подряд. */
 topicId?: string;
}

export const LiteGuide: React.FC<Props> = ({ topicId }) => {
 const list = useMemo(() => {
 if (!topicId) return chapters;
 const one = chapters.filter((c) => c.id === topicId);
 return one.length ? one : chapters;
 }, [topicId]);

 return (
 <div className="lite-wrap">

```

```

 dangerouslySetInnerHTML={{
 __html: linkifyTicketsHtml(toPlainTextHTML)
 }}
 />

 {viz && (
 <div className="lite-viz">
 <h4>Визуализация: {viz.title}</h4>
 {viz.hint && <p>{viz.hint}</p>}
 {syncKeys.map((k) => {
 const sync = PAGE_SYNC[k];
 const demo = k.includes("#") ? k.split("#")[0] : k;
 return (
 <div key={k}>
 <p>
 {demo ? `Вкладка «${demo}»` : ''}
 {sync.vizTitle ? ` — ${sync.vizTitle}` : ''}
 {sync.stepNote ? ` <> Шаг демонстрации` : ''}
 </p>
 {sync.variables.length > 0 && (

 {sync.variables.map((v) => (
 <li key={v.name}>
 <code>{v.name}</code> — {
 v.range ? ` (${v.range})` : ''
 }

))}

)}
 </div>
);
))}
 </div>
);
 }
 <p className="lite-meta" style={{ margin: 0; padding: 0; text-align: center; font-size: 0.8em; font-weight: normal; color: #ccc; border-top: 1px solid #ccc; border-bottom: 1px solid #ccc; margin-top: 10px; margin-bottom: 10px; }}>
 В интерактивной версии эта демонстрация перерисовывается по мере изменения переменных перерисовывают её напрямую
 </p>
</div>
)}

```

```

import type { DebugValue } from "../data/debugRunner";

interface LiveDistTableProps {
 /** dist из кода пользователя: имя вершины → расстояние */
 dist: Record<string, DebugValue> | null;
 /** prev/p – предок на кратчайшем пути (необязательно) */
 prev?: Record<string, DebugValue> | null;
 /** done/visited – зафиксированные вершины (необязательно) */
 done?: DebugValue[] | null;
 /** Идентификаторы вершин, нарисованных на схеме демо */
 demoIds: string[];
 /** Подпись таблицы (по умолчанию «Ваши расстояния»). */
 title?: string;
}

const fmt = (value: DebugValue | undefined): string => {
 if (value === undefined || value === null) return "—";
 if (typeof value === "number") {
 if (value === Number.POSITIVE_INFINITY) return "∞";
 if (value === Number.NEGATIVE_INFINITY) return "−∞";
 return String(value);
 }
 if (typeof value === "string") {
 const t = value.trim().toLowerCase();
 if (t === "inf" || t === "infinity") return "∞";
 if (t === "-inf" || t === "-infinity") return "−∞";
 return value;
 }
 return String(value);
};

/**
 * Таблица «ваших» расстояний, когда имена вершин в коде
 *
 * Схема демонстрации нарисована на фиксированном наборе
 * S...F у Форда–Беллмана). Если в коде пользователя верши
 * (0...6, v1...v9), картинка физически не может их показат
 * просто пропадали, и казалось, что визуализация живёт

```

```

 ? "border-emerald-500/40 bg-emerald-500"
 : "border-slate-700 bg-slate-900/70 text-slate-500"
 }` }
 title={fixed ? "вершина зафиксирована (доказано)" : null}
 >
 <b className={fixed ? "text-emerald-300" : "text-slate-500"}>
 =
 {fmt(dist[key])}
 {parent && parent !== "-" ?
 {fixed ?

 } : null}

 </div>
 </div>
);
}

```

ФАЙЛ 59/111: src/components/MnemonicCards.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import dijkstraImg from '../assets/dijkstra-freakazoid.png';
import bellmanImg from '../assets/bellman-ford-batman.jpg';
import floydImg from '../assets/floyd-network.jpg';

```

```

const cards = [
 {
 img: dijkstraImg,
 alt: 'Декстер Дуглас и Фриказоид (Дейкстра)',
 },

```

```
export default function MnemonicCards() {
 return (
 <div className="grid grid-cols-1 md:grid-cols-3 gap-4">
 {cards.map((c, i) => (
 <div key={i} className={`bg-slate-800 rounded-2xl p-4">
 <div className="h-48 overflow-hidden">
 <img src={c.img} alt={c.alt}
 className="w-full h-full object-cover group-hover:scale-110 transition duration-300"/>
 </div>
 <div>
 <h3 className="text-lg font-bold mb-1 ${c.isCurrent ? 'text-white' : 'text-slate-400'}">
 {c.title}
 </h3>
 <p className="text-slate-400 text-sm">
 {c.desc}
 {c.status}

 </p>
 <div className={`mt-3 text-xs font-mono px-2 ${c.isCurrent ? 'text-white' : 'text-slate-400'}">
 {c.content}
 </div>
 </div>
 </div>
))}
 </div>
);
}
```

---

ФАЙЛ 60/111: src/components/MobileToc.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useEffect, useState } from "react";
import { List, X } from "lucide-react";
import { Sidebar } from "../Sidebar";

interface Props {
 selectedChapterId: string;
 setSelectedChapterId: (id: string) => void;
 /** Заголовок текущей темы – показываем в кнопке, что это текущая */
 currentTitle: string;
 index: number;
```



```

 <span className="block text-[13px] font-bol

 </button>
 </div>

 {open && (
 <div className="lg:hidden fixed inset-0 z-[90]">
 <div className="absolute inset-0 bg-slate-950"
 <div className="absolute inset-y-0 left-0 w-[100%] h-[100%] flex flex-direction-column justify-center items-center text-white font-size-[18px] ease-out">
 <div className="flex items-center justify-between w-full">

 <button
 onClick={() => setOpen(false)}
 className="p-2 -mr-2 text-slate-400 hover:text-white"
 aria-label="Заккрыть содержание"
 >
 <X className="w-5 h-5" />
 </button>
 </div>
 <div className="flex-1 min-h-0">
 <Sidebar
 selectedChapterId={selectedChapterId}
 setSelectedChapterId={setSelectedChapterId}
 onNavigate={() => setOpen(false)}
 />
 </div>
 </div>
 </div>
 </div>
)}
</>
);
};

```



```

 rel="noreferrer"
 className="flex items-center gap-2 px-4 py-2 border border-sky-500/30 transition-all"
 title="Скачать полный TXT файл со всеми файлами"
 >
 <FileText className="w-3.5 h-3.5" />

 <button
 id="download-html-btn"
 onClick={saveBook}
 disabled={busy}
 className="flex items-center gap-2 w-32 h-10 text-white hover:bg-amber-600/20 border border-amber-500/20 rounded"
 title="Скачать всё пособие одним файлом"
 >
 {busy ? <Loader2 className="w-3.5 h-3.5" /> : <DownloadIcon />}
 </button>
 </div>
</details>
</div>
</div>
</div>
</header>
);
};

```

ФАЙЛ 62/111: src/components/PlanarColoringViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import React, { useCallback, useContext, useEffect, useLayoutEffect, useState } from 'react';
import { ChevronLeft, ChevronRight, Play, Pause, Rotate } from 'lucide-react';
import { PlanarityDemo } from './PlanarityDemo';
import { VizChapterContext, emitVizDemo, useVizRuntime, useVizState } from './VizChapterContext';

```



```

 });
 return frames;
}

/** Проверка двудольности BFS-ом: 2 краски нет нечётных
function bipartiteFrames(g: Graph): Frame[] {
 const frames: Frame[] = [];
 const color = new Array(g.n).fill(0);
 const queue: number[] = [];
 let conflict: [number, number] | null = null;
 frames.push({
 title: "Старт: пробуем покрасить в 2 цвета",
 note: "BFS красит вершину в цвет, противоположный р
 двудольный ($\chi > 2$).",
 color: [...color], v: null, u: null, e: null,
 });
 for (let start = 0; start < g.n && !conflict; start++)
 if (color[start] !== 0) continue;
 color[start] = 1;
 queue.push(start);
 frames.push({
 title: `Запускаем BFS из ${g.names[start]}: цвет
 note: `Вершина ${g.names[start]} ещё не покрашена
 и 2.`,
 color: [...color], v: start, u: null, e: null,
 });
 while (queue.length && !conflict) {
 const v = queue.shift() as number;
 for (const u of g.adj[v]) {
 if (color[u] === 0) {
 color[u] = 3 - color[v];
 queue.push(u);
 frames.push({
 title: `${g.names[v]} → ${g.names[u]}: цвет
 note: `Сосед не покрашен: color[${g.names[u]
 x})).join(", ") || "пуста"}].`,
 color: [...color], v, u, e: null,
 });
 }
 }
 }
 }
}

```

```

const usedB = Array.from(inc[b].values());
const used = Array.from(new Set([...usedA, ...usedB]));
let c = 1;
while (used.includes(c)) c += 1;
ecolor[i] = c;
inc[a].set(i, c);
inc[b].set(i, c);
frames.push({
 title: `Ребро (${g.names[a]}, ${g.names[b]}): цве
 note: `В вершине ${g.names[a]} заняты цвета ${us
 y) => x - y).join(", ") || "-"}}. Наименьший св
 да.`;
 color: new Array(g.n).fill(0), ecolor: [...ecolor
});
});
const chiE = Math.max(...ecolor);
frames.push({
 title: `Готово:  $\chi' = \chi_E$  при  $\Delta = \Delta$ `,
 note:
 g.key === "bip"
 ? `Двудольный граф: $\chi' = \Delta$ — это теор
 сс 1 по Визингу.`
 : ` $\chi' = \chi_E$. Теорема Визинга: $\Delta \leq \chi' \leq \Delta + 1$
 $\chi' = \Delta + 1$ — классом 2 (например, K3 и любой
 color: new Array(g.n).fill(0), ecolor: [...ecolor],
});
return frames;
}

```

```

const TABS: { id: Tab; label: string; icon: React.React
 { id: "planar", label: "1 • Планарность: K5 и K3,3",
 { id: "greedy", label: "2 • Покраска вершин ($\chi \leq \Delta + 1$)",
 { id: "bipartite", label: "3 • Двудольность (2 краски)",
 { id: "edges", label: "4 • Покраска рёбер (Визинг, Кё
];

```

```

export const PlanarColoringViz: React.FC = () => {
 const chapterId = useContext(VizChapterContext);

```

```

/* живые переменные компилятора */
const liveStep = vars ? vizNumber(vars.step) ?? vizNumber(maxStep) : null;
useEffect(() => {
 if (liveStep === null || !Number.isFinite(liveStep))
 setStep(Math.max(0, Math.min(maxStep, liveStep)));
}, [liveStep, maxStep]);

const liveColor = vars ? vizArray(vars.color) ?? vizArray(maxStep) : null;
const liveEcolor = vars ? vizArray(vars.ecolor) ?? vizArray(maxStep) : null;
const liveV = vars ? vizNumber(vars.v) : null;
const liveU = vars ? vizNumber(vars.u) : null;
const liveE = vars ? vizNumber(vars.e) : null;
const liveChi = vars ? vizNumber(vars.chi) ?? vizNumber(maxStep) : null;
const linked = Boolean(liveColor || liveEcolor || (liveV || liveU || liveE || liveChi));

const frame = frames[Math.min(step, maxStep)] ?? ({ t: 0, color: {}, ecolor: {} });
const nodeColor = (id: number): number => {
 if (liveColor && typeof liveColor[id] === "number")
 return frame.color?.[id] ?? 0;
};
const edgeColor = (i: number): number => {
 if (liveEcolor && typeof liveEcolor[i] === "number")
 return frame.ecolor?.[i] ?? 0;
};
const curV = liveV !== null && Number.isFinite(liveV);
const curU = liveU !== null && Number.isFinite(liveU);
const curE = liveE !== null && Number.isFinite(liveE);
const delta = Math.max(...graph.adj.map((a) => a.length));
const usedColors = Array.from(new Set(graph.pos.map((p) => p.color)));
const usedEcolors = Array.from(new Set(graph.edges.map((e) => e.ecolor)));

const go = useCallback((d: number) => {
 setPlaying(false);
 setStep((s) => Math.max(0, Math.min(maxStep, s + d)));
}, [maxStep]);

if (tab === "planar") {

```





```

 </div>
 <div className="bg-slate-900/70 border border
 {tab === "greedy" && <>Жадная покраска: <b
 : <b className="text-slate-200"> $\chi \leq \Delta$,
 в $\chi \leq 5$ (доказывается через $E \leq 3V - 6$ и це
 ром).</>}
 {tab === "bipartite" && <>Двудольный граф
 /DFS за $O(V + E)$. Распознавание 3-раскрашив
 {tab === "edges" && <>Рёберная покраска: <b
 Δ (Кёниг). Задача « $\chi' = \Delta$?» NP-полна даже
 </div>
 </div>
</div>

<div className="mt-3 flex flex-wrap items-center"
 <span className="text-[10px] uppercase tracking
 /> из компилятора:
 {vars && linked ? (
 ["v", "u", "e", "color", "ecolor", "chi"].map
 vars[name] === undefined ? null : (
 <span key={name} className="text-[11px] f
 y-1">
 {name} = {JSON.stringify(vars[name]).sl

)
)
) : (

 открой панель Python: массив <code classNam
 igo-300">ecolor</code> для рёбер) и вершина

)}
</div>
</div>
);
};

```

```

const Header: React.FC = () => {

```

```

<div className="space-y-4">
 <h4 className="text-base font-bold text-white">K5
 <p className="text-xs text-slate-400">Просто смотри

<div className="grid grid-cols-1 md:grid-cols-2 gap-4">
 { /* K5 */ }
 <div className="bg-slate-950 rounded-xl p-4 border border-slate-800">
 <div className="absolute top-2 left-2 bg-slate-900/30 w-auto z-20">K5 – НЕПЛАНАРЕН</div>
 <svg className="w-full h-full absolute inset-0" >
 {(() => {
 const pts = [
 {id:0,x:160,y:40},
 {id:1,x:270,y:100},
 {id:2,x:230,y:230},
 {id:3,x:90,y:230},
 {id:4,x:50,y:100}
];
 const edges = [
 [0,1],[0,2],[0,3],[0,4],
 [1,2],[1,3],[1,4],
 [2,3],[2,4],
 [3,4]
];
 function findPt(id:number) { return pts.find(p => p.id === id); }
 function intersect(a:number,b:number,c:number,d:number) {
 if (a===c||a===d||b===c||b===d) return true;
 const p1=findPt(a),p2=findPt(b),p3=findPt(c),p4=findPt(d);
 function ccw(ax:number,ay:number,bx:number,by:number,cx:number,cy:number) {
 return (cy-ay)*(bx-ax)>(by-ay)*(cx-ax);
 }
 return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.y)<0
 && ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y)<0;
 }
 const crosses: number[] = [];
 for(let i=0;i<edges.length;i++) {
 for(let j=i+1;j<edges.length;j++) {
 if (intersect(edges[i][0],edges[i][1],edges[j][0],edges[j][1]))

```

```

const edges = [
 [0,3],[0,4],[0,5],
 [1,3],[1,4],[1,5],
 [2,3],[2,4],[2,5]
];
function findPt(id:number) { return pts.f
function intersect(a:number,b:number,c:nu
 if (a===c||a===d||b===c||b===d) return
 const p1=findPt(a),p2=findPt(b),p3=find
 if (!p1||!p2||!p3||!p4) return false;
 function ccw(ax:number,ay:number,bx:num
 return (cy-ay)*(bx-ax)>(by-ay)*(cx-ax
 }
 return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.
 && ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y)
}
const crosses:number[]=[];
for(let i=0;i<edges.length;i++) for(let j
 if (intersect(edges[i][0],edges[i][1],e
 if (!crosses.includes(i)) crosses.push
 if (!crosses.includes(j)) crosses.push
 }
}
const lines = []; const circles = [];
for(let i=0;i<edges.length;i++) {
 const p1=findPt(edges[i][0]),p2=findPt(
 const xing=crosses.includes(i);
 lines.push(<line key={`l${i}`} x1={p1.x
 stroke={xing?"#f43f5e":"#4f46e5"} str
}
const labels = ["\u04141","\u04142","\u04
for(const p of pts) {
 circles.push(<g key={`c${p.id}`}>
 <circle cx={p.x} cy={p.y} r={8} fill=
 <text x={p.x} y={p.y+3} textAnchor="m
 </g>);
}
return <>{lines}{circles}</>;

```

```

 Loader2,
 Pause,
 Pencil,
 Play,
 Plus,
 RotateCcw,
 SkipBack,
 SkipForward,
 Terminal,
 Unlink,
 Variable,
 X,
} from "lucide-react";
import { buildInitTemplate, buildSyncTemplate, getPageS
import { clearVizState, emitVizState, onVizDemo } from
import {
 baseVarName,
 buildDebugRunner,
 changedVars,
 DEBUG_STEP_CAP,
 orderVars,
 type DebugResult,
} from "../data/debugRunner";
import { Tooltip } from "../Tooltip";

const PYODIDE_VERSION = "0.27.7";
const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyo
const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyo
const PLAYBACK_SPEED_KEY = "pythonCompilerPlaybackSpeed
const PLAYBACK_SPEEDS = [0.25, 0.5, 1, 1.5, 2, 4] as co

interface PythonCompilerProps {
 /** Страница, с которой синхронизируется редактор. */
 chapterId: string;
 chapterTitle: string;
 /** Размер панели (px) управляется родителем и сохран
 width?: number;
 height?: number | null;

```

```

 if (!runtimePromise) {
 runtimePromise = import(/* @vite-ignore */ PYODIDE_MODULE_NAME, {
 (module as PyodideModule).loadPyodide({ indexURL: PYODIDE_URL })
 });
 }
 return runtimePromise;
}

function errorText(error: unknown) {
 if (error instanceof Error) return error.message;
 return String(error);
}

/**
 * Предел вывода в панель «Результат».
 *
 * Трасса ограничена по ЧИСЛУ шагов (DEBUG_STEP_CAP), но не по
 * напечатанной строки: print("x" * 10_000_000) или print("x" * 10_000_000)
 * — это один шаг, после которого весь текст уезжал в <div> и не было видно
 * Обрезаем по границе строки и честно пишем, сколько э
 */
const OUTPUT_CHAR_CAP = 20_000;

function clipOutput(text: string): string {
 if (text.length <= OUTPUT_CHAR_CAP) return text;
 const cut = text.slice(0, OUTPUT_CHAR_CAP);
 const lastBreak = cut.lastIndexOf("\n");
 const head = lastBreak > OUTPUT_CHAR_CAP / 2 ? cut.slice(0, lastBreak) : cut;
 return `${head}\n... вывод обрезан: показаны первые ${head.length} символов`;
}

/** -----
 * Шаблон синхронизации: короткая шапка + минимальный шаблон
 * скелет, шаг-в-шаг с подсветкой визуализации (см. data-visualizer.js)
 */

/** -----
 * Шпаргалка Python: команды сортировки, циклы, структуры
 */

```

```

 code: "i = 1\nwhile i < 5:\n print(f\"итераци\n },\n {\n label: \"for i, x in enumerate(...)\",\n tip: \"Индекс и элемент одновременно: i — позици\n code: \"for i, x in enumerate(['a\", \"b\", \"c\n },\n},\n},\n{\n group: \"Структуры\",\n items: [\n {\n label: \"Стек (список)\",\n tip: \"LIFO, как рекурсия DFS: append = push, pop\n code: 'st = []\nst.append(\"A\") # push\nst.ap\n },\n {\n label: \"Очередь (deque)\",\n tip: \"FIFO для BFS: popleft() за O(1). list.pop\n code: 'from collections import deque\nq = deque\n t(q))\n',\n },\n {\n label: \"Куча (heapq)\",\n tip: \"Мин-куча: (вес, вершина) — как очередь пр\n code: 'import heapq\nh = []\nheapq.heappush(h,\n w, v)\n',\n },\n {\n label: \"Матрица n × m\",\n tip: \"Таблица расстояний: dist[i][j], бесконечн\n code: 'n, m = 3, 4\ndist = [[float(\"inf\")] * m\n },\n {\n label: \"Множество visited\",\n tip: \"Посещённые вершины: in/add за O(1).\",\n code: 'visited = set()\nvisited.add(\"A\")\nprint

```

```

]],
 [sync]
);
 const syncVarSet = useMemo(() => new Set(syncVarBases)
 /** Полный эталон: глаз сразу вставляет его в обычный
 const template = useMemo(() => buildSyncTemplate(chapterId)
 /** Дефолт редактора: только инициализация демо; она
 const initTemplate = useMemo(() => buildInitTemplate(chapterId)

 const [code, setCode] = useState(() => {
 const saved = savedEditors.get(chapterId);
 if (!saved) return initTemplate;
 if (saved.code === saved.base || saved.code.trim() === "")
 return saved.code;
 });
 /** «Глаз» – переключатель: эталонный код демо ↔ инициализация
 const [refOn, setRefOn] = useState(() => {
 const saved = savedEditors.get(chapterId);
 return !!saved && saved.code.trim() !== "" && saved.code !== saved.base
 });
 const [stdin, setStdin] = useState("");
 const [output, setOutput] = useState("Код выполняется");
 const [running, setRunning] = useState(false);
 const [runtimeReady, setRuntimeReady] = useState(false);
 const [runtimeMessage, setRuntimeMessage] = useState("");
 /** Принудительный повтор (глаз / Ctrl+Enter), даже если код не
 const [traceRequest, setTraceRequest] = useState(0);
 const [copied, setCopied] = useState(false);
 const [stripTab, setStripTab] = useState<"vars" | "highlight">("vars");
 const [outputOpen, setOutputOpen] = useState(true);
 const editorRef = useRef<HTMLTextAreaElement>(null);
 /**
 * Панель жива? Заккрытие (крестик, «глаз» в шапке, см.
 * компонент, но `runPythonAsync` к тому моменту может
 * этого флага доехавший до конца прогон опубликовал
 * после закрытия: панель исчезла, а визуализация прогона
 * кода, которого на экране больше нет.
 */

```

```
// Смена страницы (или вкладки демо): нетронутый редактор
// инициализацию; свой код не затираем – предлагаем сменить
useEffect(() => {
 const prev = prevTplRef.current;
 if (initTemplate === prev.base && template === prev.full) {
 prevTplRef.current = { base: initTemplate, full: template };
 setPlaying(false);
 setEditHold(null);
 setDebug(null); // трасса ссылается на строки старого кода
 setRefOn(false);
 if (code === prev.base || code === prev.full || code === prev.full) {
 setCode(initTemplate);
 setDesyncedFrom(null);
 } else {
 setDesyncedFrom(chapterId);
 }
 }
 // eslint-disable-next-line react-hooks/exhaustive-deps
}, [initTemplate, template]);

const resync = useCallback(() => {
 prevTplRef.current = { base: initTemplate, full: template };
 setCode(initTemplate);
 setRefOn(false);
 setDesyncedFrom(null);
 setStripTab("vars");
 setPlaying(false);
 setEditHold(null);
 setDebug(null);
 setTraceRequest((n) => n + 1);
}, [initTemplate, template]);

const copyCode = useCallback(async () => {
 try {
 await navigator.clipboard.writeText(code);
 setCopied(true);
 setTimeout(() => setCopied(false), 1600);
 } catch {
```



```

 const changed = changedVars(step?.locals, locals);
 return {
 line: step?.line,
 func: step?.func,
 locals,
 variables,
 changed: Object.keys(changed).filter((name) => ch
 };
 }, []));

const publishDebug = useCallback(
 (result: DebugResult, idx: number, source: string) => {
 if (!aliveRef.current) return; // панель закрыта
 if (!hasViz || result.steps.length === 0) return;
 const snapshot = snapshotAt(result, idx, source);
 emitVizState(chapterId, {
 line: snapshot.line,
 func: snapshot.func,
 variables: snapshot.variables,
 changed: snapshot.changed,
 });
 },
 [chapterId, hasViz, snapshotAt]
);

// Единственная точка публикации: любое перемещение т
// визуализации согласованный снимок, без side-effect
useEffect(() => {
 if (debug) publishDebug(debug.result, debug.idx, de
}, [debug, publishDebug]);

const goDebug = useCallback(
 (nextIdx: number) => {
 setDebug((d) => {
 if (!d || d.result.steps.length === 0) return d
 const idx = Math.max(0, Math.min(d.result.steps
 return { ...d, idx };
 });
 });

```

```

const source = code;
const input = stdin;
const request = traceRequest;
setRunning(true);
setPlaying(false);
setEditHold(null);
setDebug(null);
clearVizState(chapterId);
setOutputOpen(false);

// Пустой редактор: Pyodide не гоняем, панель в режим
// связь с визуализацией снимаем — демо показывает
if (source.trim() === "") {
 setDebug({ result: { steps: [], error: "", truncated: false } });
 setOutput("В редакторе пусто — выполнять нечего.\n" +
 «глаз».");
 setRuntimeMessage("Пустой код: связь с визуализацией снята");
 setOutputOpen(true);
 setRunning(false);
 return;
}

setRuntimeMessage(runtimeReady ? "Обновляю трассу..." : "Загружаю код");

const stdout: string[] = [];
const stderr: string[] = [];
const inputLines = input.split(/\r?\n/);

try {
 const runtime = await getPythonRuntime();
 setRuntimeReady(true);
 runtime.setStdout({ batched: (message) => stdout.push(message) });
 runtime.setStderr({ batched: (message) => stderr.push(message) });
 runtime.setStdin({ stdin: () => (inputLines.length > 0 ? inputLines.shift() : null) });

 const raw = await runtime.runPythonAsync(buildDebugCode);
 // Панель могли закрыть, пока Python считал: выходим
 if (!aliveRef.current) return;
} catch (e) {
 setRuntimeMessage("Ошибка: " + e);
 setOutputOpen(true);
 setRunning(false);
 return;
}

```

```

 if (!aliveRef.current) return;
 const message = errorText(error);
 setDebug({ result: { steps: [], error: message, t
 clearVizState(chapterId);
 setOutputOpen(true);
 setOutput(`Ошибка:\n${message}`);
 setRuntimeMessage("Не удалось выполнить код – про
 } finally {
 setRunning(false);
 }
}, [chapterId, code, stdin, traceRequest, running, ru

// Код выполняется сам: debounce даёт спокойно допечата
// изменился во время загрузки Pyodide, после завершения
useEffect(() => {
 if (running || editHold === code) return;
 if (debug?.src === code && debug.input === stdin &&
 const timer = window.setTimeout(() => void debugRun
 return () => window.clearTimeout(timer);
}, [code, stdin, traceRequest, running, runtimeReady,

// Стрелки ← → в режиме отладки листают ШАГИ (а не стр
useEffect(() => {
 if (!debug || debug.result.steps.length === 0) retu
 const onKey = (e: KeyboardEvent) => {
 const target = e.target as HTMLElement | null;
 if (target && /^(INPUT|TEXTAREA|SELECT)$/.test(ta
 if (e.metaKey || e.ctrlKey || e.altKey) return;
 if (e.key === "ArrowRight") {
 e.stopPropagation();
 e.preventDefault();
 setPlaying(false);
 stepBy(1);
 }
 if (e.key === "ArrowLeft") {
 e.stopPropagation();
 e.preventDefault();
 setPlaying(false);

```

```

// трейсером на возврате из фрейма). Значит на строке
const shownLocals = useMemo(() => {
 if (!debug || !curDebug?.step) return {};
 return snapshotAt(debug.result, debug.idx, debug.src);
}, [debug, curDebug, snapshotAt]);
const debugChanged = curDebug?.step ? changedVars(curDebug) : false;
const debugOrdered = curDebug?.step ? orderVars(shownLocals) : [];

/** Одна и та же кнопка запуска всегда видна: из редактора
 * проигрывает её, в отладчике переключает play/pause */
const runOrTogglePlayback = useCallback(() => {
 if (running) return;
 if (!debug || debug.result.steps.length === 0) {
 void debugRun(true);
 return;
 }
 if (playing) {
 setPlaying(false);
 return;
 }
 if (debug.idx >= debug.result.steps.length - 1) stepToNext();
 setPlaying(true);
}, [debug, debugRun, playing, running, stepTo]);

/** Вставка сниппета из шпаргалки в позицию курсора.
const insertSnippet = useCallback(({snippet: string}) => {
 const ta = editorRef.current;
 setCode((current) => {
 if (!ta) return current + (current.endsWith("\n") ? "" : "\n");
 const start = ta.selectionStart ?? current.length;
 const end = ta.selectionEnd ?? start;
 const padded = (start > 0 && !current.slice(0, start).endsWith("\n")) ? "\n" : "";
 const next = current.slice(0, start) + padded + snippet + current.slice(end, current.length);
 requestAnimationFrame(() => {
 ta.focus();
 ta.selectionStart = ta.selectionEnd = start + padded.length;
 });
 return next;
 });
});

```

```

 el.addEventListener("pointerup", onUp as EventListener)
 el.addEventListener("pointercancel", onUp as EventListener)
 })
>
 <div className="absolute left-1/2 top-1/2 h-10 w-10
 dingo-400 transition-colors" />
</div>
})

{/* Левый край меняет ширину на десктопе. */}
{onWidthChange && (
 <div
 role="separator"
 aria-orientation="vertical"
 aria-label="Потяните, чтобы изменить ширину панели"
 title="Потяните, чтобы изменить ширину панели"
 className="hidden lg:block absolute top-0 bottom-0
 onPointerDown={(e) => {
 const el = e.currentTarget;
 el.setPointerCapture(e.pointerId);
 const startX = e.clientX;
 const startW = width ?? 560;
 const onMove = (ev: PointerEvent) => {
 const next = Math.round(startW + (startX - ev.clientX) * 2);
 onWidthChange(Math.max(380, Math.min(next, 1000)));
 };
 const onUp = () => {
 el.removeEventListener("pointermove", onMove);
 el.removeEventListener("pointerup", onUp);
 };
 el.addEventListener("pointermove", onMove);
 el.addEventListener("pointerup", onUp as EventListener);
 })
 >
 <div className="absolute left-1/2 top-1/2 -transform: rotate(-90deg);
 dingo-500 transition-colors" />
 </div>
)}
})

```

```

 </button>
 </Tooltip>
 <Tooltip side="bottom" content="Вернуть коммент" >
 <button
 type="button"
 onClick={resync}
 className="p-1.5 rounded-lg text-slate-400"
 aria-label="Сбросить к шаблону страницы"
 >
 <RotateCcw className="w-4 h-4" />
 </button>
 </Tooltip>
 <Tooltip side="bottom" content="Скрыть панель" >
 <button
 type="button"
 onClick={onClose}
 className="p-1.5 rounded-lg text-slate-400"
 aria-label="Заккрыть компилятор"
 >
 <X className="w-4 h-4" />
 </button>
 </Tooltip>
 <Tooltip
 side="bottom"
 content={
 refOn
 ? "Убрать эталонный код: редактор станет
обственному встроенному демо. Кнопка ↺ вернёт
 : "Вставить полный эталонный код этого
удет заменён."
 }
 >
 <button
 type="button"
 onClick={() => {
 const next = !refOn;
 setRefOn(next);
 // «Убрать эталон» — значит убрать код

```

```

 <option key={speed} value={speed}>{speed}
 </option>
 </select>
 </label>
</div>
</div>

```

{/\* — Мини-вкладки: переменные страницы / шпарга.

```

<div className="shrink-0 border-b border-slate-800" >
 <div className="flex items-center gap-1 px-2 pt-2" >
 <button
 type="button"
 onClick={() => setStripTab("vars")}
 className={`inline-flex items-center gap-1.5`}
 stripTab === "vars" ? "bg-slate-800 text-white" : "text-slate-400"
 >
 <Variable className="w-3.5 h-3.5" /> Переменные
 </button>
 <button
 type="button"
 onClick={() => setStripTab("hints")}
 className={`inline-flex items-center gap-1.5`}
 stripTab === "hints" ? "bg-slate-800 text-white" : "text-slate-400"
 >
 <HelpCircle className="w-3.5 h-3.5" /> Подсказки
 </button>
 </div>
 <div className="ml-auto pr-1" >
 <Tooltip
 side="bottom"
 content={
 hasViz
 ? "Связь идёт по именам ниже: реальный тип"
 : "На выбранной странице нет интерактивных элементов"
 }
 >

```

```

т заменён)."
 >
 <button
 type="button"
 onClick={resync}
 className="flex items-center gap-1.5
0/40 text-amber-400 hover:bg-amber-500/20 t
 >
 <Unlink className="w-3.5 h-3.5 shrink
Страница сменилась – подставить её пер
 </button>
 </Tooltip>
 })
 {hasViz && (
 <div className="flex flex-wrap items-cent
 {sync.variables.map((v) => (
 <Tooltip
 key={v.name}
 side="bottom"
 content={

 <span className="block font-bol
 <code className="font-mono te

 {v.role
 {v.range && <span className="bl

 }
 }
 >
 <code
 tabIndex={0}
 className="cursor-help rounded-md
font-bold text-emerald-300 hover:bg-emeral
 >
 {v.name}
 </code>
 </Tooltip>
)})
)})
)})

```



```

 <p className="text-[10px] text-slate-600">H
 </div>
 })
</div>

{/* — Редактор (всегда на месте) + трасса + ввод
<div className="flex-1 min-h-0 flex flex-col">
 {/* Заголовок редактора: сразу видно, что выполн
 <div className="shrink-0 flex items-center gap-1
 <Pencil className="w-3.5 h-3.5 text-emerald-400" />

 Редактор • выполняется этот текст

 {state && (
 <span
 className="inline-flex items-center gap-1
 ont-bold text-amber-300 shrink-0"
 title="Трасса ниже построена по предыдущей"
 >
 <Loader2 className="w-2.5 h-2.5 animate-spin" />

)}

 {code.trim() === "" ? "пусто — ничего не выведено" : ""}

 </div>

 <textarea
 ref={editorRef}
 value={code}
 onChange={(event) => {
 setEditHold(null);
 setRefOn(false);
 setCode(event.target.value);
 }}
 onKeyDown={(event) => {
 if ((event.ctrlKey || event.metaKey) && event.key === 'Enter') {
 event.preventDefault();
 }
 }}
 />
</div>

```

```

bled={!curDebug?.prev} className="p-1 rounded-
tion-colors">
 <ChevronLeft className="w-4 h-4" />
 </button>
</Tooltip>
<Tooltip content="Шаг вперёд (→) – автопр
 <button type="button" aria-label="Шаг в
abled={!curDebug?.step || debug.idx >= debug
e hover:bg-slate-800 disabled:opacity-40 tra
 <ChevronRight className="w-4 h-4" />
 </button>
</Tooltip>
<Tooltip content="В конец трассы">
 <button type="button" aria-label="В кон
result.steps.length - 1); }} disabled={!cur
d text-slate-400 hover:text-white hover:bg-
 <SkipForward className="w-4 h-4" />
 </button>
</Tooltip>
<span className="ml-1.5 text-[10px] text-
 {curDebug?.step ? (
 <>
 шаг {debug.idx + 1}/{debug.result.s
 {curDebug.step.func !== "<module>"
 </>
) : (
 "шагов нет – исправьте ошибку и начини
)}

{debug.result.truncated && (
 <Tooltip content={`Выполнение остановлен
 <span tabIndex={0} className="cursor-l
rounded-full px-1.5 py-0.5">
 ЛИМИТ

 </Tooltip>
)}
<Tooltip content={hasViz ? "Визуализация

```

```

 </Tooltip>
 </div>

 {traceOpen && (
 <>
 {/* листинг с подсветкой текущей строки */}
 <div ref={listRef} className="flex-1 min-h-100" >
 {debug.src.split("\n").map(({line, i}) => {
 const ln = i + 1;
 const active = ln === curDebug?.step?.line;
 return (
 <div
 key={i}
 data-line={ln}
 className={`flex whitespace-pre px-1 py-1
 active ? "bg-emerald-500/15 text-emerald-300" : ""
 }`}
 >
 <span
 className={`w-7 shrink-0 select-none
 active ? "text-emerald-300" : "text-emerald-500"
 }`}
 >
 {active ? "▶" : ln}

 {line || " "}
 </div>
);
 })}
 </div>

 {/* панель переменных – «Variables», как в VS Code */}
 <div className="shrink-0 max-h-[34%] overflow-hidden" >
 <div className="flex items-center gap-1.5" >
 <Variable className="w-3 h-3 text-indigo-300" />
 Переменные на этом шаге

 </div>
 </div>

```

```

 py-2 font-mono text-[11px] leading-snug text-slate-600
 {debug.result.error}
 </pre>
) : null}
 <div className="shrink-0 border-t border-slate-800" >
 <label htmlFor="python-stdin" className="text-slate-600">
 stdin для input()
 </label>
 <input
 id="python-stdin"
 value={stdin}
 onChange={(event) => {
 setEditHold(null);
 setStdin(event.target.value);
 }}
 spellCheck={false}
 placeholder="строки ввода через ↵"
 className="w-full bg-transparent font-mono text-slate-600"
 />
 </div>
</div>

```

```

{/* — Вывод ————— */}
<div className="shrink-0 border-t border-slate-800" >
 <button
 type="button"
 onClick={() => setOutputOpen((o) => !o)}
 className="flex items-center justify-between text-slate-600"
 >

 <Terminal className="w-3.5 h-3.5 text-emerald-500" />

 {runtimeReady && <CheckCircle2 className="w-3.5 h-3.5 text-emerald-500" />}
 {outputOpen ? "открыт" : "закрыт"}

 </button>

```

```

 рцию борща!' },
 { name: 'Кот Борис', emoji: ' ', color: 'from-amber',
 bubbleText: 'Пожалуйста, пришлите мне борщ, пожалуйста.' },
 { name: 'Бабушка Люба', emoji: ' ', color: 'from-rose',
 bubbleText: 'Бабушка Люба просит борща.' },
 { name: 'Инопланетянин', emoji: ' ', color: 'from-emerald',
 bubbleText: 'Пожалуйста, пришлите мне борщ, пожалуйста.' },
 { name: 'Бизнесмен', emoji: ' ', color: 'from-slate',
 bubbleText: 'Пожалуйста, пришлите мне борщ, пожалуйста.' },
];

```

```
let counter = 3;
```

```

export const QueueViz: React.FC = () => {
 const [queue, setQueue] = useState<Customer[]>([
 { id: 'q1', name: 'Студент Вася', emoji: ' ', color: 'from-amber',
 bubbleText: 'Пожалуйста, пришлите мне борщ, пожалуйста.', animState: 'idle' },
 { id: 'q2', name: 'Кодер Семён', emoji: ' ', color: 'from-emerald',
 bubbleText: 'Пожалуйста, пришлите мне борщ, пожалуйста.', animState: 'idle' },
 { id: 'q3', name: 'Кот Борис', emoji: ' ', color: 'from-slate',
 bubbleText: 'Пожалуйста, пришлите мне борщ, пожалуйста.', animState: 'idle' },
]));

```

```

const runtime = useVizRuntime();
const liveQ = vizArray(runtime?.variables?.q);
const liveV = vizString(runtime?.variables?.v);
const displayQueue: Customer[] = liveQ
 ? liveQ.map((value, index) => ({
 id: `python-${index}`,
 name: String(value),
 emoji: " ",
 color: "from-emerald-500/20 to-emerald-600/20",
 bubbleText: `q[${index}] из Python`,
 animState: "idle",
 }))
 : queue;

```

```
const [servedHistory, setServedHistory] = useState<st
```

```

 addLog('⚠ DEQUEUE: В очереди никого нет! Повар ск
 setTimeout(() => setShake(false), 400);
 return;
 }

 setIsServing(true);
 const headGuest = queue[0];

 addLog(` DEQUEUE: Повар наливает суп первому – ${h

 // Step 1: Head guest gets eating animation
 setQueue(prev => prev.map((c, idx) => idx === 0 ? {

 setTimeout(() => {
 // Step 2: Guest leaves with happy face, queue sh
 setQueue(prev => prev.map((c, idx) => idx === 0 ?

 setServedHistory(prev => [`${headGuest.emoji} ${h

 setTimeout(() => {
 setQueue(prev => prev.slice(1));
 setIsServing(false);
 addLog(` ${headGuest.name} поел и ушел сытым.
 }, 400);
 }, 900);
}, [queue, isServing]));

const reset = () => {
 counter = 3;
 setQueue([
 { id: 'r1', name: 'Студент Вася', emoji: ' ', c
 т голода после матана!', animState: 'in' },
 { id: 'r2', name: 'Кодер Семён', emoji: ' ', c
 апишу ИИ за порцию борща!', animState: 'in' },
 { id: 'r3', name: 'Кот Борис', emoji: ' ', col
 без сметаны, плиз.', animState: 'in' },
]);
 setServedHistory([]);

```



```

 <div className={`
 absolute -top-16 left-1/2 -tr
8 text-center text-slate-200 leading-tight
 ${isHead ? 'opacity-100 borde
 `}>
 <div className="font-bold tex
 {customer.bubbleText}
 <div className="absolute -bot
it rotate-45"></div>
 </div>

 {/* Персонаж */}
 <div className={`
 w-14 h-14 rounded-2xl border-
relative
 ${customer.color}
 ${isHead ? 'ring-4 ring-amber
 `}>
 {c

 {/* Индексы HEAD/TAIL */}
 {(isHead || isTail) && (
 <span className={`
 absolute -bottom-2 text-[
 ${isHead ? 'bg-amber-500
 `}>
 {isHead && isTail ? 'h+t'

)}
 </div>
 </div>
);
 })
 })
</div>

</div>
</div>

```



```
 {/* Пол */}
 <div className="w-full h-3 bg-gradient-to-r from-blue-500 to-purple-500">
 </div>
 </div>

 {/* ЛОГ ДЕЙСТВИЙ */}
 <div className="bg-slate-900 border border-slate-800 p-4">
 <div className="text-[10px] font-mono text-slate-400">
 {log.map((entry, idx) => (
 <div
 key={idx}
 className={`text-xs font-mono flex items-center justify-between p-2`}
 >
 {idx === 0 ? Действие : }
 {entry}
 </div>
))}
 </div>
 </div>
);
};
```

---

ФАЙЛ 66/111: src/components/SalmonAutomatonWidget.tsx  
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 100

---

```
import React, { useState, useEffect, useContext, useMemo } from 'react';
import { Terminal } from 'lucide-react';
import { VizChapterContext, emitVizDemo, useVizRuntime, useVizState } from 'viz';
import { Play, RotateCcw, Plus, Volume2, Info } from 'lucide-react';
```

```
interface TrieNode {
 id: number;
 char: string;
 parent: number;
```

```

const liveStep = vars ? vizNumber(vars.step) ?? vizNumber(0) : null;
const liveQueue = vars ? vizArray(vars.queue) : null;
const liveNodes = vars ? vizNumber(vars.nodes) : null;
const liveFail = vars ? vizRecord(vars.fail) : null;

// набор образцов из кода перестраивает бор целиком
useEffect(() => {
 if (!liveWordsSig) return;
 const parsed = (JSON.parse(liveWordsSig) as unknown[])
 .map((w) => (typeof w === "string" ? w : typeof w !== "string" ? "" : w))
 .filter((w) => Boolean(w))
 .map((w) => w.toUpperCase().replace(/[^A-ZА-Я]/g, ""))
 .filter((w) => w.length > 0)
 .slice(0, 8);
 if (!parsed.length) return;
 if (parsed.join("|") === words.join("|")) return;
 setWords(parsed);
 buildInitialTrie(parsed);
 // eslint-disable-next-line react-hooks/exhaustive-deps
}, [liveWordsSig]);

// состояние автомата и шаг BFS – подсвечиваем вершины
useEffect(() => {
 if (liveState !== null && liveState >= 0 && nodes[liveState] !== null) {
 if (liveStep !== null && liveStep >= 0) {
 // simulationStep – грубый статус: 0 = бор готов,
 const total = Object.keys(nodes).length;
 const failReady = liveFail ? Object.keys(liveFail).length : 0;
 setSimulationStep(liveStep <= 0 ? 0 : failReady ? failReady : total);
 }
 if (liveQueue) {
 const q = liveQueue.map((x) => (typeof x === "number" ? x : undefined));
 if (q.length) setBfsQueue(q);
 }
 // eslint-disable-next-line react-hooks/exhaustive-deps
 }, [vars]);

```

```

 term_link: 0,
 is_terminal: false,
 words: [],
 depth: newNodes[curr].depth + 1
 };
}
curr = newNodes[curr].children[c];
}
if (!newNodes[curr].words.includes(word)) {
 newNodes[curr].words.push(word);
 newNodes[curr].is_terminal = true;
}
});

let currentX = 0;
const paddingX = 70;
const paddingY = 80;

const layoutDFS = (u: number, depth: number) => {
 const childKeys = Object.values(newNodes[u].children);
 newNodes[u].y = 40 + depth * paddingY;

 if (childKeys.length === 0) {
 newNodes[u].x = 40 + currentX * paddingX;
 currentX++;
 } else {
 let leftX = -1;
 let rightX = -1;
 childKeys.forEach(v => {
 layoutDFS(v, depth + 1);
 if (leftX === -1) leftX = newNodes[v].x!;
 rightX = newNodes[v].x!;
 });
 newNodes[u].x = leftX === -1 ? 40 + currentX * paddingX : (leftX + rightX) / 2;
 if (leftX === -1) currentX++;
 }
};
```

```

 });

 // Запуск BFS
 const startBFS = () => {
 const rootChildren = Object.values(nodes[0].children);
 const queue = [...rootChildren];
 const updatedNodes = { ...nodes };
 rootChildren.forEach((childId) => {
 updatedNodes[childId].fail = 0;
 });

 setNodes(updatedNodes);
 setBfsQueue(queue);
 setSimulationStep(1);
 setCurrentNode(null);
 setSpeechText(
 'Начинаем обход в ширину (BFS)! Все вершины на графике

 суффикса просто нет. Жми «Следующий шаг»!'
);
 };

 // Один шаг BFS
 const stepBFS = () => {
 if (bfsQueue.length === 0) {
 setSimulationStep(2);
 setCurrentNode(null);
 setSpeechText(
 'Ура! Очередь пуста! Мы успешно построили полный

 граф и полюбоваться таблицей переходов!'
);
 return;
 }

 const r = bfsQueue[0];
 const newQueue = bfsQueue.slice(1);
 const updatedNodes = { ...nodes };
 const rNode = updatedNodes[r];
 const childrenIds = Object.values(rNode.children);
 };

```

```

 <div>
 <h4 className="text-2xl font-bold text-white">
 Интерактивный конструктор: Говорящий Эхо-
 </h4>
 <p className="text-sm text-slate-400">
 Построение дерева и расчет суффиксных ссы
 </p>
 </div>
 </div>
 <div className="flex items-center gap-2">
 <button
 onClick={() => buildInitialTrie(words)}
 className="flex items-center gap-1 bg-slate
 ransition-colors"
 title="Сбросить автомат"
 >
 <RotateCcw className="w-4 h-4" /> Сбросить
 </button>
 </div>
</div>

{/* Верхняя панель: Говорящий лосось и диалоговое
<div className="grid grid-cols-1 md:grid-cols-3 g
 {/* Аватар Лосося (Сеня) с анимацией моргания и
 <div className="flex flex-col items-center just
 <div className="w-40 h-40 relative flex items
 full border border-blue-500/30 shadow-inner
 {/* SVG Лосося */}
 <svg
 viewBox="0 0 200 200"
 className={`w-36 h-36 transition-transform
 isTalking ? 'scale-105 drop-shadow-[0_0_
]`}
 >
 {/* Тело лосося */}
 <ellipse cx="100" cy="100" rx="70" ry="45
 <path d="M 40 85 Q 100 55 160 85 Q 100 115

```

```

 {/* Декоративные пузыри */}
 <div className="absolute -top-1 right-4 text-slate-400">
 <div className="absolute top-8 right-1 text-slate-400">
 </div>

 Эхо-Лосось Сеня

 </div>

 {/* Пузырь с текстом (Баббл) */}
 <div className="md:col-span-2 bg-slate-800 border border-slate-400 rounded-2xl
 transition min-h-[140px]">
 {/* Треугольник баббла */}
 <div className="absolute -left-3 top-1/2 -transform rotate(45deg)
 bg-slate-800 border-b-[12px] border-b-transparent border-l-[12px]
 border-l-transparent border-r-[12px] border-r-transparent">
 </div>

 <div className="flex items-center space-x-2 text-slate-400">
 <Volume2 className={`w-4 h-4 ${isTalking ? 'animate-pulse' : ''}` />
 Прямое вещание из аквариума:
 </div>

 <p className="text-slate-200 text-base leading-relaxed">
 {speechText}
 </p>

 <div className="mt-4 pt-3 border-t border-slate-400">
 <div className="text-xs text-slate-400 flex justify-between">
 <Info className="w-3.5 h-3.5 text-blue-400">
 Статус: {simulationStep === 0 ? 'Бор готов к бою!' : 'Бор готов к бою!'}
 </div>
 </div>

 <div className="flex gap-2">
 {simulationStep === 0 && (
 <button
 onClick={startBFS}
 className="bg-emerald-600 hover:bg-emerald-500 text-white px-2 py-1">
 Начать
 </button>
)}
 </div>
 </div>
 </div>
 </div>
 </div>
</div>

```

```

Object.values(nodes).forEach(n => {
 if (n.x && n.x > maxX) maxX = n.x;
 if (n.y && n.y > maxY) maxY = n.y;
});
const svgWidth = Math.max(maxX + 80, 400);
const svgHeight = Math.max(maxY + 60, 200);

return (
 <svg viewBox={`0 0 ${svgWidth} ${svgHeight}`}>
 <defs>
 <marker id="arrowhead" markerWidth="6" >
 <polygon points="0 0, 6 3, 0 6" fill=
 </marker>
 <marker id="fail-arrow" markerWidth="6" >
 <polygon points="0 0, 6 3, 0 6" fill=
 </marker>
 </defs>

 {/* Draw Edges */}
 {Object.values(nodes).map(node => {
 if (node.id === 0) return null;
 const parent = nodes[node.parent];
 if (!parent.x || !parent.y || !node.x || !node.y) return null;

 const isCurrent = currentNode === node.id;

 return (
 <g key={`edge-${node.id}`}>
 <line
 x1={parent.x}
 y1={parent.y + 16}
 x2={node.x}
 y2={node.y - 16}
 stroke={isCurrent ? '#60a5fa' : '#ccc'}
 strokeWidth="2"
 markerEnd="url(#arrowhead)"
 />
 <text

```

```

 { /* Draw Nodes */
 Object.values(nodes).map(node => {
 if (!node.x || !node.y) return null;
 const isRoot = node.id === 0;
 const isCurrent = currentNode === node;
 const isInQueue = bfsQueue.includes(node);

 let fill = '#1e293b';
 let stroke = '#475569';

 if (isCurrent) {
 fill = '#2563eb';
 stroke = '#60a5fa';
 } else if (isInQueue) {
 fill = '#b45309';
 stroke = '#fbbf24';
 } else if (node.is_terminal) {
 fill = '#059669';
 stroke = '#34d399';
 }

 return (
 <g key={`node-${node.id}`}>
 <circle
 cx={node.x}
 cy={node.y}
 r="16"
 fill={fill}
 stroke={stroke}
 strokeWidth="2"
 />
 <text
 x={node.x}
 y={node.y + 4}
 fill="white"
 fontSize={isRoot ? "10" : "12"}
 fontWeight="bold"
 />
 </g>
);
 });
 }
);

```



```

 onClick={() => removeWord(w)}
 className="text-slate-500 hover:text-slate-400"
 title="Удалить"
 >
 ✕
 </button>

)})
</div>
</div>

<form onSubmit={handleAddWord} className="flex flex-col gap-2" >
 <input
 type="text"
 value={newWord}
 onChange={(e) => setNewWord(e.target.value)}
 placeholder="НОВОЕ СЛОВО..."
 maxLength={8}
 className="bg-slate-800 border border-slate-700 border-radius-4px outline-none focus:border-blue-500"
 />
 <button
 type="submit"
 className="bg-slate-700 hover:bg-slate-600 text-white font-medium py-2 px-4 transition-colors"
 >
 <Plus className="w-3.5 h-3.5" /> Добавить
 </button>
</form>
</div>

{/* Таблица Вершин и суффиксных ссылок */}
<div className="lg:col-span-2 bg-slate-900/50 p-4" >
 <h5 className="text-white font-bold mb-3 flex" >

 Таблица переходов и fail-


```

```

: isCurrent
? 'bg-blue-400 animate-pi
: isInQueue
? 'bg-amber-400'
: 'bg-slate-600'
} ` }
>
#{node.id}
{node.id === 0 && <span className
</td>
<td className="py-2.5 px-3">
 <span className="bg-slate-800 b
 {node.char}

</td>
<td className="py-2.5 px-3 text-s
 {node.id === 0 ? '-' : `#{node
</td>
<td className="py-2.5 px-3">
 {node.id === 0 ? {
 <span className="text-slate-5
 } : {
 <span className="bg-indigo-95
 #{node.fail}

 }}
</td>
<td className="py-2.5 px-3">
 {node.id === 0 || node.term_lin
 <span className="text-slate-5
 } : {
 <span className="bg-rose-950 l
0 transition-colors cursor-help" title={`Ck
 #{node.term_link}

 }}
</td>
<td className="py-2.5 px-3 text-s

```

```

-2 py-1">шаг BFS = {liveStep}
{liveQueue && <span className="text-[11px]
-1">queue = {JSON.stringify(liveQueue).slice
</>
) : (

 открой панель Python: массив <code className=
nt-mono text-cyan-300">state</code>/<code c

)}
</div>
</div>
);
};

```

ФАЙЛ 67/111: src/components/SegmentTreeVisualizer.tsx  
 КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState, useEffect, useCallback, useMe
import { Play, Pause, RotateCcw, ChevronLeft, ChevronRi
import { useVizRuntime, vizNumber } from '../data/vizSt

```

```

// ===== TYPES =====
interface Step {
 id: number;
 desc: string;
 codeLine: number;
 treeState: Record<number, number>; // node ind
 highlightNodes: number[]; // current
 mergeChildren?: [number, number]; // pair be
 arrayHighlight?: [number, number]; // [L, R]
 result?: number; // partial
}

```

```

// ===== PURE LOGIC: build tree =====

```

```

 steps.push({ id: id++, desc: `build(${v}, ${l}, ${r},
 , highlightNodes: [v], arrayHighlight: [l, r] }
 tree[v] = arr[l];
 steps.push({ id: id++, desc: `tree[${v}] = ${arr[
 l, r] }));
 return;
 }
 const m = (l + r) >> 1;
 steps.push({ id: id++, desc: `build(${v}, ${l}, ${r},
 { ...tree }, highlightNodes: [v], arrayHighlight:
 go(2 * v, l, m);
 steps.push({ id: id++, desc: `Вернулись в build(${v},
 lightNodes: [v], arrayHighlight: [l, r] }));
 go(2 * v + 1, m + 1, r);
 tree[v] = (tree[2 * v] ?? 0) + (tree[2 * v + 1] ?? 0);
 steps.push({ id: id++, desc: `Объединяем: tree[${v}] =
 tate: { ...tree }, highlightNodes: [v], mergeChild
 }
 go(1, 0, n - 1);
 steps.push({ id: id++, desc: ` Дерево построено! Корень:
 Highlight: [0, n - 1] }));
 return steps;
}

```

// 2. Top-Down Query steps

```

function genQueryTopDownSteps(arr: number[], ql: number, qr: number) {
 const n = arr.length;
 const tree = buildTreeFull(arr);
 const steps: Step[] = [];
 let id = 0;

 function go(v: number, l: number, r: number, ql: number, qr: number) {
 if (ql > r || qr < l) {
 steps.push({ id: id++, desc: `query(${v}, [${l}...${r}],
 odeLine: 2, treeState: tree, highlightNodes: [v],
 return 0;
 }
 if (ql <= l && r <= qr) {

```

```

const highlights: number[] = [];
if (l & 1) {
 res += tree[l] ?? 0;
 steps.push({ id: id++, desc: `l=${l} нечётный → l
 0}. Сумма = ${res}. l++.`, codeLine: 5, treeSta
 highlights.push(l);
 l++;
}
if (r & 1) {
 r--;
 res += tree[r] ?? 0;
 steps.push({ id: id++, desc: `r=${r + 1} нечётный
 , treeState: tree, highlightNodes: [r], arrayHi
 highlights.push(r);
}
l >>= 1;
r >>= 1;
if (l < r) {
 steps.push({ id: id++, desc: `Поднимаемся: l = ${
 , arrayHighlight: [qL, qR], result: res }));
}
}
steps.push({ id: id++, desc: ` Запрос sum([${qL}..${
 ighlightNodes: [1], arrayHighlight: [qL, qR], resul
return steps;
}

// ===== TREE RENDERING HELPER =====
interface VNode { v: number; l: number; r: number; val:

function buildVisualTree(treeState: Record<number, number>) {
 if (l === r) return { v, l, r, val: treeState[v] ?? n
 const m = (l + r) >> 1;
 return { v, l, r, val: treeState[v] ?? null, left: bu
 v + 1, m + 1, r) };
}

// ===== CODE SNIPPETS =====

```

```

const [playing, setPlaying] = useState(false);
const [speed, setSpeed] = useState(700);

useEffect(() => { setIdx(0); setPlaying(false); }, [s

useEffect(() => {
 if (!playing || idx >= steps.length - 1) { if (idx :
 const t = setTimeout(() => setIdx(i => i + 1), speed
 return () => clearTimeout(t);
}, [playing, idx, steps, speed]);

const step = steps[idx] || steps[0];
if (!step) return null;

const clr = { indigo: { btn: 'bg-indigo-600 hover:bg-
 border-indigo-500 bg-indigo-600', childRing: 'ring-
 -900' }, emerald: { btn: 'bg-emerald-600 hover:bg-e
 30 border-emerald-500 bg-emerald-600', childRing: '
 -slate-900' }, amber: { btn: 'bg-amber-600 hover:bg
 der-amber-500 bg-amber-600', childRing: 'ring-rose-
 [color];

 return { step, idx, setIdx, playing, setPlaying, speed
}

// ===== SIMULATION PANEL =====
function SimPanel({ title, icon, steps, code, color, arr
 | 'emerald' | 'amber'; arr: number[] }) {
 const p = Player({ steps, color });
 if (!p) return null;
 const { step, idx, setIdx, playing, setPlaying, speed
 const n = arr.length;

 const vTree = useMemo(() => buildVisualTree(step.tree

 const renderNode = useCallback((nd: VNode): React.Reac
 const isHigh = step.highlightNodes.includes(nd.v);
 const isChild = step.mergeChildren?.includes(nd.v);

```

```

 {/* Array strip */}
 <div className="px-3 py-2 bg-slate-950 border-b b
 {arr.map((v, i) => {
 const inRange = step.arrayHighlight && i >= s
 return (
 <div key={i} className={`flex flex-col item
 go' ? 'bg-indigo-950 border-indigo-500' : c
 er-500'} -translate-y-0.5` : 'bg-slate-900 l
 <span className="text-[7px] text-slate-500
 <span className="text-[11px] font-bold for
 </div>
);
 })}
 </div>

 {/* Tree */}
 <div className="flex-1 p-2 overflow-x-auto bg-sla
 <div className="scale-[0.85] origin-top min-w-m
 </div>

 {/* Code */}
 <pre className="px-3 py-2 text-[9px] md:text-[10px]
 x">
 {code.map((line, i) => {
 const ln = i + 1;
 const active = step.codeLine === ln;
 return (
 <div key={ln} className={`px-2 py-0.5 round
 2 border-indigo-500 text-indigo-200' : colo
 ' : 'bg-amber-600/20 border-l-2 border-ambe
 <span className="inline-block w-4 text-sl
 </div>
);
 })}
 </pre>

 {/* Description + result */}
 <div className="px-3 py-2.5 bg-slate-900/60 borde

```

```

 + 1) / total) * 100 : 0}%` }} /></div>
 </div>
);
}

// =====
// MAIN EXPORTED COMPONENT
// =====
export function SegmentTreeVisualizer() {
 const runtime = useVizRuntime();
 const liveI = vizNumber(runtime?.variables?.i);
 const [arr, setArr] = useState<number[]>([5, 8, 3, 12]);
 const [customInput, setCustomInput] = useState('5, 8,');
 const [qL, setQL] = useState(1);
 const [qR, setQR] = useState(4);
 const [view, setView] = useState<'build' | 'queries'>('build');

 // Clamp query range when array changes
 useEffect(() => {
 setQL(prev => Math.min(prev, arr.length - 1));
 setQR(prev => Math.min(prev, arr.length - 1));
 }, [arr]);

 const handleApply = (e: React.FormEvent) => {
 e.preventDefault();
 const parsed = customInput.split(',').map(s => parseInt(s));
 if (parsed.length >= 2 && parsed.length <= 16) {
 setArr(parsed);
 }
 };

 const randomize = () => {
 const size = arr.length;
 const a = Array.from({ length: size }, () => Math.floor(Math.random() * 100));
 setArr(a);
 setCustomInput(a.join(', '));
 };
}

```



```

 </div>
 </form>

 { /* Query range */ }
 <div className="bg-slate-950 p-4 rounded-xl b
 <label className="text-[10px] font-bold upp
 <div className="flex items-center gap-2">
 <div className="flex items-center gap-1">
 <span className="text-xs text-slate-400
 <input
 type="number"
 min={0}
 max={arr.length - 1}
 value={qL}
 onChange={e => setQL(Math.min(Number(
 className="w-14 bg-slate-900 border b
 focus:border-indigo-500"
 />
 </div>
 <div className="flex items-center gap-1">
 <span className="text-xs text-slate-400
 <input
 type="number"
 min={0}
 max={arr.length - 1}
 value={qR}
 onChange={e => setQR(Math.min(Number(
 className="w-14 bg-slate-900 border b
 focus:border-indigo-500"
 />
 </div>
 </div>
 <div className="text-[10px] text-slate-500 p
 Запрос: sum(A[{qL}..{qR}]) = {arr.slice(q
 </div>
</div>
</div>
</div>

```

```

 icon="↑ "
 steps={queryTDSteps}
 code={QUERY_TD_CODE}
 color="emerald"
 arr={arr}
 />
 <SimPanel
 key={`qbu-${arr.join(' - ')}-${qL}-${qR}`}
 title={`Запрос sum([${qL}..${qR}]) – Снизу-вверх`}
 icon="↑ "
 steps={queryBUSteps}
 code={QUERY_BU_CODE}
 color="amber"
 arr={arr}
 />
 </div>
)}

{view === 'theory' && (
 <div className="space-y-5">
 {/* Why it splits this way */}
 <div className="bg-slate-950 p-5 rounded-xl border border-slate-800">
 <h4 className="text-base font-bold text-white">Почему разбиение?
 </h4>
 <p className="text-sm text-slate-300 leading-relaxed">
 Каждый узел берёт <code className="bg-slate-900 px-1">mid</code>
 и получает <code className="bg-slate-900 px-1">left</code> и <code className="bg-slate-900 px-1">right</code>.
 Левая часть забирает на 1 больше (округление вправо).
 </p>
 <div className="bg-slate-900 p-4 rounded-lg">
 <div className="text-slate-400">Корень: L = 0</div>
 <div className="text-slate-300">mid = 1</div>
 <div className="text-indigo-300">→ Левый: L = 0</div>
 <div className="text-emerald-300">→ Правый: R = 1</div>
 </div>
 <div className="text-slate-500 mt-2">Всего узлов: 2</div>
 </div>
 </div>
)}

```

```

 00 font-bold">х сложно</div>
 </div>
 </div>
 </div>

 {/* Verdict */}
 <div className="bg-slate-950 p-5 rounded-xl b
 <h4 className="font-bold text-blue-400 mb-2
 <p className="text-sm text-slate-300 leadin
 <strong className="text-white">Итеративны
 кальности. Но если нужны <strong className=
 ame="text-white">рекурсивный (сверху-вниз)<
 </p>
 </div>
</div>
 })
</div>
);
}

```

ФАЙЛ 68/111: src/components/Sidebar.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import React, { useMemo, useState } from "react";
import { ChevronRight, Compass, Search, Sparkles, X } f
import { chapters, SECTIONS, sectionOf } from "../data/
import { VIZ_REGISTRY } from "./vizRegistry";

```

```

interface SidebarProps {
 selectedChapterId: string;
 setSelectedChapterId: (id: string) => void;
 /** Мобильный режим: список живёт в выдвижной панели
 onNavigate?: () => void;
 /** Узкий режим: номер билета и название в одну строку
 compact?: boolean;

```

```

 className="w-full bg-slate-800/70 border border-bor
 e-500 focus:outline-none focus:border-indigo
 />
 {query && (
 <button
 onClick={() => setQuery("")}
 className="absolute right-2 top-1/2 -tran
 aria-label="Очистить поиск"
 >
 <X className="w-3.5 h-3.5" />
 </button>
)}
</div>
)}
</div>

<div className="flex-1 overflow-y-auto px-3 pb-4
 {groups.length === 0 && <p className="text-sm t

 {groups.map(({ section, items }) => (
 <div key={section.id}>
 <div className="flex items-baseline gap-1.5
 <span className="text-[10px] font-bold up
 {section.from > 0 && (
 <span className="text-[10px] text-slate
)}
 </div>
 <div className="space-y-1">
 {items.map((chapter) => {
 const isSelected = chapter.id === selec
 const hasViz = Boolean(VIZ_REGISTRY[cha
 // «18–20. Графы. Основное дерево...» → э
 const num = chapter.title.match(/^(\d+{
 const label = num ? chapter.title.slice
 return (
 <button
 key={chapter.id}
 onClick={() => {

```

```

 <div className={`shrink-0 m-3 mt-0 bg-gradient-to
 x] text-slate-300 space-y-1.5 hidden lg:block $
 <div className="font-bold text-indigo-400 flex
 <Sparkles className="w-3.5 h-3.5" /> Как поль
 </div>
 <p className="leading-relaxed">
 <b className="text-slate-200">Порядок чтения
 номерам билетов 1 → 24, каждая опирается на пр
 Следующая тема» продолжают этот же порядок.
 </p>
 <p className="leading-relaxed">
 Подчёркнутые термины в тексте раскрываются по
 симулятора.
 </p>
 <p className="text-slate-400 italic">Стрелки ←
 </div>
</aside>
);
};

```

---

ФАЙЛ 69/111: src/components/Simulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```

import React, { useState } from 'react';
import { Layers, AlignLeft, Award, Plus, ArrowRight, Ar

interface Plate {
 id: string;
 name: string;
 icon: string;
}

interface Person {
 id: string;

```

```
// Stack handlers
const addPlate = () => {
 const presets = [
 { name: 'Сковорода от омлета', icon: ' ' },
 { name: 'Кастрюля от супа', icon: ' ' },
 { name: 'Чашка от кофе', icon: '=' },
 { name: 'Миска с остатками салата', icon: ' ' },
 { name: 'Тарелка от тако', icon: ' ' },
];
 const randomPreset = presets[Math.floor(Math.random() * presets.length)];
 const newPlate = {
 id: Date.now().toString(),
 name: randomPreset.name,
 icon: randomPreset.icon
 };
 setStack(prev => [...prev, newPlate]);
 setPopMessage(` Положили поверх стопки: ${newPlate.name} ${newPlate.icon} `);
};

const popPlate = () => {
 if (stack.length === 0) {
 setPopMessage("⚠ Стопка пуста! Все тарелки вымыты");
 return;
 }
 const topPlate = stack[stack.length - 1];
 setStack(prev => prev.slice(0, prev.length - 1));
 setPopMessage(` Вымыта верхняя тарелка (LIFO): ${topPlate.name} ${topPlate.icon} `);
};

const resetStack = () => {
 setStack([
 { id: '1', name: 'Тарелка из-под пасты', icon: ' ' },
 { id: '2', name: 'Тарелка из-под пиццы', icon: ' ' },
 { id: '3', name: 'Блюдце от торта', icon: ' ' },
]);
 setPopMessage(" Стопка тарелок восстановлена.");
};
```

```
// Heap handlers
const addHypothesis = (e: React.FormEvent) => {
 e.preventDefault();
 if (!newCustomText.trim()) return;
 const item: Hypothesis = {
 id: Date.now().toString(),
 text: `Кандидат: "${newCustomText}"`,
 probability: Number(newCustomProb)
 };
 setHeap(prev => [...prev, item]);
 setNewCustomText('');
 setHeapMessage(`Добавлена гипотеза с вероятностью`);
};

const popHeap = () => {
 if (heap.length === 0) {
 setHeapMessage("⚠ Куча пуста! Нет кандидатов для");
 return;
 }
 // Find highest probability
 let maxIdx = 0;
 for (let i = 1; i < heap.length; i++) {
 if (heap[i].probability > heap[maxIdx].probability) {
 maxIdx = i;
 }
 }
 const best = heap[maxIdx];
 setHeap(prev => prev.filter((_, idx) => idx !== maxIdx));
 setHeapMessage(`Выбран самый "тяжелый" кандидат: ${best.text}`);
};

const resetHeap = () => {
 setHeap([
 { id: '1', text: 'Кандидат: "Привет, я искусственный интеллект"' },
 { id: '2', text: 'Кандидат: "Привет, я испек вкусный хлеб"' },
 { id: '3', text: 'Кандидат: "Привет, я испанский язык"' }
]);
 setHeapMessage("Гипотезы восстановлены.");
};
```

```

<button
 onClick={() => setActiveTab('queue')}
 className={`flex items-center justify-between
 activeTab === 'queue'
 ? 'bg-indigo-600/20 border-indigo-500 text-white'
 : 'bg-slate-800/60 border-slate-700/60 text-white'}
>
 <div className="flex items-center gap-3">
 <AlignLeft className={`w-5 h-5 ${activeTab === 'queue' ? 'text-white' : 'text-slate-400'} opacity-80`} />
 <div className="text-left">
 <div className="font-bold text-sm text-white">FIFO
 <div className="text-xs opacity-80">FIFO
 </div>
 </div>
 BFS

</button>

<button
 onClick={() => setActiveTab('heap')}
 className={`flex items-center justify-between
 activeTab === 'heap'
 ? 'bg-emerald-600/20 border-emerald-500 text-white'
 : 'bg-slate-800/60 border-slate-700/60 text-white'}
>
 <div className="flex items-center gap-3">
 <Award className={`w-5 h-5 ${activeTab === 'heap' ? 'text-white' : 'text-slate-400'} opacity-80`} />
 <div className="text-left">
 <div className="font-bold text-sm text-white">Приоритетная
 <div className="text-xs opacity-80">Приоритетная
 </div>
 </div>
 Beam Search

```



```

 >
 <RotateCcw className="w-5 h-5" />
 </button>
 </div>
 </div>

 {popMessage && {
 <div className="bg-blue-950/60 border border-
 imate-fade-in">
 <span className="inline-block w-2 h-2 round
 {popMessage}
 </div>
 }}

 {/* Visualization */}
 <div className="bg-slate-950/60 p-6 rounded-2
 >
 {stack.length === 0 ? {
 <div className="text-center py-12 text-sla
 <div className="text-5xl mb-3"> </div>
 <p className="text-base font-bold">Стор
 <p className="text-xs mt-1">Добавьте гр
 </div>
 } : {
 <div className="w-full max-w-sm flex flex
 <div className="absolute top-0 text-xs
 full flex items-center gap-1">
 ВЕРШИНА СТЕКА (TOP)
 <ArrowDown className="w-3.5 h-3.5 anim
 </div>

 {/* Reversed map so top of stack is vis
 {stack.slice().reverse().map(({plate, ind
 const isTop = index === 0;
 return {
 <div
 key={plate.id}
 className={`w-full flex items-cen

```

```

 /span>
 </h3>
 <p className="text-slate-400 text-xs mt-1">
 Кто первым пришел, тот первым получил суп
 </p>
</div>
<div className="flex flex-wrap items-center">
 <button
 onClick={addPerson}
 className="flex-1 md:flex-none flex item
-2.5 rounded-xl text-sm font-bold shadow-lg
 >
 <Plus className="w-4 h-4" /> Встать в очередь
 </button>
 <button
 onClick={servePerson}
 className="flex-1 md:flex-none flex item
er border-indigo-500/40 px-4 py-2.5 rounded
 >
 Налить суп первому
 </button>
 <button
 onClick={resetQueue}
 title="Сбросить"
 className="p-2.5 bg-slate-800 hover:bg-
tion-colors cursor-pointer"
 >
 <RotateCcw className="w-5 h-5" />
 </button>
</div>
</div>

```

```

{dequeueMessage && {
 <div className="bg-indigo-950/60 border border-
p-3 animate-fade-in">
 <span className="inline-block w-2 h-2 roun
 {dequeueMessage}
 </div>

```

```

 Первый (Head)

 })
 {
 <span className={`font-bold text-
 {person.name}

 <span className={`text-[10px] font
 isFirst ? 'bg-indigo-500/30 tex
 }`}>
 {isFirst ? 'Получает сун' : `В

 </div>
);
 }}}

 <div className="flex flex-col items-cen
-600 min-w-[120px] h-[120px]">
 <Plus className="w-6 h-6 mb-1" />
 <span className="text-xs font-mono up
 </div>
</div>
})
<div className="mt-6 text-xs text-slate-500
 ПЕРВЫМ ПРИШЕЛ → ПЕРВЫМ ПОЛУЧИЛ (FIFO) • И
</div>
</div>
</div>
))

{/* HEAP TAB */}
{activeTab === 'heap' && {
 <div className="space-y-6">
 <div className="bg-slate-800/80 p-5 rounded-x
 tify-between gap-4">
 <div>
 <h3 className="text-lg font-bold text-whi
 Приоритетная очередь / Куча</sp

```

```

 />
 </div>
 <div className="md:col-span-3">
 <label className="block text-xs font-bold">
 Вероятность:
 </label>
 <input
 type="range"
 min="0.01"
 max="0.99"
 step="0.01"
 value={newCustomProb}
 onChange={e => setNewCustomProb(Number(e.target.value))}
 className="w-full accent-emerald-500 cursor-pointer"
 />
 </div>
 <div className="md:col-span-3">
 <button
 type="submit"
 disabled={!newCustomText.trim()}
 className="w-full flex items-center justify-center text-sm font-medium text-emerald-500/40 disabled:opacity-50 disabled:cursor-not-allowed"
 >
 <Plus className="w-4 h-4" /> Добавить в список
 </button>
 </div>
</form>

{heapMessage && (
 <div className="bg-emerald-950/60 border border-emerald-500/40 p-3 animate-fade-in">

 {heapMessage}
 </div>
)}

{/* Visualization */}

```

```

 {isTop && (
 <span className="text-[10px]
-0.5 rounded inline-block mt-1">
 Вершина Кучи (Root) • Буд

)}
 </div>
 </div>

 <div className="flex items-center
 /* Custom progress bar */>
 <div className="hidden sm:block
 <div
 className={`h-full rounded-
 style={{ width: `${percent}%
 ></div>
 </div>
 <span className={`font-mono font
 isTop ? 'text-emerald-400 tex
 }`>
 {percent}%

 </div>
 </div>
 </div>
);
 }}}
</div>
)}
<div className="mt-6 text-xs text-slate-500
 НЕ ВАЖНО, КТО ПРИШЕЛ ПЕРВЫМ • ВАЖЕН САМЫЙ
</div>
</div>
</div>
)}
</div>
);
};

```

```

 <div>
 <h3 className="text-2xl font-bold text-white">
 <p className="text-sm text-indigo-300">Один
 </div>
 </div>
 </div>

 <div className="flex flex-wrap gap-2 mb-5">
 {[
 { id: "cases" as Tab, label: "1 · Механика: Z",
 { id: "big" as Tab, label: "2 · Большое дерево",
 { id: "play" as Tab, label: "3 · Собери сам
]).map((t) => {
 <button
 key={t.id}
 onClick={() => setTab(t.id)}
 className={`px-4 py-2 rounded-lg text-xs font-medium
 ${tab === t.id ? "bg-indigo-600 text-white" : ""}
 `}
 >
 {t.label}
 </button>
)]}
 </div>

 {tab === "cases" && <SplayRotationsViz />}
 {tab === "big" && <SplayWalkViz />}
 {tab === "play" && <SplayRotationSandbox />}

 {tab === "play" && {
 <p className="text-xs text-slate-500 mt-3">
 Порядок поворотов в песочнице сверяется с прав
 </p>
 }}
 </div>
);
};

```

```

 const n1 = leaf(1);
 const n2: TNode = { key: 2, left: n1, right: leaf(3) };
 const n4: TNode = { key: 4, left: n2, right: leaf(5) };
 return { key: 6, left: n4, right: leaf(7) };
 },
},
{
 name: "Змейка",
 hintCase: "Zig-Zag",
 target: 3,
 build: () => {
 const n3 = leaf(3);
 const n2: TNode = { key: 2, left: leaf(1), right: n3 };
 const n4: TNode = { key: 4, left: n2, right: leaf(5) };
 return { key: 6, left: n4, right: leaf(7) };
 },
},
{
 name: "Один уровень",
 hintCase: "Zig",
 target: 2,
 build: () => ({ key: 4, left: { key: 2, left: leaf(1), right: leaf(3) }, right: leaf(5) }),
},
{
 name: "Смешанный",
 hintCase: "Zig-Zag, затем Zig-Zig",
 target: 5,
 build: () => {
 const n5 = leaf(5);
 const n6: TNode = { key: 6, left: n5, right: leaf(7) };
 const n4: TNode = { key: 4, left: leaf(3), right: n6 };
 return { key: 8, left: n4, right: leaf(9) };
 },
},
];

/* ----- операции над деревом -----

```

```

const x = path[path.length - 1];
const p = path[path.length - 2];
const g = path.length >= 3 ? path[path.length - 3] : null;

if (!g) return { node: x.key, kind: "Zig" };

const xLeft = p.left === x;
const pLeft = g.left === p;
if (xLeft === pLeft) return { node: p.key, kind: "Zig" };
return { node: x.key, kind: "Zig-Zag (сначала нижняя ветвь)" };
}

/* ----- раскладка ----- */

interface Placed {
 key: number;
 x: number;
 y: number;
 depth: number;
}

function layout(root: TNode | null): { nodes: Placed[]; edges: [number, number][] } {
 const nodes: Placed[] = [];
 const edges: [number, number][] = [];
 let order = 0;

 const walk = (n: TNode | null, depth: number) => {
 if (!n) return;
 walk(n.left, depth + 1);
 nodes.push({ key: n.key, x: order++, y: depth, depth });
 if (n.left) edges.push([n.key, n.left.key]);
 if (n.right) edges.push([n.key, n.right.key]);
 walk(n.right, depth + 1);
 };
 walk(root, 0);

 const cols = Math.max(1, order);
 const rows = Math.max(1, Math.max(...nodes.map((n) => n.depth + 1)));
 const grid = Array.from({ length: rows }, () => Array.from({ length: cols }, () => null));
 for (const { key, x, y, depth } of nodes) {
 grid[depth][x] = key;
 }
 for (const [x1, x2] of edges) {
 grid[x1][y] = grid[x2][y];
 }
 return { nodes, edges };
}

```



```

const clickNode = (key: number) => {
 if (solved) return;
 const p = findPath(tree, key);
 if (p.length < 2) return; // корень крутить не с кем

 const want = canonicalNext(tree, target);
 setHistory((h) => [...h, clone(tree) as TNode]);
 setMoves((m) => [...m, { node: key, correct: want?.node === key }]);
 setTree((t) => rotateUp(clone(t) as TNode, key));
};

const undo = () => {
 if (history.length === 0) return;
 setTree(history[history.length - 1]);
 setHistory((h) => h.slice(0, -1));
 setMoves((m) => m.slice(0, -1));
};

const wrong = moves.filter((m) => !m.correct).length;
const minimal = useMemo(() => findPath(preset.build(), target));

return (
 <div className="w-full bg-slate-950 rounded-2xl border border-slate-800">
 <div className="flex items-start justify-between p-4">
 <div>
 <h4 className="text-sm sm:text-base font-bold text-slate-400">
 Клик по узлу = один поворот с его родителем
 <p className="text-[12px] text-slate-400 lead">
 Клик по узлу = один поворот с его родителем

 Клик по узлу = один поворот с его родителем
 поворотов выбираешь сам, разбор покажем в к

 </p>
 </div>
 <button
 onClick={() => reset((presetIdx + 1) % PRESET.length)}
 className="flex items-center gap-1.5 px-3 py-1.5 text-sm font-bold transition-colors shrink-0"
 >

```

```

 strokeWidth={onPath ? 3 : 1.8}
 style={{ transition: "all 700ms cubic-bezier(0.4, 0, 0.2, 1)" }}
 />
);
 }
}

{nodes.map((n) => {
 const isTarget = n.key === target;
 const clickable = !solved && n.depth > 0;
 return (
 <g
 key={n.key}
 onClick={() => clickable && clickNode(n)}
 style={{
 transform: `translate(${n.x}px, ${n.y}px)`,
 transition: "transform 700ms cubic-bezier(0.4, 0, 0.2, 1)",
 cursor: clickable ? "pointer" : "default",
 }}
 >
 {isTarget && (
 <circle r={22} fill="none" stroke="#6366f1" strokeWidth={2} />
)}
 <circle
 r={16}
 fill={isTarget ? "#6366f1" : path.has(n.key) ? "#a5b4fc" : "#475568"}
 stroke={isTarget ? "#a5b4fc" : "#475568"}
 strokeWidth={2}
 />
 <text textAnchor="middle" dominantBaseline="middle">
 {n.key}
 </text>
 </g>
);
})}
</svg>
</div>

<div className="flex flex-wrap items-center gap-2">

```

```

 {solved && (
 <div
 className={`mt-3 rounded-lg border px-3 py-2.5`}
 wrong === 0
 ? "bg-emerald-950/40 border-emerald-700/60"
 : "bg-rose-950/30 border-rose-800/60 text-rose-500"
 >
 <div className="flex items-center gap-2 font-mono">
 {wrong === 0 ? <CheckCircle2 className="w-4 h-4 text-emerald-500"/> : <X className="w-4 h-4 text-rose-500"/>}
 {wrong === 0
 ? `Узел ${target} в корне за ${moves.length} ходов`
 : `Узел ${target} в корне за ${moves.length} ходов`
 }
 </div>
 {wrong > 0 && (
 <ul className="list-disc pl-5 space-y-0.5">
 {moves.map(
 (m, i) =>
 !m.correct && (
 <li key={i}>
 ход {i + 1}: крутили {m.expected}

)
)}

)}
 </div>
)}
 </div>
);
};

```

```
}

```

```
interface Case {
 key: "zig" | "zig-zig" | "zig-zag";
 label: string;
 when: string;
 rotations: string;
 /** Номера узлов: роль → число. */
 keys: Partial<Record<NodeId, number>>;
 /** Обход слева направо – одинаковый на всех кадрах. */
 inorder: string[];
 /** Что означают поддеревья A..D. */
 ranges: string;
 frames: Frame[];
}

```

```
const VB = { w: 360, h: 285 };
const AIM_MS = 2600;
const RESULT_MS = 3400;
const MOVE = "transform 1200ms cubic-bezier(.34,.01,.2,

```

```
/* ----- Zig -----
const ZIG_BEFORE = {
 pos: { p: [195, 50], x: [105, 130], C: [280, 130], A:
 edges: [["p", "x"], ["p", "C"], ["x", "A"], ["x", "B"]
} as unknown as Pick<Frame, "pos" | "edges">;

```

```
const ZIG: Case = {
 key: "zig",
 label: "Zig",
 when: "родитель x – уже корень, деда нет",
 rotations: "1 поворот",
 keys: { x: 20, p: 40 },
 inorder: ["A", "20", "B", "40", "C"],
 ranges: "A < 20 · B между 20 и 40 · C > 40",
 frames: [
 {
 ...ZIG_BEFORE,

```

```

key: "zig-zig",
label: "Zig-Zig",
when: "x и p – дети с ОДНОЙ стороны (оба слева или оба справа)",
rotations: "2 поворота, первым – верхний",
keys: { x: 20, p: 40, g: 60 },
inorder: ["A", "20", "B", "40", "C", "60", "D"],
ranges: "A < 20 · B: 20–40 · C: 40–60 · D > 60",
frames: [
 {
 ...ZZ_S0,
 kind: "start",
 title: "Было: прямая линия 60 → 40 → 20",
 text: "Три узла выстроились в «бамбук»: каждый следующий узел
зглаживает.",
 ms: RESULT_MS,
 },
 {
 ...ZZ_S0,
 kind: "aim",
 title: "Прицел 1: ВЕРХНЕЕ ребро 60 – 40",
 text: "Главная хитрость этого случая: первым крутит 60, а не 40",
 focus: ["g", "p"],
 ms: AIM_MS,
 },
 {
 ...ZZ_S1,
 kind: "result",
 title: "Поворот 1: 40 поднялось над 60",
 text: "40 встало в корень, 60 опустилось к нему по глубине 1, а не 2.",
 moved: ["p", "g", "C"],
 ms: RESULT_MS,
 },
 {
 ...ZZ_S1,
 kind: "aim",
 title: "Прицел 2: ребро 40 – 20",
 text: "А теперь обычный одиночный поворот – тот же 40",
 },
]

```

```

 ...ZG_S0,
 kind: "start",
 title: "Было: 60 → влево на 20 → вправо на 40",
 text: "Узлы идут «змейкой»: сначала шаг влево, по
 ",
 ms: RESULT_MS,
},
{
 ...ZG_S0,
 kind: "aim",
 title: "Прицел 1: НИЖНЕЕ ребро 20 – 40",
 text: "Дед (60) стоит на месте и в этом повороте
 focus: ["p", "x"],
 ms: AIM_MS,
},
{
 ...ZG_S1,
 kind: "result",
 title: "Поворот 1: змейка выпрямилась",
 text: "40 заняло место 20 и забрало его к себе ле
 moved: ["x", "p", "C"],
 ms: RESULT_MS,
},
{
 ...ZG_S1,
 kind: "aim",
 title: "Прицел 2: ребро 60 – 40",
 text: "Остался одиночный поворот вокруг деда – и
 focus: ["g", "x"],
 ms: AIM_MS,
},
{
 pos: { x: [180, 50], p: [90, 130], g: [275, 130],
 edges: [["x", "p"], ["x", "g"], ["p", "A"], ["p",
 kind: "result",
 title: "Стало: 20 и 60 – два сына сорока",
 text: "Дерево стало идеально симметричным: все че
 очку». ",

```

```

 const from = prev[id]!;
 const to = pos[id]!;
 const r = isSubtree(id) ? 15 : 20;
 const dx = to[0] - from[0];
 const dy = to[1] - from[1];
 const len = Math.hypot(dx, dy) || 1;
 const pad = r + 8;
 return (
 <g key={`ghost-${id}`} opacity={0.55}>
 <circle cx={from[0]} cy={from[1]} r={r} fill="white" />
 {len > pad * 2 + 6 && (
 <line
 x1={from[0] + (dx / len) * pad}
 y1={from[1] + (dy / len) * pad}
 x2={to[0] - (dx / len) * pad}
 y2={to[1] - (dy / len) * pad}
 stroke="#a5b4fc"
 strokeWidth={1.5}
 strokeDasharray="4 3"
 markerEnd="url(#splay-arrow)"
 />
)}
 </g>
);
 }
}

{edges.map(([a, b], i) => {
 const pa = pos[a];
 const pb = pos[b];
 if (!pa || !pb) return null;
 const hot = Boolean(focus && focus.includes(a));
 return (
 <line
 key={`-${a}-${b}-${i}`}
 x1={pa[0]}
 y1={pa[1]}
 x2={pb[0]}
 y2={pb[1]}
 />
);
})}

```

```

 </text>
 })
 </g>
);
}
</svg>
);
};

```

```

/** Имя случая из кода пользователя → вкладка демонстрация
const CASE_BY_NAME: Record<string, number> = {
 zig: 0, "zig-zig": 1, zigzig: 1, "zig zig": 1, "zig-zigzig": 1,
};

```

```

export const SplayRotationsViz: React.FC = () => {
 const [caseIdx, setCaseIdx] = useState(0);
 const [frame, setFrame] = useState(0);
 const [playing, setPlaying] = useState(false);
 const [slow, setSlow] = useState(false);

```

```

/* — связь с Python-компилятором: case переключает в
const runtime = useVizRuntime();
const vars = runtime?.variables;
const liveCase = vars ? vizString(vars.case) ?? vizString("zig-zig") : "";
const liveStep = vars ? vizNumber(vars.step) ?? vizNumber(0) : 0;
const liveX = vars ? vizNumber(vars.xk) ?? vizNumber(0) : 0;
const liveP = vars ? vizNumber(vars.pk) ?? vizNumber(0) : 0;
const liveG = vars ? vizNumber(vars.gk) ?? vizNumber(0) : 0;
const lastSig = useRef<string | null>(null);

```

```

useEffect(() => {
 if (!liveCase && liveStep === null) return;
 const norm = (liveCase ?? "").trim().toLowerCase();
 const idx = CASE_BY_NAME[norm] ?? caseIdx;
 const sig = `${idx}:${liveStep ?? ""}`;
 if (sig === lastSig.current) return;
 const first = lastSig.current === null;
 lastSig.current = sig;

```



```

 }` }
 >
 {c.label}
 </button>
)))
 </div>

<div className="mb-3 text-xs sm:text-[13px] text-
 Кор,
 ({active.rotat
</div>

{/* шаг + пояснение стоят НАД картинкой: сначала
<div className="mb-3 rounded-xl border border-sla
 <div className="flex items-center gap-2 mb-1 fl
 <span
 className={`text-[10px] font-bold uppercase
 current.kind === "aim"
 ? "bg-amber-500/20 text-amber-300"
 : current.kind === "start"
 ? "bg-slate-700 text-slate-300"
 : "bg-indigo-500/20 text-indigo-300"
 }` }
 >
 {current.kind === "aim" ? "прицел" : curren

 <span className="text-sm font-bold text-white
 Шаг {idx + 1} из {total}. {current.title}

 </div>
 <p className="text-[13px] text-slate-400 leadin
</div>

<div className="bg-slate-900 rounded-xl border bo
 <Tree frame={current} prev={prevPos} keys={acti

 {/* обход слева направо – одинаковый на всех ка,
 <div className="mt-2 pt-2 border-t border-slate

```

```

 className="px-3 py-2 rounded-lg bg-slate-800 border-slate-700
 d: hover:text-slate-300 text-xs font-bold transition-colors"
 >
 ← Назад
</button>
<button
 onClick={() => {
 setPlaying(false);
 setFrame((f) => (f + 1) % total);
 }}
 className="px-4 py-2 rounded-lg bg-indigo-600 border-indigo-500
 w-indigo-900/40"
>
 {last ? "⏮ Сначала" : "Дальше →"}
</button>
<button
 onClick={() => setPlaying((p) => !p)}
 className="flex items-center gap-1.5 px-3 py-2 rounded-lg
 text-xs font-bold transition-colors"
>
 {playing ? <Pause className="w-3.5 h-3.5" /> : <Play />}
 {playing ? "Стоп" : "Авто"}
</button>
<button
 onClick={() => setSlow((s) => !s)}
 title="Автопрокрутка ещё медленнее"
 className={`flex items-center gap-1.5 px-3 py-2 rounded-lg
 {slow ? "bg-emerald-600/20 border-emerald-500 text-emerald-500" : "bg-slate-800 border-slate-700 text-slate-300"}
 `}
>
 <Gauge className="w-3.5 h-3.5" /> {slow ? "0.5x" : "1x"}
</button>
<button
 onClick={() => {
 setFrame(0);
 setPlaying(false);
 }}
 className="px-3 py-2 rounded-lg bg-slate-800 border-slate-700
 d: hover:text-slate-300 text-xs font-bold transition-colors"
>
 ↺ С начала
</button>

```

```

 </>
) : (

 запусти код из панели Python: переменная <с
 случай,
 а <code className="font-mono text-indigo-300">

)}
 </div>

 <p className="mt-2 text-[11px] text-slate-500">
 Листай кнопкой «Дальше» в своём темпе – кадры н
 </p>
 </div>
);
};

```

ФАЙЛ 73/111: src/components/SplayWalkViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useEffect, useMemo, useRef, useState } from 'react';

/**
 * Splay на БОЛЬШОМ дереве – по урокам treap-прохода.
 *
 * Главный принцип: НИКАКИХ «хоп». Каждый одиночный поворот
 * aim – прицел на тройку g–p–x, назван случай;
 * cut – отстёгиваем среднее поддереву β (пунктир, поворот);
 * rot – поворот: зелёным подсвечены рёбра, которых не было;
 * Рядом с логом всегда висит врезка «Механика одного поворота»
 * На каждом кадре – живая автопроверка in-order (сортировка)
 *
 * Чистые функции (splaySteps, rotateUp, bstInsert, collect)
 */

```

```

 }
 cur = cur.right;
 }
}
}

```

/\*\* Поднять child над его родителем (один атомарный пов

```

export function rotateUp(child: SN): void {
 const p = child.parent;
 if (!p) return;
 const g = p.parent;
 if (p.left === child) {
 p.left = child.right;
 if (child.right) child.right.parent = p;
 child.right = p;
 p.parent = child;
 } else {
 p.right = child.left;
 if (child.left) child.left.parent = p;
 child.left = p;
 p.parent = child;
 }
 child.parent = g;
 if (!g) return;
 if (g.left === p) g.left = child;
 else g.right = child;
}

```

/\*\* Рёбра в виде "p→c" — для diff между кадрами. \*/

```

export function collectEdges(t: SN | null): string[] {
 const es: string[] = [];
 const walk = (n: SN | null) => {
 if (!n) return;
 if (n.left) es.push(`${n.key}→${n.left.key}`);
 if (n.right) es.push(`${n.key}→${n.right.key}`);
 walk(n.left);
 walk(n.right);
 };
}

```

```

let root: SN | null = cloneWithParents(rootIn);
const steps: SplayStep[] = [];
if (!root)
 return { steps: [{ kind: "done", root: null, note:

// --- спуск ---
let cur: SN | null = root;
const pathKeys: number[] = [];
let found = false;
let last: SN = root;
while (cur) {
 last = cur;
 pathKeys.push(cur.key);
 if (key === cur.key) {
 found = true;
 break;
 }
 cur = key < cur.key ? cur.left : cur.right;
}
const x = found ? cur : last;
steps.push({
 kind: "search",
 root: cloneWithParents(root)!,
 pathKeys: [...pathKeys],
 note: isInsert
 ? `Вставили ${key} обычным BST-спуском. Теперь по
 : found
 ? `Спуск по ключу ${key}: путь ${pathKeys.join(
 : `Ключа ${key} нет. Спуск уперся в ${last.key}
 rotCount: 0,
});

// --- подъём ---
let rotCount = 0;
let target: SN = x as SN;
const fixRoot = () => {
 while (root && root.parent) root = root.parent;
};

```

```

while (target.parent) {
 const p = target.parent;
 const g = p.parent;
 if (!g) {
 emitRotation(
 target,
 "Zig",
 [p.key, target.key],
 `Zig: у ${target.key} нет деда — хватит одного
);
 continue;
 }
 const pIsLeft = g.left === p;
 const xIsLeft = p.left === target;
 if (pIsLeft === xIsLeft) {
 const side = pIsLeft ? "оба слева" : "оба справа";
 emitRotation(
 p,
 "Zig-Zig",
 [g.key, p.key],
 `Zig-Zig (${side}: линия ${g.key}—${p.key}—${ta
 ${g.key}, а ${target.key} пока не трогаем.`
);
 steps.push({
 kind: "aim",
 root: cloneWithParents(root)!,
 aimEdge: [target.parent.key, target.key],
 caseLabel: "Zig-Zig",
 note: `Теперь обычный зиг: ребро ${target.parent
 rotCount,
 });
 emitRotation(
 target,
 "Zig-Zig",
 [target.parent.key, target.key],
 `Доводим: ${target.key} над ${target.parent.key
);
 }

```

```

 let cur = t;
 while (cur) {
 if (key === cur.key) return d;
 cur = key < cur.key ? cur.left : cur.right;
 d += 1;
 }
 return -1;
 };
const heightOf = (t: SN | null): number => (t ? 1 + Math.max(
const inorderKeys = (t: SN | null): number[] => (t ? [
 ...inorderKeys(t.left),
 t.key,
 ...inorderKeys(t.right)
]);

export const INITIAL_KEYS = [50, 40, 35, 32, 30, 28, 12];

const buildInitial = (): SN => {
 let t: SN | null = null;
 for (const k of INITIAL_KEYS) t = bstInsert(t, k);
 return t!;
};

const CODE_LINES = [
 { line: 1, text: "while x has parent:" },
 { line: 2, text: " p = parent(x); g = parent(p)" },
 { line: 3, text: " if g is None: # Z" },
 { line: 4, text: " rotate(x over p)" },
 { line: 5, text: " elif p,g,x на одной линии: # " },
 { line: 6, text: " rotate(p over g) # B" },
 { line: 7, text: " rotate(x over p)" },
 { line: 8, text: " else: # Z" },
 { line: 9, text: " rotate(x over p) # H" },
 { line: 10, text: " rotate(x over g)" },
];

/* ----- врезка «механика одного по

const MechanismInset: React.FC<{ step: SplayStep | null
 const isCut = step?.kind === "cut";
 const isRot = step?.kind === "rot";
 return (

```

```

<text x="163" y="96" fill="#10b981" fontSize="20">
<g transform="translate(186,6)">
 <text y="10" fill="#94a3b8" fontSize="11" font>
 <line x1="70" y1="46" x2="45" y2="86" stroke=
 <line x1="70" y1="46" x2="95" y2="86" stroke=
 <line x1="45" y1="86" x2="27" y2="126" stroke=
 <line
 x1="45" y1="86" x2="63" y2="126"
 stroke={isRot ? "#10b981" : "#64748b"}
 strokeWidth={isRot ? 3 : 2}
 />
 <circle cx="70" cy="46" r="15" fill="#059669">
 <text x="70" y="50" textAnchor="middle" fill=
 <circle cx="45" cy="86" r="15" fill="#1e293b">
 <text x="45" y="90" textAnchor="middle" fill=
 <circle cx="27" cy="126" r="13" fill="#0f172a">
 <text x="27" y="130" textAnchor="middle" fill=
 <circle
 cx="63" cy="126" r="13"
 fill={isRot ? "#78350f" : "#0f172a"}
 stroke={isRot ? "#10b981" : "#64748b"}
 strokeWidth={isRot ? 2.5 : 1.5}
 />
 <text x="63" y="130" textAnchor="middle" fill=
 <circle cx="95" cy="126" r="13" fill="#0f172a">
 <text x="95" y="130" textAnchor="middle" fill=
 <text x="112" y="92" fill={isRot ? "#10b981"
 {isRot ? "↓ β теперь при p" : "β при p"}
 </text>
 <text x="30" y="160" fill="#64748b" fontSize=
</g>
</svg>
<p className="text-[10px] text-slate-500 leading-
 Обход слева-направо α x β p γ не меняется никог
 числа поворотов. Единственный, кто меняет хозяин
</p>
</div>
);

```



```
useEffect(() => {
 if (!playing || !steps) return;
 if (idx >= steps.length - 1) {
 setPlaying(false);
 baseRef.current = steps[steps.length - 1].root ??
 setTreeVers((v) => v + 1);
 return;
 }
 timer.current = setTimeout(() => setIdx((v) => v +
 return () => {
 if (timer.current) clearTimeout(timer.current);
 });
}, [playing, idx, steps]);
```

```
const step = steps ? steps[Math.min(idx, steps.length
const viewRoot = step ? step.root : baseRef.current;
const pos = useMemo(() => layout(viewRoot), [viewRoot
const nodes = useMemo(() => [...pos.entries()].map(({[
const edges = useMemo(() => {
 const es: { a: number; b: number }[] = [];
 const walk = (n: SN | null) => {
 if (!n) return;
 if (n.left) es.push({ a: n.key, b: n.left.key });
 if (n.right) es.push({ a: n.key, b: n.right.key });
 walk(n.left);
 walk(n.right);
 };
 walk(viewRoot);
 return es;
}, [viewRoot, treeVers, idx]);
```

```
const autoW = Math.max(VIEW_W, ...nodes.map((n) => n.p
const autoH = Math.max(300, ...nodes.map((n) => n.py
```

```
const dBefore = useMemo(() => (steps ? depthOf(steps[
const dNow = steps ? depthOf(viewRoot, steps[steps.le
const hBefore = useMemo(() => (steps ? heightOf(steps
```

```

 if (!step) return "#64748b";
 if (step.kind === "search" && step.pathKeys?.includes(k)) return "#38bdf8";
 if ((step.kind === "aim" || step.kind === "cut") &&
 const e = step.aimEdge ?? step.cutEdge!;
 if (e.includes(k)) return "#f59e0b";
 }
 if (step.pathKeys?.includes(k)) return "#38bdf8";
 return "#64748b";
};

return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border" >
 { /* Шанка */ }
 <div className="flex flex-wrap items-center justify-center" >
 <div className="flex items-center gap-3" >
 <div className="p-2.5 bg-indigo-600 text-white" >

 </div>
 <div>
 <h3 className="text-2xl font-bold text-white" >
 <p className="text-sm text-indigo-300" >Клики
 </p>
 </div>
 </div>
 <div className="flex flex-wrap items-center gap-3" >
 <input
 value={input}
 onChange={(e) => setInput(e.target.value)}
 onKeyDown={(e) => {
 if (e.key === "Enter") {
 const v = parseInt(input, 10);
 if (!Number.isNaN(v)) {
 const has = inorderKeys(baseRef.current);
 if (has) runKey(v);
 else doInsert();
 setInput("");
 }
 }
 }}
 </input>
 </div>
 </div>
)
);

```

```

 ld bg-slate-700 text-white disabled:opacity
 В начало
 </button>
 <button onClick={() => { setPlaying(false); s
 -lg text-xs font-bold bg-slate-700 text-whi
 ← Шаг
 </button>
 <button onClick={() => setPlaying(!playing)}
 er-600" : "bg-emerald-600"}`}>
 {playing ? " Пауза" : "▶ Пуск"}
 </button>
 <button onClick={() => { setPlaying(false); s
 className="px-3 py-1.5 rounded-lg text-xs fo
 Шаг →
 </button>

 Шаг {idx + 1} из {steps.length} • поворотов

 {step?.caseLabel && (
 <span className="px-2.5 py-1 rounded-full t
 00/60">
 {step.caseLabel}

)}
</div>
))

{/* Дерево */}
<div className="bg-indigo-950/20 rounded-xl p-4 b
 {!steps && (
 <div className="absolute top-3 left-3 bg-indi
 ont-bold">
 Кликни по любому узлу – splay покадрово
 </div>
)}
 <svg viewBox={`0 0 ${autoW} ${autoH}`} classNam
 {edges.map((e) => {
 const a = pos.get(e.a);

```

```

 >
 <circle
 r={isAimNode ? 24 : 20}
 fill={stepColor(n.key)}
 stroke={strokeColor(n.key)}
 strokeWidth={isAimNode ? 4 : 2}
 className="transition-all duration-300"
 />
 <text y={4} fill="#ffffff" fontSize="13"
 >
 {n.key}
 </text>
 </g>
);
}}
</svg>

```

```

{/* Лог + механика */}
<div className="w-full flex flex-col lg:flex-row"
 <div className="flex-1 bg-slate-950/80 p-4 rounded"
 <p className="font-semibold text-amber-400"
 {step?.kind === "aim"
 ? " Прицел:"
 : step?.kind === "cut"
 ? "∞ Отстёгиваем среднее поддерево:"
 : step?.kind === "rot"
 ? " Поворот:"
 : step?.kind === "search"
 ? " Спуск:"
 : "Статус:"}
 </p>
 <p className="min-h-[40px] flex items-center"
 {step ? step.note : "Кликни по узлу или в
ке справа – механика одного поворота."}
 </p>
 {steps && step && step.kind !== "search" &&
 <p className={`text-xs mt-2 ${checkOk ? "
 {checkOk ? " " : " "}} проверка кадра: o

```

```

<div className="w-full lg:w-1/2 bg-slate-900/60" >
 <h4 className="text-sm font-bold text-slate-200" >
 <div className="bg-slate-950/80 rounded-lg p-4" >
 {CODE_LINES.map((l) => {
 const zig = step?.caseLabel === "Zig" &&
 const zz1 = step?.caseLabel === "Zig-Zig" &&
 const zz2 = step?.caseLabel === "Zig-Zig" &&
 const za1 = step?.caseLabel === "Zig-Zag" &&
 const za2 = step?.caseLabel === "Zig-Zag" &&
 const active = zig || zz1 || zz2 || za1 || za2
 return (
 <div key={l.line} className={`py-1 px-2 border-400" : "text-slate-400"}` >

 {l.text}

 </div>
)
 })}
 </div>
 </div>
</div>
</div>
);
};

export default SplayWalkViz;

```

ФАЙЛ 74/111: src/components/StackViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState, useCallback } from 'react';
import { ArrowUp, Sparkles } from 'lucide-react';
import { useVizRuntime, vizArray } from '../data/vizSteps';

interface Plate {

```

```

 animState: "idle",
)))
 : dirtyStack;

const [cleanRack, setCleanRack] = useState<Plate[]>([]);
const [isWashing, setIsWashing] = useState(false);
const [showSoap, setShowSoap] = useState(false);
const [shakeEmpty, setShake] = useState(false);
const [log, setLog] = useState<string[]>([]);

const addLog = (msg: string) => setLog(prev => [...prev, msg]);

// PUSH: Положить грязную тарелку сверху стопки
const pushPlate = useCallback(() => {
 if (isWashing) return;
 if (dirtyStack.length >= 7) {
 addLog('⚠ Стопка слишком высокая, тарелки могут упасть');
 setShake(true);
 setTimeout(() => setShake(false), 400);
 return;
 }

 const preset = PLATE_PRESETS[counter % PLATE_PRESETS.length];
 counter++;

 const newPlate: Plate = {
 id: Date.now().toString(),
 ...preset,
 isDirty: true,
 animState: 'in',
 };

 setDirtyStack(prev => [...prev, newPlate]);
 addLog(`  PUSH: Положили ${newPlate.emoji} сверху с ${newPlate.preset}`);

 setTimeout(() => {
 setDirtyStack(prev => prev.map(p => p.id !== newPlate.id ? p : { ...p, animState: 'idle' }));
 }, 500);
}, [counter]);

```

```

 counter = 0;
 setDirtyStack([
 { id: 'r1', name: 'Тарелка из-под спагетти', emoji: '🍝', State: 'in' },
 { id: 'r2', name: 'Тарелка из-под пиццы', emoji: '🍕', imState: 'in' },
 { id: 'r3', name: 'Блюдец от торта', emoji: '🍰', mState: 'in' },
]);
 setCleanRack([]);
 setIsWashing(false);
 setShowSoap(false);
 setLog([' Стол сброшен. Посуда снова грязная!']);
 });

```

```

const topIndex = displayStack.length - 1;

```

```

return (
 <div className="flex flex-col gap-6">
 {/* МНЕМОНИКА / ПРАВИЛО */}
 <div className="bg-blue-950/40 border border-blue-950/40 p-6">
 <div className="text-3xl"> </div>
 <div>
 <h3 className="font-black text-blue-400 text-2xl">ПРАВИЛО</h3>
 <p className="text-xs text-slate-300 mt-1 leading-tight">
 Ты складываешь грязные тарелки друг на друга.
 А когда приходит время мыть, ты берёшь именную тарелку.
 Попытаешься вытащить нижнюю – всё рухнет!
 </p>
 </div>
 </div>
 </div>

```

```

 {/* УПРАВЛЕНИЕ */}
 <div className="flex items-center gap-3 flex-wrap">
 <button
 onClick={pushPlate}
 disabled={isWashing}
 className="flex-1 min-w-[150px] bg-gradient-to-r from-blue-500 to-blue-600 text-white font-medium rounded-md">
 Добавить тарелку
 </button>
 </div>

```

```

 <span className="w-1.5 h-1.5 rounded-full b
 ПАКОВИНА
 </div>

```

```

{ /* Мыльные пузыри при мытье */
{showSoap && {
 <div className="absolute inset-0 pointer-ev
 <div className="absolute w-6 h-6 rounded-
 "></div>
 <div className="absolute w-4 h-4 rounded-
 "></div>
 <div className="absolute w-5 h-5 rounded-
 "></div>
 <div className="absolute text-4xl anim-cr
 </div>
}}

```

```

{displayStack.length === 0 ? {
 <div className="flex-1 flex flex-col items-
 <span className="text-6xl mb-3 animate-pu
 <p className="text-white font-black text-
 <p className="text-slate-500 text-xs mt-1
 </div>
} : {

```

```

 <div className={`flex-1 flex flex-col justi

 { /* Указатель TOP */ }
 <div
 className="absolute right-8 flex items-
 style={{ bottom: `${24 + topIndex * 44}
 >
 <span className="text-xs font-mono text
er gap-1">
 <ArrowUp className="w-3.5 h-3.5" />
 ВЕРШИНА (TOP)

 <span className="text-blue-400 animate-p
 </div>

```



```

</div>

<div className="absolute top-4 right-4 flex items-center justify-center
 order-emerald-800/80 text-[9px] font-mono">
 <Sparkles className="w-3.5 h-3.5 animate-pulse">
 ЧИСТО
 </div>

<div className="flex-1 flex flex-col justify-between"
 {cleanRack.length === 0 ? {
 <div className="flex-1 flex flex-col items-center justify-center"

 <p className="text-xs font-bold font-mono">
 <p className="text-[10px] mt-1">Здесь 6
 </div>
) : {
 <div className="flex flex-col gap-2 w-full"
 {cleanRack.map(plate => {
 <div
 key={plate.id}
 className="w-full h-8 rounded-full
items-center justify-center gap-2 opacity-1
 >
 {plate.id}

 </div>
 })})}
 </div>
 })}
 </div>

 {/* Столешница сушилки */}
 <div className="w-full h-4 bg-gradient-to-r from-emerald-500 to-emerald-200"
 </div>
</div>

{/* ЛОГ ДЕЙСТВИЙ */}
<div className="bg-slate-900 border border-slate-800 p-4"

```

```

 ;

 while (j > 0 && s[i] !== s[j]) {
 steps.push({ i, j, pi: [...pi], highlight:
 яд назад: откат j = pi[j - 1] = ${pi[j -
 j = pi[j - 1];
 }

 if (s[i] === s[j]) {
 steps.push({ i, j, pi: [...pi], highlight:
 ска увеличивается.` });
 j++;
 } else {
 steps.push({ i, j, pi: [...pi], highlight:
 овпадений нет.` });
 }

 pi[i] = j;
 steps.push({ i, j, pi: [...pi], desc: `Записыва

 if (j === pattern.length) {
 steps.push({ i, j, pi: [...pi], found: true
 }
 }
 return { s, steps, patternLength: pattern.length };
}

```

```

function generateZSteps(pattern: string, text: string)
 if (!pattern) pattern = " ";
 const s = pattern + "#" + text;
 const n = s.length;
 const z = new Array(n).fill(0);
 const steps: any[] = [];

 let l = 0, r = 0;
 steps.push({ i: 0, l, r, z: [...z], desc: `Начало.

 for (let i = 1; i < n; i++) {

```

```

export function StringAlgorithmsViz({ defaultMode = "kmp"
 const [mode, setMode] = useState<"kmp" | "z">(defaultMode);
 const [pattern, setPattern] = useState("aba");
 const [text, setText] = useState("abacaba");

 // Ensure inputs are valid dynamically
 const safePattern = pattern || " ";
 const safeText = text || " ";

 const { s: demoString, steps, patternLength: demoPatternLength } =
 if (mode === "kmp") return generateKmpSteps(safePattern, safeText);
 else return generateZSteps(safePattern, safeText);
 }, [mode, safePattern, safeText]);
const runtime = useVizRuntime();
const runtimeString = vizString(runtime?.variables?.runtimeString);
const s = runtimeString ?? demoString;
const patternLength = runtimeString ? runtimeString.patternLength : demoPatternLength;

const [autoPlay, setAutoPlay] = useState(false);
const [stepIdx, setStepIdx] = useState(0);
const timer = useRef<NodeJS.Timeout | null>(null);

// Reset when inputs or mode change
useEffect(() => {
 setStepIdx(0);
 setAutoPlay(false);
}, [s, mode]);

// Timer loop for autoPlay
useEffect(() => {
 if (autoPlay) {
 timer.current = setInterval(() => {
 setStepIdx(prev => {
 if (prev >= steps.length - 1) { setAutoPlay(false); }
 return prev + 1;
 });
 }, 1200);
 }
}, [autoPlay, steps]);

```

```

 </button>
 <button onClick={() => setMode("z")}
 className={`px-3 py-1.5 text-xs
" : "text-slate-400 hover:text-slate-300"}`
 Z-ФУНКЦИЯ
 </button>
 </div>

 <div className="flex items-center gap-2"
 <div className="bg-slate-800 p-1 fl
 <span className="text-[10px] te
 <input value={pattern} onChange
xt-white text-xs font-bold font-mono px-2 p
 </div>
 <div className="bg-slate-800 p-1 fl
 <span className="text-[10px] te
 <input value={text} onChange={e
te text-xs font-bold font-mono px-2 py-1 ou
 </div>
 </div>
 </div>

 <div className="bg-slate-950 rounded-xl bor
px] flex items-center justify-start">
 <div className="flex gap-1.5 px-4 h-ful
 {s.split("").map((char, index) => {
 const isPattern = index < patte
 const isSeparator = index === p

 let borderColor = "border-slate
 let bgColor = "bg-slate-900";
 let textColor = isSeparator ? "

 // KMP Logic
 if (mode === "kmp") {
 if (step.highlight?.include
 borderColor = "border-a
 bgColor = step.match ==

```

```

 slate-700/50 font-mono font-bold">
 {{mode == "kmp" ?
 </div>

 {/* KMP fingers */}
 {mode == "kmp" && step
 <div className="abs

z-20">
 <span className=
 </div>
 }}
 {mode == "kmp" && step
 <div className="abs
 <span className=
 </div>
 }}
 {mode == "kmp" && step
 <div className="abs

20">
 <span className=
 </div>
 }}

 {/* Z fingers */}
 {mode == "z" && step.i
 <div className="abs

e z-20">
 <span className=
 </div>
 }}
 {mode == "z" && step.z
 <div className="abs
 <span className=
 </div>
 }}
 {mode == "z" && step.z
 <div className="abs
 <span className=

```

```

 { /* Description Log */
 <div className="flex-1 bg-slate-900/50 overflow-hidden">
 <div className="absolute top-0 right-0">

 {step.desc}

 </div>
 <div className="text-[10px] text-slate-400">
 {steps.length}</div>
 <div className="text-sm text-slate-400">
 {step.desc}
 </div>
 </div>
 </div>

 { /* Why this matters card */
 <div className="bg-slate-800 p-4 rounded-xl">

 <p className="text-xs text-slate-400">
 Обрати внимание на хитрость с решёткой.
 Мы пишем Шаблон + # + Текст.
 Когда значени {mode === 'kmp' ? "пробел" : "не пробел"}
 было найдено!
 Пройди шаги вручную, посмотри, как работает алгоритм
 для пропуска работы (в Z).
 </p>
 </div>

 { /* Code Sneak Peek */
 <div className="bg-slate-900 border border-slate-800 p-4">
 <div className="bg-slate-800 px-4 py-2 border border-slate-400 uppercase">
 Код C++ ({mode === 'kmp' ? 'КМП' : 'ЗКМП'})
 </div>
 <pre className="p-4 text-xs font-mono text-slate-400">
{mode === 'kmp' ? `vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
 int j = pi[i-1];

```

```

* Появляется по наведению курсора и по фокусу с клавиатуры
*/
export const Tooltip: React.FC<TooltipProps> = ({ content }) => {
 const [open, setOpen] = useState(false);

 const hiddenShift = side === "top" ? "translate-y-1" : "translate-y-0";

 return (
 <span
 className={`relative inline-flex ${className ?? ""}
 onMouseEnter={() => setOpen(true)}
 onMouseLeave={() => setOpen(false)}
 onFocus={() => setOpen(true)}
 onBlur={() => setOpen(false)}
 >
 {children}
 <span
 role="tooltip"
 className={`pointer-events-none absolute left-1/2 top-1/2
 bg-slate-900 px-3 py-2 text-left text-[11px]
 transition duration-150 ${
 side === "top" ? "bottom-full mb-2" : "top-full mt-2"
 } ${open ? "visible translate-y-0 opacity-100" : "invisible"}
 >
 {content}
 <span
 className={`absolute left-1/2 top-1/2 h-2 w-2 -translate-x-50%
 ${side === "top" ? "-bottom-[5px] border-b border-slate-500" : ""}
 >

);
};

```

```
 { u: 3, v: 4 },
];
```

```
interface TStep {
 desc: string;
 visited: Set<number>;
 fullyProcessed: Set<number>;
 currentNode: number | null;
 topoList: number[];
 dfsStack: number[];
}
```

```
export const TopologicalSortViz: React.FC = () => {
 const [steps, setSteps] = useState<TStep[]>([]);
 const [currentStepIdx, setCurrentStepIdx] = useState(0);
 useVizStepSync(currentStepIdx, setCurrentStepIdx, steps, (vars) => {
 const v = vizNumber(vars.v);
 const orderLength = vizArray(vars.order)?.length;
 if (v === null && orderLength === undefined) return null;
 const candidates = steps
 .map((step, index) => ({ step, index }))
 .filter(({ step }) => (v === null || step.currentNode !== v));
 return candidates.at(-1)?.index ?? null;
 });
 const [isPlaying, setIsPlaying] = useState(false);

 useEffect(() => {
 const listSteps: TStep[] = [];
 const visited = new Set<number>();
 const fullyProcessed = new Set<number>();
 const topoList: number[] = [];
 const dfsStack: number[] = [];

 const recordStep = (desc: string, currentNode: number) => {
 listSteps.push({
 desc,
 visited: new Set(visited),
 fullyProcessed: new Set(fullyProcessed),
 currentNode,
 topoList,
 dfsStack
 });
 };
 });
}
```



```

 topoList.push(u); // prepend behavior: we write it to the beginning
 recordStep(
 `DFS: закончили обработку '${dagNodes[u].name}'
 ,
 u,
);
 };
};

// Run DFS for all unvisited
for (let i = 0; i < 5; i++) {
 if (!visited.has(i)) {
 recordStep(
 `Запускаем новый обход DFS от нетронутой вершины
 null,
);
 dfs(i);
 }
}

recordStep(
 "Топологическая сортировка завершена! Полученный список:
 null,
);
setSteps(listSteps);
setCurrentStepIdx(0);
}, []));

const step = steps[currentStepIdx] || {
 desc: "Запуск...",
 visited: new Set(),
 fullyProcessed: new Set(),
 currentNode: null,
 topoList: [],
 dfsStack: [],
};

useEffect(() => {
 let timer: NodeJS.Timeout;

```

```

 <Pause className="w-4 h-4 ml-1" />
) : {
 <Play className="w-4 h-4 ml-1" />
 }}
 {isPlaying ? "Пауза " : "Старт "}
 </button>

 <button
 onClick={reset}
 aria-label="Сбросить сортировку"
 title="Сбросить сортировку"
 className="flex items-center justify-center
 ors"
 >
 <RotateCcw className="w-4 h-4" />
 </button>
 </div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-12" >
 {/* Playfield SVG */}
 <div className="lg:col-span-8 bg-[#0B1120] relative" >
 <svg className="w-full h-full max-w-[650px]" >
 <defs>
 <marker
 id="dag-arrow"
 viewBox="0 0 10 10"
 refX="24"
 refY="5"
 markerWidth="6"
 markerHeight="6"
 orient="auto-start-reverse"
 >
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#4F81BD" />
 </marker>
 <marker
 id="dag-arrow-active"
 viewBox="0 0 10 10"

```

```
const isProcessed = step.fullyProcessed.h
```

```
let fill = "#1e293b";
let stroke = "#334155";
let textColor = "text-slate-400";
void textColor;
```

```
if (isCurrent) {
 fill = "#312e81";
 stroke = "#818cf8";
 textColor = "text-white font-bold";
} else if (isProcessed) {
 fill = "#022c22";
 stroke = "#059669";
 textColor = "text-emerald-300";
} else if (isVisited) {
 fill = "#3730a3";
 stroke = "#6366f1";
 textColor = "text-indigo-200";
}
```

```
return (
 <g key={node.id}>
 <rect
 x={node.x - 55}
 y={node.y - 22}
 width="110"
 height="44"
 rx="8"
 fill={fill}
 stroke={stroke}
 strokeWidth="2"
 className="transition-all duration-!
 />
 <text
 x={node.x}
 y={node.y - 4}
 textAnchor="middle"
```



```
* Интерактивный Tgear – шесть режимов:
* build – сборка: ВИДИМАЯ сортировка точек (и чем с
* координатная плоскость НАВЕРХУ (ассоциаци
* чек-лист инвариантов (автопроверка, как у
* split – разрез по x с двумя корзинами (грамотная
* merge – склейка двух деревьев по y.
* erase – удаление ПРОИЗВОЛЬНОЙ точки: найти по x,
* layers – миф «дети в массиве по $2k/2k+1$ »: работает
* search – «отсортирую бинарным поиском»: поиск не с
* сортировка платит $\geq n \cdot \log n$ сравнений, со
*
* Канонический набор: все x и все y различны $\Rightarrow$ дерево
* Узел идентифицируется парой $id = "x:y"$.
*/
```

```
export interface Pair {
 key: number;
 grade: number;
}
type NodeId = string;
const pairId = (key: number, grade: number): NodeId =>

interface TNode {
 id: NodeId;
 key: number;
 grade: number;
 left: TNode | null;
 right: TNode | null;
}

export const POINTS: Pair[] = [
 { key: 45, grade: 8 },
 { key: 3, grade: 6 },
 { key: 6, grade: 5 },
 { key: 2, grade: 4 },
 { key: 9, grade: 3 },
 { key: 7, grade: 2 },
 { key: 12, grade: 0 },
```

```

 trace: { atId: NodeId; goLeft: boolean; note: string
}
export function splitTree(root: TNode | null, x: number) {
 const trace: SplitResult["trace"] = [];
 const go = (t: TNode | null): { l: TNode | null; r: TNode | null } {
 if (!t) return { l: null, r: null };
 if (t.key <= x) {
 const sub = go(t.right);
 trace.push({ atId: t.id, goLeft: true, note: `(${t.key}
 о правое поддерево.` });
 t.right = sub.l;
 return { l: t, r: sub.r };
 }
 const sub = go(t.left);
 trace.push({ atId: t.id, goLeft: false, note: `(${t.key}
 о левое поддерево.` });
 t.left = sub.r;
 return { l: sub.l, r: t };
 };
 const res = go(root);
 return { l: res.l, r: res.r, trace };
}

export interface MergeResult {
 root: TNode | null;
 trace: { aId: NodeId | null; bId: NodeId | null; chosenId: NodeId | null;
}
export function mergeTree(a: TNode | null, b: TNode | null) {
 const trace: MergeResult["trace"] = [];
 const go = (a: TNode | null, b: TNode | null): TNode | null {
 if (!a || !b) {
 trace.push({ aId: a?.id ?? null, bId: b?.id ?? null, chosenId: null,
 урое целиком.` });
 return a ?? b;
 }
 if (a.grade > b.grade) {
 trace.push({ aId: a.id, bId: b.id, chosenId: a.id,
 ем его правое поддерево с (${b.key}, ${b.grade}

```

```
export function checkTreap(root: TNode | null): { bst: boolean; heap: boolean } {
 let bst = true;
 let heap = true;
 let prev: number | null = null;
 const walk = (n: TNode | null, min: number, max: number, parentGrade: number | null) => {
 if (!n) return;
 if (n.key <= min || n.key > max) bst = false;
 if (parentGrade !== null && n.grade > parentGrade) heap = false;
 walk(n.left, min, n.key, n.grade);
 if (prev !== null && n.key < prev) bst = false;
 prev = n.key;
 walk(n.right, n.key, max, n.grade);
 };
 walk(root, -Infinity, Infinity, null);
 return { bst, heap };
}
```

/\* ----- раскладка сцены дерева ----- \*/

```
const layoutTree = (root: TNode | null) => {
 const pos = new Map<NodeId, { px: number; py: number }>();
 let slot = 0;
 const walk = (n: TNode | null, depth: number) => {
 if (!n) return;
 walk(n.left, depth + 1);
 pos.set(n.id, { px: 64 + slot * 92, py: 44 + depth * 100 });
 slot += 1;
 walk(n.right, depth + 1);
 };
 walk(root, 0);
 return pos;
};
```

```
const treeEdges = (root: TNode | null): { from: NodeId; to: NodeId }[] => {
 const es: { from: NodeId; to: NodeId }[] = [];
 const walk = (n: TNode | null) => {
 if (!n) return;
 if (n.left) es.push({ from: n.id, to: n.left.id });
 if (n.right) es.push({ from: n.id, to: n.right.id });
 walk(n.left);
 walk(n.right);
 };
 walk(root);
 return es;
};
```

```

x1={a.px} y1={a.py} x2={b.px} y2={b.py}
stroke={edgeColor ? edgeColor(e.from, e.to) : "#ccc"}
strokeWidth={2.5}
strokeDasharray={edgeDash && edgeDash(e.from, e.to)}
className="transition-all duration-500"
 />
);
 });
 {nodes.map((n) => {
 const p = pairParts(n.id);
 const pulse = pulseId === n.id;
 return (
 <g key={`tn-${n.id}`} transform={`translate(${p[0]px, ${p[1]px})`}
 <circle r={pulse ? 24 : 20} fill={nodeColor(n)}
 > strokeWidth={pulse ? 4 : 2} className="transition-all duration-500"
 <text y={-2} fill="#fff" fontSize="13" fontStyle="italic"
 <text y={13} fill="#cbd5e1" fontSize="10" fontStyle="italic"
 </g>
);
 });
</svg>
{subtitle && <p className="text-xs text-slate-400">
</div>
);
};

const pairParts = (id: NodeId): [number, number] => {
 const [a, b] = id.split(":").map(Number);
 return [a, b];
};

/* ----- плоскость (клетчатая бумага) -----

const PL_W = 780;
const PL_H = 300;
const PX = (key: number) => 42 + (key - 1) * ((PL_W - 6) / 8);
const PY = (grade: number) => 24 + (8 - grade) * ((PL_H - 24) / 7);

```



```

{drawnEdges.map((e, i) => {
 const a = byPair(e.a);
 const b = byPair(e.b);
 const isBad = badEdge && ((badEdge.a.key === e.a.key && badEdge.b.key === e.b.key) || (badEdge.a.key === e.b.key && badEdge.b.key === e.a.key));
 return <line key={`pe-${i}`} x1={a.x} y1={a.y} x2={b.x} y2={b.y} strokeDasharray={isBad ? "4,3" : "none"} />
})}

{/* временная линия игры */}
{interactive && selected && cursor && (
 <line
 x1={PX(selected.key)} y1={PY(selected.grade)}
 stroke="#818cf8" strokeWidth={2.5} strokeDasharray={[]} />
)}

{points.map((p) => {
 const done = drawnEdges.some((e) => (e.a.key === p.key && e.b.key === p.key));
 const isSel = selected && selected.key === p.key;
 return (
 <g
 key={`pp-${p.key}:${p.grade}`}
 className="transition-all duration-500"
 onPointerDown={interactive && onPointDown ? () => onPointDown(p) : undefined}
 onPointerUp={interactive && onPointUp ? () => onPointUp(p) : undefined}
 style={interactive ? { cursor: "pointer" } : undefined}
 >
 <circle cx={PX(p.key)} cy={PY(p.grade)} r={10}
 stroke={isSel ? "#a5b4fc" : done ? "#38bdf8" : "#ccc"} strokeWidth={2} />
 <text x={PX(p.key)} y={PY(p.grade) + 3.5} font-size={12} font-weight="bold">{p.key}</text>
 </g>
);
})}

```

```

 if (sameP(cur, target)) return true;
 if (seen.has(gpid(cur))) return false;
 seen.add(gpid(cur));
 cur = parentOf(edges, gpid(cur)).a;
 }
 return false;
};

/** Проверка BST леса: у узла ≤1 ребёнка слева и ≤1 справа
export function forestIsBst(edges: GEdge[], all: Pair[]) {
 const kids = new Map<string, Pair[]>();
 for (const e of edges) kids.set(gpid(e.a), [...(kids.get(gpid(e.b)) || [])]);
 const hasParent = new Set(edges.map((e) => gpid(e.b)));
 const roots = all.filter((p) => !hasParent.has(gpid(p.a)));
 const ok = (n: Pair | undefined, lo: number, hi: number) => {
 if (!n) return true;
 if (n.key <= lo || n.key > hi) return false;
 const ch = (kids.get(gpid(n)) ?? []).slice().sort((a, b) => a.key - b.key);
 const left = ch.filter((c) => c.key < n.key);
 const right = ch.filter((c) => c.key > n.key);
 if (left.length > 1 || right.length > 1) return false;
 return ok(left[0], lo, n.key) && ok(right[0], n.key, hi);
 };
 return roots.every((r) => ok(r, -Infinity, Infinity));
}

/** Полный вердикт по текущим линиям. */
export function gameVerdict(edges: GEdge[], all: Pair[]) {
 const heap = edges.every((e) => e.a.grade > e.b.grade);
 const oneParent = new Set(edges.map((e) => gpid(e.b)));
 const acyclic = !edges.some((e) => reaches(edges, e.a, e.b));
 const bst = forestIsBst(edges, all);
 const roots = all.filter((p) => !edges.some((e) => gpid(e.a) === p.a));
 const complete = edges.length === all.length - 1 && roots.length === 1;
 return { heap, oneParent, acyclic, bst, complete, win };
}

/** Попытка хода: почему нельзя — или ОК. */

```

```

 }
 seen.add(id);
 pairs.push({ key: nums[0], grade: nums[1] });
 }
 const xs = pairs.map((p) => p.key);
 const ys = pairs.map((p) => p.grade);
 return { pairs, errors, equalX: new Set(xs).size !== 0 };
};

```

```
const DEMO RAW = "45 8, 3 6, 6 5, 2 4, 9 3, 7 2, 12 0,
```

/\* ----- игра «соедини мышкой»: чистая логика.

```
/** Ребро игры: a = родитель (выше по y), b = ребёнок.
```

```
export interface GameEdge {
```

a: Pair:

b: Pair:

1

```
const pid = (p: Pair) => `${p.key}:${p.grade}`;
```

```
export function canConnect()
```

```
edges: GameEdge[],
```

x: Pair.

v: Pair,

```
): { ok: boolean; edge?: GameEdge; reason?: string } {
```

```
if (x.key == y.key && x.grade == y.grade) return {
```

```
for (const e of edges) {
```

```
if ((pid(e.a) == pid(x) && pid(e.b) == pid(y)) ||
```

```
return { ok: false, reason: "такая линия уже пров
```

1

```
if (x.grade === y.grade) return { ok: false, reason:
```

11 } :

```
const [parent, child] = x.grade > y.grade ? [x, y] :
```

```
if (edges.some((e) => pid(e.b) === pid(child)))
```

```
return { ok: false, reason: `у точки ${child.key};`
```

```
// цикл: родитель уже сидит в компоненте ребёнка (реб.
```

```
const childComp = new Set<string>([pid(child)]);
```

```
let grew = true;
```

```

 const ra = find(pid(e.a));
 const rb = find(pid(e.b));
 if (ra !== rb) comp.set(ra, rb);
}
const comps = new Set(points.map((p) => find(pid(p))))
const connected = comps.size === 1;
const countOk = edges.length === n - 1;
// BST: дети по x, inorder не убывает; корень один; э
const children = new Map<string, Pair[]>();
for (const e of edges) children.set(pid(e.a), [...(ch
const roots = points.filter((p) => !parentCount.has(p
let bstOk = roots.length === 1;
if (bstOk) {
 const seen = new Set<string>();
 const inorder: number[] = [];
 const walk = (p: Pair): void => {
 if (seen.has(pid(p))) {
 bstOk = false; // цикл
 return;
 }
 seen.add(pid(p));
 const kids = children.get(pid(p)) ?? [];
 if (kids.length > 2) {
 bstOk = false;
 return;
 }
 const sorted = [...kids].sort((u, v) => u.key - v
 let left: Pair | null = null;
 let right: Pair | null = null;
 if (sorted.length === 2) {
 if (sorted[0].key === sorted[1].key || sorted[0
 bstOk = false; // двое детей с равным x или с
 return;
 }
 left = sorted[0];
 right = sorted[1];
 } else if (sorted.length === 1) {
 // одиночный ребёнок: сторона важна – определяе

```

```

 const right = kids.filter((k) => k.key > p.key);
 if (left.length > 1 || right.length > 1) return node;
 if (left[0]) node.left = build(left[0]);
 if (right[0]) node.right = build(right[0]);
 return node;
 };
 return build(rootPair);
}

```

```

const BuildMode: React.FC = () => {
 const [raw, setRaw] = useState(DEMO_RAW);
 const live = useTreapLive();

 // набор точек из Python-кода пересобирает плоскость
 useEffect(() => {
 if (!live.pairsText || live.pairsText === raw) return;
 setRaw(live.pairsText);
 setPhase("connect");
 setOrder("y-desc");
 setSortIdx(3);
 setConnIdx(10000); // все рёбра проведены – ср
 setPlaying(false);
 // eslint-disable-next-line react-hooks/exhaustive-
 }, [live.pairsText]);
 const [phase, setPhase] = useState<BuildPhase>("sort");
 const [algo, setAlgo] = useState<SortAlgo>("none");
 const [order, setOrder] = useState<ConnectOrder>("y-d
 const [sortIdx, setSortIdx] = useState(0);
 const [connIdx, setConnIdx] = useState(0);
 const [playing, setPlaying] = useState(false);
 const [sortCost, setSortCost] = useState<string | nul

 const parsed = useMemo(() => parsePairs(raw), [raw]);
 const userPairs = parsed.errors.length ? [] : parsed.p

 const n = userPairs.length;

 const userTree = useMemo(() => (userPairs.length >= 1

```

```

const connEdgesAll = order === "y-desc" ? canonEdges
const connEdges = connEdgesAll.slice(0, connIdx);
const connDone = phase === "connect" && n >= 1 && conn

// --- игра «соедини сам» ---
const [gameEdges, setGameEdges] = useState<GameEdge[]>();
const [sel, setSel] = useState<Pair | null>(null);
const [cursor, setCursor] = useState<{ x: number; y: number}>();
const [gameMsg, setGameMsg] = useState<string | null>(null);
const [badEdge, setBadEdge] = useState<GameEdge | null>(null);
const [hintUsed, setHintUsed] = useState(false);

const resetGame = () => {
 setGameEdges([]);
 setSel(null);
 setCursor(null);
 setGameMsg(null);
 setBadEdge(null);
 setHintUsed(false);
};

const startGame = () => {
 resetGame();
 setPhase("game");
 setPlaying(false);
};

const verdict = useMemo(() => checkGameState(userPair), [userPair]);
const gameWon = phase === "game" && verdict.won;
const gameTree = useMemo(() => (gameWon ? edgesToTree(connEdgesAll) : null), [connEdgesAll, gameWon]);

const attemptConnect = (x: Pair, y: Pair) => {
 const r = canConnect(gameEdges, x, y);
 if (r.ok && r.edge) {
 setGameEdges((prev) => [...prev, r.edge!]);
 setGameMsg(r.reason ?? null);
 setBadEdge(null);
 } else {
 setGameMsg(r.reason ?? "так нельзя");
 }
};

```



```

 onClick={startGame}
 disabled={n < 2}
 className={`px-3 py-1.5 rounded-lg text-
go-600 text-white" : "bg-indigo-500/80 text
 >
 соединить сам
 </button>
 </>
 })
</div>
</div>
<Plane
 points={userPairs}
 drawnEdges={phase === "game" ? gameEdges : ph
 pending={null}
 interactive={phase === "game"}
 selected={sel}
 cursor={cursor}
 badEdge={badEdge}
 onPointDown={pointDown}
 onPointUp={pointUp}
 onMove={planeMove}
/>
<div className="mt-2 text-xs text-slate-300 bg-
 {phase === "sort" ? (
 algo === "none" ? (
 <>Ввод приходит как попало ({n} па
 ть.</>
) : sortIdx < 3 ? (
 algo === "quicksort" ? (
 <>⚡ quicksort: счётчик сравнений: <b cl
 {comparisons} (n·log2n).</>
) : (
 <> counting: точки в корзины по значени
 , потому что у – маленькие целые.</>
)
) : (
 <>Отсортировано ({sortCost ?? "способ не

```



```

 </>
 })

{phase === "game" && (
 <div className="flex flex-wrap gap-2 mt-3">
 <button onClick={undoEdge} disabled={!gameE
white disabled:opacity-30">↵ Убрать линию</
 <button onClick={resetGame} disabled={!game
ate-700 text-white disabled:opacity-30"> С
 <button onClick={showSolution} disabled={ga
te disabled:opacity-30"> Показать решение<
 </div>
 })
</div>

{/* Победное дерево в игре */}
{phase === "game" && gameTree && (
 <div className="bg-emerald-950/20 rounded-xl p-
 <h4 className="text-sm font-bold text-emerald
 <TreeView root={gameTree} subtitle="Слева-нап
ществует." />
 </div>
 })

{/* Дерево */}
{order === "y-desc" && userTree && (
 <div className="bg-slate-900/40 rounded-xl p-4
 <h4 className="text-sm font-bold text-slate-3
 <TreeView root={userTree} subtitle="Слева-нап
 </div>
 })
</div>
);
};

/* ===== РЕЖИМ SPLIT ===== */

const SPLIT KEYS = [2, 3, 6, 7, 9, 12, 45];

```

```

 </select>
 <button onClick={() => setStepIdx(Math.max(0, i
 bg-slate-700 text-white disabled:opacity-30">
 <button onClick={() => setStepIdx(Math.min(step
 font-bold bg-indigo-600 text-white disabled:op
 шаг {i
 </div>

 <div className="bg-slate-900/40 rounded-xl p-4 bo
 <h4 className="text-sm font-bold text-slate-300
 , розовые → R
 <TreeView
 root={fullTree}
 nodeColor={(id) => (inL.has(id) ? "#1e3a8a" :
 nodeStroke={(id) => (inL.has(id) ? "#60a5fa"
 subtitle={done ? "Каждый узел покрашен ровно
 />
 </div>

 <div className="text-xs text-slate-300 bg-slate-9
 {steps[idx]?.note}
 </div>

 {done && (
 <div className="grid grid-cols-1 md:grid-cols-2
 <div className="bg-sky-950/20 rounded-xl p-3
 <h5 className="text-sm font-bold text-sky-3
 <TreeView root={lTree} width={480} height={
 </div>
 <div className="bg-rose-950/20 rounded-xl p-3
 <h5 className="text-sm font-bold text-rose-
 <TreeView root={rTree} width={480} height={
 </div>
 </div>
 </div>
)}

 <div className="text-xs text-slate-400 bg-slate-9
 Как это работает на самом деле: идём с

```

```

 <TreeView root={l} width={480} height={220} n
 </div>
 <div className="bg-rose-950/20 rounded-xl p-3 b
 <h5 className="text-sm font-bold text-rose-300
 <TreeView root={r} width={480} height={220} n
 </div>
</div>

<div className="text-xs text-slate-300 bg-slate-900
 {cur?.chosenId ? {
 <=> {cur.note}</>
 } : {
 <=>Готово: {done ? "деревья склеены обратно в
 />
 }}
</div>

{done && {
 <div className="bg-emerald-950/20 rounded-xl p-3
 <h5 className="text-sm font-bold text-emerald-500
 <TreeView root={merged.root} subtitle="merge(
 </div>
)}

 <div className="text-xs text-slate-400 bg-slate-900
 Грамотно про «слияние»: у корней сравн
 в (то, что сохраняет порядок x) отправляется
 </div>
</div>
);
};

/* ===== РЕЖИМ ERASE ===== */

const EraseMode: React.FC = () => {
 const [key, setKey] = useState(9);
 const [stepIdx, setStepIdx] = useState(0);
 const live = useTreapLive();

```

```

 шаг {id}
 </div>

 {!done ? (
 <div className="bg-slate-900/40 rounded-xl p-4" >
 <TreeView
 root={fullTree}
 nodeColor={({id} => (pathSet.has(id) ? "#783"
 nodeStroke={({id} => (pathSet.has(id) ? "#f5
 subtitle="Куча сама умеет удалять только вер
 />
 </div>
) : (
 <div className="bg-slate-900/40 rounded-xl p-4" >
 <h4 className="text-sm font-bold text-slate-300">
 <TreeView root={res.root} subtitle="Валидность" >
 <ul className="mt-2 space-y-1 text-sm">
 обход слева-направо по-прежнему сортир
 каждое ребро вниз по у (куча целая) –

 </div>
))

 <div className="text-xs text-slate-300 bg-slate-900/40" >
 {steps[idx]?.note}
 </div>

 <div className="text-xs text-slate-400 bg-slate-900/40" >
 Ответ на «убрать можно только самый верхний
 ягивает merge двух детей. Удаляется любая<
 </div>
</div>
);
};

/* ===== РЕЖИМ LAYERS (2k/2k+1) =====

const LayersMode: React.FC = () => {

```

```

 }}}
 </div>
 <p className="text-xs text-slate-400">Полное
 → $2i/2i+1$ попадает в узел.</p>
 </div>
) : (
 <div className="bg-slate-900/40 rounded-xl p-4" >
 <h4 className="text-sm font-bold text-rose-300">
 >
 <TreeView root={full} subtitle="Попробуй «пос
 " />
 <div className="flex gap-2 overflow-x-auto mt-2" >
 {[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13]
 .map((i) => {
 const id = slots[i];
 return (
 <div key={i} className={`px-2.5 py-2 rounded
 border-slate-700` : "bg-rose-950/40 border-rose-900"} >
 <div className="text-slate-500 text-sm">
 {id ? <div className="text-white font-sm">
 </div>
 } : ""
 </div>
 </div>
);
 })
 }
 </div>
 <p className="text-xs text-rose-300 mt-2">См
 то левая ветка ушла на 4 уровня и заняла да
 казателями (left/right), а не арифметикой.</p>
 </div>
)}
}

<div className="text-xs text-slate-400 bg-slate-900/40 p-4" >
 Грабля «дети в массиве по $2k/2k+1$ »: форм
 ивым: слои дырявые, и «адрес» ребёнка посчита
</div>
</div>
);
};

/* ===== РЕЖИМ SEARCH (поиск ≠ сортировка) =====

```

```

return (
 <div className="space-y-4">
 <div className="flex gap-2">
 {stage < 2 && (
 <button onClick={() => { setStage(stage === 0
 digo-600 text-white">
 {stage === 0 ? "Найти у для x=7 бинарным по
 </button>
)}
 {stage === 2 && (
 <button onClick={() => setBidx(Math.min(bsSte
 -1.5 rounded-lg text-xs font-bold bg-indigo
 Шаг поиска →
 </button>
)}
 <button onClick={() => { setStage(0); setBidx(0
 Сброс</button>
 </div>

 <div className="flex gap-1.5 overflow-x-auto">
 {(stage === 2 ? sortedByKey : shuffled).map((p,
 const highlight = stage === 1 && i === wrongM
 const inBs = stage === 2 && (() => { const s
 const isMid = stage === 2 && (() => { const s
 const isFound = stage === 2 && bidx >= bsStep
 return (
 <div key={` ${p.key}:${p.grade}-${i}`} class
 nsition-all ${highlight ? "border-rose-500
 r-amber-500 bg-amber-950/30" : inBs ? "bord
 <div className="text-white font-bold">{p.
 <div className="text-slate-400 text-[10px
 </div>
)
);
)}}
 </div>

 <div className="text-xs text-slate-300 bg-slate-9
 {stage === 0 && <>Массив пар не отсортирован

```

```

 setSel(null);
 return;
 }
 const res = tryConnect(edges, sel, p);
 if (res.ok) {
 const parent = sel.grade > p.grade ? sel : p;
 const child = sel.grade > p.grade ? p : sel;
 setEdges([...edges, { a: parent, b: child }]);
 setMsg(`  Линия (${parent.key}; ${parent.grade})`);
 setSel(null);
 } else {
 setMsg("x " + res.reason);
 setSel(null);
 }
};

const win = v.win;

return (
 <div className="space-y-4">
 <div className="flex flex-wrap items-center gap-2">
 <button onClick={() => { setEdges(edges.slice(0, 3)); setSel(null); }}>
 <button onClick={() => { setEdges([]); setSel(null); }}>
 <button onClick={() => { setEdges(CANON_GAME.map(e => ({ a: e.a, b: e.b, grade: 1 }))); setSel(null); }}>
 </div>
 линий: {edges.length}
 </div>

 <div className="bg-emerald-950/20 rounded-xl p-4">
 <svg viewBox={`0 0 ${PL_W} ${PL_H + 20}`} className="w-full h-full">
 <defs>
 <pattern id="gm-grid" width="14" height="14">
 <path d="M 14 0 L 0 0 0 14" fill="none" stroke="black" stroke-width="1"/>
 </pattern>
 </defs>
 <rect x="34" y="16" width={PL_W - 46} height={PL_H - 16}/>
 </svg>
 </div>
 </div>
);

```

```

<ul className="space-y-1.5 text-sm">
 {v.hear ? " " : " "} каждое ребро – вниз
 {v.oneParent ? " " : " "} у каждой точки
 {v.acyclic ? " " : " "} циклов нет
 {v.bst ? " " : " "} обход слева-направо –
 {v.complete ? " " : " "} линий $n-1 = \{POI$

{win && {
 <p className="mt-3 text-sm font-bold text-eme
 ения просто не существует.</p>
 }}
</div>

<div className="text-xs text-slate-300 bg-slate-9
</div>
});
};

/* ===== СВЯЗЬ С КОМПИЛЯТОРОМ =====

/**
 * Живые переменные Python-кода страницы treap:
 * points / pairs – набор пар (x; y): по нему ПЕРЕСТРАИ
 * mid – разрез для вкладки Split;
 * key – удаляемая точка для вкладки Erase;
 * step / i – номер шага (листаёт кадры Split / E
 * height, n – высота дерева и число узлов (показ
 */
const useTreapLive = () => {
 const runtime = useVizRuntime();
 const vars = runtime?.variables;

 const pairsText = useMemo(() => {
 if (!vars) return null;
 const rawPts = vizArray(vars.points) ?? vizArray(va
 if (!rawPts || rawPts.length === 0) return null;
 const parts: string[] = [];
 rawPts.forEach((it) => {

```



```
// вкладка = подрежим страницы: панель Python подстав.
useEffect(() => {
 if (chapterId) emitVizDemo(chapterId, tab);
}, [chapterId, tab]);

return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border">
 <div className="flex flex-wrap items-center justify">
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-emerald-600 text-white">

 </div>
 <div>
 <h3 className="text-2xl font-bold text-white">
 <p className="text-sm text-emerald-300">Пар.
 </div>
 </div>
 </div>
 </div>

 <div className="flex flex-wrap gap-2 mb-5">
 {TABS.map((t) => {
 <button
 key={t.id}
 onClick={() => setTab(t.id)}
 className={`px-4 py-2 rounded-lg text-xs font-
 900/60 text-slate-400 hover:text-white border
 >
 {t.label}
 </button>
)})}
 </div>

 {/* что сейчас сообщил компилятор */}
 <div className="mb-4 flex flex-wrap items-center">
 <span className="text-[10px] uppercase tracking-
 <Terminal className="w-3.5 h-3.5" /> из компи.

 </div>
)}
)
```

```
import { liveMatrix, pickLiveVar, useVizRuntime } from
import { VizSourceBadge } from "../VizSourceBadge";

interface VizLiveStatusProps {
 /** Название демонстрации – подписывается в режиме вс
 demoTitle?: string;
 className?: string;
}

/** Имена, по которым визуализации обычно читают матрицу
const MATRIX_NAMES = ["d", "dist", "dp", "D", "S", "P",

/**
 * Строка состояния связи «компилятор → демонстрация» д
 *
 * Раньше компонент молча проигрывал собственный встроен
 * редакторе Python был пустой код, код с другими именами
 * размерности, – выглядело так, будто компилятор исполн
 * картинка живёт отдельной жизнью. Теперь источник данн
 * «данные из компилятора (i, j, dist 5x5)», «компилятор
 * пока нет» (отладчик стоит до первой строки) или «встр
 */
export function VizLiveStatus({ demoTitle, className =
 const runtime = useVizRuntime();
 const vars = runtime?.variables;
 const names = vars ? Object.keys(vars).filter((name)
 const linked = names.length > 0;

 const matrixSource = pickLiveVar(vars, MATRIX_NAMES);
 const matrix = liveMatrix(matrixSource?.value);

 const detail = matrix
 ? `${matrixSource?.name} ${matrix.sizeLabel}`
 : names.length
 ? names.slice(0, 6).join(", ") + (names.length >
```

```

import { QueueViz } from "../QueueViz";
import { SalmonAutomatonWidget } from "../SalmonAutomaton";
import { SegmentTreeVisualizer } from "../SegmentTreeViz";
import SplayPlayground from "../SplayPlayground";
import { StackViz } from "../StackViz";
import { StringAlgorithmsViz } from "../StringAlgorithms";
import { TopologicalSortViz } from "../TopologicalSortViz";
import { WaterfallAnimationWidget } from "../WaterfallAn";
import { Simulator } from "../Simulator";
import { PrefixSumViz } from "../visualizers/PrefixSumViz";
import { SparseTableViz } from "../visualizers/SparseTab";
import { TreapBuildViz } from "../TreapBuildViz";
```

```
export interface VizEntry {
 /** Заголовок панели/модалки. */
 title: string;
 /** Короткое пояснение, что здесь можно потыкать. */
 hint?: string;
 Component: React.FC;
}
```

```
/**
 * Единый реестр интерактивных демонстраций.
 * Используется и для панели под главой, и для всплывающ
 * и для модального окна «Открыть симулятор».
 */
```

```
export const VIZ_REGISTRY: Record<string, VizEntry> = {
 "stack-dfs": {
 title: "Стек и обход в глубину",
 hint: "Кладите и снимайте тарелки – это ровно то, ч
 Component: StackViz,
 },
 "queue-bfs": {
 title: "Очередь и обход в ширину",
 hint: "Очередь слева, живой BFS по графу справа.",
 Component: () => (
 <div className="space-y-10">
 <QueueViz />
```

песочница «подними узел сам» с проверкой порядка.

```
Component: SplayPlayground,
},
"graph-dfs-bfs": { title: "DFS и BFS на графе", Component: SplayPlayground,
"top-sort": { title: "Топологическая сортировка", Component: SplayPlayground,
"scc-kosaraju": { title: "Компоненты сильной связности", Component: SplayPlayground,
"graph-articulation": { title: "Точки сочленения", Component: SplayPlayground,
"bridges-code": { title: "Мосты: DFS + tin/low", Component: SplayPlayground,
"euler-path-vs-cycle": { title: "Эйлеров путь и цикл", Component: SplayPlayground,
"planarity-euler-formula": {
 title: "Планарность и покраска: $K_5/K_{3,3}$, $\chi \leq \Delta+1$, дельта-краска,
 hint: "Четыре вкладки: непланарные K_5 и $K_{3,3}$, жадная покраска,
 ашего кода красит вершины напрямую.",
 Component: PlanarColoringViz,
},
dijkstra: {
 title: "Дейкстра",
 hint: "Жадно забираем ближайшую вершину и релаксируем соседей.",
 Component: DijkstraViz,
},
"bellman-ford": {
 title: "Форд–Беллман",
 Component: BellmanFordViz,
},
floyd: {
 title: "Флойд–Уоршелл",
 Component: FloydViz,
},
"johnson-algo": { title: "Алгоритм Джонсона", Component: JohnsonViz,
mst: {
 title: "Остовное дерево: Краскал, Прима, Борувка",
 hint: "Три алгоритма (билеты 18–20) на одном графе. Переменные parent/mst/taken/u/v/w перекрашивают граф.",
 Component: KruskalSimulator,
},
"graph-components": {
 title: "Компоненты связности: обход и DSU",
 hint: "Две вкладки: покраска обходом (каждый новый узел красится в цвет соседа) и DSU.",
 Component: GraphComponentsViz,
},
}
```

```
import { Cpu, Film } from "lucide-react";
import { Tooltip } from "../Tooltip";

interface VizSourceBadgeProps {
 /**
 * true – картинку рисуют переменные из выполненного кода
 * false – компонент показывает собственную встроенную картинку
 */
 linked: boolean;
 /**
 * Компилятор уже выполнил код и прислал шаг, но переключатель не нажат
 * (отладчик стоит ДО первой строки или в коде другие комментарии)
 * подпись должна объяснять, почему картинка ещё не инициализирована
 */
 awaiting?: boolean;
 /** Строка пользовательского кода, на которой стоит отладчик */
 line?: number;
 /** Короткая подпись данных компилятора: «матрица 5x5» */
 detail?: string;
 /** Как называется встроенная демонстрация (для подписи) */
 demoTitle?: string;
 /** Размеры демо-сетки и данных пользователя не совпадают */
 mismatch?: string | null;
 className?: string;
}

/**
 * Явный индикатор источника данных визуализации.
 *
 * Без него демонстрация молча проигрывала собственный код
 * когда редактор Python пуст или в нём другие имена/переменные
 * так, будто компилятор «исполняет какой-то свой код».
 * «из компилятора» или «встроенное демо».
 */
export function VizSourceBadge({ linked, awaiting = false, line, detail, demoTitle, mismatch, className }) {
 const tip = linked
```

```
 className="cursor-help rounded-full border
 0"
 >
 размерность не совпадает

 </Tooltip>
) : null}
</div>
);
}
```

ФАЙЛ 82/111: src/components/WaterfallAnimationWidget.tsx  
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useContext, useEffect, useMemo, useState } from 'react';
import { ArrowRight, CheckCircle2, AlertCircle, Lightbulb } from 'lucide-react';
import { VizChapterContext, emitVizDemo, useVizRuntime, useVizState } from 'viz';

// Структура заранее подготовленного красивого бора для
// Словарь: ["HE", "SHE", "HIS", "HERS"]
interface WNode {
 id: number;
 label: string; // путь от корня
 char: string;
 x: number; // Координаты для SVG водопада
 y: number;
 depth: number;
 fail: number;
 term_link: number;
 is_terminal: boolean;
 words: string[];
}

const WATERFALL_NODES: { [key: number]: WNode } = {
 0: { id: 0, label: 'ROOT', char: 'ø', x: 400, y: 60,
```

```

const vars = runtime?.variables;

useEffect(() => {
 if (chapterId) emitVizDemo(chapterId, "aho");
}, [chapterId]);

const liveText = vars ? vizString(vars.text) ?? vizString("") : null;
const liveState = vars ? vizNumber(vars.state) ?? vizNumber(0) : null;
const liveI = vars ? vizNumber(vars.i) ?? vizNumber(0) : null;
const liveMode = vars ? vizString(vars.mode) : null;
const liveMatchesRaw = vars ? vizArray(vars.matches) : null;
const liveMatchesSig = useMemo(() => (liveMatchesRaw ? liveMatchesRaw.map((it) => it).length : 0), [liveMatchesRaw]);
const liveFail = vars ? vars.fail : undefined;

useEffect(() => {
 if (!vars) return;
 if (liveMode && /search|поиск/i.test(liveMode)) setLiveMode("search");
 else if (liveMode && /train|обуч|bfs/i.test(liveMode)) setLiveMode("train");
 if (liveText) {
 const clean = liveText.toUpperCase().replace(/[^A-Za-z0-9_]/g, "");
 if (clean && clean !== textToScan) {
 setTextToScan(clean);
 setCurrentIndex(0);
 setCurrentNode(0);
 setFoundMatches([]);
 setIsSearchDone(false);
 setJumpPath(null);
 setActiveMode("search");
 }
 }
 if (liveState !== null && liveState >= 0 && WATERFA)
 if (liveI !== null && liveI >= 0) setCurrentIndex(Math.min(liveI, textToScan.length));
 if (liveMatchesSig) {
 const parsed = (JSON.parse(liveMatchesSig) as unknown[]).map((it) => {
 if (Array.isArray(it) && it.length >= 2) {
 const idx = vizNumber(it[0]);

```

```

const resetSearchSimulation = () => {
 setCurrentIndex(0);
 setCurrentNode(0);
 setIsSearchDone(false);
 setSearchLog('Симуляция сброшена. Лосось вернулся на старт');
 setJumpPath(null);
 setFoundMatches([]);
 setActiveSearchCodeLine(1);
};

const stepSearchSimulation = () => {
 if (currentIndex >= textToScan.length) {
 setIsSearchDone(true);
 setSearchLog('Мы проплыли весь текст! Лосось доволен');
 setActiveSearchCodeLine(4);
 return;
 }

 const char = textToScan[currentIndex];
 let curr = currentNode;
 let logMsg = `[Символ '${char}']`;

 let jumpType: 'normal' | 'fail' | null = null;
 void jumpType;
 let originalCurr = curr;

 if (TRIE_TRANSITIONS[curr][char] !== undefined) {
 const nextNode = TRIE_TRANSITIONS[curr][char];
 logMsg += `У пусла #${curr} (${WATERFALL_NODES[curr].label})`;
 setJumpPath({ from: curr, to: nextNode, type: 'normal' });
 curr = nextNode;
 setActiveSearchCodeLine(3);
 } else {
 setActiveSearchCodeLine(2);
 let jumps = 0;
 while (curr !== 0 && TRIE_TRANSITIONS[curr][char] === undefined) {
 curr = TRIE_TRANSITIONS[curr][fail];
 jumps++;
 }
 if (curr === 0) {
 setSearchLog('Лосось не нашел пути');
 return;
 }
 setJumpPath({ from: originalCurr, to: curr, type: 'fail' });
 curr = curr;
 setActiveSearchCodeLine(3);
 }
};

```



```

 if (currentMatches.length > 0) {
 setFoundMatches((prev) => [...prev, ...currentMat
 }
};

// ПОЛНАЯ ПОШАГОВАЯ ИНФОРМАЦИЯ О ПОСТРОЕНИИ ДЕРЕВА (В
const TRAINING_STEPS_INFO = [
 {
 targetNodeId: 0,
 title: 'Шаг 0 (Depth 0): Вершина #0 (ROOT)',
 desc: 'Начало алгоритма BFS. Корень (ROOT) не обр
 рние вершины сразу инициализируются с fail = RO
 codeLine: 1,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'ø',
 checkingBranchesFrom: null,
 },
 {
 targetNodeId: 1,
 title: 'Шаг 1 (Depth 1): Вершина #1 (H)',
 desc: 'Мы обрабатываем первую вершину из очереди
 просто не существует. fail[H] автоматически нап
 codeLine: 5,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'H',
 checkingBranchesFrom: null,
 },
 {
 targetNodeId: 7,
 title: 'Шаг 2 (Depth 1): Вершина #7 (S)',
 desc: 'Обрабатываем дочернюю вершину #7 (S) на De
 ,
 codeLine: 5,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'S',
 },

```

```

 title: 'Шаг 6 (Depth 3): Вершина #3 (HER)',
 desc: 'Переходим на Depth 3 к #3 (HER). Родитель
 е подходят. fail[HER] = ROOT.',
 codeLine: 3,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'R',
 checkingBranchesFrom: 0,
},
{
 targetNodeId: 6,
 title: 'Шаг 7 (Depth 3): Вершина #6 (HIS)',
 desc: 'Обрабатываем #6 (HIS) на Depth 3. Родитель
 но ветка "S" → #7 (S) совпадает! fail[HIS] = #7
 codeLine: 4,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'S',
 checkingBranchesFrom: 0,
},
{
 targetNodeId: 9,
 title: 'Шаг 8 (Depth 3): Вершина #9 (SHE) — КАК С
 desc: ' МАГИЯ АХО-КОРАСИК! Обрабатываем #9 (SHE)
 ба плывет в #1 (H) и ищет ветку "E". У вершины
 codeLine: 4,
 scoutPos: 1,
 inspectedParentFail: 1,
 checkTargetChar: 'E',
 checkingBranchesFrom: 1,
},
{
 targetNodeId: 4,
 title: 'Шаг 9 (Depth 4): Вершина #4 (HERS)',
 desc: 'Финальная вершина #4 (HERS) на Depth 4. Ро
 #7 (S). fail[HERS] = #7 (S). Автомат полностью
 codeLine: 4,
 scoutPos: 0,

```

```

 </button>
 <button
 onClick={() => setActiveMode('search')}
 className={`flex items-center space-x-1.5 p-2
 ${activeMode === 'search'
 ? 'bg-blue-600 text-white shadow-lg shadow-blue-500'
 : 'text-slate-400 hover:text-white'}
 `}
 >
 <Play className="w-4 h-4" />
 Режим 2: Поиск в тексте
 </button>
 </div>
</div>

```

```

{/* ПАНЕЛЬ УПРАВЛЕНИЯ В ЗАВИСИМОСТИ ОТ РЕЖИМА */}
{activeMode === 'training' ? (
 /* ПАНЕЛЬ РЕЖИМА ОБУЧЕНИЯ */
 <div className="grid grid-cols-1 lg:grid-cols-3" >
 {/* Управление шагами обучения */}
 <div className="bg-slate-900/90 p-5 rounded-x" >
 <div>
 <h5 className="text-indigo-300 font-bold" >
 Полный обход в ширину (В
 </h5>
 <p className="text-xs text-slate-300 mb-3" >
 Показаны все шаги очереди BFS (начиная с
 ар 8 (SHE → HE), где наглядно видно
 </p>
 <div className="space-y-1.5 mb-6 overflow" >
 {TRAINING_STEPS_INFO.map((step, idx) =>
 <button
 key={idx}
 onClick={() => setTrainStep(idx)}
 className={`w-full text-left px-3 p-2
 ${
 idx === trainStep
 ? 'bg-indigo-600 text-white font-weight-bold'

```

```

 <span className="text-xs bg-indigo-950 p-1.5 rounded-l-4 border-indigo-500 text-indigo-300 font-
 Вершина: #{targetTrainingNode.id} ({t

 </h5>
 <div className="bg-slate-950 p-4 rounded-l-4 border-indigo-500 text-indigo-300 font-
 <div className={`p-1.5 rounded flex item-l-4 border-indigo-400 text-indigo-200 font-
 1. let u = trie[r][char]; // Те
 {currentTrainInfo.codeLine === 1 && <
 тут}
 </div>
 <div className={`p-1.5 rounded flex item-l-4 border-indigo-400 text-indigo-200 font-
 2. let v = fail[r]; // Подъем к
 ainInfo.inspectedParentFail].label}}
 {currentTrainInfo.codeLine === 2 && <
 к родителю}
 </div>
 <div className={`p-1.5 rounded flex item-l-4 border-amber-500 text-amber-300 font-bo
 3. while (v !== root && trie[v]
 {currentTrainInfo.codeLine === 3 && <
 т, возврат в корень}
 </div>
 <div className={`p-1.5 rounded flex item-r-l-4 border-emerald-500 text-emerald-300 f
 4. if (trie[v][char] !== undefin
 /span>
 {currentTrainInfo.codeLine === 4 && <
 ан IF!}
 </div>
 <div className={`p-1.5 rounded flex item-l-4 border-blue-500 text-blue-300 font-bold'
 5. else fail[u] = root; // Bero
 {currentTrainInfo.codeLine === 5 && <
 root}
 </div>
 </div>

```

```

 {textToScan.split('').map((char, idx) =>
 <span
 key={idx}
 className={`w-7 h-8 flex items-center justify-center
 ${idx === currentIndex
 ? 'bg-blue-600 text-white shadow-lg'
 : idx < currentIndex
 ? 'bg-emerald-950/60 text-emerald-500'
 : 'bg-slate-800 text-slate-500'}
 }`}
 >
 {char}

)}
 </div>

 {currentIndex >= textToScan.length && (

 ГOTOB0 ✓

)}
 </div>

 <button
 onClick={stepSearchSimulation}
 disabled={isSearchDone || textToScan.length === 0}
 className={`w-full py-2.5 rounded-lg font-medium text-sm
 ${isSearchDone || textToScan.length === 0
 ? 'bg-slate-700 text-slate-500 cursor-not-allowed'
 : 'bg-gradient-to-r from-blue-600 to-blue-900/40 active:scale-[0.98]'}
 }`}
 >
 {currentIndex === 0 ? 'Начать план' : 'Продолжить'}
 <ArrowRight className="w-4 h-4" />
 </button>
 </div>
</div>

<div className="lg:col-span-2 bg-slate-900/60" data-bbox="233 975 1000 1000">

```

```

 {searchLog}
 </div>
 </div>
 </div>
</div>
})

{/* SVG Анимация водопада */}
<div className="bg-gradient-to-b from-blue-950 via-blue-900 to-transparent pointer-events-none">
 <div className="absolute inset-0 opacity-10 bg-blue-900 to-transparent pointer-events-none"></div>
 <div className="absolute top-0 left-1/2 -translate-x-50% w-8 bg-blue-400 h-full animate-pulse">
 <div className="w-12 bg-blue-300 h-full animate-pulse">
 <div className="w-6 bg-blue-500 h-full animate-pulse"></div>
 </div>
 </div>

 <div className="flex items-center justify-between p-4">
 <h5 className="text-white font-bold text-base">
 Ветвящийся синий водопад (Борис)
 </h5>
 <div className="flex flex-wrap gap-4 text-xs">

 </div>
 </div>

 </div>

{/* Само SVG дерево водопада */}
<div className="w-full overflow-x-auto custom-scrollbar">
 <svg viewBox="0 0 800 520" className="w-full h-full">
 <!-- Горизонтальные линии и подписи уровней -->

```

```

 let isMatchBranch = isTrainingExamined && (
 isMatchBranch ? ' ' : ' '

return (
 <g key={`edge-${fromNode.id}-${toNode.id}` /* Линия течения воды */>
 <line
 x1={fromNode.x}
 y1={fromNode.y}
 x2={toNode.x}
 y2={toNode.y}
 stroke={isHighlighted ? '#3b82f6' : '#f43f5e'}
 strokeWidth={isHighlighted ? 4 : 2}
 className={isHighlighted ? 'highlighted' : ''}
 />
 /* Символ перехода */
 <circle cx={{(fromNode.x + toNode.x) / 2}} cy={{(fromNode.y + toNode.y) / 2}}
 r={10} fill={isTrainingExamined ? (isMatchBranch ? '#10b981' : '#f43f5e') : '#f43f5e'}
 stroke={isHighlighted ? '#3b82f6' : '#f43f5e'} strokeWidth={isHighlighted ? 4 : 2}
 />
 <text
 x={{(fromNode.x + toNode.x) / 2}}
 y={{(fromNode.y + toNode.y) / 2} + 15}
 fill={isTrainingExamined ? (isMatchBranch ? '#10b981' : '#f43f5e') : '#f43f5e'}
 fontSize="12"
 fontWeight="bold"
 textAnchor="middle"
 />
 {char}
 </text>

 /* ВИЗУАЛЬНЫЕ МЕТКИ ПРОВЕРКИ IF В ПЕРВОМ ПЕРИОДЕ
 {isTrainingExamined && (
 <g transform={`translate(${(fromNode.x + toNode.x) / 2}, ${
 (fromNode.y + toNode.y) / 2})`}
 <rect x="-15" y="-12" width="30" height="24"
 stroke={isHighlighted ? '#3b82f6' : '#f43f5e'}
 strokeWidth="2" />
 <text x="0" y="5" fontSize="14"
 {isMatchBranch ? ' ' : ' '}
 />
 </g>
)}
 </g>

```

```

 {isJustEstablished && (
 <text x={cx} y={cy - 10} fill="#10b080"
 >
 fail[{node.label}] = {targetNode.label}
 </text>
)}
 </g>
);
}
}
}

```

```

{/* Рендер вершин (бассейнов водопада) */}
{Object.values(WATERFALL_NODES).map((node) => {
 const isCurrent = activeFishPosNode.id === node.id;
 const isTargetChild = activeMode === 'transition';
 const hasWords = node.words.length > 0;

```

```

 return (
 <g key={`node-${node.id}`} className="node" >
 {/* Внешнее свечение для активной вершины */}
 {isCurrent && (
 <circle cx={node.x} cy={node.y} r={node.radius * 1.5} fill="none" stroke="#10b080" stroke-width={2} />
)}
 {isTargetChild && (
 <circle cx={node.x} cy={node.y} r={node.radius * 1.5} fill="none" stroke="#f08080" stroke-width={2} />
)}

```

```

 {/* Основной круг вершины */}
 <circle
 cx={node.x}
 cy={node.y}
 r={isCurrent || isTargetChild ? '24px' : node.radius}
 fill={isCurrent ? '#2563eb' : isTargetChild ? '#f08080' : node.fill}
 stroke={isCurrent ? 'transparent' : isTargetChild ? 'transparent' : '#ffffff'}
 strokeWidth={isCurrent || isTargetChild ? 0 : 1}
 className="transition-all duration-300"
 />

```

```

 {/* Текст внутри вершины */}

```



```

 cx={activeFishPosNode.x + 15}
 cy={activeFishPosNode.y - 25}
 r="18"
 fill={activeMode === 'training' ? '#8b5'
 stroke="#ffffff"
 strokeWidth="2"
 style={{ transition: 'cx 0.8s cubic-bez
 className="shadow-lg"
 />
 {/* Сама рыба */}
 <text
 x={activeFishPosNode.x + 15}
 y={activeFishPosNode.y - 19}
 fontSize="18"
 textAnchor="middle"
 style={{ transition: 'x 0.8s cubic-bezi
 className="animate-float select-none"
 >
 {activeMode === 'training' ? ' ♂ ' :
 </text>
 </g>

 </svg>
 </div>
 </div>

 {/* Список найденных совпадений (только в режиме
 {activeMode === 'search' && (
 <div className="mt-6 bg-slate-900/50 p-5 rounded
 <h5 className="text-white font-bold mb-3 text
 Улов лосося (Найденные слова
 </h5>
 {foundMatches.length === 0 ? (
 <p className="text-sm text-slate-400 italic
 Пока ничего не поймали. Запустите шаги си
 </p>
) : (
 <div className="flex flex-wrap gap-2">

```

```
 </>
) : (

 открой панель Python и запусти Ахо–Корасик:
 "font-mono text-cyan-300">state</code>, <code>
 an-300">matches</code> поведут лосося по эт

)}
 </div>
 </div>
);
};
```

---

## РАЗДЕЛ: ВИЗУАЛИЗАТОРЫ (ВЛОЖЕННЫЕ)

---

---

ФАЙЛ 83/111: src/components/visualizers/PrefixSumViz.tsx  
КАТЕГОРИЯ: Визуализаторы (вложенные) | СТРОК: 466 | РАЗМЕР: 10.5 КБ

---

```
import React, { useContext, useEffect, useMemo, useState } from 'react';
import {
 emitVizDemo,
 liveMatrix,
 pickLiveVar,
 useVizRuntime,
 vizArray,
 vizNumber,
 VizChapterContext,
} from '../../../data/vizStepBus';
```

```
/**
 * Префиксные суммы: 1D и 2D.
 *
 * Главная фишка — наведение на число:
```

```

 key={key}
 onClick={() => setMode(key)}
 className={`px-3 py-2 rounded-lg text-xs sm
 mode === key ? "bg-indigo-600 text-white"
 : ""
 }`
 >
 {label}
 </button>
)))
</div>
{mode === "1d" ? <Prefix1D /> : <Prefix2D />}
</div>
);
};

```

```

/* ----- 1D -----
const Prefix1D: React.FC = () => {
 const pref = useMemo(
 () => A1.reduce<number[]>((acc, v) => [...acc, acc[0] + v],
 []
);
};

```

```

/** Что сейчас под курсором: префикс P[i] или элемент A[i]
const [hover, setHover] = useState<{ kind: "pref" | "elem" }>({ kind: "pref" });
const [range, setRange] = useState<{ l: number; r: number }>({ l: 0, r: 0 });
/** Снимок i/P из автоматически выполненного Python-кода
const runtime = useVizRuntime();
const vars = runtime?.variables;
const liveI = vizNumber(vars?.i);
const liveP = vizArray(vars?.P);
const hasLiveP = !!vars && Object.prototype.hasOwnProperty.call(vars, "P");
const compilerLinked = liveI !== null || hasLiveP;
// i/P управляют ячейками напрямую; индекс Python-строки
const shownDriven = compilerLinked
 ? liveI !== null
 ? Math.max(0, Math.min(A1.length, Math.trunc(liveI)))
 : Math.max(0, Math.min(A1.length, (liveP?.length || 0)))
 : null;

```

```

 ? "bg-amber-500 text-black ring-2"
 : inCover
 ? "bg-indigo-500 text-white ring-2"
 : inRange
 ? "bg-indigo-900/70 text-indigo-300"
 : "bg-slate-800 text-slate-300"
)` }
 title={`a[${i}] = ${v}`}
 >
 {v}
 </div>
);
}
</div>

```

```

{ /* массив P */ }
<div className="flex gap-1.5 items-center">
 <div className="w-[46px] shrink-0 text-right">
 {pref.map((v, i) => {
 const active = hover?.kind === "pref" && i === range.l;
 const isL = i === range.l;
 const isR = i === range.r + 1;
 const liveValue = liveP?.[i];
 const blank = hasLiveP && (liveValue === null);
 const masked = !hasLiveP && shownDriven !== null;
 const isDrivenCur = shownDriven !== null && i === range.l;
 const shownValue = hasLiveP ? liveValue : null;
 return (
 <div
 key={i}
 onMouseEnter={() => setHover({ kind: "pref", i })}
 onMouseLeave={() => setHover(null)}
 className={` ${cellBase} cursor-help ${
 isDrivenCur
 ? "bg-emerald-600 text-white ring-2"
 : blank
 ? "bg-slate-950 text-slate-600"
 : masked
 }
 `}
 >

```

```

 P[{hover.i}] = {pref[hover.i]}
 {" "}
 – это сумма всего, что набежало от начала

 a[0..{hover.i - 1}] = {A1.slice(0, hove

 </p>
)
) : hover?.kind === "a" ? (
 <p className="text-slate-300">
 <b className="text-indigo-400">
 a[{hover.i}] = {A1[hover.i]}
 {" "}
 входит во все префиксы, начиная с{" "}

 подвинуть границу запроса.
 </p>
) : (
 <p className="text-slate-500">
 Наведите курсор на любое число: у <b classN
 у <b className="text-indigo-400">a – ку
 </p>
)}
</div>

{/* Запрос */}
<div className="bg-slate-900 border border-indigo
 <div className="text-xs text-slate-400 mb-2">
 Запрос суммы на отрезке (кликайте по массиву
 </div>
 <p className="font-mono text-sm sm:text-base te
 sum(a[{range.l}..{range.r}]) = P[{range.r + 1
 {pref[rang
 {pref[range.l
 {
 </p>
 <p className="text-[11px] text-slate-500 mt-1">
 Один вычитание вместо цикла: запрос за $O(1)$ п

```

```

const liveArrSource = pickLiveVar(vars, ["a", "A", "g"]);
const liveA = liveMatrix(liveArrSource?.value);

const prefRows = liveS?.rows ?? P.length;
const prefCols = liveS?.cols ?? (P[0]?.length ?? 0);
const grid: (number | null)[][] = liveS
 ? Array.from({ length: prefRows }, (_, i) =>
 Array.from({ length: prefCols }, (_, j) => liveS[i][j]
)
 : P;

const srcRows = liveA?.rows ?? n;
const srcCols = liveA?.cols ?? m;
const srcGrid: (number | null)[][] = liveA
 ? Array.from({ length: srcRows }, (_, i) => Array.from(
 { length: srcCols }, (_, j) => liveA[i][j]
)
 : A2;

const cellAt = (i: number, j: number): number | null {
 if (i >= 0 && j >= 0 && i < prefRows && j < prefCols) {
 return srcGrid[i][j];
 }
 return null;
}

const sizeMismatch =
 hasLiveS && liveS && (liveS.rows !== P.length || liveS.cols !== P[0].length)
 ? `Демо построено на матрице ${n}x${m} (сетка преобразована из матрицы ${liveS.sizeLabel}). Ниже нарисована сетка с размерами ${liveS.sizeLabel}.`
 : null;

useEffect(() => {
 const r1 = vizNumber(vars?.r1);
 const c1 = vizNumber(vars?.c1);
 const r2 = vizNumber(vars?.r2);
 const c2 = vizNumber(vars?.c2);
 if (r1 === null || c1 === null || r2 === null || c2 === null) return;
 setRect({
 r1: Math.max(0, Math.min(srcRows - 1, Math.trunc(r1))),
 c1: Math.max(0, Math.min(srcCols - 1, Math.trunc(c1))),
 r2: Math.max(0, Math.min(srcRows - 1, Math.trunc(r2))),
 c2: Math.max(0, Math.min(srcCols - 1, Math.trunc(c2)))
 });
}, [vars?.r1, vars?.c1, vars?.r2, vars?.c2]);

```

```

<div className="inline-block">
 {srcGrid.map((row, i) => {
 <div key={i} className="flex gap-1.5 mb-1">
 {row.map((v, j) => {
 const inRect = i >= r1 && i <= r2 &&
 return (
 <div
 key={j}
 onClick={() =>
 setRect((r) =>
 i < r.r1 || j < r.c1
 ? { r1: i, c1: j, r2: Math.max(r.r2, i), c2: Math.max(r.c2, j) }
 : { ...r, r2: i, c2: j }
)
 }
 className={` ${cellBase} cursor-pointer ${
 inRect ? "bg-indigo-500 text-white" : ""
 }`}
 >
 {v === null ? "." : v}
 </div>
)
 })}
 </div>
)}
</div>
</div>

```

```

{/* Матрица префиксных сумм */}
<div className="overflow-x-auto">
 <div className="text-xs text-slate-400 mb-2">
 Префиксные суммы {liveS ? `${livePrefSource}` : ""}
 </div>
 <div className="inline-block">
 {grid.map((row, i) => {
 <div key={i} className="flex gap-1.5 mb-1">
 {row.map((v, j) => {
 const blank = v === null || v === und

```

```

 </div>
)}}
 </div>
 </div>
</div>

<div className="min-h-[54px] bg-slate-900 border border-indigo-500"
 {hover ? {
 <p className="text-slate-300">
 <b className="text-amber-400">
 {liveS ? livePrefSource?.name ?? "S" : "P"}
 {" "}
 – сумма всего прямоугольника от левого верхнего угла
 </p>
 } : {
 <p className="text-slate-500">Наведите курсор на элемент
 }}
</div>

<div className="bg-slate-900 border border-indigo-500"
 <p className="font-mono text-sm sm:text-base text-slate-300">
 сумма = A + <span
 C + <span
 ={" "}
 {
 </p>
 <p className="font-mono text-[11px] text-slate-300">
 прямоугольник ({r1}, {c1}) ... ({r2}, {c2}) · c
 </p>
 <p className="text-[11px] text-slate-500 mt-1">
 Вычли два «хвоста» и вернули дважды вычитенный
 </p>
</div>
</div>
);
};

```



```
// Data for 2D Matrix (Sparse Table 2D)
const val2D = [
 [45, 12, 88, 34, 11, 76, 23, 90],
 [67, 19, 44, 55, 33, 21, 65, 87],
 [14, 51, 99, 13, 22, 64, 43, 76],
 [89, 32, 54, 71, 15, 88, 29, 60],
 [25, 41, 16, 92, 9, 17, 56, 31],
 [59, 18, 77, 24, 61, 82, 35, 12],
 [73, 8, 38, 85, 47, 95, 19, 58],
 [39, 81, 62, 28, 51, 42, 85, 14]
];

const ST2D: number[][][][] = Array.from({length: 8}, () =>
 Array.from({length: 8}, () =>
 Array.from({length: 4}, () =>
 Array(4).fill(0)
)
)
);

for (let r = 0; r < 8; r++) {
 for (let c = 0; c < 8; c++) {
 ST2D[r][c][0][0] = val2D[r][c];
 }
}

for (let kx = 0; kx <= 3; kx++) {
 for (let ky = 0; ky <= 3; ky++) {
 if (kx === 0 && ky === 0) continue;
 for (let r = 0; r + (1 << kx) <= 8; r++) {
 for (let c = 0; c + (1 << ky) <= 8; c++) {
 if (kx > 0) {
 ST2D[r][c][kx][ky] = Math.min(
 ST2D[r][c][kx-1][ky],
 ST2D[r + (1 << (kx-1))][c][kx-1][ky]
);
 } else {
 ST2D[r][c][kx][ky] = Math.min(
```

```

 onClick={() => setTab("2d_query")}
 className={`px-3 sm:px-4 py-2 rounded-lg text-
 se-600 text-white" : "bg-slate-800 text-sla
 >
 3. Запрос 2D (Формула)
 </button>
 </div>

 <AnimatePresence mode="wait">
 {tab === "1d" ? (
 <Viz1D key="1d" />
) : tab === "2d_build" ? (
 <Viz2DBuild key="2d_build" />
) : (
 <Viz2DQuery key="2d_query" />
)}
 </AnimatePresence>
 </div>
);
};

```

```

const Viz1D: React.FC = () => {
 const [step, setStep] = useState(0);
 const [hover, setHover] = useState<{ i: number; j: number }>({ i: 0, j: 0 });
 const runtime = useVizRuntime();
 const runtimeVars = runtime?.variables;
 const liveI = vizNumber(runtimeVars?.i);
 const liveJ = vizNumber(runtimeVars?.j);
 const liveSource = vizArray(runtimeVars?.a);
 const liveRows = vizArray(runtimeVars?.st);
 const hasLiveSource = !!runtimeVars && Object.prototype.hasOwnProperty.call(runtimeVars, 'a');
 const hasLiveTable = !!runtimeVars && Object.prototype.hasOwnProperty.call(runtimeVars, 'st');
 const compilerLinked = liveI !== null || liveJ !== null;
 const shownSource = hasLiveSource
 ? liveSource?.length
 ? liveSource
 : Array.from({ length: N }, () => null)
 : arr1D;

```

```

 </h3>
 <p className="text-sm text-slate-400">
 Формула: $ST[i][j] = \min(ST[i][j-1], ST[i + 1][j])$
 </p>
 {dimMismatch ? {
 <p className="mt-2 max-w-[560px] rounded-lg">
 {dimMismatch}
 </p>
 } : null}
 </div>
 <div className="flex gap-2">
 <button
 onClick={() => setStep(Math.max(0, step - 1))}
 className="p-2 bg-slate-800 hover:bg-slate-700"
 disabled={step === 0}
 >
 <ChevronLeft size={20} />
 </button>
 <button
 onClick={() => setStep(Math.min(maxStep, step + 1))}
 className="p-2 bg-indigo-600 hover:bg-indigo-500"
 disabled={step === maxStep}
 >
 <ChevronRight size={20} />
 </button>
 <button
 onClick={() => setStep(0)}
 className="p-2 bg-slate-800 hover:bg-slate-700"
 >
 <RotateCcw size={20} />
 </button>
 </div>
</div>

<div className="bg-slate-900 rounded-xl border border-slate-800">
 <div className="text-[11px] uppercase tracking-wider">
 <div className="flex gap-1 sm:gap-1.5 overflow-x-auto">
 {shownSource.map((v, idx) => {

```

```

 <p className="text-slate-500">
 Наведите курсор (или тапните) на любое чи
 </p>
)}
 </div>
 </div>

<div className="overflow-x-auto pb-2">
 <div className="inline-block min-w-full bg-slate-500">
 <div className="grid grid-cols-[auto_repeat(4,1fr)]">
 { /* Headers */ }
 <div className="text-slate-500 font-mono text-sm">
 i \ j
 </div>
 <div className="text-center font-bold text-slate-500">
 2^0 (L=1)
 </div>
 <div className="text-center font-bold text-slate-500">
 2^1 (L=2)
 </div>
 <div className="text-center font-bold text-slate-500">
 2^2 (L=4)
 </div>
 <div className="text-center font-bold text-slate-500">
 2^3 (L=8)
 </div>

 { /* Matrix */ }
 {Array.from({ length: N }).map((_, i) => {
 <React.Fragment key={i}>
 <div className="text-slate-500 font-mono text-sm">
 {i}
 </div>
 {Array.from({ length: LOG }).map((_, j) => {

```

```

 return (
 <div key={j} className="relative flex flex-col items-center justify-center"
 {!isValid ? (
 <div className="w-[clamp(30px,9vw,17px)] font-bold transition-colors"
) : (
 <motion.div
 onMouseEnter={() => (compilerLink ? setCompilerLink(compilerLink) : null)}
 onMouseLeave={() => setCompilerLink(null)}
 onClick={() => (compilerLink ? compilerLink() : null)}
 >
 {
 initial={{ opacity: 0, scale: 1 }}
 animate={{
 // При st = [] каркас уже сфокусирован
 opacity: compilerLink ? 1 : 0,
 scale: compilerLink || isSrc1 ? 1 : 0,
 }}
 className={`w-[clamp(30px,9vw,17px)] font-bold transition-colors
 hover && hover.i === i && hover.s === s && hover.v === v &&
 ? "bg-sky-500 text-white"
 : isActive
 ? "bg-indigo-500 text-white"
 : isSrc1
 ? "bg-emerald-500 text-white"
 : isSrc2
 ? "bg-amber-500 text-white"
 : isVisible
 ? "bg-slate-800 text-white"
 : compilerLink
 ? "bg-slate-950 text-white"
 : "bg-transparent"}
 }
 {title}
 </motion.div>
)}
)
)
}

```

```

- активна ячейка <span className="font-mono
 {(hasLiveTable || hasLiveSource) && <> =
 ?.liveTarget.j] ?? (liveTarget.j === 0 ? 1
 </p>
) : step > 0 && step <= maxStep ? (
 (() => {
 const c = computeSteps[step - 1];
 const half = 1 << (c.j - 1);
 return (
 <p className="text-lg text-slate-300">

 ST[{c.i}][{c.j}]
 {" "}
 = min(
 <span className="text-emerald-400 font-l
 ST[{c.i}][{c.j - 1}]

 ,
 <span className="text-amber-400 font-bo
 ST[{c.i + half}][{c.j - 1}]

) → min(
 {st1D

 {st1D[c.i + half][c.j - 1]}

) ={" "}
 <span className="text-indigo-400 font-b
 {st1D[c.i][c.j]}

 </p>
);
 })()
) : (
 <p className="text-slate-500 italic">
 Нажмите далее, чтобы увидеть, как блоки сво
 </p>
)
})

```

```

 ? 0
 : k;

/** 2D-демонстрация нарисована на фиксированной сетке
const liveSource2D = liveMatrix(vars?.A);
const dimMismatch2D = liveSource2D
 ? liveSource2D.rows !== 8 || liveSource2D.cols !== 8
 ? `Демо 2D построено на матрице 8x8, а в вашем ко
 не её не отрисовываются, значения вашей матрицы
 : null
 : null;

// Уровень и клетка берутся прямо из k/r/c; номер стро
useEffect(() => {
 setHover(null);
 setLocked(null);
}, [k]);

const L = 1 << shownK;
const activeCell = codeCell || locked || hover;

return (
 <div className="flex flex-col xl:flex-row gap-6">
 <div className="flex-1 min-w-0 bg-slate-900 p-3 s
 <h3 className="text-xl font-bold text-indigo-400
 <p className="text-sm text-slate-400 mb-6 text-
 Для наглядности показываем только <b classNam

 <span className="text-rose-300 animate-pulse
 иц!
 </p>
 {dimMismatch2D ? (
 <p className="mb-4 max-w-[520px] rounded-lg b
 {dimMismatch2D}
 </p>
) : null}

 <div className="flex bg-slate-950 p-1 rounded-l

```

```

eColumns: `repeat(${stepSize}, max-content)
 {Array.from({length: stepSize * st
 const r = Math.floor(idx / step
 const c = idx % stepSize;

 const liveValue = liveTable?.[`
 // Базовый слой (блоки 1x1) – э
 // одного элемента равен ему са
 // показываем из A сразу, как A
 // цикла построения st2. Уровни
 const sourceValue = step === 0
 const shownValue = compilerLink
 const isBlank = shownValue ===
 let highlightClass = isBlank
 ? 'bg-slate-950 text-slate-600'
 : isCurr
 ? 'bg-slate-800/80 text-slate-600'
 : 'bg-slate-800/50 text-slate-600'
 let zIndex = 'z-0';

 const isActive = !!activeCell &&
 ll.c + L) && ((r - activeCell.r) % (1 << st

 if (isActive) {
 if (isCurr) {
 // Значения в ячейке ещё не
 // контур «сюда смотрит код»
 // залитая пустая клетка вы
 highlightClass = isBlank
 ? `bg-indigo-500/10 text-white`
 : `bg-indigo-500 text-white`
 == c ? 'ring-[3px] ring-white ring-offset-2'
 && locked.c === c ? 'ring-[3px] ring-white'
 zIndex = 'z-10';
 } else {
 const prevL = 1 << (shownK
 const isTop = r < activeCel
 const isLeft = c < activeCe

```





</p>

```

<div className="space-y-2 pt-3 pl-2"
 {(() => {
 const prevL = 1 << (shownK - 1)
 return (
 <>
 <div className="flex items-center"
 <span className="flex-grow-1"
 hadow-[0_0_8px_#f43f5e]"></div> ST[{activeCell.r}][activeCell.c]
 <span className="text-center"
 tiveCell.r][activeCell.c][shownK-1][shownK-1]
 </div>
 <div className="flex items-center"
 <span className="flex-grow-1"
 hadow-[0_0_8px_#3b82f6]"></div> ST[{activeCell.r}][activeCell.c + prevL][shownK-1][shownK-1]
 </div>
 <div className="flex items-center"
 <span className="flex-grow-1"
 -500 shadow-[0_0_8px_#10b981]"></div> ST[{activeCell.r}][activeCell.c + prevL][shownK-1][shownK-1]
 <span className="text-center"
 >{ST2D[activeCell.r + prevL][activeCell.c][shownK-1][shownK-1]}
 </div>
 <div className="flex items-center"
 <span className="flex-grow-1"
 shadow-[0_0_8px_#f59e0b]"></div> ST[{activeCell.r}][activeCell.c + prevL][shownK-1][shownK-1]
 <span className="text-center"
 [activeCell.r + prevL][activeCell.c + prevL][shownK-1][shownK-1]
 </div>
 </>
)
 })}
 </div>

```

```

<div className="pt-3 mt-3 border-top"
) = <span className="text-white"

```

```

const queryVars = runtime?.variables;
const liveCoords = ["r1", "c1", "r2", "c2"].map((name, i) => {
const queryMismatch = liveCoords.some((v) => v !== queryVars[name]);
 ? `Координаты запроса в вашем коде выходят за пределы диапазона [1; 8]`
 : null;

const H = Math.max(1, r2 - r1 + 1);
const W = Math.max(1, c2 - c1 + 1);
const kx = Math.floor(Math.log2(H));
const ky = Math.floor(Math.log2(W));
const Lx = 1 << kx;
const Ly = 1 << ky;

const matRows = 8 - Lx + 1;
const matCols = 8 - Ly + 1;

return (
 <div className="flex flex-col xl:flex-row gap-6">
 <div className="flex-1 min-w-0 bg-slate-900 text-white">
 <h3 className="text-xl font-bold text-rose-500">Запрос
 <p className="text-slate-400 text-[13px] font-normal">
 (r1, c1 - левый верх; r2, c2 - правый низ).</p>
 {queryMismatch ? (
 <p className="mb-3 rounded-lg border border-rose-500 p-2">
 {queryMismatch}
 </p>
) : null}

 <div className="flex flex-col items-center">
 <div className="grid grid-cols-[repeat(8, 1fr)] gap-1">
 <div className="active shadow-inner w-max max-w-full">
 {Array.from({length: 64}).map((_, idx) => {
 const r = Math.floor(idx / 8);
 const c = idx % 8;
 const isInQuery = r >= r1 && r <= r2 && c >= c1 && c <= c2;
 })}
 </div>
 </div>
 </div>
 </div>
 </div>
);

```

```

er-slate-800 shadow-[inset_0_2px_10px_rgba(
 <label className="flex items-cen
 <span className="text-slate-
 <div className="flex gap-2">
 <input type="number" min={
14 bg-slate-900 border border-slate-700 py-
 <input type="number" min={
14 bg-slate-900 border border-slate-700 py-
 </div>
 </label>
 <label className="flex items-cen
 <span className="text-slate-
 <div className="flex gap-2">
 <input type="number" min={
14 bg-slate-900 border border-slate-700 py-
 <input type="number" min={
14 bg-slate-900 border border-slate-700 py-
 </div>
 </label>
 </div>
 </div>
 </div>

<div className="flex-1 bg-slate-900 p-6 rou
 <h3 className="text-xl font-bold text-em
 <p className="text-sm font-sans text-sla
 Мы знаем, что максимальная степень дв
rounded text-white">{Lx}. Аналогично
te">{Ly}.
 </p>
 <p className="text-sm font-sans text-sla
 Чтобы покрыть весь прямоугольник, мы
Name="font-mono bg-[#0a0fle] px-1 rounded b
роса. (Цветные прямоугольники слева).
 </p>

 <div className="bg-[#0a0fle] rounded-xl
text-slate-300 shadow-inner overflow-x-auto

```

```

 Откуда берутся эти 4 числа: матрица
border-slate-700 shadow-sm">ST[][][kx]][ky]
 </h4>
 <div className="grid gap-[2px] p-2.5
Columns: `repeat(${matCols}, 1fr)`}>
 {Array.from({length: matRows * matCols}, (v, idx) => {
 const r = Math.floor(idx / matCols);
 const c = idx % matCols;
 let isTL = r === r1 && c === c1;
 let isTR = r === r1 && c === c2;
 let isBL = r === r2 - Lx + 1 && c === c1;
 let isBR = r === r2 - Lx + 1 && c === c2;

 let color = "bg-slate-800 text-white";
 let txt = ST2D[r][c][kx][ky];

 let badge = null;
 if (isTL) { color = "bg-rose-500 border-2"; badge = "TL"; }
 else if (isTR) { color = "bg-lime-300 border-2"; badge = "TR"; }
 else if (isBL) { color = "bg-emerald-300 border-2"; badge = "BL"; }
 else if (isBR) { color = "bg-amber-300 border-2"; badge = "BR"; }

 return (
 <div key={idx} className={color}
 transition-all duration-300 relative shrink-0 ${c === c1 ? 'w-1/2' : 'w-1/3'}
 {txt}
 {badge && <div className="bg-slate-700 text-white px-1 font-bold rounded
 </div>
)
 })
 </div>
</div>
</div>

```

```

<script>
 function setMode(mode) {
 document.getElementById('btn-mode-normal').className =
 ? 'px-3 py-1 rounded text-sm font-bold' : 'px-3 py-1 rounded text-sm font-bold';

 document.getElementById('btn-mode-wtf').className =
 ? 'px-3 py-1 rounded text-sm font-bold' : 'px-3 py-1 rounded text-sm font-bold';

 const aside = document.querySelector('aside');
 const headerMain = document.querySelector('header');
 const questionsContainer = document.getElementById('questions-container');
 const wtfContainer = document.getElementById('wtf-container');

 if (mode === 'wtf') {
 document.body.classList.add('wtf-mode');
 if (aside) aside.style.display = 'none';
 if (headerMain) headerMain.style.display = 'none';
 if (questionsContainer) questionsContainer.style.display = 'none';
 if (wtfContainer) wtfContainer.classList.remove('hidden');

 // Remove the URL hash to avoid scrolling
 history.pushState("", document.title, window.location.pathname);
 window.scrollTo(0, 0);
 } else {
 document.body.classList.remove('wtf-mode');
 if (aside) aside.style.display = '';
 if (headerMain) headerMain.style.display = '';
 if (questionsContainer) questionsContainer.style.display = '';
 if (wtfContainer) wtfContainer.classList.add('hidden');
 }
 }

 function setTheme(theme) {
 const body = document.body;
 body.classList.remove('theme-dark', 'theme-light');
 body.classList.add('theme-' + theme);
 }

```

```

 // Timeout allows display:block
 setTimeout(() => drawerOverlay.classList.add('display-block'), 100);
 }
 document.body.style.overflow = "hidden";
} else {
 mobileDrawer.classList.add("-translate-x-100%");
 if (drawerOverlay) {
 drawerOverlay.classList.add("opacity-0");
 setTimeout(() => drawerOverlay.classList.add('display-block'), 100);
 }
 document.body.style.overflow = "";
}
}

if (mobileMenuBtn && mobileDrawer) {
 mobileMenuBtn.addEventListener("click", () => {
 toggleDrawer();
 });
}
if (closeDrawerBtn && mobileDrawer) {
 closeDrawerBtn.addEventListener("click", () => {
 toggleDrawer();
 });
}
if (drawerOverlay) {
 drawerOverlay.addEventListener("click", () => {
 toggleDrawer();
 });
}

// Close on link click
document.querySelectorAll("#mobile-drawer a").forEach(link => {
 link.addEventListener("click", () => {
 if (mobileDrawer && !mobileDrawer.classList.contains('display-block')) {
 toggleDrawer();
 }
 });
});

const observerOptions = {
 root: null,
 rootMargin: '-10% 0px -70% 0px',
 threshold: 0
};

```

```

<script src="https://polyfill.io/v3/polyfill.min.js
<script id="MathJax-script" async src="https://cdn.
<script>
 window.MathJax = {
 tex: {
 inlineMath: [['$', '$'], ['\\(', '\\)']]
 displayMath: [['$$', '$$'], ['\\[', '\\
 }
 };
</script>
<style>
 @import url('https://fonts.googleapis.com/css2?
 @reference "./src/index.css";
 .math-font { font-family: 'Fira Code', monospac
 .uno-card {
 width: 70px; height: 105px; border-radius:
 box-shadow: 0 4px 8px rgba(0,0,0,0.5); posi
 align-items: center; justify-content: cente
 color: white; overflow: hidden; flex-shrink
 text-shadow: 2px 2px 0 #000, -1px -1px 0 #0
 font-family: 'Arial Black', Impact, sans-se
 }
 .uno-card::before, .uno-card::after {
 content: attr(data-val); position: absolute
 text-shadow: 1px 1px 0 #000, -1px -1px 0 #0
 }
 .uno-card::before { top: 2px; left: 4px; }
 .uno-card::after { bottom: 2px; right: 4px; tra
 .uno-card.mini { width: 45px !important; height
 .uno-card.mini::before, .uno-card.mini::after {

/* Global formula scroll settings */
.formula-scroll {
 overflow-x: auto;
 overflow-y: hidden;
 -webkit-overflow-scrolling: touch;
 scrollbar-width: none; /* Firefox */
 -ms-overflow-style: none; /* IE and Edge */

```



```

}
.uno-inner {
 position: absolute; width: 110%; height: 75%;
 border-radius: 50%; transform: rotate(-30deg);
 box-shadow: inset 0 0 5px rgba(0,0,0,0.3);
}
.uno-val { position: relative; z-index: 2; }
.card-red { background-color: #e52521; } .card-
.card-blue { background-color: #004dcf; } .card-
.card-green { background-color: #339e35; } .card-
.card-yellow { background-color: #ffc400; } .ca
.card-black { background-color: #222; } .card-b
.card-black::before, .card-black::after { text-
.info-block { @apply bg-slate-800/80 border-l-4
.info-title { @apply font-bold mb-2 flex items-

/* Визуализация (Аналогии) */
.analogy-block { @apply bg-slate-900 border-2 b
 ems-center gap-6; }
.analogy-badge { @apply absolute -top-3 left-4 l
 king-widest border border-slate-500 shadow-md
.img-placeholder { @apply flex flex-col items-c
 r shrink-0 shadow-lg w-full md:w-auto; }
.img-caption { @apply font-mono text-sm mt-4 fo

/* THEME OVERRIDES */
body.theme-light {
 background-color: #f8fafc !important;
 color: #0f172a !important;
}
body.theme-light .bg-slate-900, body.theme-ligh
body.theme-light .bg-slate-800 { background-col
body.theme-light .bg-slate-700, body.theme-ligh
 important; }
body.theme-light .text-slate-200, body.theme-li
body.theme-light .text-slate-400 { color: #47556
body.theme-light .border-slate-600, body.theme-
body.theme-light .border-slate-500 { border-col

```

```

 animation-delay: 1.5s;
 }
 .creepy-emoji {
 display: inline-block;
 transform-origin: center;
 animation: creepy-blink 3.8s infinite;
 }
</style>
</head>
<body class="bg-slate-900 text-slate-200 font-sans lead">

 <div class="sticky top-0 z-50 w-full bg-slate-950/90
 center items-center gap-4 sm:gap-8 transition-colors">
 <div class="flex items-center gap-3">
 ТЕМ
 <div class="bg-slate-800 p-1 rounded-lg flex items-center gap-2
 transition-colors">
 <button onclick="setMode('normal')" id="btn-normal">Список тем</button>
 <button onclick="setMode('wtf')" id="btn-wtf">WTF
 </div>
 </div>
 <div class="flex items-center gap-3">
 ТЕМА
 <div class="bg-slate-800 p-1 rounded-lg flex items-center gap-2
 transition-colors">
 <button onclick="setTheme('dark')" id="btn-dark">Темная</button>
 <button onclick="setTheme('light')" id="btn-light">Светлая</button>
 <button onclick="setTheme('notebook')" id="btn-notebook">Заметки
 </div>
 </div>
 </div>

 <!-- ОПРЕДЕЛЕНИЕ МАРКЕРОВ SVG -->
 <svg width="0" height="0" class="absolute">
 <defs>

```

```

 <rect x="26" y="-5" width="8" height="6"
 <line x1="2" y1="2" x2="38" y2="2" stro
 </g>
 <!-- МЯСОРУБКА -->
 <g id="meat-grinder">
 <path d="М 10 40 L 15 20 Q 25 15 35 20
 <rect x="5" y="40" width="40" height="2
 <!-- Ручка -->
 <path d="М 45 50 Q 60 50 60 30 L 75 30"
 <circle cx="75" cy="30" r="4" fill="#33
 <!-- Выход -->
 <path d="М 5 45 L -5 45 L -5 55 L 5 55
 <circle cx="25" cy="50" r="15" fill="#f
 </g>
 <g id="lego-green">
 <rect x="0" y="10" width="20" height="1
 <rect x="3" y="5" width="14" height="5"
 </g>
 <g id="lego-white">
 <rect x="0" y="0" width="40" height="20
 <rect x="6" y="-5" width="8" height="6"
 <rect x="26" y="-5" width="8" height="6"
 <line x1="0" y1="2" x2="40" y2="2" stro
 <text x="20" y="14" fill="#475569" font
 </g>
</defs>
</svg>

<!-- Мобильная кнопка меню (Гамбургер) -->
<button id="mobile-menu-btn" class="lg:hidden fixed
 0/50 z-50 hover:bg-blue-500 transition-transform
 <svg xmlns="http://www.w3.org/2000/svg" class="
 <path stroke-linecap="round" stroke-linejoin
 </svg>
</button>

<!-- Затемнение фона на мобилках при открытом меню
<div id="drawer-overlay" class="lg:hidden fixed ins

```

```

 <summary class="list-none cursor-pointer-indigo-500 pl-3 font-bold text-indigo-300">
 2. Комплексные числа

 </summary>
 <div class="pl-6 space-y-2 text-indigo-300">

 </div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer-teal-500 pl-3 font-bold text-teal-300">
 3. Тригонометрическая форма

 </summary>
 <div class="pl-6 space-y-2 text-teal-300">

 </div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer-amber-500 pl-3 font-bold text-amber-300">
 4. Степени и корни

 </summary>
 <div class="pl-6 space-y-2 text-amber-300">


```

```
<details class="group">
 <summary class="list-none cursor-pointer text-pink-500 pl-3 font-bold text-pink-300">
 7. Взаимно простые числа
 </summary>
 <div class="pl-6 space-y-2 text-pink-300">

 </div>
</details>
```

```
<details class="group">
 <summary class="list-none cursor-pointer text-violet-500 pl-3 font-bold text-violet-300">
 8. Сравнимость и Поля Вейля
 </summary>
 <div class="pl-6 space-y-2 text-violet-300">

 </div>
</details>
```

```
<details class="group">
 <summary class="list-none cursor-pointer">

```

```


нственности НОД

ПИРАЛЬ)

вращается!)
 </div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer text-emerald-500 pl-3 font-bold text-emerald-400">
 12. Взаимно простые мно

 </summary>
 <div class="pl-6 space-y-2 text-emerald-500">

деление)
 </div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer text-emerald-500 pl-3 font-bold text-emerald-400">
 13. Корни многочленов.

 </summary>
 <div class="pl-6 space-y-2 text-emerald-500">

 </div>
</details>
```

```

 </summary>
 <div class="pl-6 mt-2 space-y-2">

 </div>
 </details>

 <div class="mt-4 mb-2 text-xs font-l
 <a href="#question-17" class="block
-500 pl-3 text-slate-400">
 <span class="font-bold text-tea

 <a href="#question-18" class="block
-500 pl-3 text-slate-400">
 <span class="font-bold text-blur

 <a href="#question-19" class="block
ald-500 pl-3 text-slate-400">
 <span class="font-bold text-eme

 <a href="#question-20" class="block
-500 pl-3 text-slate-400">
 <span class="font-bold text-ros

 <a href="#question-21" class="block
go-500 pl-3 text-slate-400">
 <span class="font-bold text-ind

 <a href="#question-22" class="block
sia-500 pl-3 text-slate-400">
 <span class="font-bold text-fuch

 <a href="#proof-algorithms" class="
-red-500 pl-3 mt-4 text-red-500 font-bold m
 ▲ Ликбез: Алгоритмы выживания


```

```

 document.querySelector(
 a.style.fontWeight + ' ' +
 if (a.href.endsWith('#')) {
 a.style.fontSize + 'px';
 }
));

 // Open the parent modal
 const correspondingLink = a.querySelector(
 if (correspondingLink) {
 const details = document.querySelector(
 if (details && details.open) {
 document.querySelector(
 if (other) {
 other.open = true;
 }
));
 }
 }
 }
));
 }, {
 root: null,
 rootMargin: '-10% 0px -70% 0px',
 threshold: [0, 0.5]
 });

 sections.forEach(section => {
 observer.observe(section);
 });
});

function fallbackModal(contentRaw,
 const modal = document.createElement('div');
 modal.style.position = 'fixed';
 modal.style.top = '0';
 modal.style.left = '0';

```



```

const btnContainer = document.c
btnContainer.style.display = 'f
btnContainer.style.gap = '10px'
btnContainer.style.marginTop =

const copyBtn = document.create
copyBtn.innerText = 'Копировать'
copyBtn.style.padding = '10px';
copyBtn.style.backgroundColor =
copyBtn.style.color = '#fff';
copyBtn.style.border = 'none';
copyBtn.style.borderRadius = '5
copyBtn.style.cursor = 'pointer'
copyBtn.onclick = () => {
 if (navigator.clipboard) {
 navigator.clipboard.wri
 copyBtn.innerText = 'Ск
 } else {
 textarea.select();
 document.execCommand('c
 copyBtn.innerText = 'Ск
 }
 setTimeout(() => copyBtn.inn
};

const closeBtn = document.creat
closeBtn.innerText = 'Закрыть';
closeBtn.style.padding = '10px'
closeBtn.style.backgroundColor :
closeBtn.style.color = '#fff';
closeBtn.style.border = 'none';
closeBtn.style.borderRadius = '5
closeBtn.style.cursor = 'pointe
closeBtn.onclick = () => modal.

box.appendChild(header);
box.appendChild(subHeader);
if (title !== 'PDF') {

```

```

 link.href = url;
 link.download = 'vyisshaya_algebra.pdf';
 document.body.appendChild(link);
 link.click();
 document.body.removeChild(link);
 }

 function setTheme(theme) {

 const body = document.body;
 body.classList.remove('bg-slate-900');

 document.getElementById('theme-dark').checked = theme === 'dark';
 document.getElementById('theme-light').checked = theme === 'light';
 document.getElementById('theme-auto').checked = theme === 'auto';

 let oldStyle = document.getElementById('theme-style');
 if (oldStyle) oldStyle.remove();
 const style = document.createElement('style');
 style.id = 'theme-style';

 if (theme === 'dark') {
 body.classList.add('bg-slate-900');
 document.getElementById('theme-dark').checked = true;
 } else if (theme === 'light') {
 body.classList.add('bg-white');
 document.getElementById('theme-light').checked = true;
 style.innerHTML = `
 body.bg-white section h1 { color: #000000; }
 body.bg-white main { color: #000000; }
 body.bg-white main p { color: #000000; }
 /* Universal text color */
 body.bg-white .text-slate-900 { color: #000000; }
 body.bg-white .text-slate-900 { color: #000000; }
 body.bg-white .text-slate-900 { color: #000000; }
 body.bg-white [class*="text-slate-900"] { color: #000000; }
 `;
 }
 }

```

```

 body.bg-white .text-amber
 body.bg-white .text-fuchsia
 body.bg-white .text-limegreen
 body.bg-white .text-blue
 body.bg-white .text-green
 body.bg-white .text-indigo
 `;
 } else if (theme === 'terminal') {
 body.classList.add('bg-black');
 document.getElementById('theme')
 .style.innerHTML = `
 body.bg-black .bg-slate-950 {
 background-color: transparent;
 }
 body.bg-black section header {
 color: #4ade80 !important;
 border-color: #22c55e !important;
 font-family: monospace;
 }
 body.bg-black svg path {
 stroke: #22c55e !important;
 fill: transparent !important;
 }
 `;
 }

 document.head.appendChild(style);
}
</script>

```

```

<!-- ФИКСИРОВАННЫЕ БЛОКИ: СМЕНА ТЕМЫ И ...
<div class="mt-8 flex flex-col gap-4 bo
<!-- Кнопки Конвертации / Экспорта

```

```
</aside>
```

```
<!-- Основной контент -->
```

```
<main class="flex-1 bg-slate-800 p-4 sm:p-8 md:p-12 lg:p-16" style="background-color: #1a2b3c; color: white; min-width: 0">
```

```
 <header class="text-center mb-12 border-b border-white pb-4">
```

```
 <h1 class="text-4xl md:text-5xl font-bl" style="font-family: monospace; font-size: 2em; margin: 0">
```

```
 ВЫСШАЯ АЛГЕБРА
```

```
 </h1>
```

```
 <p class="text-slate-400 italic text-lg" style="font-style: italic; font-size: 1.2em; margin: 0">

```

```
 <!-- Print Table of Contents -->
```

```
 <div class="hidden print:block mt-8 text-slate-400" style="display: none; font-family: monospace; font-size: 0.9em; margin-top: 20px">
```

```
 <h2 class="font-bold text-xl mb-4" style="font-weight: bold; font-size: 1.2em; margin-bottom: 10px">
```

```
 <ul class="text-base space-y-2" style="list-style-type: none; padding-left: 0; margin: 0">
```

```
 1. Ал
```

```
 2. Ко
```

```
 3. Тр
```

```
 4. Ст
```

```
 5. Гр
```

```
 6. Де
```

```
 7. Ал
```

```
 8. Кл
```

```
 9. Ко
```

```
 10. ,
```

```
 11. <
```

```
 12. l
```

```
 13. l
```

```
 14. l
```

```
 15. <
```

```
 17. l
```

```
 18. `
```

```
 19. l
```

```
 20. l
```

```
 21. l
```

```
 22. l
```

```

 <div class="text-xs text-yellow
 <div class="font-bold text-slate

 <a href="#question-10" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t
 <div class="text-xs text-orange
 <div class="font-bold text-slate

 <a href="#question-3" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans.
 <div class="text-xs text-cyan-5
 <div class="font-bold text-slate

 <!-- Row 3: НОД и НОК / Алгоритм Ев
 <a href="#question-7" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra
 <div class="text-xs text-yellow
 <div class="font-bold text-slate

 <a href="#question-12" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t
 <div class="text-xs text-orange
 <div class="font-bold text-slate

 <!-- Empty 3rd row cell -->
 <div class="hidden md:block"></div>

 <!-- Row 4: Факторизация / Структур
 <a href="#question-8" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra
 <div class="text-xs text-yellow
 <div class="font-bold text-slate

 <a href="#question-11" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t
 <div class="text-xs text-orange
 <div class="font-bold text-slate

```

```
<!-- КЛАСТЕР 3: НЕПРИВОДИМОСТЬ -->
<div class="col-span-1 md:col-span-1
b-2 border-b border-emerald-500/30 pb-2">Кл

 <a href="#question-17" onclick="setl
-l-4 border-teal-500 hover:bg-slate-700 tra
 <div class="text-xs text-teal-500
 <div class="font-bold text-slate-700

 <a href="#question-18" onclick="setl
-l-4 border-blue-500 hover:bg-slate-700 tra
 <div class="text-xs text-blue-500
 <div class="font-bold text-slate-700

 <a href="#question-19" onclick="setl
-l-4 border-emerald-500 hover:bg-slate-700
 <div class="text-xs text-emerald-500
 <div class="font-bold text-slate-700

 <a href="#question-20" onclick="setl
-l-4 border-rose-500 hover:bg-slate-700 tra
 <div class="text-xs text-rose-500
 <div class="font-bold text-slate-700

 <a href="#question-21" onclick="setl
-l-4 border-indigo-500 hover:bg-slate-700 t
 <div class="text-xs text-indigo-500
 <div class="font-bold text-slate-700

 <a href="#question-22" onclick="setl
-l-4 border-fuchsia-500 hover:bg-slate-700
 <div class="text-xs text-fuchsia-500
 <div class="font-bold text-slate-700

<!-- КЛАСТЕР 4: АЛГОРИТМЫ ВЫЖИВАНИЯ
<div class="col-span-1 md:col-span-1
```

```

k$).
 <span class="text-w
 <span class="text-w
ово.

 </div>

<!-- От противного -->
<div class="bg-slate-800/80 p-6
 <h3 class="text-xl text-fuch
 <p class="text-sm text-slate
 <ul class="list-disc pl-5 sp
 <span class="text-w
i>
 <span class="text-w
т.д.).
 <span class="text-w
что так и задумано: "Ой, противоречие, теор

 </div>

<!-- Единственность -->
<div class="bg-slate-800/80 p-6
 <h3 class="text-xl text-eme
 <p class="text-sm text-slate
 <ul class="list-disc pl-5 sp
 <span class="text-w
, r_2$).
 <span class="text-w
 <span class="text-w
счет ограничений (размер остатка) докажи, ч

 </div>

<!-- Индукция -->
<div class="bg-slate-800/80 p-6
 <h3 class="text-xl text-yel
 <p class="text-sm text-slate

```

```
<p class="text-sm text-slate-400">
 <p class="text-xs font-slate-400">
 c. Пока получается 0 – корень жив. Как то
 корня минус один.</p>
 </div>
```

```
<div class="bg-slate-800 p-4">
 <h4 class="text-red-400">
 <p class="text-sm text-slate-400">
 <p class="text-xs font-slate-400">
 ициентов (они делятся на p), кроме старшего
 зложить.</p>
 </div>
```

```
<div class="bg-slate-800 p-4">
 <h4 class="text-red-400">
 <p class="text-sm text-slate-400">
 <p class="text-xs font-slate-400">
 . Чаще всего после этого Фейс-контроль p .
 </div>
 </div>
```

```
<h2 class="text-2xl font-bold text-slate-400">
(Правила экстрасенса)</h2>
<p class="text-sm text-slate-400">
отношения), а ты их не помнишь. Юзай базовые
нейность) для делимости.</p>
```

```
<ul class="space-y-4 pb-10">
 <li class="bg-slate-800/50 p-4">

 <div>
 <h4 class="text-cyan-400">
 <p class="text-sm text-slate-400">
 <code class="text-x"
 </div>

```



---

## РАЗДЕЛ: УТИЛИТЫ

---

---

ФАЙЛ 87/111: src/utils/cn.ts

КАТЕГОРИЯ: Утилиты | СТРОК: 7 | РАЗМЕР: 169 В

---

```
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

export function cn(...inputs: ClassValue[]) {
 return twMerge(clsx(inputs));
}
```

---

ФАЙЛ 88/111: src/utils/exportHtml.ts

КАТЕГОРИЯ: Утилиты | СТРОК: 341 | РАЗМЕР: 15.5 KB

---

```
import type { Chapter } from "../types";
import { GLOSSARY_BY_ID } from "../data/glossary";
import { annotateTerms } from "../hooks/useTermHints";
import { annotateTicketLinks } from "../ticketLinks";
```

```
/**
```

```
 * Выгрузка главы (или всего пособия) в самостоятельный
 * файл
```

```
 * Файл получается автономным: стили зашиты внутрь, интерактив
 * его можно открыть с флешки, отправить в мессенджер и т.д.
 * Живые React-визуализации в статику не переносятся — вместо них
 * подставляется заметка со ссылкой на онлайн-версию, а комментарии
 * которые в приложении всплывают по наведению, собираются в блок
 * в конце документа.
```

```

.export-term .formal { color: #94a3b8; font-size: .8rem; }
.export-term .cx { color: #6ee7b7; font-size: .78rem; }
.export-foot { margin-top: 3rem; border-top: 1px solid #ccc; }
body { font-size: 15px; line-height: 1.6; }
.export-shell p, .export-shell li { line-height: 1.65; }
.export-shell pre { font-size: 12px; line-height: 1.5; }
.export-figs { margin: 0 0 1.6rem; }
.export-figs-title { color: #e2e8f0; font-size: .95rem; }
.export-fig { margin: 0 0 .9rem; }
.export-fig img { width: 100%; border: 1px solid #1e293b; }
.export-fig figcaption { color: #94a3b8; font-size: .75rem; }
@media print {
 body { background: #fff; color: #0f172a; }
 .export-note, details { break-inside: avoid; }
 details { display: block; }
 details > div, details > p { display: block !important; }
 pre { white-space: pre-wrap; }
};

```

```

const escapeHtml = (s: string) =>
 s.replace(/&/g, "&").replace(/</g, "<").replace(/>/g, ">");

/**
 * Размечает термины и превращает их в якорные ссылки на
 * Возвращает готовый HTML главы и список найденных терминов
 */
function prepareBody(html: string, opts: { selfId?: string;
 body: string;
 termIds: string[];
}) {
 const host = document.createElement("div");
 host.innerHTML = html;

 try {
 annotateTerms(host);
 } catch {
 /* если браузер не осилил регулярку – выгружаем текст как есть */
 }
}

```

```

function glossarySection(termIds: string[], book = false) {
 if (!termIds.length) return "";
 const items = termIds
 .map((id) => {
 const e = GLOSSARY_BY_ID.get(id);
 if (!e) return "";
 return `<div class="export-term" id="term-${escapeHtml(id)}">
 <h3>${escapeHtml(e.title)}</h3>
 <p>${escapeHtml(e.short)}</p>
 ${e.formal ? `<p class="formal">${escapeHtml(e.formal)}</p>` : ""}
 ${e.complexity ? `<p class="cx">Сложность: ${escapeHtml(e.complexity)}</p>` : ""}
 </div>`;
 })
 .join("\n");

 return `<section class="export-gloss">
 <h2>${book ? "Словарик терминов пособия" : "Словарик терминов"}</h2>
 <p style="font-size:.8rem;color:#94a3b8;margin:0 0 1rem 0">
 ${book ? "Словарик терминов пособия" : "Словарик терминов"}
 </p>
 ${items}
 </section>`;
}

```

```

function wrap(opts: { title: string; subtitle: string; css: string; extraCss: string }) {
 const stamp = new Date().toLocaleString("ru-RU");
 return `<!doctype html>
<html lang="ru">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>${escapeHtml(opts.title)}</title>
<style>${opts.css}</style>
<style>${EXTRA_CSS}</style>
</head>
<body>
<div class="export-shell">
 <header class="export-head">
 <div style="font-size:.7rem;text-transform:uppercase">

```

```

 if (blob.size > 4_000_000) return;
 const data = await new Promise<string>((resolve, reject) => {
 const fr = new FileReader();
 fr.onload = () => resolve(String(fr.result));
 fr.onerror = () => reject(fr.error);
 fr.readAsDataURL(blob);
 });
 img.setAttribute("src", data);
 } catch {
 /* картинка недоступна — глава останется со ссылкой */
 }
 })
};
return doc.body.innerHTML;
}

```

/\*\*

\* Кадры визуализаций: книга встраивает снимки демонстраций, которые  
 \* файл показывал, что крутилось в приложении. Снимки делались скриптом  
 \* (capture-viz-shots.mjs) и публикует в /export/viz-shots. Если не удалось  
 \* без них (офлайн-сборка, локальный дев) книга остаётся без картинок,  
 \* нормально и молча пропускается.  
 \*/

```

async function vizFigures(chapterId: string): Promise<string> {
 try {
 const base = import.meta.env.BASE_URL || "/";
 const manifestRes = await fetch(`${base}export/viz-manifest.json`);
 if (!manifestRes.ok) return "";
 const manifest = (await manifestRes.json()) as { files: string[] };
 const mine = manifest.filter((m) => m.file === `chapters/${chapterId}.mjs`);
 const figures: string[] = [];
 for (const m of mine.slice(0, 4)) {
 const res = await fetch(`${base}export/viz-shots/${m.file}`);
 if (!res.ok) continue;
 const blob = await res.blob();
 if (blob.size > 3_000_000) continue;
 const data = await new Promise<string>((resolve, reject) => {
 const fr = new FileReader();
 fr.onload = () => resolve(String(fr.result));
 fr.onerror = () => reject(fr.error);
 fr.readAsDataURL(blob);
 });
 figures.push(data);
 }
 return figures.join("\n");
 } catch {
 /* ... */
 }
}

```

```

const css = await collectCss();
const { body: rawBody, termIds } = prepareBody(chapter);
const body = (await vizFigures(chapter.id)) + (await inlineFigures(chapter.id));
return wrap({
 title: chapter.title,
 subtitle: chapter.category || "Универсальное пособие",
 css,
 body,
 gloss: glossarySection(termIds),
 origin: window.location.origin,
});
}

```

```

/** Выгрузить одну главу. */
export async function downloadChapterHtml(chapter: Chapter) {
 triggerDownload(await buildChapterHtml(chapter), safeDownloadUrl);
}

```

```

/** Собрать книгу: всё пособие одним файлом со сквозным индексом. */
export async function buildBookHtml(chapters: Chapter[]) {
 const css = await collectCss();
 const allTerms: string[] = [];
 const bodies: string[] = [];

```

```

 const toc = chapters
 .map((c) => `<li style="margin:.2rem 0"><a href="#ch-${c.id}">${c.title}</a></li>`)
 .join("\n");

```

```

 for (const c of chapters) {
 const { body: rawBody, termIds } = prepareBody(c);
 // картинки глав встраиваются сразу: книга одна, а не много
 // снимки демонстраций подтягиваются из /export/viz
 const body = (await vizFigures(c.id)) + (await inlineFigures(c.id));
 termIds.forEach((t) => {
 if (!allTerms.includes(t)) allTerms.push(t);
 });
 bodies.push(`<div id="ch-${escapeHtml(c.id)}" style="border: 1px solid #ccc; padding: 10px; margin-bottom: 10px;">
 <p style="text-align:right;font-size:.72rem;margin:0 0 10px 0">

```

```

* или «сердце Краскала (билет 18)». Раньше это был про
* номер, но попасть на него не мог — ссылка была «битой
* в никуда. Здесь такие упоминания превращаются в насто
*
* * в приложении — `

```

```

const ticketToChapter = new Map<number, string>();
for (const [id, tickets] of Object.entries(chapterTopics))
 for (const n of tickets) if (!ticketToChapter.has(n))
}

```

```

/** id страницы по номеру билета (undefined, если такого
export const chapterForTicket = (ticket: number): string |

```

```

/** «билет», «билеты», «билетов», «билете», «билетам»...
const TICKET_RE = /билет(?:ы|ов|ам|ах|ом|у|а|е)?\s*(\d{

```

```

/**
* Константы TreeWalker числами: глобальный `NodeFilter`
* во всех окружениях (в jsdom его нет) — с числами разн
*/

```

```

const SHOW_TEXT = 4;
const FILTER_ACCEPT = 1;
const FILTER_REJECT = 2;

```

```

/** Внутри чего ссылки не ставим: код, уже существующие
const SKIP_SELECTOR = "code, pre, script, style, a, but

```

```

/** Одна страница на диапазон? «билеты 18–20» → mst, а

```

```

 if (inCode > 0 || inLink > 0) return part;
 return part.replace(TICKET_RE, (whole, a: string,
 const from = Number(a);
 const to = b ? Number(b) : from;
 const id = chapterForRange(from, to);
 if (!id || id === opts.selfId) return whole;
 return `<a class="ticket-link" data-goto="${id}
 });
 });
 .join("");
}

/**
 * DOM-версия – для интерактивной страницы. Проходит то
 * (как `annotateTerms`), поэтому атрибуты и разметка не
 * Возвращает число созданных ссылок – его проверяют DOM
 */
export function annotateTicketLinks(
 root: HTMLElement,
 opts: { selfId?: string; lite?: boolean } = {}
): number {
 if (typeof document === "undefined") return 0;
 const walker = document.createTreeWalker(root, SHOW_TEXT,
 acceptNode(node) {
 const parent = (node as Text).parentElement;
 if (!parent) return FILTER_REJECT;
 if (parent.closest(SKIP_SELECTOR)) return FILTER_REJECT;
 const text = node.nodeValue ?? "";
 if (/билет/i.test(text)) return FILTER_REJECT;
 return FILTER_ACCEPT;
 },
);
 let count = 0;
 while (current = walker.nextNode()) {
 targets.push(current as Text);
 current = walker.nextNode();
 }
 return count;
}

```

```

 const el = ref.current;
 if (el) annotateTicketLinks(el, opts);
 // eslint-disable-next-line react-hooks/exhaustive-
 }, deps);
}
```

## =====

## РАЗДЕЛ: СКРИПТЫ СБОРКИ И ЭКСПОРТА

## =====

-----

ФАЙЛ 90/111: scripts/audit-coverage.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 83 | РАЗМ

-----

```
/**
 * Аудит покрытия глав пособия: у каких тем нет «объясн
 * (блок «Аналогия») и/или 2D-визуализации (интерактивн
 *
 * node scripts/audit-coverage.mjs # отчёт
 * node scripts/audit-coverage.mjs --md # markd
 */
import fs from "node:fs";
import path from "node:path";
import { fileURLToPath } from "node:url";
import { build } from "esbuild";

const ROOT = path.resolve(path.dirname(fileURLToPath(import.meta.url)),
const TMP = path.join(ROOT, "tmp", "audit");

fs.mkdirSync(TMP, { recursive: true });
const bundle = path.join(TMP, "content.bundle.mjs");

await build({
 entryPoints: [path.join(ROOT, "src/data/content.ts")],
 bundle: true,
 format: "esm",
```



```

if (process.argv.includes("--md")) {
 console.log("| # | Глава | Аналогия | 2D-виз. | SVG |");
 console.log("| --- | --- | --- | --- | --- |");
 for (const r of rows) {
 console.log(
 `| ${r.num} || "-" | ${r.title} | ${r.hasAnalogy}
 ge)) |`
);
 }
} else {
 console.log(`Глав в пособии: ${rows.length}\n`);
 const dump = (title, list) => {
 console.log(`${title} (${list.length}):`);
 list.forEach((r) => console.log(` * ${r.title} [$
 console.log();
 });
 dump("НЕТ НИ АНАЛОГИИ, НИ 2D", gapBoth);
 dump("НЕТ АНАЛОГИИ (2D есть)", gapAnalogy);
 dump("НЕТ 2D (аналогия есть)", gapViz);
 console.log(`Пропущенные номера билетов 1-24: ${missi
 console.log(`Визуализаторы без главы: ${orphanViz.join
}

```

---

ФАЙЛ 91/111: scripts/audit-terms.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 68 | РАЗМ

---

```
/**
```

```

 * Проверка покрытия глав всплывающими подсказками:
 * в каждой ли теме находятся термины из глоссария.
 *
```

```

 * node scripts/audit-terms.mjs

```

```
*/
```

```
import { build } from "esbuild";
```

```
import path from "node:path";
```

```

 const id = map.get(surface);
 if (!id) continue;
 const ut = perTerm.get(id) ?? 0;
 const us = perSurface.get(surface) ?? 0;
 if (ut >= MAX_PER_TERM || us >= MAX_PER_SURFACE) con
 perTerm.set(id, ut + 1);
 perSurface.set(surface, us + 1);
 highlights += 1;
 ids.add(id);
 used.add(id);
 }

 if (ids.size === 0) bad.push(c.title);
 console.log(
 String(ids.size).padStart(3), "|", String(highlight
 c.title.slice(0, 46).padEnd(47), [...ids].slice(0,
);
}
console.log("\nГлав без единой подсказки:", bad.length,
console.log("Терминов в глоссарии:", GLOSSARY.length, "
console.log("Не встретились:", GLOSSARY.filter((g) => !

```

ФАЙЛ 92/111: scripts/export/build-guide-pdf.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 692 | РАЗМ

```

/**
 * Компактный PDF пособия: public/export/guide_lite.pdf
 *
 * текст всех тем (без служебной разметки)
 * + снимки визуализаций, если они отрендерены (tmp/viz
 * capture-viz-shots.mjs в CI через headless-браузер)
 * + иллюстрации страниц (jpg из src/assets)
 * + текстовое описание каждой демонстрации и переменны
 * Python-компилятор (работает и без браузера – локалн
 * – БЕЗ кода панели компилятора и без навигации прилож

```

```

 "/System/Library/Fonts/Supplemental/Arial Unicode.ttf",
 "C:/Windows/Fonts/arial.ttf",
],
 bold: [
 "/usr/share/fonts/truetype/dejavu/DejaVuSans-Bold.ttf",
 "/usr/share/fonts/dejavu/DejaVuSans-Bold.ttf",
 "/usr/local/share/fonts/DejaVuSans-Bold.ttf",
 "C:/Windows/Fonts/arialbd.ttf",
],
 oblique: [
 "/usr/share/fonts/truetype/dejavu/DejaVuSans-Oblique.ttf",
 "/usr/share/fonts/dejavu/DejaVuSans-Oblique.ttf",
],
 mono: [
 "/usr/share/fonts/truetype/dejavu/DejaVuSansMono.ttf",
 "/usr/share/fonts/dejavu/DejaVuSansMono.ttf",
 "/usr/local/share/fonts/DejaVuSansMono.ttf",
 "C:/Windows/Fonts/consola.ttf",
],
};

```

```
const pick = (list) => list.find((f) => fs.existsSync(f))
```

```
/**
```

```

 * Убираем символы, которых нет в DejaVu (эмодзи и пиктограммы)
 * на их месте появляется «пустой» глиф. Стрелки, математические
 * геометрические фигуры DejaVu знает – их оставляем.
 */

```

```
const UNSUPPORTED = /[\u{1F000}-\u{1FAFF}\u{2700}-\u{2700}]/gu;
```

```
/** Русское склонение счётных существительных: 1 страница
```

```

const plural = (n, one, few, many) => {
 const mod10 = Math.abs(n) % 10;
 const mod100 = Math.abs(n) % 100;
 if (mod10 === 1 && mod100 !== 11) return one;
 if (mod10 >= 2 && mod10 <= 4 && (mod100 < 10 || mod100 > 100))
 return many;
};

```

```

const dirs = [SHOTS_DIR, path.join(ROOT, "public", "e
for (const dir of dirs) {
 if (fs.existsSync(dir)) {
 const files = fs
 .readdirSync(dir)
 .filter((f) => /\.(png|jpe?g)$/i.test(f))
 .filter((f) => f === `${chapterId}.png` || f.st
 .sort();
 if (files.length > 0) {
 return files.map((f) => ({
 file: path.join(dir, f),
 name: f.replace(/\.(png|jpe?g)$/i, "").split(
 }));
 }
 }
}
return [];
}

```

/\* \_\_\_\_\_ отрисовка PDF \_\_\_\_\_

```

class Renderer {
 constructor(doc, fonts) {
 this.doc = doc;
 this.fonts = fonts;
 this.footerTitle = "";
 this.pages = 0;
 this.cy = TOP;
 }

 /**
 * Колонтитул текущей страницы (печатаем до перехода к
 *
 * Важно: на время печати нижнее поле страницы уменьш
 * решает, что текст не влез, и молча добавляет пусты
 */
 stampFooter() {
 if (this.pages === 0 || !this.doc.page) return;
 }

```

```

}

height(text, { font, size, width, lineGap = 0 }) {
 return this.doc.font(font).fontSize(size).heightOfS
}

/**
 * Режет текст на фрагменты не выше max пунктов: снач
 * предложений, слишком длинное предложение – по слов
 *
 * Без этого pdfkit сам переносит остаток на новую стр
 * создаётся в обход рендерера – без нижнего колонтит
 * из-за чего в файле оказывалось на одну страницу бо
 */
splitToFit(text, opts, max) {
 const measure = (t) => this.height(t, opts);
 const source = String(text);
 if (measure(source) <= max) return [source];
 const chunks = [];
 let current = "";
 const flush = () => {
 if (current.trim()) chunks.push(current.trim());
 current = "";
 };
 for (const sentence of source.split(/(?<=[.!?…;:])\s/)) {
 const withSentence = current ? `${current} ${sentence}` : sentence;
 if (measure(withSentence) <= max) {
 current = withSentence;
 continue;
 }
 flush();
 if (measure(sentence) <= max) {
 current = sentence;
 continue;
 }
 for (const word of sentence.split(/\s+/)) {
 const withWord = current ? `${current} ${word}` : word;
 if (measure(withWord) <= max) current = withWord;
 }
 }
}

```

```

 const text = plain(rawText);
 if (!text) return;
 const h = this.height(text, { font: this.fonts.bold
 this.ensure(h + 4, 16);
 this.cy += 4;
 this.doc.font(this.fonts.bold).fontSize(size).fillC
 this.cy = Math.max(this.cy, this.doc.y) + 2;
}

```

```

/** Абзац: режим по высоте страницы сами – автоперенос
para(rawText, { size = 10.0, indent = 0, color = "#1f
 const text = plain(rawText);
 if (!text) return;
 const font = mono ? this.fonts.mono : italic && thi
 const width = CONTENT_W - indent;
 const opts = { font, size, width, lineGap };
 const chunks = this.splitToFit(text, opts, BOTTOM -
 chunks.forEach((chunk, index) => {
 const last = index === chunks.length - 1;
 const h = this.height(chunk, opts);
 this.ensure(h + (last ? gapAfter : 0), 6);
 const y0 = this.cy;
 this.doc.font(font).fontSize(size).fillColor(col
 this.cy = Math.max(y0 + h, this.doc.y) + (last ?
 if (this.cy > BOTTOM) this.newPage();
 }));
}

```

```

bullet(rawText, { ordinal = null, depth = 0, size = 9
 const text = plain(rawText);
 if (!text) return;
 const indent = 16 + depth * 14;
 const marker = ordinal ? `${ordinal}.` : "*";
 const width = CONTENT_W - indent - 4;
 const opts = { font: this.fonts.regular, size, width
 const chunks = this.splitToFit(text, opts, BOTTOM -
 chunks.forEach((chunk, index) => {
 const h = this.height(chunk, opts);

```

```

 this.ensure(boxH);
 const y0 = this.cy;
 this.doc.roundedRect(PAGE.margin, y0, CONTENT_W, 1);
 this.doc.font(font).fontSize(size).fillColor("#1f2937");
 this.cy = y0 + boxH + (i < lines.length ? 2 : 7);
 if (i < lines.length) this.newPage();
 }
}

```

```

/** Таблица: строки режутся по страницам, заголовок по ширине
table(rawRows, { size = 8.5 } = {}) {
 const rows = rawRows.map((r) => r.map((c) => plain(c)));
 if (!rows.length) return;
 const cols = Math.max(...rows.map((r) => r.length));
 const widths = new Array(cols).fill(CONTENT_W / cols);
 const pad = 5;
 const header = rows[0];

 const rowHeight = (cells) => {
 let max = 0;
 for (let i = 0; i < cols; i++) {
 const h = this.height(cells[i] ?? "", { font: this.font, size });
 max = Math.max(max, h);
 }
 return max + pad * 2;
 };
}

```

```

const drawRow = (cells, y, isHeader, zebra) => {
 const h = rowHeight(cells);
 this.doc.rect(PAGE.margin, y, CONTENT_W, h).fill(zebra ? "#f3f4f5" : "#fff");
 for (let i = 0; i < cols; i++) {
 const x = PAGE.margin + widths.slice(0, i).reduce((a, b) => a + b, 0);
 this.doc
 .font(isHeader ? this.fonts.bold : this.fonts.regular)
 .fontSize(size)
 .fillColor("#1f2937")
 .text(String(cells[i] ?? ""), x, y + pad, { width: widths[i] });
 }
}

```

```

 });
}

image(file, { maxWidth = CONTENT_W, maxHeight = 320,
 if (!fs.existsSync(file)) return false;
 let img;
 try {
 img = this.doc.openImage(file);
 } catch {
 return false;
 }
 const capH = caption ? 14 : 0;
 const scale = Math.min(maxWidth / img.width, (maxHeight - capH) / img.height);
 const w = img.width * scale;
 const h = img.height * scale;
 this.ensure(h + capH + 10, 20);
 const y0 = this.cy;
 const x0 = PAGE.margin + (CONTENT_W - w) / 2;
 this.doc.image(file, x0, y0, { width: w, height: h });
 this.doc.rect(x0, y0, w, h).lineWidth(0.6).strokeColor('black');
 this.cy = y0 + h + 4;
 if (caption) {
 this.doc.font(this.fonts.regular).fontSize(8).fill('black').text(caption, x0, y0 + h + 10, 'center');
 this.cy = Math.max(this.cy, this.doc.y) + 6;
 }
 return true;
}

```

```

/** Бокс с заголовком и строками; длинные режутся на две строки
box(rawTitle, rawLines, { color = "#eef2ff", border = true,
 const title = plain(rawTitle);
 const lines = rawLines.map((l) => plain(l)).filter((l) => l.length > 0);
 const inner = CONTENT_W - 20;
 let rest = lines.slice();
 let first = true;
 while (rest.length || first) {
 first = false;
 const x0 = PAGE.margin + (inner - rest[0].length) / 2;
 const y0 = this.doc.y;
 this.doc.rect(x0, y0, inner, 1.2).lineWidth(0.6).strokeColor('black');
 this.doc.font(this.fonts.regular).fontSize(12).fill('black').text(title, x0, y0 + 10, 'center');
 for (let i = 0; i < rest.length; i++) {
 this.doc.font(this.fonts.regular).fontSize(12).fill('black').text(rest[i], x0, y0 + 20 + i * 1.2, 'left');
 }
 this.doc.y = y0 + 1.2 + 1.2;
 }
 return true;
}

```



```
fonts.bold = fonts.bold ?? fonts.regular;
fonts.mono = fonts.mono ?? fonts.regular;

const { chapters, chapterTopics, PAGE_SYNC, VIZ_REGISTRY } = require('./data');
ensureDir(OUT_DIR);

const doc = new PDFDocument({
 size: [PAGE.width, PAGE.height],
 margin: PAGE.margin,
 autoFirstPage: false,
 info: {
 Title: "Универсальное пособие – алгоритмы и структуры данных",
 Author: "algv0 · автоэкспорт",
 Subject: "Текст всех тем + визуализации, без кода",
 CreationDate: new Date(),
 },
});

const stream = fs.createWriteStream(GUIDE_PDF_PATH);
doc.pipe(stream);

const R = new Renderer(doc, fonts);
const today = new Date().toISOString().slice(0, 10);
const totalTopics = new Set(Object.values(chapterTopics));
const vizCount = chapters.filter((c) => VIZ_REGISTRY[c]).length;

/* — обложка — */
R.firstPage("Обложка");
R.cy = TOP + 90;
R.h1("Универсальное пособие", { size: 26, rule: false });
R.h1("Алгоритмы и структуры данных", { size: 17, color: "#4b5563" });
R.para(
 `Билеты 1-${totalTopics.size} · ${chapters.length} ${pluralize(
 Count, "интерактивная демонстрация", "интерактивных визуализаций"
)} { size: 10.5, color: "#4b5563" }`,
);
R.para(
 "Автоматический компактный сборник: весь текст тем + визуализации. Навигация приложения и код панели Python",
);
```

```

 : 2 }));
 }
 R.h1(chapter.title.replace(/^\d+[\.-]?s*/, ""), {

const chapterTitle = chapter.title.replace(/^\d+[\.-]?s*/, "");
for (const b of htmlToBlocks(chapter.content)) {
 if (b.type === "p" && /^Билет\s/i.test(b.text)) c
 if (b.type === "heading" && b.level <= 2 && b.tex

 if (b.type === "heading") {
 if (b.level <= 2) R.h2(b.text);
 else if (b.level === 3) R.h3(b.text);
 else R.para(b.text, { size: 10.2, color: "#33415F" });
 } else if (b.type === "summary") {
 R.para("► " + b.text, { size: 9.8, color: "#373737" });
 } else if (b.type === "li") {
 R.bullet(b.text, { ordinal: b.ordinal, depth: b
 } else if (b.type === "code") {
 R.code(b.text);
 } else if (b.type === "table") {
 R.table(b.rows);
 } else if (b.type === "quote") {
 R.quote(b.text);
 } else if (b.type === "caption") {
 R.para(b.text, { size: 8.3, color: "#6b7280", i
 } else {
 R.para(b.text);
 }
}

/* иллюстрация страницы */
const img = CHAPTER_IMAGES[chapter.id];
if (img && fs.existsSync(path.join(ROOT, img))) {
 R.h2("Иллюстрация-аналогия");
 const captions = {
 dijkstra: "✂ Декстер Дуглас у CRT-монитора и ви
 "bellman-ford": " Бэтмен патрулирует Готэм в м
 floyd: " Все вершины связаны со всеми",

```

```

 { size: 8.5, color: "#6b7280", italic: true }
 });
 }
 }

 /* блиц: те же вопросы, что в приложении, но сразу
 const quiz = (quizzes && quizzes[chapter.id]) || []
 if (quiz.length) {
 R.h2("Блиц: вопросы по теме");
 quiz.forEach((q, qi) => {
 R.para(`${qi + 1}. ${q.question}`, { size: 9.6,
 q.options.forEach((option, oi) => {
 R.bullet(oi === q.correctIndex ? `${option} –
 });
 R.para(q.explanation, { size: 8.8, color: "#4b5
 });
 }
}

R.stampFooter();
doc.end();
await new Promise((resolve, reject) => {
 stream.on("finish", resolve);
 stream.on("error", reject);
});

const size = fs.statSync(GUIDE_PDF_PATH).size;
console.log("[guide] PDF пособия собран");
console.log(` тем: ${chapters.length} • снимков: ${chapters
console.log(`      выход: ${path.relative(ROOT, GUIDE_PDF_PATH)}`

// Самопроверка: страниц в файле должно быть ровно столько, сколько
// рендерер. Расхождение означает, что pdftk создал PDF-файл, который
// в файл без нижнего колонтитула и без нумерации.
const pdfLib = await import("pdf-lib");
const physical = (await pdfLib.PDFDocument.load(fs.readFile(GUIDE_PDF_PATH)))
console.log(`      страниц: ${R.pages} (рендер) / ${physical.getPages().length}`
if (R.pages > 100) console.warn(`      ВНИМАНИЕ: ${R.pages} страниц`);

```

```
 .filter((f) => f.endsWith(".png"))
 .sort();

if (pages.length === 0) throw new Error("Не найдено н

ensureDir(OUT_DIR);

const doc = new PDFDocument({
 size: [A4.width, A4.height],
 margin: 0,
 autoFirstPage: false,
 info: {
 Title: "Универсальное пособие – полный исходный к
 Author: "CI: rebuild-pdf",
 Subject: "Экспорт всего кода проекта (код -> txt
 CreationDate: new Date(),
 },
});

const stream = fs.createWriteStream(PDF_PATH);
doc.pipe(stream);

for (const file of pages) {
 doc.addPage({ size: [A4.width, A4.height], margin: 0
 doc.image(path.join(PAGES_DIR, file), 0, 0, { width:
}

doc.end();
await new Promise((resolve, reject) => {
 stream.on("finish", resolve);
 stream.on("error", reject);
});

console.log("[3/3] png -> pdf");
console.log(`      страниц: ${pages.length}`);
console.log(` выход: ${path.relative(ROOT, PDF_P
}
```

```
mst: ["Краскал", "Прим", "Борувка"],
treap: ["Собрать", "Split", "Merge"],
"splay-tree": ["Обход", "Повороты", "Прогулка"],
};

const slug = (s) =>
 s
 .toLowerCase()
 .replace(/[^a-zA-яё0-9]+/gi, "-")
 .replace(/^-+|-+$/g, "")
 .slice(0, 24);

async function vizIds() {
 const bundle = await build({
 stdin: {
 contents: `export { activeVizIds } from "../../src/...";`,
 resolveDir: path.join(ROOT, "scripts", "export"),
 loader: "ts",
 },
 bundle: true,
 write: false,
 format: "esm",
 platform: "node",
 logLevel: "warning",
 });
 const mod = await import("data:text/javascript;base64...");
 return mod.activeVizIds;
}

fs.mkdirSync(OUT_DIR, { recursive: true });

const ids = await vizIds();
console.log(`[export:shots] страниц со визуализациями: ${ids.length}`);

const browser = await puppeteer.launch({
 headless: true,
 args: ["--no-sandbox", "--disable-setuid-sandbox", "--no-dev-shm-usage"],
});
```

```
 } catch (error) {
 failed += 1;
 console.log(`[export:shots] FAIL ${id}: ${String(error)}`);
 }
 }

const manifest = fs
 .readdirSync(OUT_DIR)
 .filter((f) => /\.(png|jpe?g)$/i.test(f))
 .sort()
 .map((f) => ({ file: f, name: f.replace(/\.(png|jpe?g)/, ".txt") }));
fs.writeFileSync(path.join(OUT_DIR, "manifest.json"), JSON.stringify(manifest));

await browser.close();
console.log(`[export:shots] снимков: ${done}, пропусков: ${failed}`);
// Пропущенные страницы не повод ронять пайплайн: PDF д
// текстовое описание демонстрации.
process.exit(0);
```

---

ФАЙЛ 95/111: scripts/export/collect-code.mjs  
КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 196 | РАЗМЕР: 1.5 КБ

---

```
/**
 * ШАГ 1: КОД -> TXT
 *
 * Собирает ВСЕ исходный код репозитория в один текстовый файл
 * public/export/full_code_all_pages.txt
 *
 * Список файлов берётся из git (чтобы ничего не забыть)
 * с фоллбэком на обход файловой системы, если git недоступен
 */
import fs from "node:fs";
import path from "node:path";
import { execFileSync } from "node:child_process";
import { ROOT, OUT_DIR, TXT_PATH, HR, SUBHR, ensureDir,
```

```

 "CI / автоматизация",
 "Прочее",
];

function listFiles() {
 let files;
 try {
 files = execFileSync("git", ["ls-files", "--cached"], {
 cwd: ROOT,
 encoding: "utf8",
 })
 .split("\n")
 .filter(Boolean);
 } catch {
 files = [];
 const walk = (dir) => {
 for (const e of fs.readdirSync(dir, { withFileTypes: true })) {
 const abs = path.join(dir, e.name);
 const rel = path.relative(ROOT, abs).split(path.sep);
 if (EXCLUDE_DIRS.some((d) => rel === d || rel.startsWith(d + path.sep))) continue;
 if (e.isDirectory()) walk(abs);
 else files.push(rel);
 }
 };
 walk(ROOT);
 }

 return files
 .filter((rel) => !EXCLUDE_DIRS.some((d) => rel.startsWith(d + path.sep)))
 .filter((rel) => !EXCLUDE_FILES.has(path.basename(rel)))
 .filter((rel) => CODE_EXT.has(path.extname(rel)))
 .filter((rel) => fs.existsSync(path.join(ROOT, rel)))
 .sort((a, b) => {
 const ia = ORDER.indexOf(categoryOf(a));
 const ib = ORDER.indexOf(categoryOf(b));
 return ia !== ib ? ia - ib : a.localeCompare(b);
 });
}

```

```

push();
push(`Дата генерации:           ${stamp}`);
push(`Файлов исходного кода:    ${files.length}`);
push(`Суммарный объём кода: ${humanSize(totalByteSize)});
push(`Глав/билетов в пособии:    ${chapters.length}`);
push();

```

```

push(HR);
push(" ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ");
push(HR);
chapters.forEach((c, i) => {
 push(`  ${String(i + 1).padStart(3, " ")} ${c.title}`);
});
push();

```

```

push(HR);
push(" ОГЛАВЛЕНИЕ: ФАЙЛЫ ИСХОДНОГО КОДА");
push(HR);
let current = null;
files.forEach((rel, i) => {
 const cat = categoryOf(rel);
 if (cat !== current) {
 current = cat;
 push();
 push(`  # ${cat}`);
 }
 push(`  ${String(i + 1).padStart(3, " ")} ${rel}`);
});
push();
push();

```

```

current = null;
files.forEach((rel, i) => {
 const cat = categoryOf(rel);
 if (cat !== current) {
 current = cat;
 push(HR);
 push(`РАЗДЕЛ: ${cat.toUpperCase()}`);
 }
});

```



```
* Общие настройки и утилиты пайплайна экспорта.
*
* код -> full_code_all_pages.txt (collect-code.mjs)
* txt -> page-XXXX.png (render-pages.mjs)
* png -> full_code_all_pages.pdf (build-pdf.mjs)
*/
import fs from "node:fs";
import path from "node:path";
import { fileURLToPath } from "node:url";
import { execFileSync } from "node:child_process";

export const ROOT = path.resolve(path.dirname(fileURLToPath(import.meta.url)), "..");

/** Куда кладём финальные артефакты (эта папка отдаётся клиенту) */
export const OUT_DIR = path.join(ROOT, "public", "export");

/** Промежуточные PNG-страницы (в .gitignore, в CI уезжают) */
export const PAGES_DIR = path.join(ROOT, "tmp", "export");

export const TXT_PATH = path.join(OUT_DIR, "full_code_all_pages.txt");
export const PDF_PATH = path.join(OUT_DIR, "full_code_all_pages.pdf");

/** Параметры вёрстки страницы (подобраны под A4 @150dpi) */
export const PAGE = {
 /** DPI раstra. Можно переопределить: EXPORT_DENSITY=100 */
 density: Number(process.env.EXPORT_DENSITY ?? 150),
 /** 1-bit ч/б страницы весят в 2-4 раза меньше. EXPORT_GRAYSCALE=1 */
 monochrome: process.env.EXPORT_GRAYSCALE !== "1",
 size: "A4",
 font: "DejaVu-Sans-Mono",
 fontFile: "/usr/share/fonts/truetype/dejavu/DejaVuSansMono.ttf",
 pointsize: 8,
 /** Максимум символов в строке (далее – жёсткий перебор) */
 cols: 138,
 /** Строк содержимого на странице (+2 служебные: шапка, подвал) */
 rows: 76,
};

export const HR = "=".repeat(PAGE.cols);
```

```

* таблицы, код, спойлеры <details>. Служебная разметка
* svg, кнопки) отбрасывается, содержимое спойлеров – не
*
* Никаких внешних зависимостей: собственный мини-парсер
* куски хранятся отдельными узлами #text, поэтому инлайн
* (, <code>,) не «уезжает» в конец абзаца.
*/
```

```
/** Содержимое этих тегов не нужно в текстовой версии.
const SKIP = new Set([
 "svg", "script", "style", "button", "iframe", "video",
 "path", "circle", "rect", "line", "polyline", "polygon",
 "defs", "marker", "tspan", "ellipse", "use", "clipPath",
]);
```

```
/** Пустые элементы (без закрывающего тега). */
const VOID = new Set(["br", "hr", "img", "input", "meta",
```

```
/** Блочные теги: их содержимое собирается в отдельный блок.
const BLOCK = new Set([
 "h1", "h2", "h3", "h4", "h5", "h6", "p", "li", "pre",
 "summary", "figcaption", "table", "tr", "td", "th", "thead",
 "article", "ul", "ol", "details", "figure", "header",
]);
```

```
const HEADING = new Set(["h1", "h2", "h3", "h4", "h5",
```

```
/**
* Разбор HTML в дерево: { tag, attrs, children }, текст
* Намеренно простой: страницы пособия генерируются нами
*/
```

```
export function parseHtml(html) {
 const root = { tag: "#root", attrs: {}, children: [] };
 const stack = [root];
 const re = /<!--[\s\S]*?-->|<\/([a-zA-Z0-9]+)\s*>|<([a-zA-Z0-9]+)\s*/;
 let m;
 while ((m = re.exec(html)) !== null) {
 const [full, closeTag, openTag, attrsStr, selfClose] = m;
```

```

 .replace(/'/g, "'")
 .replace(/ /g, "\u00a0")
 .replace(/—/g, "—")
 .replace(/–/g, "—")
 .replace(/«/g, "«")
 .replace(/»/g, "»")
 .replace(/…/g, "...")
 .replace(/×/g, "×")
 .replace(/≤/g, "≤")
 .replace(/≥/g, "≥")
 .replace(/≠/g, "≠")
 .replace(/∞/g, "∞")
 .replace(/→/g, "→")
 .replace(/←/g, "←")
 .replace(/&/g, "&")
 .replace(/&#(\d+)/g, (_, d) => String.fromCharCode

```

```

const clean = (s) => decodeEntities(s).replace(/\s+/g,

```

```

const isEl = (n) => n.tag !== "#text";

```

```

const isBlockEl = (n) => isEl(n) && BLOCK.has(n.tag) &&

```

```

/** Есть ли у узла блочные потомки (тогда его текст «ра

```

```

const hasBlockChild = (node) => node.children.some(isBl

```

```

/** Текст инлайнового поддерева в порядке документа. */

```

```

function inlineText(node) {
 let out = "";
 for (const c of node.children) {
 if (c.tag === "#text") {
 out += c.text;
 } else if (SKIP.has(c.tag)) {
 continue;
 } else if (BLOCK.has(c.tag)) {
 continue;
 } else if (c.tag === "br") {
 out += "\n";
 } else {

```

```
* Обход дерева: собираем блоки в порядке документа.
* @returns {Array<{type:string,text?:string,rows?:string[]}>}
*/
export function htmlToBlocks(html) {
 const root = parseHtml(html);
 const out = [];

 const push = (block) => {
 const prev = out[out.length - 1];
 if (prev && prev.type === block.type && prev.text === block.text) {
 out.push(block);
 }
 };

 const emitListItem = (li, ordinal, depth) => {
 const text = collectText(li);
 if (text) push({ type: "li", text, ordinal: ordinal });
 // вложенные списки внутри li
 li.children.forEach((c) => {
 if (c.tag === "ul" || c.tag === "ol") emitList(c, depth + 1);
 });
 };

 const emitList = (list, depth) => {
 let i = 0;
 for (const li of list.children) {
 if (li.tag !== "li") continue;
 i += 1;
 emitListItem(li, list.tag === "ol" ? i : null, depth);
 }
 };

 const walk = (node, listDepth = 0) => {
 if (node.tag !== "#root" && SKIP.has(node.tag)) return;

 if (node.tag === "table") {
 const rows = tableRows(node);
 if (rows.length) push({ type: "table", rows });
 return;
 }
 };
}
```

```

 const text = inlineText(node);
 if (!text) return;
 if (HEADING.has(node.tag)) push({ type: "heading",
 else if (node.tag === "li") emitListItem(node, null);
 else if (node.tag === "summary") push({ type: "summary",
 else if (node.tag === "blockquote") push({ type: "quote",
 else if (node.tag === "figcaption") push({ type: "caption",
 else push({ type: "p", text });
 });

 root.children.forEach((c) => walk(c, 0));
 return out;
}

/** То же самое, но простым текстом (для .txt-дампа и в
export function htmlToPlainText(html) {
 const lines = [];
 for (const b of htmlToBlocks(html)) {
 if (b.type === "heading") lines.push("", "#".repeat
 else if (b.type === "summary") lines.push("", "▶ ");
 else if (b.type === "li") lines.push(" ".repeat(b.l
 else if (b.type === "code") lines.push("", ...b.tex
 else if (b.type === "table") {
 lines.push("");
 b.rows.forEach((r) => lines.push("| " + r.join("
 lines.push("");
 } else if (b.type === "quote") lines.push("> " + b.
 else if (b.type === "caption") lines.push(" " + b
 else lines.push(b.text, "");
 }
 return lines.join("\n").replace(/\n{3,}/g, "\n\n").tr
}

```

```

const source = txt.replace(/\r\n/g, "\n").split("\n");
const pages = [];
for (let i = 0; i < source.length; i += PAGE.rows) {
 pages.push(source.slice(i, i + PAGE.rows));
}
return pages;
}

async function main() {
 if (!fs.existsSync(TXT_PATH)) {
 throw new Error(`Нет ${path.relative(ROOT, TXT_PATH)}
 }

 const im = imageMagickCmd();
 const pages = paginate(fs.readFileSync(TXT_PATH, "utf8"));

 fs.rmSync(PAGES_DIR, { recursive: true, force: true });
 ensureDir(PAGES_DIR);

 const total = pages.length;
 const jobs = pages.map((lines, idx) => {
 const num = String(idx + 1).padStart(4, "0");
 const header = `Универсальное пособие • полный исходный код
 Math.max(1, PAGE.cols - 44 - `стр. ${idx + 1} / ${total}`);
 const body = [header, "-".repeat(PAGE.cols), ...lines];
 const txtFile = path.join(PAGES_DIR, `page-${num}.txt`);
 const pngFile = path.join(PAGES_DIR, `page-${num}.png`);
 fs.writeFileSync(txtFile, body + "\n", "utf8");
 return { txtFile, pngFile, num };
 });

 const args = (job) => [
 "-density", String(PAGE.density),
 "-page", PAGE.size,
 "-font", fs.existsSync(PAGE.fontFile) ? PAGE.fontFile : "DejaVuSansMono.ttf",
 "-pointsize", String(PAGE.pointsize),
 `text:${job.txtFile}`,
];
}

```

```

 console.error("Ошибка рендера страниц:", err.message)
 process.exit(1);
});

```

ФАЙЛ 99/111: scripts/verify/browser-checks.tsx

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 1065 | ПА

```

/**
 * DOM-харнес: настоящее приложение в jsdom + настоящий
 * Монтирует App, кликает кнопки, печатает в редакторе
 * визуализациям матрицы чужих размерностей и ловит пад
 */
import { act } from "react";
import { createRoot } from "react-dom/client";
import type { Root } from "react-dom/client";
import App from "../../src/App";
import { chapters, chapterTopics, SECTIONS, sectionOf }
import { quizzes } from "../../src/data/quizzes";
import { VIZ_REGISTRY } from "../../src/components/vizR
import { VizLiveStatus } from "../../src/components/Viz
import { PAGE_SYNC, buildInitTemplate } from "../../src
import { VizChapterContext, emitVizState, clearVizState
import { buildDebugRunner, type DebugResult } from "../

type Py = { runPythonAsync: (src: string) => Promise<un

export interface TestResult {
 group: string;
 name: string;
 ok: boolean;
 ms: number;
 detail: string;
 consoleErrors: string[];
}

```

```

}

function findButton(match: string | RegExp): HTMLButtonElement {
 const list = Array.from(document.querySelectorAll("button"));
 return list.find((b) => {
 const hay = [b.getAttribute("aria-label"), b.getAttribute("value")];
 return typeof match === "string" ? hay.includes(match) : match.test(hay);
 });
}

async function click(el: Element | undefined | null, label: string) {
 if (!el) throw new Error(`не найдена ${label}`);
 await act(async () => {
 el.dispatchEvent(new window.MouseEvent("click", { bubbles: true }));
 await sleep(20);
 });
}

/** Панель компилятора – все поиски внутри неё, чтобы не искать по всему документу */
function panel(): HTMLElement | null {
 return document.querySelector<HTMLElement>('aside[aria-label="Compiler Panel"]');
}

function panelText(): string {
 return (panel()?.textContent ?? "").replace(/\s+/g, " ");
}

function textarea(): HTMLTextAreaElement | undefined {
 return panel()?.querySelector<HTMLTextAreaElement>("text-area");
}

function typeInto(ta: HTMLTextAreaElement, value: string) {
 const setter = Object.getOwnPropertyDescriptor(window, "document").value;
 if (!setter) throw new Error("нет нативного setter'a для document");
 act(() => {
 setter.call(ta, value);
 ta.dispatchEvent(new window.Event("input", { bubbles: true }));
 });
}

```



```

async function mountApp(topic: string) {
 if (root) {
 await act(async () => {
 root?.unmount();
 });
 root = null;
 }
 if (host) {
 host.remove();
 }
 host = freshHost();
 window.history.replaceState({}, "", `/?topic=${encodeURIComponent(topic)}`);
 root = createRoot(host);
 await act(async () => {
 root!.render(<App />);
 });
 await settle(80);
 return host;
}

```

/\*\* Смонтировать одну визуализацию так, как это делает

```

async function mountViz(vizId: string) {
 if (root) {
 await act(async () => {
 root?.unmount();
 });
 root = null;
 }
 if (host) host.remove();
 host = freshHost();
 const entry = VIZ_REGISTRY[vizId];
 if (!entry) throw new Error(`в реестре нет визуализации с id ${vizId}`);
 const Comp = entry.Component;
 root = createRoot(host);
 await act(async () => {
 root!.render(
 <VizChapterContext.Provider value={vizId}>
 <VizLiveStatus demoTitle={entry.title} />
 </VizChapterContext.Provider>
);
 });
 await settle(80);
 return host;
}

```

```

consoleErrors = [];
const t0 = Date.now();
let ok = true;
let detail = "";
try {
 const d = await fn();
 detail = d ?? "";
} catch (error) {
 ok = false;
 detail = error instanceof Error ? `${error.message}` :
}
const ms = Date.now() - t0;
const errs = consoleErrors.filter((e) => !/not wrapped/.test(e));
if (errs.length > 0) {
 ok = false;
 detail = `${detail}\nconsole.error (${errs.length})
 errs.join("\n")
}
results.push({ group, name, ok, ms, detail, consoleErrors });
const mark = ok ? "PASS" : "FAIL";
realConsoleError === console.error;
process.stdout.write(`${mark} [${group}] ${name}${ok ? "" : " failed"}\n`);
if (!ok) process.stdout.write(detail.split("\n").map((e) => `  ${e}\n`));
return ok;
}

/* -----
/* сценарии
/* -----

/** Фильтры прогона: HAR_CHAPTERS=segment-trees,floyd и
const onlyChapters = (process.env.HAR_CHAPTERS || "").split(",");
const skipGroups = new Set((process.env.HAR_SKIP || "").split(","));
/** HAR_SYNC=floyd,dijkstra – прогнать только эти записи
const onlySync = new Set((process.env.HAR_SYNC || "").split(","));

export async function runSuite({ py }: { py: Py }): Promise<void> {
 const chapterList = onlyChapters.length ? chapters.filter((c) => onlyChapters.includes(c)) : chapters;
 const want = (group: string) => !skipGroups.has(group) && (onlySync.size ? onlySync.has(group) : true);
 const groups = chapterList.filter((c) => want(c.group)).map((c) => c.group);
 const results = [];
 for (const group of groups) {
 const name = ` ${group} `;
 const fn = () => {
 const c = chapterList.find((c) => c.group === group);
 if (!c) throw new Error(`chapter ${group} not found`);
 return c.fn();
 };
 const ok = await runTest(group, name, fn);
 results.push({ group, name, ok, ms, detail, consoleErrors });
 }
 const mark = results.length ? "PASS" : "FAIL";
 process.stdout.write(`${mark} [${groups.join(", ")}] ${results.length} tests${results.length > 1 ? "s" : ""} ${ok ? "" : " failed"}\n`);
 if (!ok) results.forEach((r) => process.stdout.write(`${r.detail}\n`));
}

```

```

const done = await waitFor(() => RUN_DONE.some((m) =>
 if (!done) throw new Error(`запуск не завершился`);
const msg = RUN_DONE.find((m) => bodyText().includes(m));
const sync = PAGE_SYNC[chapter.id];
if (!sync?.code) return `кода синхронизации нет`;
if (/Ошибка|Не удалось/.test(msg)) throw new Error(msg);
// после настоящего прогона визуализация обязана быть
const linked = await waitFor(() => /данные из коммента/);
if (!linked) return `${msg}; визуализация осталась прежней`;
return `${msg} → визуализация переключилась на данные`;
});
}

```

/\* — 2. Флойд: матрицы ЛЮБОЙ размерности из кода полнотестов \*/

```

const floydCases: { name: string; vars: Record<string, any> } = [
 { name: "5x5 (меньше демо-7x7)", vars: { n: 5, k: 1, i: 0, j: 0,
 [9, 6, 9, 9, 0] } }, expect: "5x5" },
 { name: "10x10 (больше демо – старый краш nodeNames)", vars: {
 from({ length: 10 }, (_, c) => (r === c ? 0 : (r + c) % 10)) },
 expect: "10x10" },
 { name: "3x3 с бесконечностями", vars: { n: 3, k: 0, i: 0, j: 0,
 [0, 1, 2] }, expect: "3x3" },
 { name: "не квадратная 4x6", vars: { n: 4, k: 1, i: 0, j: 0,
 [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 0, 20] } },
 expect: "Демо-схема графа" },
 { name: "словарь словарей d[0][1]", vars: { n: 4, k: 0, i: 0, j: 0,
 2: { 0: 9, 1: 9, 2: 0, 3: 1 }, 3: { 0: 3, 1: 9, 2: 0, 3: 0 } },
 expect: "словарь словарей" },
 { name: "ключи-пары \"(i, j)\", vars: { n: 3, k: 0, i: 0, j: 0,
 "(1, 2)": 2, "(2, 0)": 1, "(2, 1)": 9, "(2, 2)": 0 },
 expect: "ключи-пары" },
 { name: "плоский список – это НЕ матрица", vars: { n: 0, k: 0, i: 0, j: 0,
 [0] }, expect: "плоский список" },
 { name: "80x80 (обрезка до 64)", vars: { n: 80, k: 0, i: 0, j: 0,
 (_, c) => (r === c ? 0 : 1 + ((r + c) % 50))) },
 expect: "80x80" },
 { name: "пустая матрица", vars: { n: 0, k: 0, i: 0, j: 0,
 [0] }, expect: "пустая матрица" },
 { name: "матрица под именем D и k/i/j вне диапазона", vars: {
 [10, 11, 12, 0] }, expect: "4x4" },
];

```

```

for (const c of floydCases) {
 await test("В. Флойд: размерности", c.name, async () => {
 await mountViz("floyd");
 });
}

```

```

 if (!c.expect.test(txt)) throw new Error(`ожидали
const defects = domDefects(host);
if (defects.length) throw new Error(`артефакты в
return /размерность не совпадает/.test(txt) ? "пр
 });
}

await test("С. матричные страницы", "2D-построение: о
await mountViz("sparse-table");
const A8 = Array.from({ length: 8 }, (_, i) => Array
await act(async () => {
 emitVizState("sparse-table", { line: 7, func: "<math>
 });
 await sleep(40);
});
const grids = () => Array.from((host ?? document).q
let g0 = grids()[0];
if (!g0) throw new Error("нет ни одной матрицы блок
const firstCell = () => (g0.children[0]?.textContent
// «Оригинал (1x1)» = сама матрица A: значения видн
if (firstCell() !== "1") throw new Error(`оригинал
if ((g0.children[7]?.textContent ?? "").trim() !==
// шаги кликаются даже под live-k из компилятора
await click(findButton("Шаг 2 (4x4)"), "вкладка шаг
const txt = (host?.textContent ?? "").replace(/\s+/
if (!/Матрица блоков 4x4/.test(txt)) throw new Error
g0 = grids()[0];
const cls = (g0.children[0] as HTMLElement)?.classN
// пустая активная клетка (код смотрит на (0,0), st
if (!/border-dashed/.test(cls) || !/indigo/.test(cl
 throw new Error(`пустая активная клетка нарисован
}
if (domDefects(host).length) throw new Error(`артеф
return "оригинал из A с первого шага, вкладка «Шаг
});

/* — 4. Дейкстра / Форд-Беллман: чужие имена вершин
for (const [viz, vars] of [

```

```

const seen = new Set<string>();
for (const s of result.steps) for (const n of Object.keys(result.finalValues ?? {}))
for (const n of Object.keys(result.finalValues ?? {}))
// в документации имя может быть видом «хк / х»,
// проверяем, что хотя бы один python-идентификатор
const declared = sync.variables.map({v} => v.name);
const ident = (t: string) => /^[A-Za-z_][A-Za-z0-9_]*$/;
const missing = declared
 .map({raw} => raw.split(/[/[\],\s+или\s+/).map(s => s.trim()))
 .filter({alts} => alts.some(ident))
 .filter({alts} => !alts.some(a => ident(a) && !seen.add(a)))
 .map({alts} => alts.join(" / "));
if (missing.length) throw new Error(`код не создан`);

```

```

if (!VIZ_REGISTRY[chapterId]) return `шагов ${result.steps.length}`;
await mountViz(chapterId);
const finalVars = result.finalValues ?? result.steps[result.steps.length - 1].finalValues;
await act(async () => {
 emitVizState(chapterId, { line: result.steps[result.steps.length - 1].line,
 ed: Object.keys(finalVars) });
 await sleep(40);
});
const txt = (host?.textContent ?? "").replace(/[/\s]/g, " ");
if (!/данные из компилятора/.test(txt)) throw new Error(`ошибка в коде`);
const defects = domDefects(host);
if (defects.length) throw new Error(`артефакты в коде`);
return `шагов ${result.steps.length}${result.steps[result.steps.length - 1].finalValues ? " с переменными " : ""} `;

```

```

});

```

```

}

```

/\* — 6. Поведение редактора: пустой код, ошибки, гла...

```

const ch = "dijkstra";

```

```

await test("F. редактор", "только комментарии: редактор", async () => {
 await mountApp(ch);
 await click(findButton("Боковая панель Python"), "п...

```

```

 if (domDefects(host).length) throw new Error(`артефакты: ${domDefects(host).length}
 return `вывод обрезан до ${out.length} знаков, хвост: ${out.length - 1}
 });

```

```

await test("F. редактор", "полная очистка: ничего не осталось", async (ch) => {
 await mountApp(ch);
 await click(findButton("Боковая панель Python"), "поиск");
 const ta = textarea();
 if (!ta) throw new Error("нет редактора");
 typeInto(ta, "");
 const done = await waitFor(() => bodyText().include("очистка"));
 if (!done) throw new Error(`на пустой редактор панель не исчезла`);
 if (!textarea()) throw new Error("редактор исчез после очистки");
 if (!/встроенное демо/.test(bodyText())) throw new Error("не встроенное демо");
 if (allButtons().length < 5) throw new Error(`кнопок: ${allButtons().length}, не 5`);
 return `пустой код: кнопок ${allButtons().length}, не 5`;
});

```

```

await test("F. редактор", "глаз: эталон попадает в редактор", async (ch) => {
 await mountApp(ch);
 await click(findButton("Боковая панель Python"), "поиск");
 const ta = textarea();
 if (!ta) throw new Error("нет редактора");
 const init = ta.value;
 const editorButtons = allButtons();
 const ref = PAGE_SYNC[ch]?.code ?? "";
 if (!ref) throw new Error("у страницы нет эталонного кода");
 const done0 = await waitFor(() => RUN_DONE.some((m) => m === "глаз"));
 if (!done0) throw new Error(`инициализация не выполнена`);

```

```

// — глаз ВКЛ: эталон лежит В РЕДАКТОРЕ, и редактор не пустой
await click(findButton("Вставить эталонный код"), "поиск");
const trace = await waitFor(() => !!findButton(/Остаток/));
if (!trace) throw new Error(`эталон не запустился. ${findButton(/Остаток/).length} кнопок`);
if (!textarea()) throw new Error("редактор исчез во время теста");
if (/Ошибка:/.test(panelText())) throw new Error(`эталон не вставлен`);
const ta2 = textarea();
const firstRefLine = ref.split("\n").find((l) => l.startsWith("// —"));

```

```

 if (!back) throw new Error(`\u0438 не вернул инициализацию`);
 return "эталон вставляется в редактор, ведёт демонстрацию";
 });

```

```

await test("F. редактор", "код не перетекает между страницами");
await mountApp(ch);
await click(findButton("Боковая панель Python"), "показать панель");
const ta = textarea();
if (!ta) throw new Error("нет редактора");
typeInto(ta, "# MY-MARKER-14\nx = 41\nx += 1\nprint x\n");
await waitFor(() => RUN_DONE.some((m) => bodyText(m).includes("42")));
if (!bodyText().includes("42")) throw new Error("свернуть не удалось");
// переходим на другую страницу, не закрывая панель
await mountApp("floyd");
await click(findButton("Боковая панель Python"), "показать панель");
const ta2 = textarea();
if (!ta2) throw new Error("на новой странице нет редактора");
if (ta2.value.includes("MY-MARKER-14")) throw new Error("код перетек");
if (!ta2.value.includes("# «")) throw new Error("на новой странице нет редактора");
await mountApp(ch);
await click(findButton("Боковая панель Python"), "показать панель");
const ta3 = textarea();
if (!ta3?.value.includes("MY-MARKER-14")) throw new Error("код перетек");
return "у каждой страницы свой код, возврат сохраняется";
});

```

```

await test("F. редактор", "редактор живёт и во время прогона трассы");
await mountApp(ch);
await click(findButton("Боковая панель Python"), "показать панель");
const done = await waitFor(() => bodyText().includes("трасса собралась"));
if (!done) throw new Error(`трасса не собралась. Панель не появилась`);
if (!textarea()) throw new Error("во время прогона редактор не появился");
if (!findButton("/Остановить прогон трассы/")) throw new Error("кнопка не появилась");
// Листинг сворачивается (освобождая место редактору)
// и редактор остаётся на месте.
const codeLines = () => panel()?.querySelectorAll("pre");
if (codeLines() === 0) throw new Error("в раскрытой панели нет кода");
await click(findButton("Свернуть листинг трассы"), "свернуть");

```

```

const listing = await waitFor(() => !!findButton(/0
if (!listing) throw new Error("панель не показала тр
const namelessListing = namelessButtons(panel() ??
const listingBtns = allButtons().length;
if (namelessEditor.length) throw new Error(`в редак
 , ")}`);
if (namelessListing.length) throw new Error(`в листин
 (" , ")}`);
const required = [/Остановить прогон трассы/, /Сверн
 начало трассы/, /В конец трассы/, /автопроход/i]
const btns = allButtons().join(" | ");
const missing = required.filter((re) => !re.test(btns)
if (missing.length) throw new Error(`в листинге нет
return `редактор: ${editorBtns} кнопок, листинг: ${
});

```

```

await test("Н. доступность", "бейдж честно различает
 await mountApp(ch);
// 1) демо: компилятор ещё ничего не присылал
if (!/встроенное демо/.test(bodyText())) throw new Error
if (/переменных пока нет/.test(bodyText())) throw new Error
await click(findButton("Боковая панель Python"), "панель")
const listing = await waitFor(() => !!findButton(/0
if (!listing) throw new Error("панель не показала тр
// 2) код выполнен, отладчик на первой строке: переменных пока нет
const awaiting = await waitFor(() => /переменных пока нет/
if (!awaiting) throw new Error(`после запуска бейджа не
 `${(bodyText().match(/встроенное демо|данные из компилятора/))}`
// 3) дошли до конца трассы – картинка обязана жить
await click(findButton("В конец трассы"), "в конец трассы")
const linked = await waitFor(() => /данные из компилятора/
if (!linked) throw new Error("в конце трассы визуализация не
// 4) сняли связь (Esc + пустой код) – снова демо
await key("Escape");
await settle(80);
const ta = textarea();
if (!ta) throw new Error("редактор не вернулся по expected type
typeInto(ta, "");

```



```

 if (!el) throw new Error(`в шапке нет кнопки «${w}»`);
 const href = el.getAttribute("href") ?? "";
 if (!href.endsWith(w.file)) throw new Error(`«${w}» не имеет файла`);
 const name = [el.getAttribute("aria-label"), el.getAttribute("title")].filter(Boolean).join(" ");
 if (name.trim().length < 5) throw new Error(`у кнопки нет названия`);
 }
 if (!menu.querySelector("#download-html-btn")) throw new Error("нет кнопки загрузки HTML");
 const py = document.querySelector<HTMLButtonElement>("#download-python");
 if (!py) throw new Error("нет кнопки панели Python");
 if (!py.getAttribute("aria-label")) throw new Error("у кнопки нет aria-label");
 if (!py.getAttribute("title")) throw new Error("у кнопки нет title");
 return "4 загрузки на месте, href ведут на файлы, кнопки работают";
});

```

```

await test("I. ссылки", "упоминание «билет 5» кликабельно", async () => {
 await mountApp("sparse-table");
 const links = Array.from(document.querySelectorAll("a"));
 if (links.length === 0) throw new Error("на странице нет ссылок");
 const toPrefix = links.find((a) => a.dataset.goto === "5");
 if (!toPrefix) throw new Error(`нет ссылки на билет 5`);
 if (!/билет 5/i.test(toPrefix.textContent ?? "")) throw new Error(`ссылка не содержит «билет 5»`);
 const href = toPrefix.getAttribute("href") ?? "";
 if (!href.includes("topic=prefix-sums-2d")) throw new Error("ссылка не содержит «topic=prefix-sums-2d»");
 await click(toPrefix, "ссылка на билет 5");
 await settle(120);
 const txt = bodyText();
 if (!/Двумерная матрица префиксных сумм/.test(txt))
 throw new Error(`клик по «билет 5» не открыл страницу`);
 return `ссылка на страницу: ${links.length}, переход работает`;
});

```

```

await test("I. ссылки", "содержание: разделы по порядку", async () => {
 await mountApp("bridges-code");
 const sidebar = document.querySelector("#sidebar-content");
 if (!sidebar) throw new Error("нет сайдбара с содержанием");
 const headers = Array.from(sidebar.querySelectorAll("h2, h3"));
 if (headers.length !== 5) throw new Error(`ожидалось 5 заголовков, получено ${headers.length}`);
 return "содержание соответствует";
});

```

```

await test("I. ссылки", "введение: базовая база, обзор");
await mountApp("intro");
const txt = bodyText();
for (const must of ["Дерево как массив", "Куча", "Обзор"])
 if (!txt.includes(must)) throw new Error(`во введении не найдено: ${must}`);
if (!txt.includes("2n") || !txt.includes("2n+1")) throw new Error("не найдено: 2n или 2n+1");
if (!txt.includes("≤ 1")) throw new Error("не найдено: ≤ 1");
if (/Обзор: Кратчайшие пути/.test(txt)) throw new Error("неверно: кратчайшие пути");
const goto = Array.from(document.querySelectorAll("a[href]"));
if (!goto.includes("dijkstra")) throw new Error("не найдено: дикий");
if (domDefects(document.querySelector("#chapter-viz")))
 throw new Error("артефакты в DOM введения");
return "введение = куча/очередь/бинпоиск/сортировка";
});

```

```

await test("I. блиц", "вопросы отвечают, объясняют и");
await mountApp("sparse-table");
const list = quizzes["sparse-table"] ?? [];
if (list.length === 0) throw new Error("у билета 2 вопроса");
const section = Array.from(document.querySelectorAll("div"));
if (!section) throw new Error("блиц не отрисован на странице");
const ask = (label: string | RegExp) =>
 Array.from(section.querySelectorAll("button")).find(
 b => b.getAttribute("aria-label").includes(label)
);
const hay = [b.getAttribute("aria-label"), b.textContent];
return typeof label === "string" ? hay.includes(label) : false;
});
const shown = () => (section.textContent ?? "").includes(list[0].question.slice(0, 24));
if (!shown()) throw new Error(`первый вопрос не показан: ${list[0].question}`);
const wrong = list[0].options.findIndex((_, i) => i !== 0);
await click(ask(list[0].options[wrong].slice(0, 20)));
await settle(80);
if (!/Неверно/.test(shown())) throw new Error("на странице не показан ответ");

```



```

const ta = textarea();
if (!ta) throw new Error("нет редактора");
typeInto(
 ta,
 [
 "a = [5, 1, 9, 3, 7, 2]",
 "n = len(a)",
 "LOG = 3",
 "i, j = 0, 0",
 "st = []",
 "for row in a:",
 " st.append([row] + [None] * (LOG - 1))",
 "for j in range(1, LOG):",
 " for i in range(n - (1 << j) + 1):",
 " st[i][j] = min(st[i][j - 1], st[i + (1 << j) - 1])",
 "print(n, st[0][2], st[1][1])",
].join("\n")
);
const ready = await waitFor(() => bodyText().includes("Готово"));
if (!ready) throw new Error("своя таблица не построена");
if (!/6 1 1/.test(panelText())) throw new Error("отсутствует таблица");
await click(findButton("В конец трассы"), "в конец трассы");
const linked = await waitFor(() => /данные из компилятора/);
if (!linked) throw new Error("демонстрация не принята");
const defects = domDefects(document.querySelector("#defects"));
if (defects.length) throw new Error("артефакты в демонстрации");
return "свой массив из 6 чисел: st[0][2] = 1, st[1][1] = 1";
});

await test("К. свой код", "Косарайю: эталон сам считает");
await mountApp("scc-kosaraju");
await click(findButton("Боковая панель Python"), "панель Python");
if (!textarea()) throw new Error("нет редактора");
await click(findButton("Вставить эталонный код"), "вставить код");
const ready = await waitFor(() => bodyText().includes("Готово"));
if (!ready) throw new Error("эталон Косарайю не выполнен");
if (/Ошибка:/.test(panelText())) throw new Error("эталон не выполнен");
const out = panelText();

```

```

const unknown = liteLinks.filter((a) => !chapters.s
if (unknown.length) throw new Error(`в lite-версии
if (!/Блиц: вопросы по теме/.test(txt)) throw new E
if (!/Верно:/.test(txt)) throw new Error("в lite-ве
return `${txt.length} знаков, ссылок в lite ${liteL
});

await test("G. lite-режим", "HTML-книга: оглавление,
const { buildBookHtml } = await import("../src/u
const html = await buildBookHtml(chapters);
if (!html.includes('<nav id="toc"')) throw new Erro
const missing = chapters.filter((c) => !html.includ
if (missing.length) throw new Error(`в книге потеря
const tocLinks = (html.match(/href="#ch-/g) ?? []).
if (tocLinks < chapters.length) throw new Error(`сс
if (!/Словарик терминов пособия/.test(html)) throw
if (!/↑ к оглавлению/.test(html)) throw new Error("
if (html.length < 200_000) throw new Error(`книга п
return `${(html.length / 1024).toFixed(0)} КБ, глав
});

console.error = realConsoleError;
return results;
}

/** Сводка для оркестратора. */
export function summary() {
 const byGroup = new Map<string, { ok: number; fail: n
 for (const r of results) {
 const g = byGroup.get(r.group) ?? { ok: 0, fail: 0
 g[r.ok ? "ok" : "fail"]++;
 byGroup.set(r.group, g);
 }
 return { total: results.length, passed: results.filte
 .fromEntries(byGroup) };
}

/** Инициализация страницы из vizSync – нужна для прове

```



```

const withoutLinks = [];
for (const chapter of chapters) {
 const html = chapter.content ?? "";
 let mentionsHere = 0;

 // 3. упоминания «билет N» — только внутри текста, не
 const textOnly = html
 .replace(/<pre[\s\S]*?</pre>/gi, " ")
 .replace(/<code[\s\S]*?</code>/gi, " ")
 .replace(/<[>]*>/g, " ");
 for (const m of textOnly.matchAll(TICKET_RE)) {
 const from = Number(m[1]);
 const to = m[2] ? Number(m[2]) : from;
 mentionsHere += 1;
 for (let n = from; n <= to; n += 1) {
 ok(n >= 1 && n <= 24 && covered.has(n), `«${chapter
 }
 }
 ticketMentions += mentionsHere;

 // 4. явные переходы
 for (const m of html.matchAll(/data-goto="([^\"]+)"/g))
 explicitLinks += 1;
 ok(chapterIds.has(m[1]), `«${chapter.id}»: data-goto
 }
 for (const m of html.matchAll(/href="\?(?:lite=1&)?to
 explicitLinks += 1;
 ok(chapterIds.has(decodeURIComponent(m[1])), `«${ch
 }

 // 5. якоря
 for (const m of html.matchAll(/href="#([^\"]+)"/g)) {
 ok(html.includes(`id="${m[1]}"`), `«${chapter.id}»:
 }

 if (mentionsHere === 0 && !/data-goto=/.test(html)) w
}

```

```

for (const id of withoutLinks) NOTES.push(`нет перекрёс
const noQuiz = chapters.filter((c) => !(quizzes[c.id] ?

console.log("----- ссылки и структура содержания —
console.log(` страниц: ${chapters.length} • разделов:
console.log(` упоминаний «билет N» в тексте: ${ticketM
console.log(` визуализаций в реестре: ${registryIds.le
 GLOSSARY.length}`);
console.log(` блиц есть у ${Object.keys(quizzes).length
if (NOTES.length) console.log(` заметки:\n ${NOTES.
if (noQuiz.length) console.log(` без блица: ${noQuiz.j

if (FAILS.length) {
 console.log(`\nПРОВАЛЕНО ${FAILS.length}:`);
 FAILS.slice(0, 40).forEach((f) => console.log(` x ${f
 process.exit(1);
}
console.log("\nвсе ссылки целы: 0 битых переходов");
```

```

ФАЙЛ 101/111: scripts/verify/check-reference-code.mjs
КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 133 | РАЗМ
```

```

/**
 * Контракт «эталонный код ↔ страница ↔ визуализация».
 *
 * npm run verify:code
 *
 * Каждый референсный Python-код из PAGE_SYNC выполняет
 * (pyodide из node_modules) под тем же трейсером, что
 * проверяется:
 * 1. код не падает и не обрывается по лимиту шагов;
 * 2. трасса не пустая (иначе панель показала бы пуст
 * 3. каждая переменная, обещанная странице, реально
 * иначе визуализация не может за ним следовать;
 * 4. в снимках нет «мусорных» значений, которые урон
```



```

py.setStderr({ batched: (m) => printed.push("[stderr] "

const only = process.argv.slice(2);
const ident = (t) => /^[A-Za-z_][A-Za-z0-9_]*$/.test(t)
/** «xk / x», «dist[v]», «(i-1)//2», «π» → набор python
const alts = (raw) => raw.split(/[\/[\],]|\s+или\s+/).m

const rows = [];
let problems = 0;
for (const [key, entry] of Object.entries(mod.sync)) {
 if (only.length && !only.some((o) => key.includes(o)))
 printed.length = 0;
 let result;
 try {
 result = JSON.parse(String(await py.runPythonAsync(
 } catch (error) {
 problems++;
 rows.push({ key, steps: 0, status: `СБОЙ ЗАПУСКА: $
 continue;
 }
 const seen = new Set();
 for (const step of result.steps) for (const name of 0)
 for (const name of Object.keys(result.finalValues ??

const issues = [];
if (result.error) issues.push(`ошибка Python: ${result
if (result.truncated) issues.push("трасса оборвана по
if (result.steps.length === 0) issues.push("нет ни од
const missing = (mod.declared[key] ?? [])
 .map(alts)
 .filter((a) => a.some(ident))
 .filter((a) => !a.some((x) => ident(x) && seen.has(
 .map((a) => a.join(" / "));
if (missing.length) issues.push(`код не создаёт обеща
/*
* Обещанная переменная не должна быть пустой на ВСЕХ
* объявил и не наполнил – визуализации нечего показы
* «защитый» результат вместо работы алгоритма. Именн

```

```

===== "ок" ? "" : r.status}`});
}
console.log("=".repeat(96));
console.log(`всего: ${rows.length} | с проблемами: ${problems}`);
writeFileSync(`${OUT_DIR}/reference-code.json`, JSON.stringify(rows));
process.exit(problems > 0 ? 1 : 0);

```

ФАЙЛ 102/111: scripts/verify/dom-env.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 107 | РАЗМЕР: 1.5 КБ

```

/**
 * Браузерное окружение для проверок: jsdom + те API, которые есть в браузере
 * Создается ПОСЛЕ загрузки Pyodide (иначе pyodide решит, что это не браузер)
 */
import { JSDOM, VirtualConsole } from "jsdom";

const OBSERVER_STUB = class {
 observe() {}
 unobserve() {}
 disconnect() {}
 takeRecords() {
 return [];
 }
};

export function createDom({ url = "http://localhost/?to=reference-code" }) {
 const virtualConsole = new VirtualConsole();
 const jsdomErrors = [];
 virtualConsole.on("jsdomError", (error) => {
 jsdomErrors.push(error?.message || String(error));
 onError?.(error);
 });
 // console.error/warn приложения собираем отдельно – чтобы не мешало
 const consoleErrors = [];
 virtualConsole.on("error", (...args) => consoleErrors.push(...args));

```

```

 "navigator",
 "HTMLElement",
 "HTMLInputElement",
 "HTMLTextAreaElement",
 "HTMLButtonElement",
 "HTMLAnchorElement",
 "HTMLDivElement",
 "HTMLSelectElement",
 "Element",
 "Node",
 "Event",
 "CustomEvent",
 "MouseEvent",
 "KeyboardEvent",
 "InputEvent",
 "getComputedStyle",
 "requestAnimationFrame",
 "cancelAnimationFrame",
 "DOMParser",
 "SVGElement",
 "Text",
 "DocumentFragment",
 "NodeList",
 "HTMLCollection",
 "Range",
 "MutationObserver",
 "localStorage",
 "sessionStorage",
 "Blob",
 "File",
 "FileReader",
];
 for (const key of copy) {
 try {
 Object.defineProperty(globalThis, key, { value: w
 } catch {
 /* read-only в Node – не критично */
 }
 }

```

```

 logLevel: "warning",
 });
const { liveMatrix, pickLiveVar, clampIndex } = await import('live-matrix');

let passed = 0;
const failures = [];
function check(name, actual, expected) {
 const a = JSON.stringify(actual);
 const e = JSON.stringify(expected);
 if (a === e) passed++;
 else failures.push(`${name}\n          получено: ${a}\n          ожидалось: ${e}`);
}
const m = (v) => liveMatrix(v);
const shape = (v) => {
 const r = m(v);
 return r ? { rows: r.rows, cols: r.cols, label: r.size } : null;
};
const cells = (v) => {
 const r = m(v);
 if (!r) return null;
 return Array.from({ length: r.rows }, (_, i) => Array.from({ length: r.cols }, (_, j) => r.get(i, j)));
};

/* — liveMatrix: формы данных —————
check("квадратная 3×3 (список списков)", shape([[1, 2, 3], [4, 5, 6], [7, 8, 9]]), [[1, 2, 3], [4, 5, 6], [7, 8, 9]]);
check("не квадратная 2×4", shape([[1, 2, 3, 4], [5, 6, 7, 8]]), [[1, 2, 3, 4], [5, 6, 7, 8]]);
check("значения ячеек 2×2", cells([[0, 1], [2, 3]]), [[0, 1], [2, 3]]);
check("рванные строки: cols = максимум", shape([[1, 2, 3], [4, 5, 6], [7, 8, 9, 10]]), [[1, 2, 3], [4, 5, 6], [7, 8, 9, 10]]);
check("рванные строки: дырка = null", cells([[1, 2, 3], [4, 5, 6], [7, 8, 9, null]]), [[1, 2, 3], [4, 5, 6], [7, 8, 9, null]]);
check("at() вне диапазона → null", m([[1, 2], [3, 4]]).at(10, 10), null);
check("at() с отрицательным индексом → null", m([[1, 2], [3, 4]]).at(-10, -10), null);
check("список словарей с числовыми ключами", cells([{ 0: 1, 1: 2 }, { 0: 3, 1: 4 }]), [{ 0: 1, 1: 2 }, { 0: 3, 1: 4 }]);
check("словарь списков", cells({ 0: [1, 2], 1: [3, 4] }), { 0: [1, 2], 1: [3, 4] });
check("словарь словарей", cells({ 0: { 0: 7, 1: 8 }, 1: { 0: 9, 1: 10 } }), { 0: { 0: 7, 1: 8 }, 1: { 0: 9, 1: 10 } });
check('ключи-пары "(i, j)" после снапшота tuple', cells({ 0: { 0: 7, 1: 8 }, 1: { 0: 9, 1: 10 } }, tuple), { 0: { 0: 7, 1: 8 }, 1: { 0: 9, 1: 10 } });
check('ключи-пары без скобок "0, 1"', cells({ "0, 0": 5, "0, 1": 6, "1, 0": 7, "1, 1": 8 }, tuple), { "0, 0": 5, "0, 1": 6, "1, 0": 7, "1, 1": 8 });

/* — liveMatrix: что матрицей НЕ является —————

```

```

process.exit(failures.length ? 1 : 0);

```

ФАЙЛ 104/111: scripts/verify/probe-compiler.tsx

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 121 | РАЗМЕР: 1.2 КБ

```

/**
 * Зонд состояния панели Python во времени: что происходит
 * Запуск: node tmp/harness/run-probe.mjs
 */
import { act } from "react";
import { createRoot } from "react-dom/client";
import App from "../../src/App";

const sleep = (ms: number) => new Promise((r) => setTimeout(r, ms));
async function settle(ms = 40) {
 await act(async () => {
 await sleep(ms);
 });
}
function panel(): HTMLElement | null {
 return document.querySelector<HTMLElement>('aside[aria-label="Panel"]');
}
function editor(): HTMLTextAreaElement | undefined {
 return panel()?.querySelector<HTMLTextAreaElement>('text-area');
}
function footer(): string {
 const divs = Array.from(panel()?.querySelectorAll("div"));
 const f = divs.find((d) => d.className.includes("text"));
 return (f?.textContent ?? "").replace(/\s+/g, " ");
}
function state(tag: string) {
 const p = panel();
 const ed = editor();
 const txt = (p?.textContent ?? "").replace(/\s+/g, " ");
 const step = txt.match(/war[^]*\s*\d+\s*\/\s*\d+/i);
}
```

```
await click(findButton(/Боковая панель Python/), "панель Python",
state("t=0"));
for (const t of [300, 1000, 3000, 8000]) {
 await settle(t - (t === 300 ? 0 : 0));
 state(`t+${t}`);
 if (/Трасса готова|Нет шагов|Пустой код|Ошибка/.test(state)) {
 return;
 }
}
```

```
process.stdout.write("\n=== 2. вводим код ТОЛЬКО с кодами\n");
const ed = editor();
if (!ed) {
 process.stdout.write(" редактора нет – жмём Esc и ждём\n");
 await act(async () => {
 document.dispatchEvent(new window.KeyboardEvent("keydown", {
 key: "Esc",
 }));
 await sleep(30);
 });
}
const ed2 = editor();
if (!ed2) throw new Error("редактор так и не появился\n");
typeInto(ed2, "# только комментарий\n# и ещё\n");
state("ввели");
for (const t of [200, 800, 2000, 6000]) {
 await settle(t);
 state(`t+${t}`);
}
```

```
process.stdout.write("\n=== 3. вводим рабочий код x=41\n");
const ed3 = editor();
if (!ed3) throw new Error("редактор пропал после комментария\n");
typeInto(ed3, "x = 41\nx += 1\nprint(x)\n");
state("ввели");
for (const t of [200, 800, 2000, 6000]) {
 await settle(t);
 state(`t+${t}`);
}
```

```
process.stdout.write("\n=== 4. очищаем редактор полностью\n");
const ed4 = editor();
```

Универсальное пособие • полный исходный код

-----  
помарочно по DOM-сценариям, `reference-code.json` — по

## Что именно проверяется

### `verify:unit` — `live-matrix.test.mjs`

Разбор данных, которые присылает компилятор, в матрицу 1000 размерности: список списков, рванные строки, словарь слов, `"(0, 3)"` (так снапшотятся tuple-ключи Python), `"inf"`, числа-строки, обрезка до 64x64. Отдельно — что матрицей, список и словарь по именам (`dist`, `prev`): иначе визуализация чужие данные за сетку. Плюс `pickLiveVar` (приоритет имени, индексы пользователя вне диапазона демо).

### `verify:code` — `check-reference-code.mjs`

Каждый референсный код из `PAGE\_SYNC` выполняется настоящим `sys.settrace`-трейсером, что и панель компилятора. Протокол трасса не оборвана по лимиту шагов, шагов больше нуля (или пустой листинг), и \*\*каждая переменная, обещанная строкой кодом\*\* — иначе визуализация физически не может за ним следовать. Предупреждения интерпретатора, которые иначе пойдут в «Результат».

### `verify:dom` — `browser-checks.tsx`

Настоящее приложение монтируется в jsdom (`App` целиком в первой главе, визуализация, панель Python), сценарии кликают кнопки в редакторе.

| Группа | Что делает |

| --- | --- |

| А. страницы (22 × 3) | каждая глава монтируется; бейдж автозапуска инициализации настоящим Python не роняет и не возвращает `[object Object]`; нет кнопок без имени |

| В. Флойд: размерности (10) | 5×5, 10×10 (больше демо — 100×100), 100×100 (обрезка), пустая матрица, чужое имя, список, 80×80 (обрезка), пустая матрица, чужое имя

| С. матричные страницы (6) | префиксные суммы 2D и разности

| D. графы путей (2) | Дейкстра/Форд-Беллман с вершинами, гашения гаснут и оживают обратно |

| Е. эталонный код (46) | прогон кода в CPython → финальный

Универсальное пособие • полный исходный код

-----  
\* window.\_\_algv0PythonRuntime и собирает реальные комп  
\*/

```
import * as esbuild from "esbuild";
import { writeFileSync, mkdirSync } from "fs";
import { loadPyodide } from "pyodide";
import { createDom } from "./dom-env.mjs";
```

```
const OUT_DIR = "tmp/verify";
const t0 = Date.now();
const jsdomErrors = [];
```

```
log("гружаю CPython (pyodide из node_modules)...");
const py = await loadPyodide({ indexURL: "node_modules/" });
log(` Python готов за ${Date.now() - t0} мс`);
```

```
log("поднимаю jsdom...");
const { window, document, consoleErrors } = createDom({
window.__algv0PythonRuntime = py;
```

```
log("собираю приложение esbuild'ом..");
mkdirSync(OUT_DIR, { recursive: true });
await esbuild.build({
 entryPoints: ["scripts/verify/browser-checks.tsx"],
 bundle: true,
 outfile: `${OUT_DIR}/bundle.mjs`,
 format: "esm",
 platform: "browser",
 target: "es2022",
 jsx: "automatic",
 sourcemap: true,
 // Vite-специфичные глобалы, которых нет вне dev-серв
 define: {
 "process.env.NODE_ENV": '"development"',
 "import.meta.env.BASE_URL": '"/"',
 "import.meta.env.MODE": '"development"',
 "import.meta.env.DEV": "true",
 "import.meta.env.PROD": "false",
 "import.meta.env.SSR": "false",
```



Универсальное пособие • полный исходный код

```

process.exit(sum.failed > 0 || jsdomErrors.length > 0 ?
```

```
function log(message) {
 process.stdout.write(`[verify:dom] ${message}\n`);
}
```

```

ФАЙЛ 107/111: scripts/verify/run-probe.mjs
```

```
КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 45 | РАЗМ
```

```

/**
 * Зонд панели Python: печатает состояние редактора/тран
 * Не тест, а диагностика — когда нужно понять, что пан
 * ввода кода (автозапуск, переход в листинг, сообщения
 *
 * npm run verify:probe
 */
```

```
import * as esbuild from "esbuild";
import { mkdirSync } from "fs";
import { loadPyodide } from "pyodide";
import { createDom } from "../dom-env.mjs";
```

```
const OUT_DIR = "tmp/verify";
mkdirSync(OUT_DIR, { recursive: true });
const log = (m) => process.stdout.write(`[verify:probe]
```

```
log("гружаю CPython...");
const py = await loadPyodide({ indexURL: "node_modules/"
log("поднимаю jsdom...");
const { window, document } = createDom();
window.__algv0PythonRuntime = py;
```

```
await esbuild.build({
 entryPoints: ["scripts/verify/probe-compiler.tsx"],
 bundle: true,
 outfile: `${OUT_DIR}/probe-bundle.mjs`,
```

```
иначе деплой из новой рабочей ветки отклоняется .
```

```
on:
```

```
 push:
```

```
 branches:
```

```
 - main
```

```
 # Позволяет проверить Pages прямо из рабочей ветки
```

```
 - "arena/**"
```

```
 workflow_dispatch:
```

```
permissions:
```

```
 contents: read
```

```
 pages: write
```

```
 id-token: write
```

```
concurrency:
```

```
 group: pages
```

```
 cancel-in-progress: true
```

```
jobs:
```

```
 build:
```

```
 name: Build static site
```

```
 runs-on: ubuntu-latest
```

```
 steps:
```

```
 - name: Checkout
```

```
 uses: actions/checkout@v6
```

```
 - name: Setup Node.js
```

```
 uses: actions/setup-node@v7
```

```
 with:
```

```
 node-version: 24
```

```
 cache: npm
```

```
 - name: Install dependencies
```

```
 run: npm ci --no-audit --no-fund
```

```
 - name: Build for the repository subpath
```

```
 env:
```

Универсальное пособие • полный исходный код

ФАЙЛ 109/111: .github/workflows/rebuild-pdf.yml

КАТЕГОРИЯ: CI / автоматизация | СТР0К: 155 | РАЗМЕР: 6.1

name: Rebuild code PDF

# Пайплайн пересборки PDF при каждом коммите:

#

#   исходный код   →   full\_code\_all\_pages.txt   (scrip

#                   └─→   page-0001.png ... page-NNNN.png   (

#                                                   └─→   full\_code\_all\_page

#

# Готовые TXT и PDF коммитаются обратно в ветку (их отда

# промежуточные PNG сохраняются как artifacts запуска.

on:

  push:

    branches:

      - "\*\*"

    paths-ignore:

      # чтобы коммит самого workflow не запускал бескон

      - "public/export/\*\*"

      - "\*\*/\*.md"

  workflow\_dispatch:

    inputs:

      density:

        description: "DPI растеризации страниц (по умол

        required: false

        default: "150"

concurrency:

  group: rebuild-pdf-\${{ github.ref }}

  cancel-in-progress: true

permissions:

- name: Step 1 – код → txt  
run: npm run export:txt
- name: Step 2 – txt → png  
env:  
  EXPORT\_DENSITY: \${github.event.inputs.density}  
run: npm run export:png
- name: Step 3 – png → pdf  
run: npm run export:pdf
- name: Step 4 – собрать приложение и поднять pre  
run: |  
  # headless-браузер для снимков: chrome ставит  
  # этот вызов – страховка, если кэш оказался п  
  npx puppeteer browsers install chrome || true  
  npm run build  
  mkdir -p tmp  
  nohup npx vite preview --host 127.0.0.1 --por  
  for i in \$(seq 1 40); do  
    if curl -sf http://127.0.0.1:4173/ > /dev/n  
    sleep 1  
  done
- name: Step 5 – снимки визуализаций (headless-бр  
run: npm run export:shots  
continue-on-error: true
- name: Step 6 – PDF-конспект со снимками  
run: |  
  npm run export:guide  
  # снимки публикуются вместе с приложением: HT  
  # в автономный файл прямо из /export/viz-shot  
  mkdir -p public/export/viz-shots  
  cp tmp/viz-shots/\*.png tmp/viz-shots/manifest
- name: Type-check приложения  
run: npx tsc --noEmit

```

git add -- public/export/full_code_all_pages.
t/viz-shots
if git diff --cached --quiet; then
 echo "Экспорт не изменился — коммит не нужен"
 exit 0
fi
git commit -m "chore(export): пересборка full
git push origin "HEAD:${GITHUB_REF_NAME}"

```

ФАЙЛ 110/111: .github/workflows/verify.yml

КАТЕГОРИЯ: CI / автоматизация | СТРОК: 71 | РАЗМЕР: 2.3

name: Verify

# Проверки приложения на каждом пуше и PR:

```
#
verify:unit — разбор «живых» данных компилятора (l
verify:code — все эталонные Python-коды страниц вы
под трейсером панели и создают обеща
verify:dom — приложение монтируется в jsdom: стра
матрицы чужих размерностей, кнопки,
#
Headless-браузер не нужен: jsdom + pyodide из node_modules
```

on:

push:

branches:

- "\*\*"

paths-ignore:

- "public/export/\*\*"

- "\*\*/\*.md"

pull\_request:

workflow\_dispatch:

concurrency:

```
run: npm run verify:dom
```

- name: Отчёт проверок  
if: always()  
uses: actions/upload-artifact@v4  
with:  
 name: verify-report  
 path: tmp/verify/report.json  
 if-no-files-found: ignore

---

## РАЗДЕЛ: ПРОЧЕЕ

---

---

ФАЙЛ 111/111: .ai\_skills/universal\_guide\_authoring/SKIL  
КАТЕГОРИЯ: Прочее | СТРОК: 252 | РАЗМЕР: 13.3 KB

---

# Universal Guide Authoring – шаблоны страниц пособия

Обязательный справочник для **\*\*создания и правки контента** репозитории. На него ссылаются `AGENTS.md` и `GEMINI.md` собираем **\*\*только\*\*** из блоков ниже – не придумываем новый «голый» markdown.

Контент живёт в `src/data/\*\*/\*.ts` как массив объектов `src/data/content.ts` (при одинаковом `id` побеждает бо.

---  
## 1. Объект главы

```
```ts
```

```
import { Chapter } from "../../types";
```

```
export const myChapters: Chapter[] = [
```

```
...определение, абзацы, стат-сетка, аналогии, спол
    </div>
  </div>
</section>
...

```

Цвет левой полосы и заголовка блока – семантика раздела

Полоса / цвет	Смысл раздела
---	---
`border-blue-500` + `text-blue-400`	определение, «о
`border-emerald-500` + `text-emerald-400`	алгоритм,
`border-indigo-500` + `text-indigo-400`	теория, док
`border-amber-500` + `text-amber-400`	ловушки, част
`border-rose-500` + `text-rose-400`	ограничения, «к

3. Библиотека блоков (точные классы)

3.1 Определение / строгое правило – всегда по центру

```
```html
<div class="bg-slate-900 p-4 rounded-lg mb-6 text-center">
 <p class="text-lg text-emerald-300 italic mb-2">Строг
 <p class="text-xl font-mono text-white">if d[v] > d[u]
</div>
...

```

### ### 3.2 Обычный абзац

```
```html
<p class="text-slate-300 text-sm mb-4">
  Текст с <b>акцентом</b> и инлайн-кодом
  <span class="font-mono text-emerald-300">dist[v]</span>
</p>
...

```

```

    </div>
</div>
```

```

Второй карточкой может быть «Аналогия 2», «Ограничения» «Когда не работает» — маккуп тот же (пунктирная rose-ра

### ### 3.5 Инфо-блок с таблицей

```

```html
<div class="bg-slate-800/70 p-5 rounded-lg border borde
  <p class="text-sm font-bold text-white mb-2">  Заголо
  <p class="text-slate-300 text-sm mb-3">Пояснение пере
  <table class="w-full text-xs text-left border-collaps
    <thead>
      <tr class="text-slate-500 border-b border-slate-70
        <th class="py-1.5 pr-2 font-bold">Колонка</th>
        <th class="py-1.5 pr-2 font-bold">Колонка</th>
      </tr>
    </thead>
    <tbody>
      <tr class="border-b border-slate-800 align-top">
        <td class="py-2 pr-3 font-bold">Значение</td>
        <td class="py-2 pr-3 font-mono">O(log n)</td>
      </tr>
    </tbody>
  </table>
</div>
```

```

### ### 3.6 Спойлер `<details>` — **\*\*обязателен\*\*** для кода, и

```

```html
<details class="bg-slate-900/40 rounded-lg border borde
  <summary class="p-4 font-bold text-slate-400 outline-l
    e-700/50">
    ▶ Код: Дейкстра на куче (Python)
  </summary>

```

по `id` главы после текста.

5. Контракт «демонстрация ↔ компилятор»

Каждая страница обязана иметь и визуализацию, и синхронизацию:

- `src/components/vizRegistry.tsx` – запись с ключом ****`id`**** (``title``, опциональный ``hint``, ``Component``).
- `src/data/vizSync.ts` – запись ``PAGE_SYNC[id]`` или ``PAGE_SYNC[title]`, где ``demoId`` – второй аргумент ``emitVizDemo(demoId, title)`` (вкладки одной демонстрации).
- Поля записи: ``vizTitle``, ``stepNote`` (что считается одной страницей), ``variables: [{ name, role, range }]``, ``code`` (референс на код).
- Переменные, которые читает визуализация, должны быть ``int``, ``list``, ``dict``, ``str``: экземпляры классов ``Int``, ``List``, ``Dict``, ``Str`` до графики не доезжают.
- Стартовый шаблон в редакторе компилятора – ``buildInit``, инициализация переменных до строки ``# ---`` – тело алгоритма.

Проверка перед коммитом: ``npm run dev`` → страница открылась, код подставляет код, шаг демонстрации подсвечивает нужные переменные.

6. Экспорт в PDF (``npm run export:guide``)

Текст страниц парсится ``scripts/export/html-to-blocks.mjs``, результат – ``public/export/guide_lite.pdf`` (текст + визуализации, без кода компилятора и без навигации). Чтобы экспорт был полным:

- заголовки – только ``h2/h3/h4``; списки – ``ul/ol/li``; теги кода – ``pre``; определения – обычный ``p`` (см. 3.1);
- ****смысл не должен держаться на эмодзи****: в PDF используются только ASCII-глифы – пиктограммы вырезаются, остаётся «Операции на отрезках» доедет как «Операции на отрезках», а вот «Операции на отрезках / Операции на отрезках» потеряет смысл, пиктограммы вырезаются.